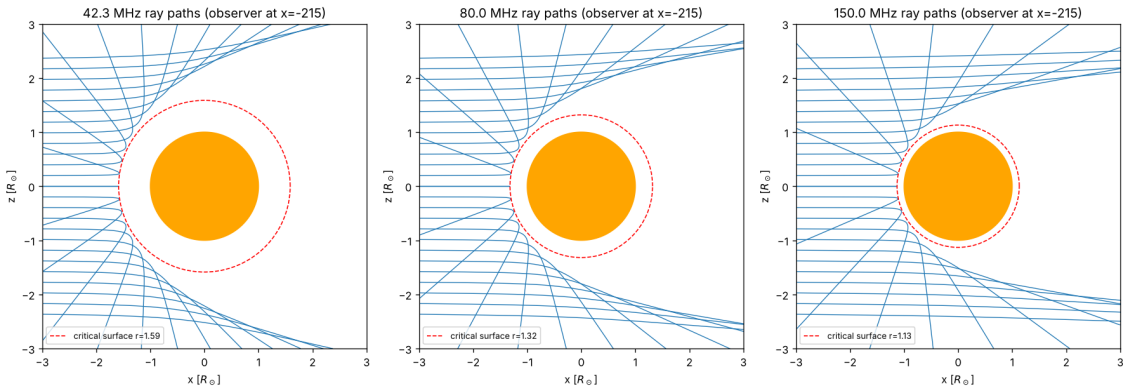
radiosynth

From MHD to MHz

Synthetic Radio Imaging of MHD Solar–Stellar Coronal Model Output and Comparison with Observations

A standalone Python implementation of the BATS-R-US refractive radio ray-tracing tool, with thermal free–free, gyroresonance and gyrosynchrotron emission and circular polarisation,
for BATS-R-US, PLUTO and other MHD codes — the Sun and other stars



Line-by-line Code Reference

Companion to the User Manual, `radiosynth_user_manual.pdf`,
to which the references to chapters, sections and equations point.

Soumitra Hazra

Second version (2.2.0) — November 2026

Free software under the GNU General Public License, version 3 or later

Algorithm after Benkevitch et al. (2010, ApJ 725, 447) and Moschou et al. (2018, ApJ 867, 51)

Contents

How this document is organised **1**

1 common/constants.py — physical constants **2**

Module header and imports . . . 2
Constant definitions . . . 2
critical_density() . . . 2
plasma_frequency() . . . 3

2 common/magnetoionic.py — cold-plasma magneto-ionic theory **4**

Module docstring and imports . . . 4
Gyrofrequency . . . 5
gyrofrequency() and magnetoionic_params() . . . 5
axial_ratio_x() . . . 5
refractive_index2() . . . 5
cutoff() . . . 6

3 common/grid.py — model grids and interpolation **7**

Module docstring and import . . . 7
MHDGrid: class docstring . . . 7
MHDGrid.__init__: input checks . . . 8
MHDGrid.__init__: gradient and storage . . . 8
MHDGrid.__init__: helpers for fast lookup . . . 9
MHDGrid: properties . . . 9
MHDGrid.inside() . . . 10
MHDGrid._locate(): find the cell and the fractional position . . . 10
MHDGrid.sample(): trilinear interpolation . . . 10
MHDGrid: magnetic field . . . 11
MHDGrid.box_intersection(): where a straight ray enters and leaves the box . . . 12
MHDGrid: common API . . . 13
SphericalGrid: class docstring . . . 13
SphericalGrid.__init__: fields . . . 14
SphericalGrid.__init__: domain extent . . . 14
SphericalGrid.__init__: gradient components . . . 15
SphericalGrid: description and properties . . . 16
SphericalGrid._sph(): Cartesian to spherical . . . 16

SphericalGrid._phi_rel() and inside() . . . 17
SphericalGrid._locate() . . . 17
SphericalGrid.sample(): locate in r and theta . . . 17
SphericalGrid.sample(): longitude handling . . . 18
SphericalGrid.sample(): trilinear interpolation in index space . . . 18
SphericalGrid.sample(): gradient to Cartesian, cell size . . . 18
SphericalGrid: magnetic field . . . 19
SphericalGrid.domain_intersection() . . . 21
SphericalGrid.to_npz() . . . 21

4 common/readers.py — BATS-R-US and generic input 22
Module docstring and imports . . . 22
Variable-name helpers . . . 22
Unit tables . . . 23
find_magnetic_field(): Bx, By, Bz in gauss . . . 23
_density_to_ne(): density in any unit to n_e . . . 24
_pressure_si() . . . 25
derive_ne_te(): electron density . . . 25
derive_ne_te(): electron temperature . . . 25
_unique_axis() . . . 26
points_to_grid(): tensor grid or resampling . . . 27
points_to_grid(): resampling scattered points . . . 27
build_grid() and grid_from_arrays() . . . 28
_standardise_xyz() . . . 29
read_tecplot_ascii() . . . 30
read_pytecplot() . . . 31
_read_records(): Fortran unformatted files . . . 31
read_batsrus_idl(): binary IDL files . . . 32
read_batsrus_idl(): ASCII files and units . . . 33
load_mhd(): the general entry point . . . 34

5 common/pluto.py — PLUTO input 36
Module docstring . . . 36
PLUTO constants and default units . . . 37
parse_definitions() . . . 37
read_grid_out() . . . 38
_read_out_file() . . . 38
_stag_shape() . . . 39
read_pluto_binary() . . . 39

read_pluto_vtk(): header and helpers . . . 40
read_pluto_vtk(): sections . . . 41
read_pluto_vtk(): geometry . . . 42
_import_pypluto(): find a pyPLUTO module . . . 42
_to_n123(): put arrays in (n1, n2, n3) order . . . 43
read_pluto_pypluto(): read through pyPLUTO . . . 43
_find_var() . . . 44
load_pluto(): read and find the units . . . 45
load_pluto(): physical fields . . . 46
load_pluto(): grid . . . 46
load_pluto(): magnetic field . . . 47
load_pluto(): report . . . 48

6 common/fields.py — derived fields (current density) 49
Module docstring and constants . . . 49
_grad() . . . 49
curl_b() . . . 49
current_density() . . . 50

7 common/mechanism.py — the mechanism registry 51
Module docstring . . . 51
Registry storage . . . 51
class Mechanism . . . 52
register() . . . 52
get_mechanism() . . . 52
available_mechanisms() . . . 52

8 common/emission.py — the emission-model interface 54
Module docstring and imports . . . 54
Constructor . . . 54
Evaluation: ___call___(ne, te, freq, length_unit) . . . 55

9 common/raytrace.py — the ray tracer 56
Module docstring and imports . . . 56
Ray status codes . . . 57
_emission_function(): the emission model as a function . . . 57
_add_emission(): radiative transfer over a segment . . . 57
_predictor(): half step and plasma at the mid point . . . 58
_terminations(): rays that end at the mid point . . . 58

_integrate_to_body(): the last piece before the photosphere . . . 58
_dielectric(): permittivity and its gradient . . . 59
_snell_fallback(): rays that crossed the critical surface (default mode) . . . 59
_check_step(): precision control . . . 60
_parabolic_step(): reflection at the critical surface . . . 61
_boris_step(): Boris (CYLRAD) rotation . . . 61
_advance(): move every ray . . . 62
_clip_at_body(): a step that ends inside the photosphere . . . 62
_new_step_size(): step regrowth . . . 63
trace_rays(): signature and docstring . . . 63
trace_rays(): set-up . . . 64
trace_rays(): the main loop . . . 65
trace_rays(): return value . . . 67

10 common/straight.py — straight-ray sampler 68
Module docstring and imports . . . 68
segment_ends(): where each line ends . . . 68
default_step() . . . 69
iter_samples(): chunks of sampled rays . . . 69

11 common/image.py — image set-up and driver 71
Module docstring and imports . . . 71
parse_frequencies() . . . 71
freq_label() . . . 72
image_geometry(): pixels and ray directions . . . 72
start_points(): where each ray starts . . . 73
Parallel worker . . . 74
RadioImage: container . . . 74
RadioImage: derived quantities . . . 75
RadioImage.summary() . . . 75
Output methods . . . 76
RadioImage.plot() . . . 77
make_radio_image(): the main entry point . . . 78
make_radio_image(): tracing . . . 79
make_radio_image(): result . . . 80
trace_paths(): individual trajectories . . . 80
compare_refraction(): refracted vs straight lines of sight . . . 81

12 common/symmetric.py — 1D (spherically symmetric) models **83**
Module docstring and imports . . . 83
grid_from_profile() . . . 83
read_profile(): text tables . . . 84
is_symmetric() . . . 85
_rotate() . . . 85
make_symmetric_image(): one row of rays . . . 85
make_symmetric_image(): rotation and flux . . . 86
plot_profile() . . . 87

13 common/figures.py — figures in the style of Moschou et al. (2018) **89**
Module docstring and imports . . . 89
earth_observer() . . . 90
_julian_day() . . . 90
sun_ephemeris(): B0, L0, P and distance . . . 90
observer_for_date() . . . 92
rsun_to_arcsec(), image_extent(), limb_radius() . . . 92
Helper functions . . . 93
read_fits_map(): observed FITS maps . . . 94
figure_single_map(): Fig. 3 style . . . 95
figure_compare_observation(): Fig. 2 style . . . 96
make_time_series() . . . 98
figure_time_series(): Fig. 5 style . . . 98
animate_time_series() . . . 100
_regrid_to_obs() . . . 101
fit_longitude_offset() . . . 101
compare_with_observation() . . . 103

14 common/stellar.py — stellar radio emission **106**
Module docstring and constants . . . 106
stellar_observer() . . . 106
_default_fov() . . . 107
make_stellar_image() . . . 107
spectral_index() . . . 107
stellar_spectrum() . . . 107
rotational_modulation() . . . 108
Re-export of the wind solution . . . 109

15 common/fitsio.py — FITS input/output without astropy **110**
Constants . . . 110
_parse_value() . . . 110
read_fits() . . . 110
write_fits() . . . 111

16 common/cli.py — command-line interface **113**
Module docstring and imports . . . 113
Options: image and physics . . . 113
Options: 1D models . . . 116
Options: PLUTO and output . . . 116
Running . . . 117
Imaging and output . . . 118

17 simplistic/emission.py — the simplistic mechanism **122**
Module docstring and imports . . . 122
simplistic_coefficients() . . . 122

18 simplistic/___init___.py — package interface **123**
Mechanism registration . . . 123

19 bremsstrahlung/emission.py — thermal free–free coefficients **124**
Module docstring and imports . . . 124
The prefactor . . . 124
bremsstrahlung_coefficients() . . . 124

20 bremsstrahlung/magnetized.py — free–free emission of the two magneto-ionic modes **126**
Module docstring and imports . . . 126
magnetized_ff_coefficients() . . . 126
rayleigh_jeans_mode() . . . 127

21 bremsstrahlung/wind.py — analytic stellar-wind solution **128**
Module docstring and imports . . . 128
wind_optical_depth(): analytic reference . . . 128
wind_flux_density(): analytic flux of an optically thick wind . . . 129

22 bremsstrahlung/___init___.py — package interface **130**
Mechanism registration . . . 130

23 gyroresonance/opacity.py — optical depth of a harmonic layer 132
Module docstring and imports . . . 132
Prefactor . . . 133
mode_factor() . . . 133
layer_optical_depth() . . . 133
harmonic_field() . . . 134

24 gyroresonance/transfer.py — polarised transfer along straight rays 135
Module docstring . . . 135
Imports . . . 135
transfer_chunk(): fields along the rays . . . 136
transfer_chunk(): harmonic layers . . . 137
transfer_chunk(): choice of the two polarisations . . . 138
transfer_chunk(): formal solution . . . 138
transfer_chunk(): Stokes V and diagnostics . . . 139

25 gyroresonance/image.py — make_gr_image and GRImage 141
Module docstring and imports . . . 141
GRImage: attributes . . . 141
GRImage: derived quantities . . . 142
GRImage: summary and output . . . 143
GRImage.plot() . . . 143
make_gr_image(): set-up . . . 145
make_gr_image(): loop over frequencies and chunks . . . 146

26 gyroresonance/___init___.py — package interface 148
Mechanism registration . . . 148

27 gyrosynchrotron/library.py — the fast GS codes as a library 149
Module docstring and imports . . . 149
_candidate_names() . . . 150
find_library() . . . 150
GSLibrary: ctypes binding . . . 150
load_library() . . . 152
build_library() . . . 152
library_version() . . . 153
Command line . . . 153

28 gyrosynchrotron/electrons.py — the fast-electron prescription **154**
Module docstring . . . 154
Index tables . . . 155
FastElectrons: constructor . . . 155
FastElectrons.from_array() . . . 156
Evaluation of numbers and rules . . . 156
Energy check . . . 157
Rules for n_b . . . 157

29 gyrosynchrotron/image.py — make_gs_image and GSImage **159**
Module docstring and imports . . . 159
GSImage . . . 160
_fields(): voxel quantities . . . 160
voxel_parms(): the parameters of the codes . . . 161
make_gs_image(): set-up . . . 161
make_gs_image(): one chunk of lines of sight . . . 163
make_gs_image(): the image . . . 165

30 gyrosynchrotron/___init___.py — package interface **166**
Mechanism registration . . . 166

31 combined/coefficients.py — gyrosynchrotron coefficients per voxel **168**
Module docstring and constants . . . 168
gs_voxel_coefficients() . . . 168
source_weights() . . . 169

32 combined/image.py — make_combined_image and CombinedImage **171**
Module docstring and imports . . . 171
CombinedImage . . . 172
make_combined_image(): set-up . . . 174
make_combined_image(): one chunk . . . 175
make_combined_image(): refraction correction of the free-free part . . . 177
make_combined_image(): the image . . . 177

33 combined/___init___.py — package interface **179**
Mechanism registration . . . 179

34 radiosynth/___init___.py — package interface **180**
Docstring: the package layout . . . 180
Public names . . . 181

Short module paths 181

35 radiosynth/___main___.py — python -m radiosynth 183

Entry point 183

How this document is organised

This document describes the source code of `radiosynth` line by line; how to use the package and the physics are in the User Manual. The chapters follow the folders of the package (User Manual, Section 1.2): first the common machinery (`radiosynth/common/`), then one group of chapters per emission mechanism (`simplistic/`, `bremsstrahlung/`, `gyroresonance/`), and finally the package files `__init__.py` and `__main__.py`. Each chapter lists the source in blocks, with the line numbers of the file, followed by a commentary that refers to individual lines. The listings are generated from the source files at the time the manual is built, so the line numbers always match the distributed code. `radiosynth` is written by Soumitra Hazra and is distributed under the GNU General Public License, version 3 or later; every source file starts with the corresponding copyright and licence lines.

Chapter 1

common/constants.py — physical constants

File: `radiosynth/common/constants.py` — 38 lines.

All numbers in SI units. The solar radius is the BATS-R-US value (6.96e8 m), so that lengths in $R_\odot$ convert exactly as in the Fortran code.

Module header and imports

lines 4–10

```
4  """Physical constants (SI) used by radiosynth.
5
6  Values follow the BATS-R-US ModConst conventions where it matters
7  (solar radius 6.96e8 m) so that results can be compared one-to-one.
8  """
9  import numpy as np
10 from scipy import constants as _c
```

L.10 `scipy.constants` provides CODATA values of k_B, m_e, m_p, e, c, ε_0.

Constant definitions

lines 12–25

```
12 PI = np.pi
13 K_B = _c.k                      # Boltzmann constant [J/K]
14 M_E = _c.m_e                    # electron mass [kg]
15 M_P = _c.m_p                    # proton mass [kg]
16 E_CHARGE = _c.e                 # elementary charge [C]
17 C_LIGHT = _c.c                  # speed of light [m/s]
18 EPS0 = _c.epsilon_0
19 # e^2/(4 pi eps0)  [J m]  == cElectronChargeSquaredJm in BATS-R-US
20 E2_JM = E_CHARGE**2 / (4.0 * PI * EPS0)
21 R_SUN = 6.96e8                   # BATS-R-US rSun [m]
22 AU = 1.495978707e11              # [m]
23 AU_RSUN = AU / R_SUN            # ~215 R_sun
24 JANSKY = 1.0e-26                # W m^-2 Hz^-1
25 SFU = 1.0e-22                   # solar flux unit
```

L.13 Boltzmann constant [J/K]; used in B_ν and in $T_e = P/(nk_B)$.

L.14–17 Electron and proton masses, elementary charge, speed of light.

L.20 $e^2/(4\pi\varepsilon_0)$ in J m. This is `cElectronChargeSquaredJm` of BATS-R-US and appears in $n_{\rm cr}$ and in κ_ν.

L.21 Solar radius used to convert path lengths in $R_\odot$ to metres.

L.22–23 1 AU in metres and in solar radii (about 215); a natural observer distance.

L.24–25 Flux units: 1 Jy $= 10^{-26}$ and 1 sfu $= 10^{-22}$ W m^{-2} Hz^{-1}.

critical_density()

lines 28–33

```
28 def critical_density(freq_hz):
29     """Electron density [m^-3] at which the plasma frequency equals freq_hz.
30 
31     n_cr = pi m_e f^2 / (e^2/(4 pi eps0))   (identical to BATS-R-US NElectronSiCr)
32     """
33     return PI * M_E * np.asarray(freq_hz, dtype=float)**2 / E2_JM
```

L.33 Eq. (18.1): $n_{\mathrm{cr}} = \pi m_e f^2/(e^2/4\pi\varepsilon_0)$, identical to `NElectronSiCr = cPi*cElectronMass*RadioFrequency**2/cElectronChargeSquaredJm`. It accepts scalars or arrays.

plasma_frequency()

lines 36–38

```
36 def plasma_frequency(ne_si):
37     """Electron plasma frequency [Hz] for density [m^-3]."""
38     return np.sqrt(np.asarray(ne_si) * E2_JM / (PI * M_E))
```

L.38 The inverse relation, $f_p = \sqrt{n_e e^2/(4\pi\varepsilon_0\,\pi m_e)} \approx 8.98\sqrt{n_e[\mathrm{m}^{-3}]}$ Hz.

Chapter 2

common/magnetoionic.py — cold-plasma magneto-ionic theory

File: `radiosynth/common/magnetoionic.py` — 82 lines.

Shared by the magnetised free–free coefficients and the gyroresonance opacity: the parameters X, Y, the axial ratio of the modes, the Appleton–Hartree refractive index and the cutoffs.

Module docstring and imports

lines 4–36

```
4  """Cold-plasma magneto-ionic theory shared by the polarized mechanisms.
5
6  Notation (all dimensionless):
7
8      X = (nu_p / nu)^2 = n_e / n_cr          (n_cr: critical density)
9      Y = nu_B / nu,      nu_B = e B / (2 pi m_e c) = 2.7992 MHz * B[G]
10     cos(theta) = B . k / (|B| |k|),   k pointing towards the observer
11
12 The two natural (magneto-ionic) modes are the extraordinary (x) mode, whose
13 electric vector rotates in the same sense as the gyrating electrons, and the
14 ordinary (o) mode.  Appleton-Hartree refractive indices:
15
16     n^2 = 1 - X (1-X) / [ (1-X) - Y^2 sin^2/2  -/+ sqrt(Y^4 sin^4/4 + (1-X)^2 Y^2 cos^2) ]
17
18 (upper sign: o-mode, lower sign: x-mode).  Cutoffs: o-mode X = 1, x-mode
19 X = 1 - Y (for Y < 1).
20
21 Mode polarisation: in the frame with B along z and k in the x-z plane the
22 transverse electric vector of mode sigma is e ~ (T_sigma cos(theta), i, -T_sigma
23 sin(theta)) / sqrt(1 + T_sigma^2) (longitudinal part neglected), with axial
24 ratio T_x T_o = -1 and
25
26     |T_x| = 2 (1-X) |cos| / [ Y sin^2 + sqrt(Y^2 sin^4 + 4 (1-X)^2 cos^2) ]
27
28 so |T_x| = 1 (circular) for parallel propagation and 0 (linear, E perp B) for
29 perpendicular propagation.  Handedness: for B . k > 0 the x-mode is
30 right-hand circularly polarised in the IAU/IEEE radio convention (Stokes V > 0).
31 """
32 import numpy as np
33
34 from .constants import E_CHARGE, M_E, PI, critical_density
35
36 #: electron gyrofrequency per gauss [Hz/G]:  e B / (2 pi m_e)  with B = 1e-4 T
```

Defines the notation (X, Y, θ with $\mathbf{k}$ towards the observer), the modes, the Appleton–Hartree formula, the polarisation vector $(T\cos\theta,\ i,\ -T\sin\theta)/\sqrt{1+T^2}$ and the handedness convention: for $\mathbf{B}\cdot\mathbf{k} > 0$ the x-mode is right-hand circularly polarised (IAU/IEEE), $V > 0$.

Gyrofrequency

lines 37–37

```
37 NU_B_PER_GAUSS = E_CHARGE * 1e-4 / (2.0 * PI * M_E)        # 2.799249e6 Hz/G
```

L.37 $\nu_B = eB/(2\pi m_e)$ per gauss: 2.7992 MHz/G.

gyrofrequency() and magnetoionic_params()

lines 40–49

```
40 def gyrofrequency(b_gauss):
41     """Electron gyrofrequency [Hz] for |B| in gauss."""
42     return NU_B_PER_GAUSS * np.asarray(b_gauss, dtype=float)
43
44
45 def magnetoionic_params(ne, b_gauss, freq):
46     """X = n_e/n_cr and Y = nu_B/nu for density [m^-3], |B| [G] and frequency [Hz]."""
47     X = np.asarray(ne, dtype=float) / critical_density(freq)
48     Y = gyrofrequency(b_gauss) / freq
49     return X, Y
```

L.47 $X = n_e/n_{\rm cr} = (\nu_p/\nu)^2$ (same $n_{\rm cr}$ as the ray tracer).

L.48 $Y = \nu_B/\nu$.

axial_ratio_x()

lines 52–60

```
52 def axial_ratio_x(X, Y, cos_theta):
53     """|T_x|, the axial ratio of the x-mode (see module docstring);
54     |T_o| = 1/|T_x|."""
55     c = np.abs(np.asarray(cos_theta, dtype=float))
56     s2 = np.maximum(1.0 - c * c, 0.0)
57     a = np.maximum(1.0 - np.asarray(X, dtype=float), 1e-12)
58     Y = np.asarray(Y, dtype=float)
59     den = Y * s2 + np.sqrt(Y * Y * s2 * s2 + 4.0 * a * a * c * c)
60     return np.where(den > 0, 2.0 * a * c / np.where(den > 0, den, 1.0), 1.0)
```

L.59 $|T_x| = 2(1-X)|\cos\theta|/[Y\sin^2\theta + \sqrt{Y^2\sin^4\theta + 4(1-X)^2\cos^2\theta}]$: 1 (circular) along $\mathbf{B}$, 0 (linear, $\mathbf{E}\perp\mathbf{B}$) across it.

L.60 Guard against $0/0$ when $Y = 0$ and $\cos\theta = 0$.

refractive_index2()

lines 63–74

```
63 def refractive_index2(X, Y, cos_theta, mode):
64     """Appleton-Hartree n^2 for mode 'o' or 'x' (cold, collisionless plasma)."""
65     X = np.asarray(X, dtype=float)
66     Y = np.asarray(Y, dtype=float)
67     c2 = np.asarray(cos_theta, dtype=float) ** 2
68     s2 = 1.0 - c2
69     a = 1.0 - X
70     root = np.sqrt(0.25 * Y**4 * s2 * s2 + a * a * Y * Y * c2)
71     sgn = 1.0 if mode == "o" else -1.0
```

```
72      den = a - 0.5 * Y * Y * s2 + sgn * root
73      with np.errstate(divide="ignore", invalid="ignore"):
74          return 1.0 - X * a / den
```

L.70–72 Appleton–Hartree with the + root for the o-mode and − for the x-mode: along $\mathbf{B}$, $n^2 = 1 - X/(1 \pm Y)$.

cutoff()

lines 77–82

```
77  def cutoff(X, Y, mode):
78      """True where mode 'o' (X >= 1) or 'x' (X >= 1 - Y) cannot propagate."""
79      X = np.asarray(X, dtype=float)
80      if mode == "o":
81          return X >= 1.0
82      return X >= 1.0 - np.asarray(Y, dtype=float)
```

o-mode: $X \geq 1$; x-mode: $X \geq 1 - Y$. The transfer stops a polarisation at its cutoff.

Chapter 3

common/grid.py — model grids and interpolation

File: `radiosynth/common/grid.py` — 523 lines.

The ray tracer asks a grid object for five things at arbitrary points: n_e, ∇n_e, T_e, the local cell size, and whether the point is inside the domain. `MHDGrid` (Cartesian, rectilinear) and `SphericalGrid` (r, θ, ϕ) provide the same interface: `sample(p)`, `inside(p)`, `domain_intersection(o, d)`, `r_body`, `to_npz()`. This replaces BATS-R-US's `get_local_density`, `interpolate_grid_amr_gc`, `calc_gradient_ghost` and `CellSize_DB`.

Module docstring and import

lines 4–19

```
"""Cartesian (rectilinear) MHD grid with fast vectorised trilinear sampling.

This replaces the BATS-R-US machinery used by ModRadioWaveImage
(``interpolate_grid_amr_gc``, ``calc_gradient_ghost``, ``CellSize_DB``):

* electron density n_e [m^-3] and its gradient [m^-3 per R_sun] are stored
  on the grid and trilinearly interpolated to the ray position, exactly as
  BATS-R-US interpolates State_VGB and GradVar_DGB;
* the recommended step ``DeltaSNew`` is the local minimum cell size.

Coordinates are in units of R_sun (as in BATS-R-US).  The axes may be
non-uniform (stretched) but must be monotonically increasing.
"""
import numpy as np

from .constants import R_SUN
```

Explains the correspondence with the BATS-R-US interpolation routines. The density gradient is pre-computed on the grid and interpolated, exactly as BATS-R-US interpolates `GradVar_DGB`.

MHDGrid: class docstring

lines 22–39

```
class MHDGrid:
    """Electron density and temperature on a rectilinear 3D grid.

    Parameters
    ----------
    x, y, z : 1D arrays, strictly increasing, in R_sun
    ne      : (nx, ny, nz) electron density [m^-3]
    te      : (nx, ny, nz) electron temperature [K]  (may be None for the
              'simplistic' emission model)
    r_body  : radius of the inner boundary (photosphere) in R_sun; rays that
              reach it are stopped (IsBody in BATS-R-US).
    dtype   : storage precision of the interpolated fields (float32 halves
              memory; arithmetic is always done in float64).
```

```
35      length_unit : metres per coordinate unit (default R_sun; for a star with
36                coordinates in stellar radii use r_star * R_SUN)
37      """
38
39      NFIELD = 5   # ne, dne/dx, dne/dy, dne/dz, Te
```

L.39 Five fields are stored per grid point, interleaved in one array: n_e, $\partial_x n_e$, $\partial_y n_e$, $\partial_z n_e$, T_e. Interleaving means one memory fetch per cell corner returns all five values.

MHDGrid.___init___: input checks

lines 41–58

```
41      def __init__(self, x, y, z, ne, te=None, r_body=1.0, dtype=np.float32, length_unit=R_SUN)
        ↪ :
42          self.x = np.asarray(x, dtype=float)
43          self.y = np.asarray(y, dtype=float)
44          self.z = np.asarray(z, dtype=float)
45          for a, name in ((self.x, "x"), (self.y, "y"), (self.z, "z")):
46              if a.ndim != 1 or a.size < 2 or np.any(np.diff(a) <= 0):
47                  raise ValueError(f"axis {name} must be 1D, strictly increasing, size>=2")
48          shape = (self.x.size, self.y.size, self.z.size)
49          ne = np.asarray(ne, dtype=float)
50          if ne.shape != shape:
51              raise ValueError(f"ne has shape {ne.shape}, expected {shape}")
52          ne = np.where(np.isfinite(ne), np.maximum(ne, 0.0), 0.0)
53          if te is None:
54              te = np.ones(shape)
55          te = np.asarray(te, dtype=float)
56          if te.shape != shape:
57              raise ValueError(f"te has shape {te.shape}, expected {shape}")
58          te = np.where(np.isfinite(te), te, 1.0)
```

L.42–44 Axes as float arrays (in $R_\odot$).

L.45–47 Each axis must be 1D, strictly increasing and have at least two points; otherwise interpolation is undefined.

L.48–51 The density must be shaped (nx, ny, nz) in `ij` ordering, the same order as the axes.

L.52 NaN or infinite values become 0 and negative densities are clipped to 0. This guards against ghost-cell or undefined values in model output.

L.53–58 Temperature is optional (not needed for the simplistic mechanism); it defaults to 1 K. Non-finite values become 1 K.

MHDGrid.___init___: gradient and storage

lines 60–72

```
60          self.shape = shape
61          self.r_body = float(r_body)
62          self.length_unit = float(length_unit)
63          # density gradient per R_sun (2nd order central, like BATS-R-US)
64          gx, gy, gz = np.gradient(ne, self.x, self.y, self.z, edge_order=2)
65          f = np.empty(shape + (self.NFIELD,), dtype=dtype)
66          f[..., 0] = ne
67          f[..., 1] = gx
68          f[..., 2] = gy
69          f[..., 3] = gz
```

```
70          f[..., 4] = te
71          self._flat = f.reshape(-1, self.NFIELD)
72          del f, gx, gy, gz
```

L.61 Inner boundary radius: a ray stops when it reaches it (BATS-R-US `IsBody`).

L.62 Metres per coordinate unit: $R_\odot$ by default, R_* for stellar models. The tracer converts path lengths with it.

L.64 Density gradient with second-order central differences in the interior and second-order one-sided differences at the edges. Works for non-uniform axes. The unit is m^{-3} per $R_\odot$, matching positions in $R_\odot$.

L.65–70 Fill the interleaved array, by default in float32 to halve memory (5 fields $\times$ 4 bytes per point).

L.71 Flatten to (Npoints, 5) so that a single integer index addresses a grid point.

L.72 Free the temporary arrays.

MHDGrid.___init___: helpers for fast lookup

lines 73–85

```
73          self.dx = np.diff(self.x)
74          self.dy = np.diff(self.y)
75          self.dz = np.diff(self.z)
76          self.lo = np.array([self.x[0], self.y[0], self.z[0]])
77          self.hi = np.array([self.x[-1], self.y[-1], self.z[-1]])
78          self._uniform = all(np.allclose(d, d[0], rtol=1e-6) for d in (self.dx, self.dy, self.
        ↪ dz))
79          self._strides = np.array([shape[1] * shape[2], shape[2], 1], dtype=np.int64)
80          offs = []
81          for di in (0, 1):
82              for dj in (0, 1):
83                  for dk in (0, 1):
84                      offs.append(di * self._strides[0] + dj * self._strides[1] + dk)
85          self._corner_off = np.array(offs, dtype=np.int64)
```

L.73–75 Cell widths along each axis; used for interpolation weights and the local cell size.

L.76–77 Corners of the domain box.

L.78 If all three axes are uniform, locating a cell is a division (`floor`) rather than a binary search.

L.79 Flat-index strides for C-ordered (nx, ny, nz) arrays: index $= i\,n_y n_z + j\,n_z + k$.

L.80–85 Pre-compute the 8 flat-index offsets of the cell corners $(i+d_i, j+d_j, k+d_k)$, $d \in \{0,1\}$.

MHDGrid: properties

lines 87–99

```
87      # ------------------------------------------------------------------
88      @property
89      def nbytes(self):
90          return self._flat.nbytes
91
92      def __repr__(self):
93          return (f"MHDGrid(shape={self.shape}, x=[{self.x[0]:.3g},{self.x[-1]:.3g}], "
94                  f"y=[{self.y[0]:.3g},{self.y[-1]:.3g}], z=[{self.z[0]:.3g},{self.z[-1]:.3g}],
        ↪  "
```

```
95                 f"min cell={self.min_cell:.3g} R_sun, r_body={self.r_body})")
96
97     @property
98     def min_cell(self):
99         return float(min(self.dx.min(), self.dy.min(), self.dz.min()))
```

L.89 Memory used by the stored fields.

L.92 One-line description printed by the driver (shape, extent, smallest cell, r_{body}).

L.98 Smallest cell width over all axes.

MHDGrid.inside() lines 101–103

```
101     def inside(self, p):
102         """Boolean mask: points (N,3) lie within the grid box."""
103         return np.all((p >= self.lo) & (p <= self.hi), axis=1)
```

L.103 A point is inside if all three coordinates lie within $[\mathrm{lo}, \mathrm{hi}]$. Vectorised over N points.

MHDGrid._locate(): find the cell and the fractional position lines 105–112

```
105     def _locate(self, axis, d, u):
106         if self._uniform:
107             i = np.floor((u - axis[0]) / d[0]).astype(np.int64)
108         else:
109             i = np.searchsorted(axis, u, side="right") - 1
110         i = np.clip(i, 0, axis.size - 2)
111         t = (u - axis[i]) / d[i]
112         return i, np.clip(t, 0.0, 1.0)
```

L.106–107 Uniform axis: $i = \lfloor (u - x_0)/\Delta x \rfloor$.

L.109 Non-uniform axis: binary search, $O(\log n)$ per point.

L.110 Clip to valid cells $[0, n-2]$; points outside are evaluated at the boundary value.

L.111–112 Fractional position $t \in [0, 1]$ inside the cell, used as the linear weight.

MHDGrid.sample(): trilinear interpolation lines 114–144

```
114     def sample(self, p):
115         """Trilinear interpolation at points p (N,3).
116
117         Returns
118         -------
119         ne   : (N,)   electron density [m^-3]
120         grad : (N,3)  density gradient [m^-3 / R_sun]
121         te   : (N,)   electron temperature [K]
122         cell : (N,)   local minimum cell size [R_sun]  (BATS-R-US DeltaSNew)
123         inside : (N,) bool, point within the grid box
124         """
125         p = np.asarray(p, dtype=float)
126         ins = self.inside(p)
```

```
        i, tx = self._locate(self.x, self.dx, p[:, 0])
        j, ty = self._locate(self.y, self.dy, p[:, 1])
        k, tz = self._locate(self.z, self.dz, p[:, 2])
        base = i * self._strides[0] + j * self._strides[1] + k
        out = np.zeros((p.shape[0], self.NFIELD))
        wx = (1 - tx, tx)
        wy = (1 - ty, ty)
        wz = (1 - tz, tz)
        c = 0
        for di in (0, 1):
            for dj in (0, 1):
                wxy = wx[di] * wy[dj]
                for dk in (0, 1):
                    w = wxy * wz[dk]
                    out += w[:, None] * self._flat[base + self._corner_off[c]]
                    c += 1
        cell = np.minimum(np.minimum(self.dx[i], self.dy[j]), self.dz[k])
        return np.maximum(out[:, 0], 0.0), out[:, 1:4], out[:, 4], cell, ins
```

L.126 Domain flag returned to the tracer.

L.127–129 Cell indices and weights along x, y, z.

L.130 Flat index of the lower corner of each point's cell.

L.131–134 Accumulator for the 5 fields, and the linear weights $(1-t, t)$ along each axis.

L.136–142 Sum over the 8 corners: weight $w = w_x w_y w_z$ times the corner's 5 field values. Each step gathers N rows of the field table at once. This is the trilinear interpolation of BATS-R-US (`Weight_I` in `get_local_density`).

L.143 Local cell size $\min(\Delta x, \Delta y, \Delta z)$: the target step `DeltaSNew`.

L.144 Return n_e (clipped ≥ 0), ∇n_e (N,3), T_e, cell size and the inside flag.

MHDGrid: magnetic field

lines 146–197

```
    # ---- magnetic field (gyroresonance, polarisation) -------------------
    @property
    def has_field(self):
        return getattr(self, "_bflat", None) is not None

    def add_field(self, bx, by, bz, dtype=np.float32):
        """Attach the magnetic field (Cartesian components, gauss) on the grid nodes."""
        b = np.stack([np.asarray(c, dtype=float) for c in (bx, by, bz)], axis=-1)
        if b.shape[:3] != self.shape:
            raise ValueError(f"B has shape {b.shape[:3]}, expected {self.shape}")
        b = np.where(np.isfinite(b), b, 0.0)
        self._bflat = b.astype(dtype).reshape(-1, 3)
        return self

    def _interp_flat(self, flat, p):
        """Trilinear interpolation of a (N_nodes, k) array at points p (N,3)."""
        p = np.asarray(p, dtype=float)
        i, tx = self._locate(self.x, self.dx, p[:, 0])
        j, ty = self._locate(self.y, self.dy, p[:, 1])
        k, tz = self._locate(self.z, self.dz, p[:, 2])
        base = i * self._strides[0] + j * self._strides[1] + k
        out = np.zeros((p.shape[0], flat.shape[1]))
        wx, wy, wz = (1 - tx, tx), (1 - ty, ty), (1 - tz, tz)
```

```
169         c = 0
170         for di in (0, 1):
171             for dj in (0, 1):
172                 for dk in (0, 1):
173                     out += (wx[di] * wy[dj] * wz[dk])[:, None] * flat[base + self._corner_off
    ↪ [c]]
174                     c += 1
175         return out
176 
177     def sample_field(self, p):
178         """Trilinear interpolation of B [G] at points p (N,3) -> (N,3) Cartesian."""
179         if not self.has_field:
180             raise ValueError("this grid has no magnetic field: use add_field() or "
181                              "load_mhd(..., with_b=True)")
182         return self._interp_flat(self._bflat, p)
183 
184     # ---- extra scalar fields (e.g. current density, fast-electron density) --
185     def add_scalar(self, name, values, dtype=np.float32):
186         """Attach a named scalar field on the grid nodes (any quantity, e.g. 'j')."""
187         v = np.asarray(values, dtype=float)
188         if v.shape != self.shape:
189             raise ValueError(f"{name} has shape {v.shape}, expected {self.shape}")
190         if not hasattr(self, "scalars"):
191             self.scalars = {}
192         self.scalars[name] = np.where(np.isfinite(v), v, 0.0).astype(dtype).reshape(-1, 1)
193         return self
194 
195     def sample_scalar(self, name, p):
196         """Trilinear interpolation of the scalar field ``name`` at points p (N,3)."""
197         return self._interp_flat(self.scalars[name], p)[:, 0]
```

The field is optional and stored separately from n_e and T_e, so grids without a field use no extra memory and the refractive tracer is unchanged.

- **L.148** True once a field has been attached.
- **L.151–157** Stack (B_x, B_y, B_z) [G] on the grid nodes, check the shape, replace non-finite values by 0 and keep them in `float32` as a (N,3) array.
- **L.160–175** Generic trilinear interpolation of any (nodes, k) array, with the same cell location and the same eight weights as `sample()`.
- **L.177** **B** [G] at the points, Cartesian components.
- **L.185–193** Any named scalar field on the nodes (stored in `grid.scalars`), e.g. the current density `'j'` (common/fields.py), for use in fast-electron rules.
- **L.195** Its value at the points.

MHDGrid.box_intersection(): where a straight ray enters and leaves the box

lines 199–215

```
199     def box_intersection(self, origin, direction):
200         """Slab-method ray/box intersection.  Returns (t_enter, t_exit) arrays;
201         a ray misses the box when t_exit < max(t_enter, 0)."""
202         o = np.asarray(origin, dtype=float)
203         d = np.asarray(direction, dtype=float)
204         o = np.broadcast_to(o, d.shape)
205         par = np.abs(d) < 1e-300
```

```
206         dsafe = np.where(par, 1.0, d)
207         t1 = (self.lo - o) / dsafe
208         t2 = (self.hi - o) / dsafe
209         tmin = np.minimum(t1, t2)
210         tmax = np.maximum(t1, t2)
211         # ray parallel to a slab: either always inside it or never
212         in_slab = (o >= self.lo) & (o <= self.hi)
213         tmin = np.where(par, np.where(in_slab, -np.inf, np.inf), tmin)
214         tmax = np.where(par, np.where(in_slab, np.inf, -np.inf), tmax)
215         return tmin.max(axis=1), tmax.min(axis=1)
```

Slab method: for each axis, the ray $\mathbf{o} + t\mathbf{d}$ is between the two faces for $t \in [t_1, t_2]$; the intersection over the three axes is $[\max t_{\min}, \min t_{\max}]$.

L.204 Same observer for all rays (broadcast).

L.205–206 Avoid division by zero for directions parallel to a face.

L.207–210 Face crossings along each axis.

L.212–214 Parallel rays are either always inside that slab ($\pm\infty$) or never (empty interval).

L.215 Entry and exit parameters; the ray misses if exit $<$ entry.

MHDGrid: common API

lines 217–236

```
217     # common API ----------------------------------------------------
218     domain_intersection = box_intersection
219 
220     def to_npz(self, path):
221         """Save n_e, T_e (and B) (reload with radiosynth.load_mhd(path))."""
222         extra = {}
223         if self.has_field:
224             extra["b"] = self._bflat.reshape(self.shape + (3,))
225         np.savez(path, geometry="cartesian", x=self.x, y=self.y, z=self.z,
226                  ne=self._flat[:, 0].reshape(self.shape),
227                  te=self._flat[:, 4].reshape(self.shape), r_body=self.r_body,
228                  length_unit=self.length_unit, **extra)
229 
230     @property
231     def r_outer(self):
232         """Largest distance from the origin inside the domain (corner of the box)."""
233         return float(np.linalg.norm(np.maximum(np.abs(self.lo), np.abs(self.hi))))
234 
235 
236 # ============================================================================
```

L.218 Generic name used by `image.start_points`, so Cartesian and spherical grids are interchangeable.

L.231 Largest radius inside the box; used to start parallel rays outside the model.

L.220–227 Save n_e, T_e and the axes. `load_mhd('file.npz')` rebuilds the grid without re-reading the original model.

SphericalGrid: class docstring

lines 237–260

```
237 class SphericalGrid:
238     """Electron density and temperature on a spherical (r, theta, phi) grid.
239 
240     Native sampler for spherical MHD output (PLUTO, BATS-R-US 'sph', ...):
241     no resampling to a Cartesian box, so the fine radial resolution near the
242     Sun is preserved.
243 
244     Parameters
245     ----------
246     r      : 1D cell-centre radii [R_sun], increasing
247     theta  : 1D cell-centre colatitudes [rad] in [0, pi], increasing
248     phi    : 1D cell-centre longitudes [rad], increasing; a single value
249              (or None) means an axisymmetric 2.5D model (fields independent of phi)
250     ne, te : (nr, ntheta, nphi) electron density [m^-3] and temperature [K]
251     r_edges, theta_edges, phi_edges : optional cell-edge arrays used for the
252              domain extent (default: half a cell beyond the centres)
253     r_body : inner boundary [R_sun] where rays stop (default inner r edge)
254     length_unit : metres per coordinate unit (default R_sun)
255 
256     Cartesian convention: x = r sin(theta) cos(phi), y = r sin(theta) sin(phi),
257     z = r cos(theta)   (PLUTO / BATS-R-US convention).
258     """
259 
260     NFIELD = 5   # ne, dn/dr, (1/r) dn/dtheta, 1/(r sin) dn/dphi, Te
```

Used for PLUTO (and any) spherical output. Sampling is done in (r, θ, ϕ) index space, so the stretched radial grid near the Sun is used as is.

L.260 Stored fields: n_e, $\partial_r n_e$, $r^{-1}\partial_\theta n_e$, $(r\sin\theta)^{-1}\partial_\phi n_e$, T_e. These are the spherical components of the gradient vector, converted to Cartesian after interpolation.

SphericalGrid.__init__: fields

lines 262–273

```
262     def __init__(self, r, theta, phi, ne, te=None, r_edges=None, theta_edges=None,
263                  phi_edges=None, r_body=None, dtype=np.float32, length_unit=R_SUN):
264         self.r = np.asarray(r, float)
265         self.theta = np.asarray(theta, float)
266         self.axisym = phi is None or np.size(phi) == 1
267         self.phi = np.zeros(1) if phi is None else np.atleast_1d(np.asarray(phi, float))
268         nr, nt, nph = self.r.size, self.theta.size, self.phi.size
269         ne = np.asarray(ne, float).reshape(nr, nt, nph)
270         ne = np.where(np.isfinite(ne), np.maximum(ne, 0.0), 0.0)
271         te = np.ones_like(ne) if te is None else np.asarray(te, float).reshape(nr, nt, nph)
272         te = np.where(np.isfinite(te), te, 1.0)
273         self.shape = (nr, nt, nph)
```

L.266 A single (or absent) ϕ value means a 2.5D axisymmetric model.

L.267 Store at least one ϕ value so the array is always 3D.

L.269–272 Reshape to (nr, nth, nph) and clean invalid values, as in `MHDGrid`.

SphericalGrid.__init__: domain extent

lines 275–291

```
275         def edges(c, e):
276             if e is not None:
277                 return np.asarray(e, float)
278             if c.size == 1:
279                 return np.array([c[0], c[0]])
280             m = 0.5 * (c[1:] + c[:-1])
281             return np.concatenate([[c[0] - (m[0] - c[0])], m, [c[-1] + (c[-1] - m[-1])]])
282 
283         self.r_edges = edges(self.r, r_edges)
284         self.theta_edges = np.clip(edges(self.theta, theta_edges), 0.0, np.pi)
285         self.phi_edges = edges(self.phi, phi_edges)
286         self.r_lo, self.r_hi = self.r_edges[0], self.r_edges[-1]
287         self.full_theta = self.theta_edges[0] <= 1e-6 and self.theta_edges[-1] >= np.pi - 1e
        ↪ -6
288         span = self.phi_edges[-1] - self.phi_edges[0]
289         self.full_phi = self.axisym or span >= 2 * np.pi * (1 - 1e-6)
290         self.r_body = float(self.r_lo if r_body is None else r_body)
291         self.length_unit = float(length_unit)
```

L.275–281 Use the given cell edges, or reconstruct them as mid-points between centres plus half a cell at each end.

L.284 Colatitude edges are limited to $[0, \pi]$.

L.286 Inner and outer radii of the domain.

L.287 Does the grid cover the whole colatitude range? If not, rays leaving the wedge stop.

L.288–289 Full 2π in longitude (or axisymmetric) means ϕ is periodic.

L.290 Rays stop at the inner boundary, by default the inner radial edge (usually 1 $R_\odot$).

SphericalGrid.___init___: gradient components

lines 293–316

```
293         # gradient in spherical components
294         R = self.r[:, None, None]
295         S = np.maximum(np.sin(self.theta), 1e-6)[None, :, None]
296         g_r = np.gradient(ne, self.r, axis=0, edge_order=2 if nr > 2 else 1)
297         g_t = (np.gradient(ne, self.theta, axis=1, edge_order=2 if nt > 2 else 1) / R
298                if nt > 1 else np.zeros_like(ne))
299         if nph > 1:
300             if self.full_phi:
301                 pad = np.concatenate([ne[:, :, -1:], ne, ne[:, :, :1]], axis=2)
302                 ph = np.concatenate([[self.phi[-1] - 2 * np.pi], self.phi, [self.phi[0] + 2 *
        ↪ np.pi]])
303                 g_p = np.gradient(pad, ph, axis=2)[:, :, 1:-1]
304             else:
305                 g_p = np.gradient(ne, self.phi, axis=2, edge_order=2 if nph > 2 else 1)
306             g_p = g_p / (R * S)
307         else:
308             g_p = np.zeros_like(ne)
309         f = np.empty(self.shape + (self.NFIELD,), dtype=dtype)
310         f[..., 0], f[..., 1], f[..., 2], f[..., 3], f[..., 4] = ne, g_r, g_t, g_p, te
311         self._flat = f.reshape(-1, self.NFIELD)
312         del f, g_r, g_t, g_p
313         self.dr = np.diff(self.r_edges)
314         self.dth = np.diff(self.theta_edges)
315         self.dph = np.diff(self.phi_edges) if nph > 1 else np.array([np.inf])
316         self._strides = np.array([nt * nph, nph, 1], dtype=np.int64)
```

L.294–295 Broadcast helpers r and $\sin\theta$ ($\geq 10^{-6}$ to avoid division by zero at the poles).

L.296 $\partial n_e/\partial r$.

L.297–298 $r^{-1}\partial n_e/\partial\theta$.

L.300–303 Periodic ϕ: pad one cell on each side with the wrapped values so that the central difference is correct at $\phi = 0$ and 2π.

L.305 Non-periodic wedge: ordinary one-sided differences at the edges.

L.306 $(r\sin\theta)^{-1}\partial n_e/\partial\phi$.

L.308 Axisymmetric: no ϕ gradient.

L.309–311 Interleaved storage as in `MHDGrid`.

L.313–315 Cell widths in r, θ, ϕ (used for the step size).

L.316 Flat-index strides for the (nr, nth, nph) array.

SphericalGrid: description and properties

lines 318–334

```
318     def __repr__(self):
319         kind = "axisymmetric (r,theta)" if self.axisym else "3D (r,theta,phi)"
320         return (f"SphericalGrid({kind}, shape={self.shape}, r=[{self.r_lo:.3g},{self.r_hi:.3g
        ↪ }] R_sun, "
321                 f"theta=[{np.degrees(self.theta_edges[0]):.1f},{np.degrees(self.theta_edges
        ↪ [-1]):.1f}] deg"
322                 + ("" if self.axisym else
323                    f", phi=[{np.degrees(self.phi_edges[0]):.1f},{np.degrees(self.phi_edges
        ↪ [-1]):.1f}] deg")
324                 + f", min dr={self.dr.min():.3g} R_sun, r_body={self.r_body})")
325
326     @property
327     def nbytes(self):
328         return self._flat.nbytes
329
330     @property
331     def min_cell(self):
332         return float(min(self.dr.min(), self.r_lo * self.dth.min()))
333
334     @staticmethod
```

L.319 Reports whether the model is axisymmetric or 3D, and its extent in degrees.

L.331 Smallest of Δr and $r_{\rm lo}\Delta\theta$.

SphericalGrid._sph(): Cartesian to spherical

lines 335–339

```
335     def _sph(p):
336         r = np.sqrt(np.einsum("ij,ij->i", p, p))
337         th = np.arccos(np.clip(p[:, 2] / np.maximum(r, 1e-300), -1.0, 1.0))
338         ph = np.mod(np.arctan2(p[:, 1], p[:, 0]), 2 * np.pi)
339         return r, th, ph
```

L.336 $r = |\mathbf{p}|$.

L.337 Colatitude $\theta = \arccos(z/r)$ (clipped for round-off).

L.338 Longitude $\phi = \operatorname{atan2}(y, x)$ mapped to $[0, 2\pi)$.

SphericalGrid._phi_rel() and inside()

lines 341–355

```
341     def _phi_rel(self, ph):
342         """phi relative to the first edge, wrapped to [0, 2pi)."""
343         return np.mod(ph - self.phi_edges[0], 2 * np.pi) + self.phi_edges[0]
344 
345     def inside(self, p):
346         r, th, ph = self._sph(np.asarray(p, float))
347         ok = (r >= self.r_lo) & (r <= self.r_hi)
348         if not self.full_theta:
349             ok &= (th >= self.theta_edges[0]) & (th <= self.theta_edges[-1])
350         if not self.full_phi:
351             phr = self._phi_rel(ph)
352             ok &= phr <= self.phi_edges[-1]
353         return ok
354 
355     @staticmethod
```

L.343 Longitude measured from the first edge, so a wedge starting at any angle can be tested with one comparison.

L.347 Inside the radial shell ...

L.348 ... and within the colatitude range, if limited ...

L.350 ... and within the longitude range, if limited.

SphericalGrid._locate()

lines 356–362

```
356     def _locate(c, u):
357         n = c.size
358         if n == 1:
359             return np.zeros(u.size, np.int64), np.zeros(u.size)
360         i = np.clip(np.searchsorted(c, u, side="right") - 1, 0, n - 2)
361         t = np.clip((u - c[i]) / (c[i + 1] - c[i]), 0.0, 1.0)
362         return i, t
```

L.358 A single cell (axisymmetric ϕ): index 0, weight 0.

L.360–361 Binary search for the bracketing cell centres and the linear weight. The spherical axes are often stretched, so no uniform shortcut is used.

SphericalGrid.sample(): locate in r and theta

lines 364–372

```
364     def sample(self, p):
365         p = np.asarray(p, float)
366         r, th, ph = self._sph(p)
367         ins = (r >= self.r_lo) & (r <= self.r_hi)
368         if not self.full_theta:
369             ins &= (th >= self.theta_edges[0]) & (th <= self.theta_edges[-1])
370         i, ti = self._locate(self.r, r)
371         j, tj = self._locate(self.theta, th)
```

```
372         nph = self.phi.size
```

L.366 Spherical coordinates of the sample points.

L.367–369 Domain flag (radial shell, colatitude range).

L.370–371 Bracketing indices and weights in r and θ.

SphericalGrid.sample(): longitude handling

lines 373–390

```
373         if nph == 1:
374             k0 = np.zeros(r.size, np.int64)
375             k1 = k0
376             tk = np.zeros(r.size)
377         elif self.full_phi:
378             # periodic: cells between phi[-1] and phi[0]+2pi wrap around
379             phr = np.mod(ph - self.phi[0], 2 * np.pi)
380             rel = self.phi - self.phi[0]
381             k0 = np.searchsorted(rel, phr, side="right") - 1
382             k0 = np.clip(k0, 0, nph - 1)
383             k1 = (k0 + 1) % nph
384             nxt = np.where(k0 == nph - 1, 2 * np.pi, rel[np.minimum(k0 + 1, nph - 1)])
385             tk = np.clip((phr - rel[k0]) / (nxt - rel[k0]), 0.0, 1.0)
386         else:
387             phr = self._phi_rel(ph)
388             ins &= phr <= self.phi_edges[-1]
389             k0, tk = self._locate(self.phi, phr)
390             k1 = k0 + 1
```

L.373–376 Axisymmetric: both ϕ corners are the single cell, so the field does not depend on ϕ.

L.377–385 Periodic longitude: the last cell centre is followed by the first one plus 2π. `k1 = (k0+1) % nph` wraps around, and the weight uses a 2π-shifted upper centre.

L.386–390 Limited wedge: ordinary search; points outside the wedge are flagged.

SphericalGrid.sample(): trilinear interpolation in index space

lines 391–399

```
391         st = self._strides
392         out = np.zeros((r.size, self.NFIELD))
393         for di, wi in ((0, 1 - ti), (1, ti)):
394             ii = np.minimum(i + di, self.r.size - 1)
395             for dj, wj in ((0, 1 - tj), (1, tj)):
396                 jj = np.minimum(j + dj, self.theta.size - 1)
397                 for kk, wk in ((k0, 1 - tk), (k1, tk)):
398                     w = wi * wj * wk
399                     out += w[:, None] * self._flat[ii * st[0] + jj * st[1] + kk]
```

L.393–399 Weighted sum over the 8 corner cells. `np.minimum(..., size-1)` keeps the upper index valid when a point is beyond the last centre; the weight is then zero.

SphericalGrid.sample(): gradient to Cartesian, cell size

lines 400–413

```
400         ne, gr, gt, gp, te = out.T
401         sth, cth = np.sin(th), np.cos(th)
402         sph, cph = np.sin(ph), np.cos(ph)
403         grad = np.empty((r.size, 3))
404         grad[:, 0] = gr * sth * cph + gt * cth * cph - gp * sph
405         grad[:, 1] = gr * sth * sph + gt * cth * sph + gp * cph
406         grad[:, 2] = gr * cth - gt * sth
407         dr = self.dr[np.clip(np.searchsorted(self.r_edges, r) - 1, 0, self.dr.size - 1)]
408         rdt = r * self.dth[np.clip(np.searchsorted(self.theta_edges, th) - 1, 0, self.dth.
        ↪ size - 1)]
409         cell = np.minimum(dr, rdt)
410         if nph > 1:
411             dph = self.dph[np.clip(k0, 0, self.dph.size - 1)]
412             cell = np.minimum(cell, np.maximum(r * np.abs(sth) * dph, 0.25 * rdt))
413         return np.maximum(ne, 0.0), grad, te, cell, ins
```

L.400–402 Unpack the interpolated fields and the trigonometric factors at each point.

L.403–406 Transform (g_r, g_θ, g_ϕ) to Cartesian with the unit vectors $\hat{r} = (s_\theta c_\phi, s_\theta s_\phi, c_\theta)$, $\hat{\theta} = (c_\theta c_\phi, c_\theta s_\phi, -s_\theta)$, $\hat{\phi} = (-s_\phi, c_\phi, 0)$.

L.407–409 Local cell size $\min(\Delta r, r\Delta\theta)$.

L.410–412 Include $r \sin\theta\, \Delta\phi$ in 3D, floored at a quarter of $r\Delta\theta$. The cells near the poles are tiny, and without the floor the step size would collapse there.

L.413 Same return signature as `MHDGrid.sample`.

SphericalGrid: magnetic field

lines 415–493

```
415     # ---- magnetic field (gyroresonance, polarisation) -------------------
416     @property
417     def has_field(self):
418         return getattr(self, "_bflat", None) is not None
419
420     def add_field(self, b1, b2, b3, components="spherical", dtype=np.float32):
421         """Attach the magnetic field [G] at the cell centres.
422
423         components='spherical': (B_r, B_theta, B_phi) (PLUTO bx1, bx2, bx3);
424         'cartesian': (B_x, B_y, B_z)."""
425         b = np.stack([np.asarray(c, dtype=float).reshape(self.shape) for c in (b1, b2, b3)],
426                      axis=-1)
427         b = np.where(np.isfinite(b), b, 0.0)
428         if components not in ("spherical", "cartesian"):
429             raise ValueError("components must be 'spherical' or 'cartesian'")
430         self._bcomp = components
431         self._bflat = b.astype(dtype).reshape(-1, 3)
432         return self
433
434     def _interp_flat(self, flat, p):
435         """Trilinear interpolation (in r, theta, phi index space) of a
436         (N_cells, k) array at points p (N,3).  Returns (values, theta, phi)."""
437         p = np.asarray(p, float)
438         r, th, ph = self._sph(p)
439         i, ti = self._locate(self.r, r)
440         j, tj = self._locate(self.theta, th)
441         nph = self.phi.size
```

```
442         if nph == 1:
443             k0 = np.zeros(r.size, np.int64)
444             k1 = k0
445             tk = np.zeros(r.size)
446         elif self.full_phi:
447             phr = np.mod(ph - self.phi[0], 2 * np.pi)
448             rel = self.phi - self.phi[0]
449             k0 = np.clip(np.searchsorted(rel, phr, side="right") - 1, 0, nph - 1)
450             k1 = (k0 + 1) % nph
451             nxt = np.where(k0 == nph - 1, 2 * np.pi, rel[np.minimum(k0 + 1, nph - 1)])
452             tk = np.clip((phr - rel[k0]) / (nxt - rel[k0]), 0.0, 1.0)
453         else:
454             k0, tk = self._locate(self.phi, self._phi_rel(ph))
455             k1 = k0 + 1
456         st = self._strides
457         out = np.zeros((r.size, flat.shape[1]))
458         for di, wi in ((0, 1 - ti), (1, ti)):
459             ii = np.minimum(i + di, self.r.size - 1)
460             for dj, wj in ((0, 1 - tj), (1, tj)):
461                 jj = np.minimum(j + dj, self.theta.size - 1)
462                 for kk, wk in ((k0, 1 - tk), (k1, tk)):
463                     out += (wi * wj * wk)[:, None] * flat[ii * st[0] + jj * st[1] + kk]
464         return out, th, ph
465 
466     def sample_field(self, p):
467         """Trilinear interpolation of B [G] at points p (N,3) -> (N,3) Cartesian."""
468         if not self.has_field:
469             raise ValueError("this grid has no magnetic field: use add_field() or "
470                              "load_mhd(..., with_b=True)")
471         out, th, ph = self._interp_flat(self._bflat, p)
472         if self._bcomp == "cartesian":
473             return out
474         br, bt, bp = out.T
475         sth, cth = np.sin(th), np.cos(th)
476         sph, cph = np.sin(ph), np.cos(ph)
477         b = np.empty((out.shape[0], 3))
478         b[:, 0] = br * sth * cph + bt * cth * cph - bp * sph
479         b[:, 1] = br * sth * sph + bt * cth * sph + bp * cph
480         b[:, 2] = br * cth - bt * sth
481         return b
482 
483     def add_scalar(self, name, values, dtype=np.float32):
484         """Attach a named scalar field at the cell centres (e.g. 'j')."""
485         v = np.asarray(values, dtype=float).reshape(self.shape)
486         if not hasattr(self, "scalars"):
487             self.scalars = {}
488         self.scalars[name] = np.where(np.isfinite(v), v, 0.0).astype(dtype).reshape(-1, 1)
489         return self
490 
491     def sample_scalar(self, name, p):
492         """Trilinear interpolation of the scalar field ``name`` at points p (N,3)."""
493         return self._interp_flat(self.scalars[name], p)[0][:, 0]
```

L.420–431 The field is given at the cell centres either as spherical components (B_r, B_θ, B_ϕ) — PLUTO's `bx1`, `bx2`, `bx3` — or as Cartesian components. The choice is remembered in `_bcomp`.

L.434–464 Generic interpolation in (r, θ, ϕ) index space of any (cells, k) array, with the same

longitude handling as `sample()` (axisymmetric, periodic, or a longitude patch); also returns θ, ϕ of the points.

L.466–473 **B** at the points; Cartesian components are returned as they are.

L.474–481 Spherical to Cartesian at the sample point: $B_x = B_r \sin\theta\cos\phi + B_\theta \cos\theta\cos\phi - B_\phi \sin\phi$, and similarly for B_y, B_z. Interpolating the spherical components and rotating afterwards keeps a radial field radial between grid points.

L.483–491 Named scalar fields at the cell centres, as for the Cartesian grid.

SphericalGrid.domain_intersection() lines 495–507

```
495     def domain_intersection(self, origin, direction):
496         """Entry/exit parameters of straight rays with the outer sphere r_hi."""
497         o = np.broadcast_to(np.asarray(origin, float), np.shape(direction))
498         d = np.asarray(direction, float)
499         b = np.einsum("ij,ij->i", o, d)
500         c = np.einsum("ij,ij->i", o, o) - self.r_hi**2
501         disc = b * b - c
502         sq = np.sqrt(np.maximum(disc, 0.0))
503         t0 = np.where(disc > 0, -b - sq, np.inf)
504         t1 = np.where(disc > 0, -b + sq, -np.inf)
505         return t0, t1
506 
507     box_intersection = domain_intersection
```

L.499–501 Solve $|\mathbf{o} + t\mathbf{d}| = r_{\mathrm{hi}}$: $t = -b \pm \sqrt{b^2 - c}$, with $b = \mathbf{o}\cdot\mathbf{d}$ and $c = |\mathbf{o}|^2 - r_{\mathrm{hi}}^2$.

L.503–504 Entry and exit; rays that miss the sphere get an empty interval.

L.507 Alias for symmetry with `MHDGrid`.

SphericalGrid.to_npz() lines 509–523

```
509     def to_npz(self, path):
510         extra = {}
511         if self.has_field:
512             extra = dict(b=self._bflat.reshape(self.shape + (3,)), b_components=self._bcomp)
513         np.savez(path, geometry="spherical", r=self.r, theta=self.theta,
514                  phi=self.phi if not self.axisym else np.array([0.0]),
515                  r_edges=self.r_edges, theta_edges=self.theta_edges,
516                  phi_edges=self.phi_edges, axisym=self.axisym,
517                  ne=self._flat[:, 0].reshape(self.shape),
518                  te=self._flat[:, 4].reshape(self.shape), r_body=self.r_body,
519                  length_unit=self.length_unit, **extra)
520 
521     @property
522     def r_outer(self):
523         return float(self.r_hi)
```

Saves everything needed to rebuild the grid, including the `geometry='spherical'` tag that `load_mhd` checks.

Chapter 4

common/readers.py — BATS-R-US and generic input

File: `radiosynth/common/readers.py` — 567 lines.

Turns model files into dictionaries of point variables, works out n_e and T_e with the right units, arranges the points on a grid (or resamples them), and returns an `MHDGrid`. `load_mhd` is the single entry point and also dispatches to the PLUTO reader.

Module docstring and imports

lines 4–35

```
4  """Readers that turn MHD model output into an MHDGrid.
5
6  Supported inputs
7  ----------------
8  * numpy arrays                          -> ``grid_from_arrays``
9  * Tecplot ASCII (.dat/.tec), ordered (I,J,K) zones or FE point zones
10                                         -> ``read_tecplot_ascii``
11 * Tecplot binary (.plt/.szplt) via pytecplot (needs Tecplot 360)
12                                         -> ``read_pytecplot``
13 * BATS-R-US / SWMF IDL plot files (.out, ascii / real4 / real8)
14                                         -> ``read_batsrus_idl``
15 * ``load_mhd(path)`` dispatches on the extension.
16
17 All readers return a dict ``{name: array}`` of point data, which
18 ``build_grid`` converts to an MHDGrid (computing n_e [m^-3] and T_e [K]).
19 If the points are not on a tensor-product grid (e.g. BATS-R-US
20 unstructured/AMR '3d' output) they are resampled to a uniform Cartesian box.
21 """
22 import os
23 import re
24 import struct
25 import warnings
26
27 import numpy as np
28
29 from .constants import K_B, M_P, R_SUN
30 from .grid import MHDGrid
31
32
33 # ============================================================================
34 # Variable identification and unit handling
35 # ============================================================================
```

Lists the supported formats. Every reader returns `{name: array}`, which `build_grid` converts.

Variable-name helpers

lines 36–51

```
36 def _base(name):
```

```
37      """'Rho [g/cm^3]' -> 'rho'"""
38      return re.split(r"[\s\[(]", name.strip().strip('"'), maxsplit=1)[0].lower()
39
40
41  def _unit(name):
42      m = re.search(r"\[([^\]]*)\]", name)
43      return m.group(1).strip().lower().replace(" ", "") if m else None
44
45
46  def _find(vars_, candidates):
47      for c in candidates:
48          for k in vars_:
49              if _base(k) == c:
50                  return k
51      return None
```

L.36 Base name in lower case: `'Rho [g/cm^3]'` becomes `'rho'`.

L.41 Unit in square brackets, lower case without spaces: `'g/cm^3'`.

L.46 First variable whose base name is in the candidate list.

Unit tables

lines 54–59

```
54  _MASS_UNITS = {"g/cm^3": 1e3, "g/cm3": 1e3, "g/cc": 1e3, "kg/m^3": 1.0, "kg/m3": 1.0}
55  _NUM_UNITS = {"cm^-3": 1e6, "/cm^3": 1e6, "1/cm^3": 1e6, "/cc": 1e6, "n/cc": 1e6,
56                "mp/cc": 1e6, "amu/cc": 1e6, "amu/cm^3": 1e6, "amu/cm3": 1e6, "cm-3": 1e6,
57                "m^-3": 1.0, "/m^3": 1.0, "1/m^3": 1.0, "m-3": 1.0, "/m3": 1.0}
58  _PRES_UNITS = {"dyne/cm^2": 0.1, "dyn/cm^2": 0.1, "dyne/cm2": 0.1, "dyn/cm2": 0.1,
59                 "pa": 1.0, "npa": 1e-9, "j/m^3": 1.0}
```

L.54 Mass-density units and their factor to kg m^{-3}.

L.55 Number-density units and their factor to m^{-3}. BATS-R-US 'amu/cc' and 'Mp/cc' are number densities.

L.58 Pressure units and their factor to Pa (1 dyn cm^{-2} = 0.1 Pa).

find_magnetic_field(): Bx, By, Bz in gauss

lines 62–91

```
62  _B_UNITS = {"gauss": 1.0, "g": 1.0, "nt": 1e-5, "t": 1e4, "tesla": 1e4, "mt": 10.0}
63
64
65  def find_magnetic_field(vars_, b_unit=None, verbose=True):
66      """Return (bx, by, bz) in gauss from BATS-R-US-style variables Bx/By/Bz
67      (also B_x, b1..b3).  Units from the variable name ([Gauss], [nT], [T]) or
68      ``b_unit`` (number = gauss per output unit, or one of 'G', 'nT', 'T');
69      if neither is known gauss is assumed with a warning."""
70      names = None
71      for cand in (("bx", "by", "bz"), ("b_x", "b_y", "b_z"), ("b1", "b2", "b3")):
72          ks = [_find(vars_, [c]) for c in cand]
73          if all(k is not None for k in ks):
74              names = ks
75              break
76      if names is None:
77          raise KeyError("no magnetic field (Bx, By, Bz) in the MHD output; "
```

```
                        f"variables: {list(vars_)}")
    if b_unit is not None:
        fac = float(b_unit) if not isinstance(b_unit, str) else _B_UNITS[b_unit.lower()]
        src = f"b_unit={b_unit}"
    else:
        u = _unit(names[0])
        if u in _B_UNITS:
            fac, src = _B_UNITS[u], f"[{u}] in the variable name"
        else:
            fac, src = 1.0, "assumed gauss (no unit in the file; pass b_unit=)"
            warnings.warn("magnetic field unit unknown: assuming gauss (pass b_unit=)")
    if verbose:
        print(f"radiosynth: magnetic field from {names}, {src}")
    return tuple(np.asarray(vars_[k], float) * fac for k in names)
```

L.62 Gauss per unit: G = 1, nT = 10^{-5}, T = 10^{4}, mT = 10.

L.71–75 Look for `Bx, By, Bz`, then `B_x, ...`, then `b1, b2, b3` (names compared without units and case).

L.79–81 An explicit `b_unit` wins: a number (gauss per file unit) or a unit name.

L.83–88 Otherwise the unit in the variable name (e.g. `Bx [Gauss]`, BATS-R-US Tecplot) is used; without one, gauss is assumed with a warning. BATS-R-US IDL files in normalised units need `b_unit`.

_density_to_ne(): density in any unit to n_e

lines 94–120

```
def _density_to_ne(vals, unit, r=None, kind=None, mu=1.0):
    """Convert a density variable to electron number density [m^-3].

    kind: 'mass' or 'number' (None -> from the unit string, else guessed from
    the magnitude near r ~ 1.1 R_sun).  For mass density n_e = rho/(mu m_p)
    (BATS-R-US uses n_e = rho/m_p for a hydrogen plasma, mu=1)."""
    u = unit
    if u in _MASS_UNITS:
        return vals * _MASS_UNITS[u] / (mu * M_P), f"mass density [{u}]"
    if u in _NUM_UNITS:
        return vals * _NUM_UNITS[u], f"number density [{u}]"
    # guess from magnitude (typical low corona n_e ~ 1e8-1e9 cm^-3)
    if r is not None:
        sel = (r > 1.05) & (r < 1.3)
        ref = np.nanmedian(np.abs(vals[sel])) if sel.any() else np.nanmedian(np.abs(vals))
    else:
        ref = np.nanmedian(np.abs(vals))
    guesses = [(1e3 / (mu * M_P), "mass density [g/cm^3] (guessed)"),
               (1.0 / (mu * M_P), "mass density [kg/m^3] (guessed)"),
               (1e6, "number density [cm^-3] (guessed)"),
               (1.0, "number density [m^-3] (guessed)")]
    if kind == "mass":
        guesses = guesses[:2]
    elif kind == "number":
        guesses = guesses[2:]
    best = min(guesses, key=lambda g: abs(np.log10(max(ref * g[0], 1e-300)) - 14.5))
    return vals * best[0], best[1]
```

L.101–102 Mass density: $n_e = \rho/(\mu m_p)$.

L.103–104 Number density: unit conversion only.

L.106–110 Unknown unit: take the median value at $r = 1.05$–$1.3\,R_\odot$ as reference.

L.111–119 Try g/cm^3, kg/m^3, cm^{-3} and m^{-3} (or only the mass or number options if the name says which) and pick the one that gives n_e closest to $10^{14.5}$ m^{-3}, a typical low-corona value. The choice is printed, so you can check it.

_pressure_si() lines 123–127

```
123 def _pressure_si(vals, unit):
124     if unit in _PRES_UNITS:
125         return vals * _PRES_UNITS[unit]
126     # BATS-R-US dimensional output default: dyne/cm^2
127     return vals * 0.1
```

L.127 Without a recognised unit, the BATS-R-US dimensional default dyn cm^{-2} is assumed.

derive_ne_te(): electron density lines 130–150

```
130 def derive_ne_te(vars_, density_var=None, density_unit=None, te_var=None,
131                 pe_var=None, p_var=None, mu=1.0, te_const=None, verbose=True):
132     """Return (ne [m^-3], te [K], info) from a dict of point variables.
133 
134     Priority for T_e:  te_var / 'te'  >  Pe/(n_e k)  >  P/(2 n_e k) (Te=Ti)
135                        >  'T'/'temp'  >  te_const.
136     """
137     x, y, z = vars_["x"], vars_["y"], vars_["z"]
138     r = np.sqrt(x**2 + y**2 + z**2)
139     info = {}
140     dk = density_var or _find(vars_, ["ne", "n_e", "rho", "n", "dens", "density", "nrho"])
141     if dk is None:
142         raise KeyError(f"no density variable found among {list(vars_)}; use density_var=")
143     unit = density_unit.lower().replace(" ", "") if density_unit else _unit(dk)
144     if unit is None and "_units" in vars_:
145         unit = vars_["_units"].get(dk)
146     kind = None
147     if density_unit is None and unit is None:
148         kind = "number" if _base(dk) in ("ne", "n_e", "n") else ("mass" if _base(dk) == "rho"
    ↪ else None)
149     ne, how = _density_to_ne(np.asarray(vars_[dk], float), unit, r, kind, mu)
150     info["density"] = f"{dk}: {how}"
```

L.138 Radius of every point, for the unit guess and the printed check.

L.140 Density variable: user choice or the first of ne, n_e, rho, n, dens, density.

L.143–145 Unit: from the user, from the name, or from the IDL header units.

L.146–148 If there is no unit, the name decides whether it is a mass or a number density.

L.149 Convert to m^{-3} and remember how, for the report.

derive_ne_te(): electron temperature lines 152–192

```
152     units = vars_.get("_units", {})
153     te = None
154     tk = te_var or _find(vars_, ["te", "t_e"])
155     if tk is not None:
156         te = np.asarray(vars_[tk], float)
157         info["te"] = f"{tk} [K]"
158     if te is None:
159         pk = pe_var or _find(vars_, ["pe", "p_e"])
160         if pk is not None:
161             pe = _pressure_si(np.asarray(vars_[pk], float), _unit(pk) or units.get(pk))
162             te = pe / (np.maximum(ne, 1.0) * K_B)
163             info["te"] = f"{pk}/(n_e k_B)"
164     if te is None:
165         pk = p_var or _find(vars_, ["p", "pres", "pressure"])
166         if pk is not None:
167             p = _pressure_si(np.asarray(vars_[pk], float), _unit(pk) or units.get(pk))
168             te = p / (2.0 * np.maximum(ne, 1.0) * K_B)
169             info["te"] = f"{pk}/(2 n_e k_B)  (assumes T_e = T_i, no Pe in file)"
170     if te is None:
171         tk = _find(vars_, ["t", "temp", "temperature", "ti"])
172         if tk is not None:
173             te = np.asarray(vars_[tk], float)
174             info["te"] = f"{tk} [K]"
175     if te is None:
176         if te_const is None:
177             raise KeyError("no temperature/pressure variable found; pass te_const= or te_var=
    ↪ ")
178         te = np.full_like(ne, float(te_const))
179         info["te"] = f"constant {te_const} K"
180     if verbose:
181         sel = (r > 1.05) & (r < 1.15)
182         msg = ""
183         if sel.any():
184             msg = (f" (median at r=1.05-1.15: n_e={np.nanmedian(ne[sel]):.3e} m^-3, "
185                    f"T_e={np.nanmedian(te[sel]):.3e} K)")
186         print(f"radiosynth: n_e from {info['density']}; T_e from {info['te']}{msg}")
187     return ne, te, info
188
189
190 # ============================================================================
191 # Point cloud -> rectilinear grid
192 # ============================================================================
```

L.154–157 1st choice: a Te variable in K.

L.159–162 2nd: electron pressure, $T_e = P_e/(n_e k_B)$ (AWSoM writes Pe).

L.165–168 3rd: total pressure, $T = P/(2 n_e k_B)$, i.e. $T_e = T_i$ in a hydrogen plasma.

L.171 4th: a generic temperature variable.

L.176–178 Last: a constant given by the user, otherwise an error.

L.180–186 Print what was used and the median n_e, T_e near $r = 1.1\,R_\odot$. This is the best check that the units are right.

`_unique_axis()` lines 193–202

```
193 def _unique_axis(v, rtol=1e-6):
194     v = np.asarray(v, float)
195     scale = max(np.ptp(v), 1e-30)
196     q = np.round(v / (scale * rtol)).astype(np.int64)
197     uq, inv = np.unique(q, return_inverse=True)
198     # representative coordinate per unique value
199     ax = np.zeros(uq.size)
200     np.add.at(ax, inv, v)
201     ax /= np.bincount(inv)
202     return ax, inv
```

L.195–197 Round coordinates to a relative tolerance of 10^{-6} and find the distinct values. This copes with single-precision round-off in the file.

L.199–201 Each axis value is the mean of the coordinates in its group; `inv` maps every point to its axis index.

points_to_grid(): tensor grid or resampling

lines 205–229

```
205 def points_to_grid(vars_, names, resample_shape=None, resample_box=None,
206                   method="linear", verbose=True):
207     """Arrange point data on a tensor-product grid.
208 
209     If the points already form a full tensor grid, they are simply reshaped
210     (any ordering).  Otherwise they are resampled to a uniform box of shape
211     ``resample_shape`` (default 160^3) covering ``resample_box``
212     ((xmin,xmax),(ymin,ymax),(zmin,zmax)); ``method`` is 'linear'
213     (Delaunay, accurate, slow for >1e6 points) or 'nearest' (fast).
214     Returns (x, y, z, {name: 3D array}).
215     """
216     x, y, z = (np.asarray(vars_[k], float) for k in ("x", "y", "z"))
217     ax_x, ix = _unique_axis(x)
218     ax_y, iy = _unique_axis(y)
219     ax_z, iz = _unique_axis(z)
220     n = x.size
221     if ax_x.size * ax_y.size * ax_z.size == n and resample_shape is None:
222         flat = (ix * ax_y.size + iy) * ax_z.size + iz
223         if np.unique(flat).size == n:
224             out = {}
225             for k in names:
226                 a = np.empty(n)
227                 a[flat] = np.asarray(vars_[k], float)
228                 out[k] = a.reshape(ax_x.size, ax_y.size, ax_z.size)
229             return ax_x, ax_y, ax_z, out
```

L.217–220 Distinct coordinate values along each axis.

L.221–229 If the number of points equals $n_x n_y n_z$ and no two points share a cell, the data already form a tensor grid. They are placed by index, so any point order works (Tecplot I-fastest, shuffled, etc.).

points_to_grid(): resampling scattered points

lines 231–266

```
231     # ---- resample scattered data
232     shape = resample_shape or (160, 160, 160)
233     if np.isscalar(shape):
234         shape = (int(shape),) * 3
235     if resample_box is None:
236         resample_box = ((x.min(), x.max()), (y.min(), y.max()), (z.min(), z.max()))
237     gx = np.linspace(*resample_box[0], shape[0])
238     gy = np.linspace(*resample_box[1], shape[1])
239     gz = np.linspace(*resample_box[2], shape[2])
240     if verbose:
241         print(f"radiosynth: {n} scattered points -> resampling ({method}) to {shape} "
242               f"uniform grid; this can take a while")
243     from scipy.spatial import cKDTree
244     pts = np.column_stack([x, y, z])
245     G = np.stack(np.meshgrid(gx, gy, gz, indexing="ij"), axis=-1).reshape(-1, 3)
246     # density spans orders of magnitude: interpolate its logarithm
247     vals = []
248     for k in names:
249         v = np.asarray(vars_[k], float)
250         vals.append(np.log(np.maximum(v, 1e-300)) if k == "ne" else v)
251     vals = np.column_stack(vals)
252     if method == "nearest":
253         _, idx = cKDTree(pts).query(G, k=1, workers=-1)
254         res = vals[idx]
255     else:
256         from scipy.interpolate import LinearNDInterpolator
257         res = LinearNDInterpolator(pts, vals)(G)
258         bad = ~np.isfinite(res).all(axis=1)
259         if bad.any():
260             _, idx = cKDTree(pts).query(G[bad], k=1, workers=-1)
261             res[bad] = vals[idx]
262     out = {}
263     for c, k in enumerate(names):
264         a = res[:, c]
265         out[k] = (np.exp(a) if k == "ne" else a).reshape(shape)
266     return gx, gy, gz, out
```

L.232–239 Target uniform grid (default 160^3) over the given or full box.

L.244–245 Source points and target points.

L.247–251 The density is interpolated as $\log n_e$: it varies over many orders of magnitude, and linear interpolation of the logarithm keeps it positive and smooth.

L.252–254 Fast nearest-neighbour lookup with a k-d tree.

L.256–261 Linear interpolation on the Delaunay triangulation; target points outside the convex hull fall back to the nearest neighbour.

L.262–265 Undo the logarithm and reshape to 3D.

build_grid() and grid_from_arrays()

lines 269–301

```
269 def build_grid(vars_, r_body=1.0, resample_shape=None, resample_box=None,
270                resample_method="linear", dtype=np.float32, verbose=True, r_star=None,
271                with_b=False, b_unit=None, **derive_kw):
272     """dict of point variables -> MHDGrid (see derive_ne_te for keywords).
273
```

```
    r_star : stellar radius in R_sun when the coordinates are in stellar radii
             (BATS-R-US stellar runs, #STAR); default 1 (the Sun).
    with_b : also attach the magnetic field Bx, By, Bz (needed for
             gyroresonance); b_unit overrides the unit (see find_magnetic_field)."""
    ne, te, info = derive_ne_te(vars_, verbose=verbose, **derive_kw)
    pv = {"x": vars_["x"], "y": vars_["y"], "z": vars_["z"], "ne": ne, "te": te}
    names = ["ne", "te"]
    if with_b:
        pv["bx"], pv["by"], pv["bz"] = find_magnetic_field(vars_, b_unit, verbose)
        names += ["bx", "by", "bz"]
    x, y, z, f = points_to_grid(pv, names, resample_shape, resample_box,
                                resample_method, verbose)
    g = MHDGrid(x, y, z, f["ne"], f["te"], r_body=r_body, dtype=dtype,
                length_unit=(r_star or 1.0) * R_SUN)
    if with_b:
        g.add_field(f["bx"], f["by"], f["bz"])
    g.info = info
    return g


def grid_from_arrays(x, y, z, ne, te=None, r_body=1.0, **kw):
    """Convenience wrapper: 1D axes [R_sun] + 3D n_e [m^-3], T_e [K]."""
    return MHDGrid(x, y, z, ne, te, r_body=r_body, **kw)


# ============================================================================
# Tecplot ASCII
# ============================================================================
```

L.278 Physical fields at the points.

L.284 Arrange or resample.

L.286 Build the grid and attach the conversion info. With `r_star` (in $R_\odot$) the coordinates are taken to be in stellar radii, as in BATS-R-US stellar runs, and the length unit is R_*.

L.294 Convenience wrapper around `MHDGrid`.

_standardise_xyz() lines 302–310

```
def _standardise_xyz(vars_):
    out = dict(vars_)
    for c in ("x", "y", "z"):
        if c not in out:
            k = _find(vars_, [c])
            if k is None:
                raise KeyError(f"coordinate '{c}' not found in {list(vars_)}")
            out[c] = out[k]
    return out
```

L.304 Make sure the coordinates are available as plain 'x', 'y', 'z', whatever their original names (for example 'X [R]').

read_tecplot_ascii()

lines 313–373

```
313 def read_tecplot_ascii(path, zone=0):
314     """Read an ASCII Tecplot file (POINT or BLOCK packing, ordered or FE zone).
315 
316     Only nodal variables of the requested zone are returned; FE connectivity
317     is ignored (the points are re-gridded by points_to_grid)."""
318     with open(path, "r") as fh:
319         text = fh.read()
320     # header ---------------------------------------------------------------
321     mvar = re.search(r"VARIABLES\s*=\s*(.*?)(?=\n\s*(ZONE|DATASETAUXDATA|AUXDATA)\b)",
322                      text, re.S | re.I)
323     if not mvar:
324         raise ValueError("VARIABLES line not found")
325     raw = mvar.group(1)
326     names = re.findall(r'"([^"]*)"', raw) or re.split(r"[,\s]+", raw.strip())
327     names = [n.strip() for n in names if n.strip()]
328     zones = [m for m in re.finditer(r"^\s*ZONE\b", text, re.M | re.I)]
329     if not zones:
330         raise ValueError("no ZONE found")
331     zstart = zones[zone].start()
332     zend = zones[zone + 1].start() if zone + 1 < len(zones) else len(text)
333     ztext = text[zstart:zend]
334     # zone header spans until the first line that starts with a number
335     lines = ztext.split("\n")
336     hdr = []
337     i = 0
338     num_re = re.compile(r"^\s*[-+]?(\d|\.\d)")
339     while i < len(lines) and not num_re.match(lines[i]):
340         hdr.append(lines[i])
341         i += 1
342     header = " ".join(hdr)
343 
344     def geti(key):
345         m = re.search(r"\b" + key + r"\s*=\s*(\d+)", header, re.I)
346         return int(m.group(1)) if m else None
347 
348     I, J, K = geti("I"), geti("J"), geti("K")
349     N = geti("N") or geti("NODES")
350     E = geti("E") or geti("ELEMENTS")
351     block = bool(re.search(r"(DATAPACKING\s*=\s*BLOCK|F\s*=\s*BLOCK|F\s*=\s*FEBLOCK)", header
    ↪ , re.I))
352     npts = N if N is not None else (I or 1) * (J or 1) * (K or 1)
353     nv = len(names)
354     body = "\n".join(lines[i:])
355     import warnings as _w
356     with _w.catch_warnings():
357         _w.simplefilter("ignore")
358         try:
359             arr = np.fromstring(body.replace("D", "E").replace("d", "e"), sep=" ")
360         except Exception:
361             arr = np.array(body.split(), dtype=float)
362     data = arr[: npts * nv]
363     if data.size < npts * nv:
364         raise ValueError(f"expected {npts * nv} values, found {data.size}")
365     data = data.reshape(nv, npts) if block else data.reshape(npts, nv).T
366     out = {n: data[c] for c, n in enumerate(names)}
367     _ = E
```

```
368     return _standardise_xyz(out)
369
370
371 # ============================================================================
372 # Tecplot binary through pytecplot
373 # ============================================================================
```

L.321–327 Variable names from the VARIABLES line, quoted or not, possibly over several lines.

L.328–333 Select the requested zone.

L.335–342 The zone header runs until the first line that starts with a number.

L.344–350 Read I, J, K (ordered zone) or N, E (finite-element zone) from the header.

L.351 BLOCK packing stores one variable after another; POINT packing stores one point after another.

L.352 Number of points.

L.354–361 Parse all numbers in one go (Fortran 'D' exponents accepted). The connectivity list of FE zones, which follows the data, is ignored.

L.365 Arrange as (variable, point) for either packing.

read_pytecplot() lines 374–385

```
374 def read_pytecplot(path, zone=0):
375     """Read .plt/.szplt with pytecplot (requires a Tecplot 360 installation)."""
376     import tecplot as tp
377     ds = tp.data.load_tecplot(path)
378     z = ds.zone(zone)
379     out = {v.name: z.values(v.index).as_numpy_array().astype(float) for v in ds.variables()}
380     return _standardise_xyz(out)
381
382
383 # ============================================================================
384 # BATS-R-US / SWMF IDL format
385 # ============================================================================
```

L.377 The same call as in the original script `Radio_Emsssion_Calculation_batsrus.py`. Every variable of the zone is read as a NumPy array.

_read_records(): Fortran unformatted files lines 386–404

```
386 def _read_records(fh):
387     """Iterate Fortran unformatted sequential records, auto-detecting endianness."""
388     data = fh.read()
389     pos = 0
390     endian = None
391     recs = []
392     while pos < len(data):
393         if endian is None:
394             for e in ("<", ">"):
395                 (L,) = struct.unpack(e + "i", data[pos:pos + 4])
396                 if 0 < L < len(data) and data[pos + 4 + L:pos + 8 + L] == data[pos:pos + 4]:
397                     endian = e
398                     break
```

```
399             if endian is None:
400                 raise ValueError("not a Fortran unformatted file")
401         (L,) = struct.unpack(endian + "i", data[pos:pos + 4])
402         recs.append(data[pos + 4:pos + 4 + L])
403         pos += 8 + L
404     return recs, endian
```

Fortran sequential files wrap every record in 4-byte length markers. The endianness is found from the first record, whose leading and trailing markers must agree.

L.401–403 Split the file into records.

read_batsrus_idl(): binary IDL files

lines 407–448

```
407 def read_batsrus_idl(path):
408     """Read a BATS-R-US IDL plot file (.out): ascii, real4 or real8.
409 
410     Returns a dict of point variables (coordinates as 'x','y','z') plus
411     '_units' when the header line lists units.  Works for regular Cartesian
412     (cut/box with dx>0) and unstructured (dx<=0, AMR) 3D output."""
413     with open(path, "rb") as fh:
414         first = fh.read(4)
415     binary = False
416     if len(first) == 4:
417         for e in ("<", ">"):
418             if 0 < struct.unpack(e + "i", first)[0] <= 4096:
419                 binary = True
420     if binary:
421         with open(path, "rb") as fh:
422             recs, e = _read_records(fh)
423         head = recs[0].decode("ascii", "ignore").strip()
424         r2 = recs[1]
425         rsize = len(r2) - 16
426         rfmt = "f" if rsize == 4 else "d"
427         nstep, = struct.unpack(e + "i", r2[0:4])
428         time_, = struct.unpack(e + rfmt, r2[4:4 + rsize])
429         ndim, npar, nvar = struct.unpack(e + "3i", r2[4 + rsize:16 + rsize])
430         gencoord = ndim < 0
431         ndim = abs(ndim)
432         n_d = struct.unpack(e + f"{ndim}i", recs[2][:4 * ndim])
433         ir = 3
434         params = []
435         if npar > 0:
436             params = list(struct.unpack(e + f"{npar}{rfmt}", recs[ir][:npar * rsize]))
437             ir += 1
438         names = recs[ir].decode("ascii", "ignore").split()
439         ir += 1
440         npts = int(np.prod(n_d))
441         dt = np.dtype(e + rfmt)
442         coord = np.frombuffer(recs[ir], dtype=dt, count=npts * ndim).astype(float)
443         coord = coord.reshape((npts, ndim), order="F")
444         ir += 1
445         vals = []
446         for _ in range(nvar):
447             vals.append(np.frombuffer(recs[ir], dtype=dt, count=npts).astype(float))
448             ir += 1
```

L.414–419 Binary if the first 4 bytes form a plausible record length; ASCII files start with text.

L.423 Record 1: header line (often the units).

L.424–429 Record 2: nstep, time, ndim, npar, nvar. Its length gives the real size (4 bytes for real4, 8 for real8).

L.430 Negative ndim means generalised (unstructured) coordinates.

L.432 Record 3: grid dimensions.

L.435–437 Record 4 (if present): equation parameters (gamma, rbody, ...).

L.438 Next: names of coordinates, variables and parameters.

L.442–443 Coordinates, stored in Fortran order as (point, dimension).

L.446–447 One record per variable.

read_batsrus_idl(): ASCII files and units

lines 449–494

```
449     else:
450         with open(path, "r") as fh:
451             head = fh.readline().strip()
452             tok = fh.readline().split()
453             nstep, time_ = int(tok[0]), float(tok[1])
454             ndim, npar, nvar = int(tok[2]), int(tok[3]), int(tok[4])
455             gencoord = ndim < 0
456             ndim = abs(ndim)
457             n_d = [int(v) for v in fh.readline().split()[:ndim]]
458             params = [float(v) for v in fh.readline().split()] if npar > 0 else []
459             names = fh.readline().split()
460             npts = int(np.prod(n_d))
461             arr = np.loadtxt(fh, max_rows=npts)
462         coord = arr[:, :ndim]
463         vals = [arr[:, ndim + c] for c in range(nvar)]
464 
465     cnames = names[:ndim]
466     vnames = names[ndim:ndim + nvar]
467     out = {}
468     for c, cn in enumerate(cnames):
469         out[cn] = coord[:, c]
470     for c, vn in enumerate(vnames):
471         out[vn] = vals[c]
472     # units from the header line, if present  ("R R R g/cm3 km/s ...")
473     toks = head.split()
474     units = {}
475     if len(toks) >= ndim + nvar:
476         cand = toks[-(ndim + nvar + len(params)):] if len(toks) >= ndim + nvar + len(params)
    ↪ else toks
477         cand = cand[: ndim + nvar]
478         for n, u in zip(cnames + vnames, cand):
479             units[n] = u.lower().replace("cm3", "cm^3").replace("cm2", "cm^2")
480     out["_units"] = units
481     out["_meta"] = dict(header=head, nstep=nstep, time=time_, ndim=ndim,
482                         gencoord=gencoord, n_d=tuple(n_d), params=params,
483                         param_names=names[ndim + nvar:])
484     if ndim != 3:
485         raise ValueError(f"need a 3D file, got ndim={ndim}")
486     # coordinate names in BATS-R-US are x y z (or r lon lat for spherical)
487     if all(_base(c) in ("x", "y", "z") for c in cnames):
488         out["x"], out["y"], out["z"] = (out[k] for k in cnames)
```

```
489     else:
490         raise ValueError(f"non-Cartesian coordinates {cnames}: convert to x,y,z first")
491     return out
492
493
494 # ============================================================================
```

L.451–461 ASCII layout: the same header items on lines, then one row per point.

L.465–471 Name the coordinate and variable arrays.

L.473–479 If the header line lists one unit per coordinate and variable (for example 'R R R g/cm3 km/s ...'), attach them as `_units`.

L.481 Keep the header information.

L.484–490 Only 3D Cartesian coordinates are accepted here; spherical models go through `SphericalGrid`.

load_mhd(): the general entry point

lines 495–567

```
495 def load_mhd(path, reader=None, **build_kw):
496     """Read an MHD output file and return an MHDGrid.
497
498     reader: 'tecplot', 'pytecplot', 'idl', 'pluto', 'npz' or None (inferred:
499             a directory containing grid.out or a .vtk file -> PLUTO).
500     PLUTO keywords (nout, fmt, definitions, unit_density, unit_length,
501     unit_velocity, mu_e, mean_mol_weight, te_const, r_body, with_b) go to load_pluto.
502     with_b=True also reads the magnetic field (Bx, By, Bz; PLUTO bx1..bx3), in
503     gauss, for gyroresonance; b_unit sets its unit if the file does not.
504     build_kw are passed to build_grid (density_var, density_unit, te_var,
505     pe_var, p_var, te_const, mu, r_body, resample_shape, resample_box,
506     resample_method, dtype)."""
507     ext = os.path.splitext(path)[1].lower()
508     if reader is None and (ext == ".vtk" or (os.path.isdir(path) and
509                            os.path.exists(os.path.join(path, "grid.out")))):
510         reader = "pluto"
511     if reader == "pluto":
512         from .pluto import load_pluto
513         keep = ("nout", "fmt", "definitions", "unit_density", "unit_length",
514                 "unit_velocity", "mu_e", "mean_mol_weight", "te_const", "r_body", "dtype",
515                 "pluto_reader", "pypluto_path", "r_star", "with_b")
516         if "pluto_reader" in build_kw and build_kw["pluto_reader"] is not None:
517             build_kw["reader"] = build_kw.pop("pluto_reader")
518             keep = keep + ("reader",)
519         return load_pluto(path, **{k: v for k, v in build_kw.items()
520                                    if k in keep and v is not None})
521     build_kw = {k: v for k, v in build_kw.items()
522                 if k not in ("nout", "fmt", "definitions", "unit_density", "unit_length",
523                              "unit_velocity", "mu_e", "mean_mol_weight", "pluto_reader",
524                              "pypluto_path")}
525     if not build_kw.get("with_b"):
526         build_kw.pop("b_unit", None)
527     if build_kw.get("r_body") is None:
528         build_kw.pop("r_body", None)
529     if reader is None:
530         reader = {".dat": "tecplot", ".tec": "tecplot", ".plt": "pytecplot",
531                   ".szplt": "pytecplot", ".out": "idl", ".outs": "idl",
```

```
532                 ".npz": "npz"}.get(ext)
533     if reader == "tecplot":
534         v = read_tecplot_ascii(path)
535     elif reader == "pytecplot":
536         v = read_pytecplot(path)
537     elif reader == "idl":
538         v = read_batsrus_idl(path)
539     elif reader == "npz":
540         z = np.load(path)
541         if "geometry" in z.files and str(z["geometry"]) == "spherical":
542             from .grid import SphericalGrid
543             ax = bool(z["axisym"])
544             g = SphericalGrid(z["r"], z["theta"], None if ax else z["phi"], z["ne"], z["te"],
545                               r_edges=z["r_edges"], theta_edges=z["theta_edges"],
546                               phi_edges=None if ax else z["phi_edges"],
547                               r_body=float(z["r_body"]),
548                               length_unit=float(z["length_unit"]) if "length_unit" in z.
    ↪ files
549                               else R_SUN)
550             if "b" in z.files:
551                 g.add_field(*np.moveaxis(z["b"], -1, 0), components=str(z["b_components"]))
552             return g
553         if {"x", "y", "z", "ne"} <= set(z.files) and z["ne"].ndim == 3:
554             g = MHDGrid(z["x"], z["y"], z["z"], z["ne"],
555                         z["te"] if "te" in z.files else None,
556                         r_body=float(z["r_body"]) if "r_body" in z.files
557                         else build_kw.get("r_body", 1.0),
558                         length_unit=float(z["length_unit"]) if "length_unit" in z.files
559                         else (build_kw.get("r_star") or 1.0) * R_SUN)
560             if "b" in z.files:
561                 g.add_field(*np.moveaxis(z["b"], -1, 0))
562             return g
563         v = {k: z[k] for k in z.files}
564     else:
565         raise ValueError(f"cannot infer reader for '{path}'")
566     v = {k: (np.ravel(a) if isinstance(a, np.ndarray) else a) for k, a in v.items()}
567     return build_grid(v, **build_kw)
```

L.508–510 A directory containing grid.out, or a .vtk file, is PLUTO output.

L.511–520 Pass only the PLUTO-relevant keywords to `load_pluto`.

L.521–528 Drop the PLUTO keywords and unset values for the other readers.

L.530 Choose the reader from the file extension.

L.533–538 Call the reader.

L.539–563 A saved grid is rebuilt directly (spherical or Cartesian); any other npz is treated as point data.

L.566–567 Flatten the arrays and build the grid.

Chapter 5

common/pluto.py — PLUTO input

File: `radiosynth/common/pluto.py` — 512 lines.

Reads PLUTO binary (`.dbl`/`.flt`) and legacy VTK output, converts code units to physical units using `definitions.h`, and returns a `SphericalGrid` or `MHDGrid`.

Module docstring

lines 4–46

```
4  """Reader for PLUTO (Mignone et al. 2007) output.
5
6  Supported
7  ---------
8  * binary ``.dbl`` (float64) and ``.flt`` (float32) output, ``single_file`` or
9    ``multiple_files``, described by ``grid.out`` and ``dbl.out`` / ``flt.out``;
10 * legacy VTK ``.vtk`` (RECTILINEAR_GRID for Cartesian, STRUCTURED_GRID for
11   3D spherical), cell data;
12 * geometries CARTESIAN (-> MHDGrid) and SPHERICAL (-> SphericalGrid; 3D or
13   2.5D axisymmetric r-theta).
14
15 Units
16 -----
17 PLUTO stores normalised variables.  Physical values are
18     rho [g/cm^3]      = rho * UNIT_DENSITY
19     p   [dyn/cm^2]    = prs * UNIT_DENSITY * UNIT_VELOCITY^2
20     length [cm]       = x   * UNIT_LENGTH
21 The three units are read from ``definitions.h`` when it is found (the
22 expressions may use PLUTO's CONST_* names), or can be passed explicitly.
23 PLUTO's own defaults are UNIT_DENSITY = m_p, UNIT_LENGTH = 1 AU,
24 UNIT_VELOCITY = 1 km/s.
25
26 Plasma model
27 ------------
28 n_e = rho / (mu_e m_p)     (mu_e = mass per electron in m_p; 1 for pure H as
29                             in BATS-R-US, ~1.17 for solar H+He)
30 T_e = p * mu m_p / (rho k_B)  (single-fluid temperature, mu = mean molecular
31                             weight; 0.5 for fully ionised H, i.e. T_e = T_i =
32                             p / (2 n k_B)).  A 'Te' or 'T' variable in the
33                             output takes precedence, or use te_const.
34 """
35 import glob
36 import math
37 import os
38 import re
39 import struct
40
41 import numpy as np
42
43 from .constants import K_B, M_P, R_SUN
44 from .grid import MHDGrid, SphericalGrid
45
46 # PLUTO pluto.h physical constants (cgs)
```

Documents the supported formats, the unit conversion and the plasma model (n_e, T).

PLUTO constants and default units

lines 47–63

```
47 PLUTO_CONST = dict(
48     CONST_AH=1.008, CONST_AHe=4.004, CONST_AZ=30.0, CONST_amu=1.66053886e-24,
49     CONST_au=1.49597892e13, CONST_c=2.99792458e10, CONST_e=4.80320425e-10,
50     CONST_eV=1.602176463158e-12, CONST_G=6.6726e-8, CONST_h=6.62606876e-27,
51     CONST_kB=1.3806505e-16, CONST_ly=0.9461e18, CONST_mp=1.67262171e-24,
52     CONST_mn=1.67492728e-24, CONST_me=9.1093826e-28, CONST_mH=1.6733e-24,
53     CONST_Lsun=3.826e33, CONST_Mearth=5.9736e27, CONST_Msun=2.e33,
54     CONST_NA=6.0221367e23, CONST_pc=3.0856775807e18, CONST_PI=np.pi,
55     CONST_Rearth=6.378136e8, CONST_Rgas=8.3144598e7, CONST_Rsun=6.96e10,
56     CONST_sigma=5.67051e-5, CONST_sigmaT=6.6524e-25,
57 )
58 PLUTO_DEFAULT_UNITS = dict(UNIT_DENSITY=PLUTO_CONST["CONST_mp"],
59                            UNIT_LENGTH=PLUTO_CONST["CONST_au"],
60                            UNIT_VELOCITY=1.0e5)
61
62
63 # ---------------------------------------------------------------------------
```

L.47 The physical constants of PLUTO's `pluto.h` (cgs), so that `definitions.h` expressions such as `CONST_mp*1.e8` evaluate exactly as in PLUTO.

L.58 PLUTO's defaults when `definitions.h` does not set them: $U_\rho = m_p$, $U_L = 1$ AU, $U_v = 1$ km/s.

parse_definitions()

lines 64–83

```
64 def parse_definitions(path):
65     """Evaluate UNIT_DENSITY/UNIT_LENGTH/UNIT_VELOCITY (and other numeric
66     #defines) from a PLUTO definitions.h."""
67     env = dict(PLUTO_CONST)
68     env.update(sqrt=np.sqrt, pow=pow, exp=np.exp, log=np.log)
69     out = {}
70     with open(path) as fh:
71         for line in fh:
72             m = re.match(r"\s*#define\s+(\w+)\s+(.+?)\s*(/\*.*)?$", line)
73             if not m:
74                 continue
75             name, expr = m.group(1), m.group(2)
76             try:
77                 val = eval(expr, {"__builtins__": {}}, env)       # noqa: S307
78             except Exception:
79                 continue
80             if isinstance(val, (int, float, np.floating)) and not isinstance(val, bool):
81                 env[name] = float(val)
82                 out[name] = float(val)
83     return out
```

L.67–68 Evaluation namespace: the constants plus a few math functions.

L.72 Match `#define NAME expression` lines (a trailing C comment is ignored).

L.77–79 Evaluate the expression without Python built-ins. Non-numeric defines (such as `PHYSICS MHD`) fail and are skipped.

L.81–82 Numeric results are stored and become available to later defines.

read_grid_out()

lines 86–114

```
86  def read_grid_out(path):
87      """Parse PLUTO grid.out -> (geometry, [(left_edges, right_edges, centres)] x 3)."""
88      geom = "CARTESIAN"
89      dims = []
90      with open(path) as fh:
91          lines = fh.readlines()
92      k = 0
93      while k < len(lines) and lines[k].lstrip().startswith("#"):
94          m = re.search(r"GEOMETRY:\s*(\w+)", lines[k])
95          if m:
96              geom = m.group(1).upper()
97          k += 1
98      while k < len(lines) and len(dims) < 3:
99          tok = lines[k].split()
100         k += 1
101         if not tok:
102             continue
103         n = int(tok[0])
104         rows = np.array([[float(v) for v in lines[k + i].split()] for i in range(n)])
105         k += n
106         if rows.shape[1] >= 5:          # PLUTO 3: i xL xc xR dx
107             xl, xc, xr = rows[:, 1], rows[:, 2], rows[:, 3]
108         else:                           # PLUTO 4: i xL xR
109             xl, xr = rows[:, 1], rows[:, 2]
110             xc = 0.5 * (xl + xr)
111         dims.append((xl, xr, xc))
112     while len(dims) < 3:
113         dims.append((np.array([0.0]), np.array([1.0]), np.array([0.5])))
114     return geom, dims
```

L.93–96 Skip the comment header, picking up the `GEOMETRY` line.

L.103–105 For each direction: the number of cells, then one line per cell.

L.106–107 PLUTO 3 layout: i, left, centre, right, width.

L.109–110 PLUTO 4 layout: i, left, right; the centre is the midpoint.

L.112 Fill missing dimensions of 1D/2D runs with a single cell.

_read_out_file()

lines 117–126

```
117 def _read_out_file(path, nout):
118     """Parse dbl.out/flt.out: returns (nout, time, mode, endian, varnames)."""
119     rows = [ln.split() for ln in open(path) if ln.strip()]
120     if nout is None or nout < 0:
121         row = rows[nout if nout is not None else -1]
122     else:
123         row = next((r for r in rows if int(r[0]) == nout), None)
124         if row is None:
```

```
125             raise ValueError(f"output {nout} not listed in {path}")
126     return int(row[0]), float(row[1]), row[4], row[5], row[6:]
```

L.119–125 Pick the line for the requested output number, or the last line by default.
L.126 Output number, time, single/multiple files, endianness and the variable list.

_stag_shape()

lines 129–139

```
129 def _stag_shape(name, n):
130     """Size of staggered fields (bx1s etc.) in PLUTO single-file output."""
131     n1, n2, n3 = n
132     low = name.lower()
133     if low.endswith("1s"):
134         return (n1 + 1, n2, n3)
135     if low.endswith("2s"):
136         return (n1, n2 + 1, n3)
137     if low.endswith("3s"):
138         return (n1, n2, n3 + 1)
139     return n
```

Staggered magnetic-field components (e.g. `Bx1s`) have one more value along their direction. Their size is needed to skip over them in a single-file output.

read_pluto_binary()

lines 142–178

```
142 def read_pluto_binary(directory, nout=-1, fmt=None, variables=("rho", "prs", "Te", "T")):
143     """Read selected variables of a PLUTO .dbl/.flt output.
144 
145     Returns dict(geometry, dims, time, nout, vars={name: (n1,n2,n3) array})."""
146     if fmt is None:
147         fmt = "dbl" if os.path.exists(os.path.join(directory, "dbl.out")) else "flt"
148     outf = os.path.join(directory, f"{fmt}.out")
149     nout, time, mode, endian, names = _read_out_file(outf, nout)
150     geom, dims = read_grid_out(os.path.join(directory, "grid.out"))
151     n = tuple(len(d[2]) for d in dims)
152     dt = np.dtype(("<" if endian.startswith("l") else ">") + ("f8" if fmt == "dbl" else "f4")
    ↪ )
153     want = {v.lower() for v in variables}
154     out = {}
155     if mode == "single_file":
156         fn = os.path.join(directory, f"data.{nout:04d}.{fmt}")
157         with open(fn, "rb") as fh:
158             off = 0
159             for name in names:
160                 shp = _stag_shape(name, n)
161                 cnt = int(np.prod(shp))
162                 if name.lower() in want:
163                     fh.seek(off)
164                     a = np.fromfile(fh, dtype=dt, count=cnt)
165                     out[name] = a.reshape(shp[::-1]).transpose(2, 1, 0).astype(float)
166                 off += cnt * dt.itemsize
167     else:
168         for name in names:
```

```
169             if name.lower() not in want:
170                 continue
171             fn = os.path.join(directory, f"{name}.{nout:04d}.{fmt}")
172             shp = _stag_shape(name, n)
173             a = np.fromfile(fn, dtype=dt, count=int(np.prod(shp)))
174             out[name] = a.reshape(shp[::-1]).transpose(2, 1, 0).astype(float)
175     return dict(geometry=geom, dims=dims, time=time, nout=nout, vars=out, names=names)
176
177
178 # ---------------------------------------------------------------------------
```

L.147 Use .dbl if a dbl.out exists, otherwise .flt.

L.149 Description of the requested output.

L.150 Geometry and cell edges.

L.152 Data type: float64 or float32, little or big endian.

L.155–166 Single file: the variables follow each other. Seek to each wanted variable using the sizes of the preceding ones, so only rho, prs (and T/Te) are read.

L.165 PLUTO writes x_1 fastest. Read as (n3, n2, n1) and transpose to (n1, n2, n3).

L.167–174 Multiple files: one file per variable, `name.NNNN.dbl`.

read_pluto_vtk(): header and helpers

lines 179–211

```
179 def read_pluto_vtk(path, variables=("rho", "prs", "Te", "T")):
180     """Read a PLUTO legacy-VTK file (binary, big-endian).  Returns the same dict
181     as read_pluto_binary; for STRUCTURED_GRID the node coordinates are
182     converted back to spherical (r, theta, phi) edges."""
183     data = open(path, "rb").read()
184     pos = 0
185
186     def line():
187         nonlocal pos
188         e = data.index(b"\n", pos)
189         s = data[pos:e].decode("ascii", "ignore").strip()
190         pos = e + 1
191         return s
192
193     def arr(count, typ):
194         nonlocal pos
195         dt = np.dtype(">f4") if typ.startswith("float") else np.dtype(">f8") \
196             if typ.startswith("double") else np.dtype(">i4")
197         a = np.frombuffer(data, dtype=dt, count=count, offset=pos).astype(float)
198         pos += count * dt.itemsize
199         return a
200
201     line(); line()
202     if line().upper() != "BINARY":
203         raise ValueError("only BINARY legacy VTK is supported")
204     kind = line().split()[1].upper()
205     dims = None
206     coords = {}
207     pts = None
208     want = {v.lower() for v in variables}
209     out = {}
210     time = 0.0
```

```
211     b_cartesian = False
```

L.186–191 Read one text line from the byte buffer.

L.193–199 Read a big-endian binary array (VTK legacy binary is always big-endian).

L.201–204 Skip the version and title lines, check BINARY, read the dataset type.

read_pluto_vtk(): sections

lines 212–251

```
212     while pos < len(data):
213         s = line()
214         if not s:
215             continue
216         t = s.split()
217         key = t[0].upper()
218         if key == "FIELD":
219             for _ in range(int(t[2])):
220                 h = line().split()
221                 while not h:
222                     h = line().split()
223                 v = arr(int(h[1]) * int(h[2]), h[3].lower())
224                 if h[0].upper() == "TIME":
225                     time = float(v[0])
226         elif key == "DIMENSIONS":
227             dims = tuple(int(v) for v in t[1:4])
228         elif key in ("X_COORDINATES", "Y_COORDINATES", "Z_COORDINATES"):
229             coords[key[0]] = arr(int(t[1]), t[2].lower())
230         elif key == "POINTS":
231             pts = arr(3 * int(t[1]), t[2].lower()).reshape(-1, 3)
232         elif key in ("CELL_DATA", "POINT_DATA"):
233             pass
234         elif key == "SCALARS":
235             name, typ = t[1], t[2].lower()
236             line()                                  # LOOKUP_TABLE default
237             nc = [max(d - 1, 1) for d in dims]
238             a = arr(int(np.prod(nc)), typ)
239             if name.lower() in want:
240                 out[name] = a.reshape(nc[::-1]).transpose(2, 1, 0)
241         elif key == "VECTORS":
242             nc = [max(d - 1, 1) for d in dims]
243             a = arr(3 * int(np.prod(nc)), t[2].lower())
244             # VTK_VECTOR_DUMP: PLUTO writes vectors in Cartesian components
245             if "bx1" in want and t[1].lower().startswith(("magnetic", "bfield", "b_field")):
246                 a = a.reshape(nc[::-1] + [3])
247                 for c in range(3):
248                     out[f"bx{c + 1}"] = a[..., c].transpose(2, 1, 0)
249                 b_cartesian = True
250         else:
251             continue
```

L.218–225 Field data: PLUTO stores the time here.

L.226 Number of grid nodes (cells + 1) per direction.

L.228 Rectilinear grid: node coordinates along each axis.

L.230 Structured grid: Cartesian coordinates of every node.

L.234–240 Cell data: read the wanted variables, skip the others.

L.241–243 Vector data are skipped.

read_pluto_vtk(): geometry

lines 252–276

```
252     if kind == "RECTILINEAR_GRID":
253         geom = "CARTESIAN"
254         dd = []
255         for c in "XYZ":
256             e = coords[c]
257             if e.size == 1:
258                 e = np.array([e[0] - 0.5, e[0] + 0.5])
259             dd.append((e[:-1], e[1:], 0.5 * (e[:-1] + e[1:])))
260     elif kind == "STRUCTURED_GRID":
261         geom = "SPHERICAL"
262         P = pts.reshape(dims[2], dims[1], dims[0], 3).transpose(2, 1, 0, 3)
263         r = np.linalg.norm(P, axis=-1)
264         th = np.arccos(np.clip(P[..., 2] / np.maximum(r, 1e-300), -1, 1))
265         ph = np.mod(np.arctan2(P[..., 1], P[..., 0]), 2 * np.pi)
266         re_ = r[:, 0, 0]
267         te_ = th[0, :, 0] if dims[1] > 1 else np.array([0.0, np.pi])
268         pe_ = np.unwrap(ph[0, dims[1] // 2, :]) if dims[2] > 1 else np.array([0.0, 2 * np.pi
    ↪ ])
269         dd = [(e[:-1], e[1:], 0.5 * (e[:-1] + e[1:])) for e in (re_, te_, pe_)]
270     else:
271         raise ValueError(f"unsupported VTK dataset {kind}")
272     return dict(geometry=geom, dims=dd, time=time, nout=None, vars=out, names=list(out),
273                 b_cartesian=b_cartesian)
274
275
276 # ---------------------------------------------------------------------------
```

L.253–259 Cartesian: node coordinates give the cell edges directly.

L.261–269 Spherical: convert the node positions back to r, θ, ϕ and read the edges along one grid line in each direction.

_import_pypluto(): find a pyPLUTO module

lines 277–293

```
277 def _import_pypluto(pypluto_path=None):
278     """Import pyPLUTO: from ``pypluto_path`` if given, else an installed pyPLUTO
279     (classic ``pload`` version shipped with PLUTO, or the newer PyPLUTO package
280     with ``Load``), else the copy bundled in radiosynth/common/contrib/pyPLUTO.py."""
281     import importlib
282     import sys
283     if pypluto_path:
284         p = os.path.abspath(pypluto_path)
285         if os.path.isfile(p):
286             p = os.path.dirname(p)
287         if p not in sys.path:
288             sys.path.insert(0, p)
289         return importlib.import_module("pyPLUTO")
290     try:                                    # a pyPLUTO installed by the user
291         return importlib.import_module("pyPLUTO")
```

```
292     except ImportError:                     # the copy bundled with radiosynth
293         return importlib.import_module("radiosynth.common.contrib.pyPLUTO")
```

Lets radiosynth use the pyPLUTO reader that comes with PLUTO, as an alternative to the built-in reader.

L.283–289 If a directory (or the path of pyPLUTO.py) is given, put it first on `sys.path` and import from there.

L.290–291 Otherwise use an installed pyPLUTO (classic or the new PyPLUTO package) ...

L.292 ... and if none is installed, the copy bundled in `radiosynth/contrib/pyPLUTO.py` (the classic `pload` version, with the multiple-files fix).

_to_n123(): put arrays in (n1, n2, n3) order

lines 296–308

```
296 def _to_n123(a, n):
297     """Return array a as (n1, n2, n3) whatever the reader's axis order/dimension."""
298     a = np.asarray(a, dtype=float)
299     if a.shape == tuple(n):
300         return a
301     if a.ndim == 3 and a.shape == tuple(n[::-1]):
302         return a.transpose(2, 1, 0)
303     sq = tuple(k for k in n if k > 1)
304     if a.shape == sq:
305         return a.reshape(n)
306     if a.shape == sq[::-1]:
307         return a.T.reshape(n)
308     raise ValueError(f"cannot match array shape {a.shape} to grid {n}")
```

Different readers return arrays in different orders: classic pyPLUTO returns (n1, n2, n3) for 3D and (n1, n2) for 2D, while other tools return C order (n3, n2, n1). This helper accepts any of these and returns (n1, n2, n3).

L.301 Reversed (C-order) 3D array: transpose.

L.303–307 2D/1D outputs: restore the singleton dimensions.

read_pluto_pypluto(): read through pyPLUTO

lines 311–352

```
311 def read_pluto_pypluto(directory, nout=-1, fmt=None, pypluto_path=None,
312                       variables=("rho", "prs", "Te", "T")):
313     """Read PLUTO output through pyPLUTO; returns the same dict as
314     read_pluto_binary.  Works with the classic ``pyPLUTO.pload(ns, w_dir=...,
315     datatype=...)`` and with the newer ``pyPLUTO.Load(nout, path=...)``."""
316     pp = _import_pypluto(pypluto_path)
317     d = directory if directory.endswith(os.sep) else directory + os.sep
318     if fmt is None:
319         fmt = "dbl" if os.path.exists(os.path.join(d, "dbl.out")) else "flt"
320     if nout is None or nout < 0:
321         nout = _read_out_file(os.path.join(d, f"{fmt}.out"), nout)[0]
322     geom, _ = read_grid_out(os.path.join(d, "grid.out"))
323     if hasattr(pp, "pload"):                                  # classic pyPLUTO
324         D = pp.pload(nout, w_dir=d, datatype="float" if fmt == "flt" else "double")
325         names = list(getattr(D, "vars", []))
326         time = float(getattr(D, "time", getattr(D, "SimTime", 0.0)))
```

```
327     elif hasattr(pp, "Load"):                                   # PyPLUTO >= 1.0
328         D = pp.Load(nout, path=d, datatype=fmt)
329         names = list(getattr(D, "varsfile", getattr(D, "vars", [])) or [])
330         geom = str(getattr(D, "geom", geom)).upper()
331         time = float(np.atleast_1d(getattr(D, "timelist", [0.0]))[0])
332     else:
333         raise ImportError("the imported pyPLUTO has neither pload nor Load")
334     x1r, x2r, x3r = (np.atleast_1d(np.asarray(getattr(D, f"x{i}r"), float)) for i in (1, 2,
        ↪ 3))
335     if x3r.size < 2:
336         x3r = np.array([0.0, 2 * np.pi]) if geom == "SPHERICAL" else np.array([-0.5, 0.5])
337     if x2r.size < 2:
338         x2r = np.array([0.0, np.pi]) if geom == "SPHERICAL" else np.array([-0.5, 0.5])
339     dims = [(e[:-1], e[1:], 0.5 * (e[:-1] + e[1:])) for e in (x1r, x2r, x3r)]
340     n = tuple(len(dd[2]) for dd in dims)
341     want = {v.lower() for v in variables}
342     out = {}
343     for v in want:
344         for cand in (v, v.capitalize(), v.upper()):
345             if hasattr(D, cand):
346                 out[cand] = _to_n123(getattr(D, cand), n)
347                 break
348     return dict(geometry=geom, dims=dims, time=time, nout=nout, vars=out,
349                 names=names or list(out))
350
351
352 # ---------------------------------------------------------------------------
```

L.316–322 Import pyPLUTO, choose dbl/flt, find the last output number in dbl.out/flt.out if none is given, and read the geometry from grid.out (classic pyPLUTO does not report it).

L.323–326 Classic API: `pyPLUTO.pload(ns, w_dir=..., datatype='double'|'float')`. The variables become attributes (`D.rho`, `D.prs`, ...), with the list in `D.vars` and the time in `D.time`.

L.327–331 Newer PyPLUTO API: `pyPLUTO.Load(nout, path=...)`, which also gives the geometry in `D.geom`.

L.334–338 Cell edges `x1r`, `x2r`, `x3r` (pyPLUTO attributes). Missing dimensions of 2D runs get a single cell (the full ϕ or θ range for spherical runs).

L.339–340 The same (left edge, right edge, centre) triplets as `read_grid_out`.

L.343–347 Copy rho, prs and a temperature if present, in (n1, n2, n3) order.

L.348 The same dictionary as `read_pluto_binary`, so the rest of `load_pluto` is shared by both readers.

_find_var() lines 353–358

```
353 def _find_var(vars_, *names):
354     low = {k.lower(): k for k in vars_}
355     for n in names:
356         if n.lower() in low:
357             return vars_[low[n.lower()]]
358     return None
```

Case-insensitive lookup of a variable name.

load_pluto(): read and find the units

lines 361–424

```
361 def load_pluto(path, nout=-1, fmt=None, definitions=None, unit_density=None,
362                unit_length=None, unit_velocity=None, mu_e=1.0, mean_mol_weight=0.5,
363                te_const=None, r_body=None, dtype=np.float32, verbose=True,
364                reader="builtin", pypluto_path=None, r_star=None, with_b=False):
365     """Load PLUTO output and return an MHDGrid (Cartesian) or SphericalGrid.
366 
367     path        : output directory (with grid.out, dbl.out/flt.out) or a .vtk file
368     nout        : output number (default: last one listed in dbl.out/flt.out)
369     fmt         : 'dbl' or 'flt' (default: whichever .out file exists)
370     definitions : path to definitions.h (default: look in the directory and its
371                   parent); explicit unit_* arguments override it
372     mu_e        : mass per electron in m_p:  n_e = rho / (mu_e m_p)
373     mean_mol_weight : mu in T = p mu m_p / (rho k_B)
374     te_const    : use a constant T_e [K] (isothermal / no pressure output)
375     r_body      : inner boundary in R_sun (default: inner radial edge)
376     reader      : 'builtin' (radiosynth's own .dbl/.flt reader) or 'pypluto'
377                   (use the pyPLUTO module shipped with PLUTO; falls back to the
378                   built-in reader if pyPLUTO fails)
379     pypluto_path: directory containing pyPLUTO.py (if not on sys.path)
380     r_star      : stellar radius in R_sun.  If given, coordinates are expressed in
381                   stellar radii (x * UNIT_LENGTH / R_star) and the grid's
382                   length_unit is R_star; default: coordinates in R_sun
383     with_b      : also read the cell-centred magnetic field bx1, bx2, bx3 and
384                   attach it in gauss, B = b sqrt(4 pi UNIT_DENSITY) UNIT_VELOCITY
385                   (needed for gyroresonance)
386     """
387     want = ("rho", "prs", "Te", "T") + (("bx1", "bx2", "bx3") if with_b else ())
388     if os.path.isdir(path):
389         d = None
390         if reader == "pypluto":
391             try:
392                 d = read_pluto_pypluto(path, nout, fmt, pypluto_path, variables=want)
393                 if verbose:
394                     print("radiosynth: PLUTO data read with pyPLUTO")
395             except Exception as exc:                        # noqa: BLE001
396                 print(f"radiosynth: pyPLUTO failed ({type(exc).__name__}: {exc}); "
397                       f"using the built-in reader")
398         if d is None:
399             d = read_pluto_binary(path, nout, fmt, variables=want)
400         base = path
401     else:
402         d = read_pluto_vtk(path, variables=want)
403         base = os.path.dirname(os.path.abspath(path))
404 
405     units = dict(PLUTO_DEFAULT_UNITS)
406     src = "PLUTO defaults"
407     if definitions is None:
408         for cand in (os.path.join(base, "definitions.h"),
409                      os.path.join(base, "..", "definitions.h")):
410             if os.path.exists(cand):
411                 definitions = cand
412                 break
413     if definitions is not None:
414         defs = parse_definitions(definitions)
415         for k in units:
416             if k in defs:
```

```
                units[k] = defs[k]
        src = os.path.relpath(definitions)
    for k, v in (("UNIT_DENSITY", unit_density), ("UNIT_LENGTH", unit_length),
                 ("UNIT_VELOCITY", unit_velocity)):
        if v is not None:
            units[k] = float(v)
            src = "user arguments"
    UD, UL, UV = units["UNIT_DENSITY"], units["UNIT_LENGTH"], units["UNIT_VELOCITY"]
```

L.388–403 A directory is binary output; a file is VTK.

L.390–397 With `reader='pypluto'` the data are read by pyPLUTO. If that fails (missing module, unsupported layout, a bug in an old copy), the built-in reader is used and a message says so.

L.405–412 Start from PLUTO's defaults and look for definitions.h next to the data or one level up.

L.413–418 Take the units from definitions.h.

L.419–423 Explicit arguments override everything.

load_pluto(): physical fields

lines 426–445

```
    v = d["vars"]
    rho = _find_var(v, "rho")
    if rho is None:
        raise KeyError(f"no 'rho' in PLUTO output (variables: {d['names']})")
    rho_cgs = rho * UD
    ne = rho_cgs * 1e3 / (mu_e * M_P)                          # m^-3
    T = _find_var(v, "Te", "T")
    if te_const is not None:
        te = np.full_like(ne, float(te_const))
        tsrc = f"constant {te_const} K"
    elif T is not None:
        te = T
        tsrc = "T/Te variable [K]"
    else:
        prs = _find_var(v, "prs")
        if prs is None:
            raise KeyError("no 'prs' in output: pass te_const= for isothermal runs")
        p_cgs = prs * UD * UV**2
        te = p_cgs * mean_mol_weight * M_P * 1e3 / (np.maximum(rho_cgs, 1e-300) * K_B * 1e7)
        tsrc = f"p mu m_p/(rho k_B), mu={mean_mol_weight}"
```

L.430 Density in g cm^{-3}.

L.431 $n_e = \rho/(\mu_e m_p)$ in m^{-3} (g cm^{-3} $\times 10^3$ = kg m^{-3}).

L.433–438 A constant temperature, or a T/Te variable from the file.

L.443–444 Otherwise $T = p\,\mu m_p/(\rho k_B)$ in cgs (m_p in g, k_B in erg/K).

load_pluto(): grid

lines 447–476

```
    Lu = (r_star or 1.0) * R_SUN                               # metres per output unit
    L = UL / (Lu * 100.0)                                      # code length -> R_sun or
    ↪ R_star
```

```
449     (x1l, x1r, x1), (x2l, x2r, x2), (x3l, x3r, x3) = d["dims"]
450     geom = d["geometry"]
451     if geom == "SPHERICAL":
452         r, r_e = x1 * L, np.concatenate([x1l, x1r[-1:]]) * L
453         nph = ne.shape[2]
454         one_d = ne.shape[1] == 1 and nph == 1
455         if ne.shape[1] == 1 and not one_d:
456             raise ValueError("spherical output with one theta cell but several phi cells "
457                              "is not supported")
458         phi = x3 if nph > 1 else None
459         if one_d:                                   # 1D radial model: spherical symmetry
460             from .symmetric import grid_from_profile
461             g = grid_from_profile(r, ne[:, 0, 0], te[:, 0, 0], r_edges=r_e, r_body=r_body,
462                                   length_unit=Lu, dtype=dtype)
463             if verbose:
464                 print("radiosynth: 1D spherical PLUTO output - assuming spherical symmetry")
465         else:
466             g = SphericalGrid(r, x2, phi, ne, te, r_edges=r_e,
467                               theta_edges=np.concatenate([x2l, x2r[-1:]]),
468                               phi_edges=np.concatenate([x3l, x3r[-1:]]) if nph > 1 else None,
469                               r_body=r_body, dtype=dtype, length_unit=Lu)
470     elif geom == "CARTESIAN":
471         if min(ne.shape) < 2:
472             raise ValueError("Cartesian PLUTO output must be 3D for imaging")
473         g = MHDGrid(x1 * L, x2 * L, x3 * L, ne, te,
474                     r_body=1.0 if r_body is None else r_body, dtype=dtype, length_unit=Lu)
475     else:
476         raise ValueError(f"PLUTO geometry {geom} not supported (use CARTESIAN or SPHERICAL)")
```

L.447 Output length unit in metres: $R_\odot$, or R_* when `r_star` is given (stellar runs).

L.448 Code length unit expressed in that unit (U_L in cm divided by $R_\odot$ or R_* in cm).

L.451–469 Spherical: radii in $R_\odot$ (or R_*), angles unchanged; one ϕ cell means axisymmetric. The grid records its length unit.

L.470–474 Cartesian: all axes in $R_\odot$ (or R_*).

L.454–464 1D spherical output (one θ and one ϕ cell) is read as a spherically symmetric model with `grid_from_profile` (common/symmetric.py).

load_pluto(): magnetic field

lines 478–487

```
478     if with_b:
479         b = [_find_var(v, f"bx{i}") for i in (1, 2, 3)]
480         if any(c is None for c in b):
481             raise KeyError(f"with_b=True but bx1/bx2/bx3 not found (variables: {d['names']})"
    ↪ )
482         bfac = math.sqrt(4.0 * math.pi * UD) * UV                # code units -> gauss
483         b = [c * bfac for c in b]
484         if geom == "SPHERICAL":
485             g.add_field(*b, components="cartesian" if d.get("b_cartesian") else "spherical")
486         else:
487             g.add_field(*b)
```

L.479 Cell-centred `bx1`, `bx2`, `bx3` (the staggered `bx1s`... are not used).

L.482 PLUTO code units to gauss: $B = b\sqrt{4\pi\rho_0}\,v_0$ with ρ_0 = `UNIT_DENSITY` and v_0 = `UNIT_`

`VELOCITY` (cgs).

L.484–487 Spherical runs store (B_r, B_θ, B_ϕ); VTK files written with `VTK_VECTOR_DUMP` hold Cartesian vectors (`b_cartesian`). Cartesian runs store (B_x, B_y, B_z).

load_pluto(): report

lines 489–512

```
489     if verbose:
490         if with_b:
491             bmag = np.sqrt(sum(c**2 for c in b))
492             print(f"radiosynth: magnetic field bx1..bx3 x {bfac:.4e} G, |B| max {bmag.max()
    ↪ :.3e} G")
493         print(f"radiosynth: PLUTO {geom.lower()} output, t={d['time']:g} (code units), "
494               f"variables {d['names']}")
495         print(f"            units from {src}: UNIT_DENSITY={UD:.4e} g/cm^3, "
496               f"UNIT_LENGTH={UL:.4e} cm (= {L:.4g} {'R_star' if r_star else 'R_sun'}), "
497               f"UNIT_VELOCITY={UV:.4e} cm/s")
498         print(f"            n_e = rho/({mu_e} m_p); T_e from {tsrc}")
499         print(f"            {g}")
500         if geom == "SPHERICAL" and not (0.5 < g.r_lo < 3.0):
501             print(f"WARNING: inner radius is {g.r_lo:.3g} {'R_star' if r_star else 'R_sun'} -
    ↪ "
502                   f"check UNIT_LENGTH (PLUTO's default is 1 AU; pass definitions= or "
503                   f"unit_length=), or r_star= for a star")
504         rr = g.r if geom == "SPHERICAL" else None
505         if rr is not None:
506             i = np.searchsorted(rr, 1.1 * max(g.r_lo, 1.0))
507             i = min(i, rr.size - 1)
508             print(f"            at r={rr[i]:.3g} R_sun: n_e = {np.median(ne[i]):.3e} m^-3, "
509                   f"T_e = {np.median(te[i]):.3e} K (median over angles)")
510     g.info = dict(units=units, time=d["time"], nout=d["nout"], te=tsrc)
511
512     return g
```

L.493–499 Print the units used, the plasma model and the grid.

L.500–502 Warn if the inner radius is not near 1 $R_\odot$, the usual sign of a wrong length unit.

L.504–509 Typical n_e, T_e at $r \approx 1.1\,R_\odot$ as a sanity check.

Chapter 6

common/fields.py — derived fields (current density)

File: `radiosynth/common/fields.py` — 67 lines.

Computes $\nabla \times \mathbf{B}$ and $|\mathbf{J}|$ on the grid, a proxy for where magnetic energy is released, usable in fast-electron rules.

Module docstring and constants

lines 4–14

```
4  """Derived fields on the model grid.
5
6  current_density(grid) computes |J| = |curl B| / mu0 on the grid from the
7  magnetic field attached with add_field(), and stores it as the scalar field
8  'j' [A m^-2] (grid.sample_scalar('j', points)).  It is a common proxy for the
9  sites of magnetic energy release (current sheets, reconnection regions), e.g.
10 to place fast electrons for gyrosynchrotron emission.
11 """
12 import numpy as np
13
14 MU0 = 4e-7 * np.pi
```

`L.14` μ_0 [SI].

_grad()

lines 17–20

```
17 def _grad(a, x, axis):
18     if a.shape[axis] < 2:
19         return np.zeros_like(a)
20     return np.gradient(a, x, axis=axis, edge_order=2 if a.shape[axis] > 2 else 1)
```

Second-order `np.gradient` along one axis on a non-uniform axis; zero for a degenerate (single-cell) axis, e.g. ϕ of an axisymmetric grid.

curl_b()

lines 23–57

```
23 def curl_b(grid):
24     """curl B on the grid nodes, in gauss per coordinate unit.
25
26     Cartesian grids: (curl B)_x,y,z.  Spherical grids: components (r, theta,
27     phi) of curl B computed with the spherical formulas from the stored
28     (B_r, B_theta, B_phi) (or from Cartesian components converted to them)."""
29     if not getattr(grid, "has_field", False):
30         raise ValueError("the grid has no magnetic field")
31     if hasattr(grid, "x"):                                  # MHDGrid
32         b = grid._bflat.reshape(grid.shape + (3,)).astype(float)
```

```
33          bx, by, bz = b[..., 0], b[..., 1], b[..., 2]
34          x, y, z = grid.x, grid.y, grid.z
35          cx = _grad(bz, y, 1) - _grad(by, z, 2)
36          cy = _grad(bx, z, 2) - _grad(bz, x, 0)
37          cz = _grad(by, x, 0) - _grad(bx, y, 1)
38          return cx, cy, cz
39      # SphericalGrid
40      b = grid._bflat.reshape(grid.shape + (3,)).astype(float)
41      R = grid.r[:, None, None]
42      TH = grid.theta[None, :, None]
43      PH = grid.phi[None, None, :]
44      if grid._bcomp == "cartesian":
45          st, ct, sp, cp = np.sin(TH), np.cos(TH), np.sin(PH), np.cos(PH)
46          bx, by, bz = b[..., 0], b[..., 1], b[..., 2]
47          br = bx * st * cp + by * st * sp + bz * ct
48          bt = bx * ct * cp + by * ct * sp - bz * st
49          bp = -bx * sp + by * cp
50      else:
51          br, bt, bp = b[..., 0], b[..., 1], b[..., 2]
52      S = np.maximum(np.sin(TH), 1e-6)
53      phi = grid.phi
54      c_r = (_grad(S * bp, grid.theta, 1) - _grad(bt, phi, 2)) / (R * S)
55      c_t = (_grad(br, phi, 2) / S - _grad(R * bp, grid.r, 0)) / R
56      c_p = (_grad(R * bt, grid.r, 0) - _grad(br, grid.theta, 1)) / R
57      return c_r, c_t, c_p
```

L.31–38 Cartesian grid: $(\nabla \times \mathbf{B})_x = \partial_y B_z - \partial_z B_y$, etc.

L.41–49 Spherical grid: the components (B_r, B_θ, B_ϕ) (converted if the field was given in Cartesian components).

L.54–56 Spherical curl: $(\nabla\times\mathbf{B})_r = \frac{1}{r\sin\theta}[\partial_\theta(\sin\theta B_\phi) - \partial_\phi B_\theta]$, $(\nabla\times\mathbf{B})_\theta = \frac{1}{r}[\frac{1}{\sin\theta}\partial_\phi B_r - \partial_r(rB_\phi)]$, $(\nabla \times \mathbf{B})_\phi = \frac{1}{r}[\partial_r(rB_\theta) - \partial_\theta B_r]$.

current_density()

lines 60–67

```
60  def current_density(grid, store=True):
61      """|J| [A m^-2] on the grid; stored as grid scalar 'j' when store=True."""
62      c = curl_b(grid)
63      mag = np.sqrt(sum(ci * ci for ci in c))                # gauss per coordinate unit
64      j = mag * 1e-4 / (grid.length_unit * MU0)               # T/m / mu0 -> A/m^2
65      if store:
66          grid.add_scalar("j", j)
67      return j
```

L.64 $|\mathbf{J}| = |\nabla\times\mathbf{B}|/\mu_0$: gauss per coordinate unit → T/m (factor 10^{-4}/length unit), divided by μ_0, in A m^{-2}.

L.66 Stored as the grid scalar `'j'`, so rules can use `f['j']`.

Chapter 7

common/mechanism.py — the mechanism registry

File: `radiosynth/common/mechanism.py` — 70 lines.

The single place where `common/` meets the mechanism folders. A new mechanism is a new folder that calls `register()` — nothing in `common/` changes.

Module docstring

lines 4–23

```
4  """Registry of radio emission mechanisms.
5
6  Each mechanism lives in its own sub-package of radiosynth
7  (``simplistic/``, ``bremsstrahlung/``, ``gyroresonance/``) and registers
8  itself here when it is imported.  The shared machinery in ``common/`` (grids,
9  readers, ray tracer, image driver, figures) only talks to mechanisms through
10 this registry, so a new mechanism is added by creating a new folder and
11 calling :func:`register` - nothing in ``common/`` has to change.
12
13 A mechanism entry has
14
15 name          : the string used in ``emission=...`` and on the command line
16 coefficients  : callable ``(ne, te, freq, length_unit=..., **options) ->
17                 (kappa [1/m], S)`` giving the absorption coefficient and the
18                 source function at sample points, for the refractive ray
19                 tracer (None if the mechanism needs more than n_e and T_e,
20                 e.g. the magnetic field for gyroresonance)
21 make_image    : callable ``(grid, frequencies, **kw) -> RadioImage``
22 description   : one line shown by ``python -m radiosynth --list-mechanisms``
23 """
```

Describes the fields of an entry: `name`, `coefficients` (for the refractive tracer, or None), `make_image`, `description`.

Registry storage

lines 24–29

```
24 import importlib
25
26 _REGISTRY = {}
27
28 # sub-packages that provide mechanisms (imported on demand)
29 BUILTIN = ("simplistic", "bremsstrahlung", "gyroresonance", "gyrosynchrotron", "combined")
```

L.26 Name → `Mechanism`.

L.29 The folders shipped with radiosynth; they are imported on demand, which avoids circular imports between `common/` and the mechanisms.

class Mechanism

lines 32–44

```
32 class Mechanism:
33     """Description of one emission mechanism (see module docstring)."""
34
35     def __init__(self, name, coefficients=None, make_image=None, description="",
36                  options=()):
37         self.name = name
38         self.coefficients = coefficients
39         self.make_image = make_image
40         self.description = description
41         self.options = tuple(options)
42
43     def __repr__(self):
44         return f"Mechanism({self.name!r}: {self.description})"
```

L.35 A plain record; `options` names the mechanism-specific keywords (for documentation and the CLI).

register()

lines 47–50

```
47 def register(mech):
48     """Add (or replace) a mechanism in the registry."""
49     _REGISTRY[mech.name.lower()] = mech
50     return mech
```

Adds or replaces an entry (lower-case name) and returns it, so that a package can write `MECHANISM = register(Mechanism(...))`.

get_mechanism()

lines 53–61

```
53 def get_mechanism(name):
54     """Return the Mechanism called ``name``, importing its sub-package if needed."""
55     key = name.lower()
56     if key not in _REGISTRY and key in BUILTIN:
57         importlib.import_module(f"radiosynth.{key}")
58     if key not in _REGISTRY:
59         raise ValueError(f"Unknown radio emission mechanism '{name}' "
60                          f"(available: {', '.join(available_mechanisms())})")
61     return _REGISTRY[key]
```

L.56–57 Import `radiosynth.<name>` the first time a built-in mechanism is asked for.

L.58–59 Unknown names raise an error listing the available ones.

available_mechanisms()

lines 64–70

```
64 def available_mechanisms():
65     for b in BUILTIN:
66         try:
67             importlib.import_module(f"radiosynth.{b}")
```

```
68          except ImportError:                     # pragma: no cover
69              pass
70      return sorted(_REGISTRY)
```

Imports all built-in folders and returns the sorted names (`python -m radiosynth --list-mechanisms`).

Chapter 8

common/emission.py — the emission-model interface

File: `radiosynth/common/emission.py` — 48 lines.

`EmissionModel` is the callable object used by the refractive ray tracer: it returns the absorption coefficient κ [m^{-1}] and the source function S for arrays of n_e and T_e at one frequency (the Python counterpart of `user_material_properties` in `ModUserAwsom.f90`). It holds no physics itself: it looks the mechanism up in the registry and forwards the call to `simplistic/emission.py` or `bremsstrahlung/emission.py`. You can still pass any function with the same signature as `emission=`.

Module docstring and imports

lines 4–21

```
4  """Emission-model interface used by the refractive ray tracer.
5
6  ``EmissionModel(kind, **options)`` is a callable returning ``(kappa, S)``
7  at sample points:
8
9      kappa : absorption coefficient [1/m]  (1/length_unit for 'simplistic')
10     S     : source function [W m^-2 Hz^-1 sr^-1]
11
12 so that the BATS-R-US-style pixel intensity is I = sum(kappa * S * ds).
13
14 The physics lives in the mechanism folders (radiosynth/simplistic,
15 radiosynth/bremsstrahlung, ...); this class only looks the mechanism up in
16 the registry (common/mechanism.py) and forwards the call.
17 """
18 import numpy as np
19
20 from .constants import R_SUN
21 from .mechanism import get_mechanism
```

L.21 The registry (common/mechanism.py) is the only link to the mechanism folders.

Constructor

lines 24–43

```
24 class EmissionModel:
25     """Callable returning (kappa [1/m], S [SI intensity]) for given n_e, T_e, freq.
26
27     kind  : 'bremsstrahlung' or 'simplistic' (any registered mechanism that
28             provides coefficients from n_e and T_e alone)
29     gaunt : Gaunt factor for bremsstrahlung (number or 'dulk')
30     """
31
32     def __init__(self, kind="bremsstrahlung", gaunt=10.0):
33         kind = kind.lower()
34         mech = get_mechanism(kind)
35         if mech.coefficients is None:
```

```
36            raise ValueError(f"mechanism '{kind}' cannot be used with the refractive ray "
37                             f"tracer; use radiosynth.{kind}.make_image instead")
38        self.kind = kind
39        self.gaunt = gaunt
40        self._coef = mech.coefficients
41
42    def __repr__(self):
43        return f"EmissionModel(kind={self.kind!r}, gaunt={self.gaunt!r})"
```

L.33–34 Case-insensitive lookup; an unknown name raises an error listing the available mechanisms (as `CON_stop` does in Fortran).

L.35–36 Gyroresonance needs the magnetic field and has its own driver, so it cannot be used here.

L.40 Keep the coefficient function.

L.42 Readable description, printed by `make_radio_image`.

Evaluation: `__call__(ne, te, freq, length_unit)` lines 45–48

```
45    def __call__(self, ne, te, freq, length_unit=R_SUN):
46        if self.kind == "bremsstrahlung":
47            return self._coef(ne, te, freq, length_unit=length_unit, gaunt=self.gaunt)
48        return self._coef(np.asarray(ne, dtype=float), te, freq, length_unit=length_unit)
```

L.46 Bremsstrahlung also receives the Gaunt factor.

L.48 Other mechanisms get n_e, T_e, the frequency and the length unit.

Chapter 9

common/raytrace.py — the ray tracer

File: `radiosynth/common/raytrace.py` — 399 lines.

This is the core of the package. It is written following the refractive ray tracing of the BATS-R-US radio-wave imaging module (`ray_path` in `ModRadioWaveImage.f90`; Benkevitch et al. 2010; Moschou et al. 2018), but it advances *all* rays together: every quantity is an array over the currently active rays, and the decisions taken per ray become boolean masks.

The file is organised in small subroutines, one per task, and the main function `trace_rays` calls them in order in each iteration. Each subroutine acts on the rays of the current iteration only. Index conventions used throughout: `act` holds the global indices of the rays still active; arrays such as `p`, `ne`, `eps` are *local* to `act`; `adv` holds local indices of the rays that advance in this iteration, and `ia = act[adv]` their global indices. Subroutines that change the state of rays (`ds`, `slope`, `steep`, `intensity`, ...) receive the global arrays and the global indices.

Module docstring and imports

lines 4–37

```
4  """Refractive radio ray tracing with emissivity integration.
5
6  Python port of ``ray_path`` in BATS-R-US ModRadioWaveImage.f90
7  (Benkevitch et al. 2010; Moschou et al. 2018), vectorised over all rays.
8
9  Ray equations (Moschou+2018 Eq. 7-13):   dr/ds = v,  dv/ds = v x (grad n / n x v)
10 with n^2 = eps = 1 - n_e/n_cr.  Each step of trace_rays calls, in order:
11
12 1. ``_predictor``        advance half a step along the current direction and
13                          sample n_e, grad n_e, T_e and the cell size there;
14 2. ``_terminations``     find rays that left the box / integration sphere,
15                          hit the body or reached over-dense plasma;
16    ``_integrate_to_body`` add the last straight piece of rays whose mid point
17                          is already inside the body;
18 3. ``_dielectric``       eps, grad eps and eps half a step back; rays with
19                          eps <= 0 there are stopped;
20    ``_snell_fallback``   (default mode) step back / reflect rays that
21                          trespassed the critical surface instead of stopping them;
22 4. ``_check_step``       step control: if the curvature is too large shrink ds
23                          and redo; if the next step may cross the critical
24                          surface mark the ray "steep", store the distance to the
25                          critical surface, shrink ds and redo;
26 5. ``_advance``          straight line (no refraction), ``_parabolic_step``
27                          near the critical surface, or ``_boris_step``;
28    ``_clip_at_body``     shorten the last segment of rays entering the body;
29    ``_add_emission``     add the emission of the segment (thin or full transfer);
30 6. ``_new_step_size``    smoothly regrow ds towards the cell size.
31
32 Lengths are in the grid's coordinate unit (R_sun by default, or R_star; the
33 conversion to metres is grid.length_unit), physical quantities SI.
34 """
35 import numpy as np
36
```

```
37 from .constants import R_SUN, critical_density
```

Lists the ray equations and the subroutines called in one step, in order.

Ray status codes

lines 39–48

```
39 # Termination codes
40 ACTIVE, ESCAPED, HIT_BODY, OVERDENSE, NEG_EPS, OPAQUE, MAX_STEPS, MISSED = range(8)
41 STATUS_NAMES = {ACTIVE: "active", ESCAPED: "escaped", HIT_BODY: "hit body",
42                 OVERDENSE: "over-dense", NEG_EPS: "eps<=0", OPAQUE: "tau>tau_max",
43                 MAX_STEPS: "max steps", MISSED: "missed grid"}
44
45
46 # ============================================================================
47 # building blocks (all vectorised over the rays of the current iteration)
48 # ============================================================================
```

L.40 Integer codes stored per ray: 0 active, 1 escaped, 2 hit body, 3 over-dense, 4 $\varepsilon \le 0$, 5 optically thick, 6 step limit, 7 missed the grid.

L.41 Names used in printed summaries.

_emission_function(): the emission model as a function

lines 49–56

```
49 def _emission_function(emission, L):
50     """emit(ne, te, freq) -> (kappa [1/m], B); passes the length unit to the
51     built-in EmissionModel (needed by the simplistic mechanism)."""
52     if hasattr(emission, "kind"):                          # built-in EmissionModel
53         def emit(ne_, te_, f_):
54             return emission(ne_, te_, f_, length_unit=L)
55         return emit
56     return emission                                        # user function (ne, te, freq)
```

L.52–55 Built-in emission models receive the length unit (needed by the simplistic mechanism, which is defined per normalised length).

L.56 User functions are called as `f(ne, te, freq)` and must return (κ, B).

_add_emission(): radiative transfer over a segment

lines 59–67

```
59 def _add_emission(intensity, tau, idx, kappa, B, dl, transfer):
60     """Add the emission of segments of length dl [m] to rays idx.
61     thin: I += kappa B dl;  full: I += B (1 - exp(-dtau)) exp(-tau), tau += dtau."""
62     if transfer == "thin":
63         intensity[idx] += kappa * B * dl
64     else:
65         dtau = kappa * dl
66         intensity[idx] += B * np.exp(-tau[idx]) * (-np.expm1(-dtau))
67         tau[idx] += dtau
```

L.63 Optically thin: $I \mathrel{+}= \kappa B \Delta\ell$, as in BATS-R-US.

L.65–67 Full transfer: add $Be^{-\tau}(1-e^{-\Delta\tau})$, the exact solution for constant κ, B on the segment, then update τ. `-expm1(-x)` is $1-e^{-x}$, accurate for small x.

_predictor(): half step and plasma at the mid point

lines 70–75

```
70 def _predictor(grid, p, s, d):
71     """Mid point of the new segment and the plasma there."""
72     half = 0.5 * d
73     pm = p + s * half[:, None]
74     ne, gn, te, cell, inside = grid.sample(pm)
75     return half, pm, ne, gn, te, cell, inside
```

L.72 $\Delta s/2$.

L.73 Predictor: $\mathbf{r}_{1/2} = \mathbf{r}_0 + \mathbf{v}_0 \Delta s/2$.

L.74 Density, its gradient, temperature, local cell size and an inside-the-domain flag at $\mathbf{r}_{1/2}$.

_terminations(): rays that end at the mid point

lines 78–85

```
78 def _terminations(pm, ne, inside, ncr, rbody2, rint2):
79     """Termination code of every ray from its mid point (0 = continue)."""
80     r2 = np.einsum("ij,ij->i", pm, pm)
81     term = np.zeros(pm.shape[0], np.int8)
82     term[r2 < rbody2] = HIT_BODY
83     term[(term == 0) & (~inside | (r2 > rint2))] = ESCAPED
84     term[(term == 0) & (ne >= ncr)] = OVERDENSE
85     return term
```

L.80 $|\mathbf{r}_{1/2}|^2$ (`einsum` gives row-wise dot products).

L.81 Termination code (0 = continue).

L.82 Mid point inside the body: the ray reached the photosphere. Tested first, because for a spherical grid the body is also outside the domain.

L.83 Left the domain or the integration sphere: escaped.

L.84 Over-dense plasma at the mid point; in the default mode such rays are recovered by `_snell_fallback`.

_integrate_to_body(): the last piece before the photosphere

lines 88–98

```
88 def _integrate_to_body(grid, p, s, d, ncr, rbody2, emit, freq, L, idx, intensity, tau,
89                       transfer):
90     """Rays whose predictor point is already inside the body: integrate the
91     remaining straight piece from the segment start to the surface."""
92     bb = np.einsum("ij,ij->i", p, s)
93     cc = np.einsum("ij,ij->i", p, p) - rbody2
94     tt = np.clip(-bb - np.sqrt(np.maximum(bb * bb - cc, 0.0)), 0.0, d)
95     ne_h, _, te_h, _, _ = grid.sample(p + s * (0.5 * tt)[:, None])
96     ne_h = np.minimum(ne_h, ncr)
97     k_h, B_h = emit(ne_h, te_h, freq)
98     _add_emission(intensity, tau, idx, k_h, B_h, tt * L, transfer)
```

When the mid point is already inside the body, the piece of the step between $\mathbf{r}_0$ and the surface would otherwise be lost. It is integrated as a straight segment.

L.92–94 Distance to the sphere r_{body} along $\mathbf{v}$: $t = -b - \sqrt{b^2 - c}$ with $b = \mathbf{r}_0 \cdot \mathbf{v}$ and $c = r_0^2 - r_{\mathrm{body}}^2$, limited to $[0, \Delta s]$.

L.95–97 Emission at the mid point of that piece; the density is capped at n_{cr}.

L.98 Add it, in metres, in the thin or full mode.

_dielectric(): permittivity and its gradient

lines 101–109

```
101 def _dielectric(ne, gn, s, half, ncr):
102     """Dielectric permittivity eps = n^2 at the mid point, its gradient
103     (per coordinate unit), |grad eps|^2, grad eps . v and eps half a step back."""
104     eps = 1.0 - ne / ncr                        # DielPerm
105     ge = -gn / ncr                              # GradDielPerm_D  [1/R_sun]
106     ge2 = np.einsum("ij,ij->i", ge, ge)
107     gs = np.einsum("ij,ij->i", ge, s)           # GradEpsDotSlope
108     ehb = eps - gs * half                       # DielPermHalfBack
109     return eps, ge, ge2, gs, ehb
```

L.104 $\varepsilon_{1/2} = 1 - n_e/n_{\mathrm{cr}}$ at the mid point.

L.105 $\nabla\varepsilon = -\nabla n_e/n_{\mathrm{cr}}$ per coordinate unit.

L.106 $|\nabla\varepsilon|^2$.

L.107 $\nabla\varepsilon \cdot \mathbf{v}_0$: negative when the ray moves towards denser plasma.

L.108 Back-extrapolated permittivity at the step start, $\varepsilon_0 = \varepsilon_{1/2} - \nabla\varepsilon \cdot \mathbf{v}_0 \, \Delta s/2$ (Benkevitch et al. 2010, Eq. 34).

_snell_fallback(): rays that crossed the critical surface (default mode)

lines 112–130

```
112 def _snell_fallback(term, act, d, s, ge, ge2, gs, ds, slope, steep, min_ds, step_min):
113     """Default mode: rays that trespassed the critical surface (over-dense mid
114     point or eps <= 0 half a step back) are stepped back with ds/4 and, when
115     ds < 4 min_ds, specularly reflected about grad eps (Snell's law).
116     Modifies term, ds, slope, steep; returns the mask of such rays."""
117     bad = (term == OVERDENSE) | (term == NEG_EPS)
118     if not bad.any():
119         return np.zeros(act.size, bool)
120     back = bad & (d >= 4.0 * min_ds)
121     ds[act[back]] = 0.25 * d[back]
122     snell = bad & ~back & (ge2 > 0) & (gs < 0)
123     if snell.any():
124         gh = ge[snell] / np.sqrt(ge2[snell])[:, None]
125         sv = s[snell]
126         slope[act[snell]] = sv - 2.0 * np.einsum("ij,ij->i", sv, gh)[:, None] * gh
127         ds[act[snell]] = np.maximum(d[snell], step_min)
128         steep[act[snell]] = False
129     term[back | snell] = 0
130     return back | snell
```

Implements the fallback described by Benkevitch et al. (2010, Sec. 5.2) and Moschou et al. (2018, Sec. 2.2): return to the previous position, then reflect.

L.117 Rays that are over-dense at the mid point or have $\varepsilon_0 \leq 0$.

L.120–121 First choice: retry from the same start point with a quarter of the step. The position is not changed, so the next iteration probes a point closer to $\mathbf{r}_0$.

L.122–128 If the step is already tiny, reflect the direction specularly about the gradient, $\mathbf{v} \leftarrow \mathbf{v} - 2(\mathbf{v} \cdot \hat{g})\hat{g}$ with $\hat{g} = \nabla\varepsilon/|\nabla\varepsilon|$, when the ray moves into the gradient. The step is reset to at least `step_min` and the steep flag cleared.

L.129–130 These rays stay active but do not advance in this iteration; the mask is returned to the caller.

_check_step(): precision control

lines 133–159

```
133 def _check_step(ok, act, d, half, ehb, ge2, gs, steep, dist2cr, ds, tolerance, tol2,
134                 step_min, min_ds, strict_batsrus):
135     """BATS-R-US check_step for "gentle" (non-steep) rays.
136 
137     (a) curvature too large for the tolerance -> shrink ds;
138     (b) the next step may cross the critical surface -> mark steep, store the
139         distance to the critical surface, shrink ds.
140     The rays stay at the segment start (PositionHalfBack) and are retried.
141     Returns the mask of retried rays."""
142     chk = ok & ~steep[act]
143     with np.errstate(divide="ignore", invalid="ignore"):
144         curv = (0.5 * half / ehb)**2 * (ge2 - gs**2)
145         r_a = chk & (curv >= tol2)
146         newd = d.copy()
147         newd[r_a] = 0.99 * d[r_a] / (2.0 * np.sqrt(curv[r_a] / tol2))
148         if not strict_batsrus:
149             newd[r_a] = np.maximum(newd[r_a], min_ds)
150             r_a &= d > min_ds * 1.0000001
151         r_b = chk & ~r_a & (3.0 * gs * half <= -ehb)
152         d2c = ehb[r_b] / np.sqrt(ge2[r_b])
153     idx = act[r_b]
154     steep[idx] = True
155     dist2cr[idx] = d2c
156     newd[r_b] = np.maximum(0.5 * tolerance * d2c, step_min)
157     retry = r_a | r_b
158     ds[act[retry]] = newd[retry]
159     return retry
```

Applied to rays that are not marked steep, and only with refraction on.

L.142 Rays to check.

L.144 $(\Delta s/4\varepsilon_0)^2(|\nabla\varepsilon|^2 - (\nabla\varepsilon \cdot \mathbf{v})^2)$, the squared turning angle over half a step; the bracket is the squared transverse gradient.

L.145 (a) Too much bending for the tolerance (about $1/(2\,\mathrm{Tol}) = 50$ steps per radian).

L.147 Shrink the step by the factor needed to meet the tolerance, with a 1% margin.

L.148–150 Default mode: never go below `min_ds`, and stop retrying once there.

L.151 (b) Correctness criterion (Benkevitch et al. 2010, Eq. 33): a linear prediction puts $\varepsilon = 0$ within about 1.5 steps.

L.152 Distance to the critical surface $d_{\mathrm{cr}} = \varepsilon_0/|\nabla\varepsilon|$.

L.154–155 Mark the ray steep and remember d_{cr}.

L.156 New step $\max(\frac{1}{2}\mathrm{Tol}\, d_{\mathrm{cr}}, \mathrm{step_min})$.

L.157–159 These rays redo the step with the smaller Δs; their position stays at $\mathbf{r}_0$.

_parabolic_step(): reflection at the critical surface

lines 162–174

```
162 def _parabolic_step(p_start, s, g, gs, ge2, ehb):
163     """Switch to the opposite branch of the parabola near the critical surface
164     (reflection).  Returns new direction, new position and path length."""
165     lcos = -gs / ge2                                    # L cos(alpha)
166     proj = -lcos[:, None] * g                           # v_proj
167     stepy = s - proj
168     snew = s - 2.0 * proj
169     stepy = 4.0 * stepy * (lcos * ehb)[:, None]
170     pnew = p_start + stepy                              # PositionHalfBack + StepY
171     stepx = (ehb * lcos**2)[:, None] * g
172     seg = np.sqrt(np.einsum("ij,ij->i", 2 * stepx, 2 * stepx)
173                   + np.einsum("ij,ij->i", stepy, stepy))
174     return snew, pnew, seg
```

L.165 $L\cos\alpha = -\nabla\varepsilon\cdot\mathbf{v}/|\nabla\varepsilon|^2$, with $L = 1/|\nabla\varepsilon|$ and α the incidence angle.

L.166 Component of $\mathbf{v}$ along $\nabla\varepsilon$: $\mathbf{v}_\parallel = -L\cos\alpha\,\nabla\varepsilon$.

L.167 Transverse component $\mathbf{v} - \mathbf{v}_\parallel$, of length $\sin\alpha$.

L.168 Reflected direction: the parallel component is reversed.

L.169 Scale to the chord of the parabola, $|\Delta\mathbf{r}_\perp| = 4\sin\alpha\cos\alpha\,\varepsilon_0 L$.

L.170 Jump from $\mathbf{r}_0$ to the symmetric point on the other branch.

L.171 Depth of the parabola vertex along $\nabla\varepsilon$, $\varepsilon_0(L\cos\alpha)^2|\nabla\varepsilon|$.

L.172–173 Path length assigned to the arc, $\sqrt{|2\Delta\mathbf{r}_\parallel|^2 + |\Delta\mathbf{r}_\perp|^2}$.

_boris_step(): Boris (CYLRAD) rotation

lines 177–189

```
177 def _boris_step(s, pm, half, eps, g):
178     """Boris (CYLRAD) rotation of the direction by grad(n)/n over the step, then
179     the second half step from the mid point."""
180     coef = 0.5 * half / eps
181     rel = coef[:, None] * g                             # grad(n)/n * ds/2
182     om = np.cross(rel, s)
183     s1 = s + np.cross(s, om)
184     om = np.cross(rel, s1)
185     s1 = s + np.cross(s, om)
186     c2 = 2.0 / (1.0 + np.einsum("ij,ij->i", om, om))
187     snew = s + c2[:, None] * np.cross(s1, om)
188     pnew = pm + snew * half[:, None]
189     return snew, pnew
```

L.180 $\Delta s/(4\varepsilon_{1/2})$.

L.181 $\mathbf{a} = \nabla\varepsilon\,\Delta s/(4\varepsilon) = (\nabla\mu/\mu)\,\Delta s/2$.

L.182–183 First estimate: $_0 = \mathbf{a}\times\mathbf{v}_0$, $\mathbf{v}' = \mathbf{v}_0 + \mathbf{v}_0\times\ _0$.

L.184–185 Rotation vector at the half step $_{1/2} = \mathbf{a}\times\mathbf{v}'$, and $\mathbf{v}'' = \mathbf{v}_0 + \mathbf{v}_0\times\ _{1/2}$.

L.186–187 Exact-length rotation $\mathbf{v}_1 = \mathbf{v}_0 + \frac{2}{1+\Omega^2}\mathbf{v}''\times\ _{1/2}$, so $|\mathbf{v}_1| = 1$.

L.188 Corrector: $\mathbf{r}_1 = \mathbf{r}_{1/2} + \mathbf{v}_1\Delta s/2$.

_advance(): move every ray

lines 192–209

```
192 def _advance(p_start, s, pm, half, d, eps, ge, gs, ge2, ehb, no_refraction,
193              strict_batsrus, step_min):
194     """advance_ray: new direction, position and segment length of every ray."""
195     if no_refraction:
196         return s, pm + s * half[:, None], d
197     refl = 3.0 * gs * half <= -ehb
198     if strict_batsrus:
199         refl |= (d < step_min) & (gs < 0.0)
200     snew = np.empty_like(s)
201     pnew = np.empty_like(pm)
202     seg = d.copy()
203     if refl.any():
204         snew[refl], pnew[refl], seg[refl] = _parabolic_step(
205             p_start[refl], s[refl], ge[refl], gs[refl], ge2[refl], ehb[refl])
206     bo = ~refl
207     if bo.any():
208         snew[bo], pnew[bo] = _boris_step(s[bo], pm[bo], half[bo], eps[bo], ge[bo])
209     return snew, pnew, seg
```

L.196 Without refraction: direction unchanged, $\mathbf{r}_1 = \mathbf{r}_{1/2} + \mathbf{v}\Delta s/2$, segment length Δs.

L.197 The parabolic switch is used where the linear prediction says the ray would reach $\varepsilon = 0$ (Eq. 33); the other rays take a Boris step.

L.198–199 In strict mode the switch is also used when $\Delta s <$ `step_min` and $\nabla\varepsilon \cdot \mathbf{v} < 0$ (Section 6.5).

L.203–205 Parabolic switch for the selected rays.

L.206–208 Boris step for the others.

_clip_at_body(): a step that ends inside the photosphere

lines 212–227

```
212 def _clip_at_body(p_start, pnew, rbody2):
213     """Fraction of every segment outside the body (1 if it does not enter it),
214     and the mask of segments that enter it."""
215     r2e = np.einsum("ij,ij->i", pnew, pnew)
216     into_body = r2e < rbody2
217     frac = np.ones(pnew.shape[0])
218     if into_body.any():
219         ps, dd_ = p_start[into_body], pnew[into_body] - p_start[into_body]
220         aa = np.einsum("ij,ij->i", dd_, dd_)
221         bb = 2.0 * np.einsum("ij,ij->i", ps, dd_)
222         cc = np.einsum("ij,ij->i", ps, ps) - rbody2
223         disc = np.maximum(bb * bb - 4.0 * aa * cc, 0.0)
224         with np.errstate(divide="ignore", invalid="ignore"):
225             tt = (-bb - np.sqrt(disc)) / (2.0 * aa)
226         frac[into_body] = np.clip(np.nan_to_num(tt, nan=1.0), 0.0, 1.0)
227     return frac, into_body
```

L.216 Steps whose end point lies inside r_{body}.

L.219–226 Solve $|\mathbf{r}_0 + t(\mathbf{r}_1 - \mathbf{r}_0)| = r_{\mathrm{body}}$ for the first root $t \in [0, 1]$ and keep only that fraction of the segment; this matters at the disk centre at high frequency.

_new_step_size(): step regrowth

lines 230–242

```
230 def _new_step_size(ia, d, dsnew, ehb, ge2, steep, dist2cr, ds, no_refraction):
231     """get_new_step_size: regrow ds smoothly towards the local cell size, except
232     for steep rays still close to the critical surface."""
233     cond = ~steep[ia] | (ehb > dist2cr[ia] * np.sqrt(ge2))
234     if no_refraction:
235         cond[:] = True
236     ic = ia[cond]
237     steep[ic] = False
238     dd, dn = d[cond], dsnew[cond]
239     ds[ic] = np.maximum(2.0 - dd / dn, 1.0) * np.minimum(dn, dd)
240 
241 
242 # ============================================================================
```

L.233 Regrow the step for gentle rays, and for steep rays that are now farther from the critical surface than when they were marked ($\varepsilon_0 > d_{\rm cr}|\nabla\varepsilon|$).

L.235 Without refraction, always regrow.

L.237 Those rays are gentle again.

L.239 Non-linear update $\Delta s' = \max(2 - \Delta s/\Delta s_{\rm new}, 1)\min(\Delta s_{\rm new}, \Delta s)$: small steps roughly double each time and saturate smoothly at $\Delta s_{\rm new}$.

trace_rays(): signature and docstring

lines 243–293

```
243 def trace_rays(grid, pos0, dir0, freq, emission, *, transfer="thin",
244                no_refraction=False, r_integration=None, tolerance=1.0e-2,
245                step_min=1.0e-2, step_factor=1.0, ds_init=None, tau_max=30.0,
246                kappa_over_mu=False, max_steps=200000, status0=None,
247                record_ids=None, strict_batsrus=False, min_ds=1.0e-6):
248     """Trace a bunch of rays and integrate the emission along them.
249 
250     Parameters
251     ----------
252     grid      : MHDGrid
253     pos0      : (N,3) starting points [R_sun] (inside the grid box)
254     dir0      : (N,3) initial directions (pointing away from the observer)
255     freq      : radio frequency [Hz]
256     emission  : EmissionModel (or any callable(ne, te, freq) -> (kappa, B))
257     transfer  : 'thin'  -> I = sum kappa B ds  (BATS-R-US / Moschou+2018)
258                 'full'  -> formal solution of dI/ds = kappa (B - I):
259                            I = sum B (1 - exp(-dtau)) exp(-tau)
260     no_refraction : straight lines of sight (UseNoRefraction in BATS-R-US)
261     r_integration : radius of the integration sphere [R_sun]; None -> no limit
262     tolerance, step_min : BATS-R-US Tolerance (1e-2) and StepMin (1e-2 R_sun)
263     step_factor : DeltaSNew = step_factor * local cell size
264     ds_init   : initial step [R_sun]; None -> local cell size * step_factor
265     tau_max   : (full transfer only) stop a ray once tau exceeds this
266     kappa_over_mu : divide kappa by the refractive index mu=sqrt(eps)
267                     (not done in BATS-R-US; physically appropriate near the
268                     critical layer, Dulk 1985)
269     max_steps : safety cap on iterations per ray
270     status0   : optional initial status array (rays with status!=0 are skipped)
271     record_ids: optional indices of rays whose paths should be stored
```

```
    strict_batsrus : reproduce ModRadioWaveImage.f90 exactly.  Differences
                     of the default (False) mode, following Benkevitch+2010 Sec. 5:
                     (a) the parabolic switch is used only when the linear
                         prediction says the next step would cross the critical
                         surface; the extra Fortran trigger
                         "ds < StepMin .and. grad(eps).v < 0" is dropped, because
                         it is reached after curvature-driven step reduction and
                         makes rays far from the critical surface (eps ~ 0.6)
                         jump along a long, wrong parabola;
                     (b) a ray whose mid point lands in over-dense plasma (or with
                         eps<=0 at the start point) is not discarded: it is stepped
                         back with ds/4 and, when ds < 4*min_ds, specularly
                         reflected about grad(eps) ("linear reflection following
                         Snell's law", Benkevitch+2010; Moschou+2018 Sec. 2.2).
    min_ds    : smallest step used by the curvature control / fallback [R_sun]

    Returns
    -------
    dict with 'intensity' [W m^-2 Hz^-1 sr^-1], 'tau', 'status', 'nstep',
    and 'paths' / 'slopes' (dict id -> (M,3) arrays of positions and unit
    directions after every accepted step) when record_ids is given.
    """
```

L.243 Grid, starting points, directions, frequency and emission model are required; the rest are keyword options. The defaults `tolerance=1e-2` and `step_min=1e-2` are those of BATS-R-US.

L.247 Switch between the original BATS-R-US step logic and the corrected default; explained in the docstring and in Section 6.5.

trace_rays(): set-up

lines 294–329

```
    if transfer not in ("thin", "full"):
        raise ValueError("transfer must be 'thin' or 'full'")
    # ---- set-up
    pos = np.array(pos0, dtype=float)
    slope = np.array(dir0, dtype=float)
    slope /= np.linalg.norm(slope, axis=1)[:, None]
    N = pos.shape[0]
    ncr = float(critical_density(freq))
    tol2 = tolerance**2
    rint2 = np.inf if r_integration is None else r_integration**2 + 0.01
    rbody2 = grid.r_body**2
    L = float(getattr(grid, "length_unit", R_SUN))          # metres per coordinate unit
    emit = _emission_function(emission, L)

    status = np.zeros(N, np.int8) if status0 is None else np.array(status0, np.int8)
    intensity = np.zeros(N)
    tau = np.zeros(N)
    nstep = np.zeros(N, np.int64)
    dist2cr = np.zeros(N)
    steep = np.zeros(N, bool)
    ds = np.zeros(N)

    act = np.flatnonzero(status == ACTIVE)
    if ds_init is None:
```

```
318         if act.size:
319             ds[act] = step_factor * grid.sample(pos[act])[3]
320     else:
321         ds[:] = ds_init
322 
323     paths = None
324     if record_ids is not None:
325         record_ids = np.atleast_1d(record_ids)
326         rec_mask = np.zeros(N, bool)
327         rec_mask[record_ids] = True
328         paths = {int(i): [pos[i].copy()] for i in record_ids}
329         slopes = {int(i): [slope[i].copy()] for i in record_ids}
```

L.297–299 Copies of positions $\mathbf{r}$ and unit directions $\mathbf{v}$; the directions are re-normalised.

L.301 Critical density for this frequency.

L.302 Squared tolerance.

L.303 Squared integration-sphere radius plus 0.01; infinite if none.

L.304 Squared inner-boundary radius.

L.305 Metres per coordinate unit: $R_\odot$ for the Sun, R_* for a star.

L.306 The emission model as a function of (n_e, T_e, ν).

L.308 Initial status (rays that miss the grid arrive here already flagged).

L.309–314 Per-ray state: intensity, optical depth, step count, distance to the critical surface, steep flag, step size.

L.316 Indices of the rays to trace.

L.317–321 Initial step: the local cell size at the entry point (times `step_factor`), or a user-given value.

L.323–329 Optionally record the position and direction of selected rays after every accepted step (used by `trace_paths` and the tests).

trace_rays(): the main loop

lines 331–393

```
331     # ---- one iteration per loop: every active ray makes (or retries) one step
332     while act.size:
333         p, s, d = pos[act], slope[act], ds[act]
334         half, pm, ne, gn, te, cell, inside = _predictor(grid, p, s, d)
335         dsnew = step_factor * cell
336 
337         term = _terminations(pm, ne, inside, ncr, rbody2, rint2)
338         hb = np.flatnonzero(term == HIT_BODY)
339         if hb.size:
340             _integrate_to_body(grid, p[hb], s[hb], d[hb], ncr, rbody2, emit, freq, L,
341                                act[hb], intensity, tau, transfer)
342 
343         eps, ge, ge2, gs, ehb = _dielectric(ne, gn, s, half, ncr)
344         term[(term == 0) & (ehb <= 0.0)] = NEG_EPS
345 
346         if not strict_batsrus and not no_refraction:
347             ok_bad = _snell_fallback(term, act, d, s, ge, ge2, gs, ds, slope, steep,
348                                      min_ds, step_min)
349         else:
350             ok_bad = np.zeros(act.size, bool)
351         ok = (term == 0) & ~ok_bad
```

```
352
353         retry = np.zeros(act.size, bool)
354         if not no_refraction:
355             retry = _check_step(ok, act, d, half, ehb, ge2, gs, steep, dist2cr, ds,
356                                 tolerance, tol2, step_min, min_ds, strict_batsrus)
357
358         adv = np.flatnonzero(ok & ~retry)
359         if adv.size:
360             ia = act[adv]
361             kap, B = emit(ne[adv], te[adv], freq)
362             if kappa_over_mu:
363                 kap = kap / np.sqrt(np.maximum(eps[adv], 1e-6))
364             snew, pnew, seg = _advance(p[adv], s[adv], pm[adv], half[adv], d[adv], eps[adv],
365                                        ge[adv], gs[adv], ge2[adv], ehb[adv], no_refraction,
366                                        strict_batsrus, step_min)
367             frac, into_body = _clip_at_body(p[adv], pnew, rbody2)
368             _add_emission(intensity, tau, ia, kap, B, seg * frac * L, transfer)
369             pos[ia] = pnew
370             slope[ia] = snew
371
372             # rays which left the box / integration sphere, hit the body, or are opaque
373             r2n = np.einsum("ij,ij->i", pnew, pnew)
374             gone = (~grid.inside(pnew) & ~into_body) | (r2n > rint2)
375             status[ia[gone]] = ESCAPED
376             status[ia[into_body & ~gone]] = HIT_BODY
377             if transfer == "full":
378                 status[ia[(~gone) & (tau[ia] > tau_max)]] = OPAQUE
379
380             _new_step_size(ia, d[adv], dsnew[adv], ehb[adv], ge2[adv], steep, dist2cr, ds,
381                            no_refraction)
382
383             if paths is not None:
384                 for i in ia[rec_mask[ia]]:
385                     paths[int(i)].append(pos[i].copy())
386                     slopes[int(i)].append(slope[i].copy())
387
388         tmask = (term > 0) & ~ok_bad
389         status[act[tmask]] = term[tmask]
390         nstep[act] += 1
391         over = (nstep[act] >= max_steps) & (status[act] == ACTIVE)
392         status[act[over]] = MAX_STEPS
393         act = act[status[act] == ACTIVE]
```

One pass of the loop is one attempted step of every active ray. Each line calls one of the subroutines above.

L.332 Iterate until every ray has ended.

L.333–335 Local copies of start point, direction and step; predictor; desired new step from the local cell size.

L.337–341 Rays that end at the mid point; for those that reached the body, the remaining piece is integrated.

L.343–344 Permittivity and its gradient; $\varepsilon_0 \leq 0$ means the ray is already beyond the critical surface.

L.346–351 Default mode: recover the rays that crossed the critical surface; `ok` are the rays that continue normally.

L.353–356 Step control (only with refraction).

L.358–360 The rays that passed all checks advance now.

L.361–363 κ and B at the mid point, optionally κ/μ with $\mu = \sqrt{\varepsilon}$.

L.364–366 Move the rays (straight, parabolic switch or Boris).

L.367–368 Clip at the photosphere and add the emission of the segment, in metres.

L.369–370 Store the new state.

L.373–378 End-of-step tests: escaped, ended on the photosphere, or (full transfer) $\tau > \tau_{\max}$, after which nothing more can be seen.

L.380–381 Regrow the step.

L.383–386 Record the paths of the selected rays.

L.388–389 Store the termination codes decided at the mid point.

L.390 Count the iteration (including retries).

L.391–392 Safety stop for runaway rays.

L.393 Keep only the rays that are still active.

trace_rays(): return value

lines 395–399

```
395     out = {"intensity": intensity, "tau": tau, "status": status, "nstep": nstep}
396     if paths is not None:
397         out["paths"] = {k: np.array(v) for k, v in paths.items()}
398         out["slopes"] = {k: np.array(v) for k, v in slopes.items()}
399     return out
```

L.395 Intensity [W m^{-2} Hz^{-1} sr^{-1}], optical depth, status and step count per ray.

L.396–398 Recorded trajectories as (M,3) arrays.

Chapter 10

common/straight.py — straight-ray sampler

File: `radiosynth/common/straight.py` — 88 lines.

Cuts each line of sight into equal steps and samples n_e, T_e and $\mathbf{B}$ at the nodes, chunk by chunk, for mechanisms that integrate along straight rays.

Module docstring and imports

lines 4–20

```
4  """Straight-ray sampler for mechanisms that do not need refraction.
5
6  Gyroresonance emission comes from thin layers where B is large and the
7  frequency is far above the plasma frequency, so refraction is negligible and
8  the rays are straight lines (the standard approach of gyroresonance codes).
9  This module cuts every line of sight into equal steps and samples n_e, T_e and
10 B at the step nodes, chunk by chunk, so that a mechanism can do its radiative
11 transfer with vectorised numpy along the ray axis.
12
13 Each line starts where it enters both the integration sphere and the grid
14 (common/image.py start_points, as for the refractive tracer) and ends where it
15 leaves the domain or the integration sphere, or hits the body r = r_body.
16 Node 0 is the end nearest to the observer (front-to-back ordering).
17 """
18 import numpy as np
19
20 from .raytrace import ACTIVE, ESCAPED, HIT_BODY, MISSED
```

Explains why straight rays suffice for gyroresonance (the layers form where $\nu \gg \nu_p$) and the conventions: same start points as the refractive tracer, node 0 nearest the observer.

segment_ends(): where each line ends

lines 23–42

```
23 def segment_ends(grid, pos, dirs, status0, r_integration):
24     """Length of each straight line inside the domain (coordinate units) and
25     whether it ends on the body.  pos: start points, dirs: unit directions."""
26     # rays that miss the domain have non-finite start points; they get length 0
27     pos = np.where(np.isfinite(pos), pos, 0.0)
28     _, t_out = grid.domain_intersection(pos, dirs)
29     b = np.einsum("ij,ij->i", pos, dirs)
30     c = np.einsum("ij,ij->i", pos, pos)
31     # far exit of the integration sphere
32     disc = b * b - (c - r_integration**2)
33     t_sph = -b + np.sqrt(np.maximum(disc, 0.0))
34     t_end = np.minimum(t_out, t_sph)
35     # first hit of the body sphere
36     disc_b = b * b - (c - grid.r_body**2)
37     t_body = -b - np.sqrt(np.maximum(disc_b, 0.0))
38     hit = (disc_b > 0) & (t_body > 0) & (t_body < t_end)
39     t_end = np.where(hit, t_body, t_end)
40     t_end = np.where(status0 == ACTIVE, np.maximum(t_end, 0.0), 0.0)
```

```
41     status = np.where(status0 == ACTIVE, np.where(hit, HIT_BODY, ESCAPED), MISSED).astype(np.
       ↪ int8)
42     return t_end, status
```

L.28 Exit from the grid (box or outer sphere).

L.32–34 Far exit of the integration sphere, $|\mathbf{p} + t\mathbf{d}| = r_{\text{int}}$.

L.36–39 First intersection with the body sphere $r = r_{\text{body}}$, if it comes before the exit.

L.41 Status as in the tracer: escaped, hit body or missed grid.

default_step()

lines 45–50

```
45 def default_step(grid, length, ds=None, max_nodes=20000):
46     """Step length: ds if given, else half the smallest cell, but at least
47     length/max_nodes."""
48     if ds is None:
49         ds = 0.5 * float(grid.min_cell)
50     return max(float(ds), float(np.max(length, initial=0.0)) / max_nodes)
```

Half the smallest cell, but never more than 20000 nodes per ray. The harmonic layers are located between nodes, so the step only has to resolve the variation of the fields.

iter_samples(): chunks of sampled rays

lines 53–88

```
53 def iter_samples(grid, pos, dirs, t_end, ds, max_points=400000, with_field=True):
54     """Yield dict(idx, t, p, ne, te, inside, b, dl) for chunks of rays.
55 
56     t    : (nr, nk+1) node parameters along each line (coordinate units)
57     p    : (nr, nk+1, 3) node positions
58     ne, te, inside : (nr, nk+1) sampled density [m^-3], temperature [K], flag
59     b    : (nr, nk+1, 3) magnetic field [G] (if with_field)
60     dl   : (nr, nk) step lengths (coordinate units; 0 past the end of the line)
61     """
62     nray = dirs.shape[0]
63     pos = np.where(np.isfinite(pos), pos, 0.0)
64     nk_all = np.ceil(t_end / ds).astype(np.int64)
65     order = np.argsort(nk_all)              # group rays of similar length
66     start = 0
67     while start < nray:
68         nk = max(int(nk_all[order[start]]), 1)
69         stop = min(nray, start + max(1, max_points // (nk + 1)))
70         nk = max(int(nk_all[order[stop - 1]]), 1)       # longest ray of the chunk
71         stop = start + max(1, min(stop - start, max_points // (nk + 1)))
72         nr = stop - start
73         idx = order[start:stop]
74         k = np.arange(nk + 1)
75         t = np.minimum(k[None, :] * ds, t_end[idx][:, None])
76         p = pos[idx][:, None, :] + t[..., None] * dirs[idx][:, None, :]
77         flat = p.reshape(-1, 3)
78         ne, _, te, _, ins = grid.sample(flat)
79         out = dict(idx=idx, t=t, p=p,
80                    ne=np.where(ins, ne, 0.0).reshape(nr, nk + 1),
81                    te=te.reshape(nr, nk + 1), inside=ins.reshape(nr, nk + 1),
82                    dl=np.diff(t, axis=1))
```

```
83          if with_field:
84              b = grid.sample_field(flat)
85              b[~ins] = 0.0
86              out["b"] = b.reshape(nr, nk + 1, 3)
87          yield out
88          start = stop
```

L.64–65 Number of steps per ray; rays are sorted by length so that each chunk is nearly rectangular.

L.68–72 Chunk size: as many rays as fit in `max_points` nodes for the longest ray of the chunk (memory stays bounded).

L.75–76 Node parameters (the last step of a ray is shorter, later steps have zero length) and positions.

L.78–82 Sample n_e and T_e (zero density outside the domain) and the step lengths.

L.83–86 Sample $\mathbf{B}$ [G], zero outside the domain.

Chapter 11

common/image.py — image set-up and driver

File: `radiosynth/common/image.py` — 472 lines.

Builds the rays for an image (port of `write_plot_radiowave`), runs the tracer for each frequency (optionally in parallel), and wraps the result in a `RadioImage`.

Module docstring and imports

lines 4–28

```
4  """Synthetic radio image driver (port of write_plot_radiowave).
5
6  Image-plane geometry (identical to ModWritePlotRadioWave.f90):
7    * the image plane passes through the Sun centre, perpendicular to the
8      observer direction  Normal = Obs/|Obs|;
9    * in-plane unit vectors  Tau = (z x Normal)/|...|  (image x, "right"),
10                            Xi  = Normal x Tau        (image y, "up");
11   * pixel centres  X = XLower + (i-1/2) dX,  Y = YLower + (j-1/2) dY;
12   * each ray starts at the observer and points at its pixel centre; the
13     straight segment observer -> integration sphere is emission-free.
14     rIntegration = ceil(max(ImagePlaneDiagRadius + 1, 5)).
15 """
16 import math
17 import multiprocessing as mp
18 import re
19 import time
20
21 import numpy as np
22
23 from .constants import C_LIGHT, K_B, R_SUN, JANSKY, critical_density
24 from .emission import EmissionModel
25 from .raytrace import trace_rays, ACTIVE, MISSED, STATUS_NAMES
26
27
28 # ---------------------------------------------------------------------------
```

Summarises the image geometry, which is identical to BATS-R-US.

L.25 The tracer and the status codes.

parse_frequencies()

lines 29–44

```
29 def parse_frequencies(spec):
30     """'1500kHz, 11MHz, 42.7MHz, 1.08GHz' -> array of Hz (like parse_freq_string)."""
31     if np.isscalar(spec) and not isinstance(spec, str):
32         return np.array([float(spec)])
33     if not isinstance(spec, str):
34         # list of numbers [Hz] or of strings such as ["80MHz", "1.2GHz"]
35         return np.array([parse_frequencies(v)[0] if isinstance(v, str) else float(v)
36                          for v in spec])
37     scale = {"hz": 1.0, "khz": 1e3, "mhz": 1e6, "ghz": 1e9, "thz": 1e12}
```

```
38      out = []
39      for num, unit in re.findall(r"([-+0-9.eEdD]+)\s*([a-zA-Z]+)?", spec):
40          unit = (unit or "Hz").lower()
41          if unit not in scale:
42              raise ValueError(f"Unrecognized frequency unit '{unit}'. Use Hz, kHz, MHz, GHz or
    ↪ THz")
43          out.append(abs(float(num.replace("d", "e").replace("D", "e"))) * scale[unit])
44      return np.array(out)
```

Python version of `parse_freq_string`: accepts `'1500kHz, 11MHz, 42.7MHz'`, a number, or a list of numbers in Hz.

L.31–36 Numbers are taken as Hz; a list may mix numbers [Hz] and strings such as '80MHz'.

L.39 Regular expression: a number (Fortran `d` exponents allowed) followed by an optional unit.

L.40–42 Units are case-insensitive. A number without a unit is in Hz.

L.43 Negative values are made positive, as in the Fortran.

freq_label() lines 47–50

```
47  def freq_label(f):
48      for lim, u in ((1e12, "THz"), (1e9, "GHz"), (1e6, "MHz"), (1e3, "kHz"), (0, "Hz")):
49          if f >= lim:
50              return f"{f / max(lim, 1):.2f} {u}" if lim else f"{f:.2f} Hz"
```

L.48 Readable label such as '150.00 MHz' for titles and file headers.

image_geometry(): pixels and ray directions lines 53–79

```
53  def image_geometry(observer, x_size, y_size, nx, ny, up=(0.0, 0.0, 1.0), parallel=False):
54      """Return pixel coordinate axes, unit ray directions (ny*nx, 3), the image
55      unit vectors and the 3D pixel-centre positions.
56
57      Ray index = j*nx + i (x fastest), as in the Fortran code.
58      parallel=True: observer at infinity in the direction of ``observer`` (stars)."""
59      obs = np.asarray(observer, dtype=float)
60      dist = np.linalg.norm(obs)
61      normal = obs / dist
62      up = np.asarray(up, dtype=float)
63      tau = np.cross(up, normal)
64      if np.linalg.norm(tau) < 1e-12:          # observer on the 'up' axis
65          tau = np.cross(np.array([0.0, 1.0, 0.0]), normal)
66      tau /= np.linalg.norm(tau)
67      xi = np.cross(normal, tau)
68      dxp = x_size / nx
69      dyp = y_size / ny
70      xc = -0.5 * x_size + (np.arange(nx) + 0.5) * dxp
71      yc = -0.5 * y_size + (np.arange(ny) + 0.5) * dyp
72      X, Y = np.meshgrid(xc, yc)               # (ny, nx)
73      pix = X.reshape(-1, 1) * tau + Y.reshape(-1, 1) * xi
74      if parallel:                             # distant observer: parallel rays
75          d = np.broadcast_to(-normal, pix.shape).copy()
76      else:
```

```
77          d = pix - obs
78          d /= np.linalg.norm(d, axis=1)[:, None]
79      return xc, yc, d, tau, xi, normal, pix
```

L.59–61 Observer position and $\hat{n} = \mathbf{O}/|\mathbf{O}|$ (`Normal_D`).

L.63 $\hat{\tau} = \hat{z} \times \hat{n}$ (`Tau_D`), the image X axis.

L.64–65 Guard: if the observer lies on the up axis the cross product vanishes, so $\hat{y}$ is used instead.

L.67 $\hat{\xi} = \hat{n} \times \hat{\tau}$ (`Xi_D`), the image Y axis.

L.68–71 Pixel sizes and pixel-centre coordinates $X_i = X_{\rm lo} + (i - \frac{1}{2})\Delta X$.

L.72–73 3D positions of all pixel centres, ordered with X fastest (ray index $= j\,n_x + i$ as in Fortran).

L.74–75 Distant observer (stars, `parallel=True`): all rays are parallel, pointing along $-\hat{n}$.

L.77–78 Otherwise: unit directions from the observer to the pixels (`SlopeUnscaled_D` normalised), as in BATS-R-US.

L.79 The 3D pixel positions are also returned; the parallel projection uses them as ray origins.

start_points(): where each ray starts

lines 82–109

```
82 def start_points(grid, observer, dirs, r_integration):
83     """Starting points: where the straight ray first enters both the
84     integration sphere and the MHD grid box.  Returns (pos, status0)."""
85     obs = np.asarray(observer, dtype=float)
86     if obs.ndim == 1:
87         D2 = obs @ obs
88         od = dirs @ obs
89     else:                                   # one origin per ray (parallel projection)
90         D2 = np.einsum("ij,ij->i", obs, obs)
91         od = np.einsum("ij,ij->i", dirs, obs)
92     disc = r_integration**2 - D2 + od**2
93     t_sph = -od - np.sqrt(np.maximum(disc, 0.0))
94     t_in, t_out = grid.domain_intersection(obs, dirs)
95     t0 = np.maximum(np.maximum(t_sph, t_in), 0.0)
96     # far exit of the sphere
97     t_sph_out = -od + np.sqrt(np.maximum(disc, 0.0))
98     t1 = np.minimum(t_out, t_sph_out)
99     status0 = np.full(dirs.shape[0], ACTIVE, np.int8)
100    status0[(disc <= 0) | (t1 <= t0)] = MISSED
101    # nudge inside the box to avoid round-off at the faces
102    span = np.where(status0 == ACTIVE, t1 - t0, 0.0)
103    t0 = t0 + np.minimum(1e-9 * np.maximum(1.0, np.sqrt(D2)), 1e-3 * span)
104    t0 = np.where(np.isfinite(t0), t0, 0.0)          # missed rays: any finite point
105    pos = obs + t0[:, None] * dirs
106    return pos, status0
107
108
109 # ---------------------------------------------------------------------------
```

L.86–91 One observer for all rays (perspective), or one origin per ray (parallel projection).

L.92–93 Entry into the integration sphere: the smaller root of $|\mathbf{O}+t\mathbf{d}| = r_{\rm int}$ (`ObservToIntSphereDist`).

L.94 Entry into and exit from the model domain (box or outer sphere).

L.95 Start at the later of the two entries, and never behind the observer.

L.97–98 End of the useful interval.

L.100 Rays that never pass through the model inside the sphere are flagged and skipped.

L.102–103 Nudge the start point slightly inside, so that round-off does not put it just outside the box face.

Parallel worker

lines 110–118

```
110 _G = {}
111
112
113 def _worker(args):
114     idx, freq = args
115     g = _G
116     res = trace_rays(g["grid"], g["pos"][idx], g["dirs"][idx], freq, g["emission"],
117                      status0=g["status0"][idx], **g["kw"])
118     return idx, res["intensity"], res["tau"], res["status"], res["nstep"]
```

L.110 Module-level storage for the shared data. With the `fork` start method the worker processes inherit it without copying or pickling the grid.

L.113 Traces one chunk of ray indices and returns the results with the indices.

RadioImage: container

lines 121–148

```
121 class RadioImage:
122     """Container for a multi-frequency synthetic radio image.
123
124     Attributes
125     ----------
126     intensity : (nfreq, ny, nx)  [W m^-2 Hz^-1 sr^-1] (dimensionless for 'simplistic')
127     tau       : (nfreq, ny, nx)  optical depth along the ray (full transfer)
128     status    : (nfreq, ny, nx)  termination code of each ray
129     x, y      : pixel-centre coordinates in the image plane [R_sun]
130     freqs     : [Hz]
131     """
132
133     def __init__(self, intensity, tau, status, nstep, x, y, freqs, observer, meta,
134                  distance=None, length_unit=R_SUN):
135         self.intensity = intensity
136         self.tau = tau
137         self.status = status
138         self.nstep = nstep
139         self.x = x
140         self.y = y
141         self.freqs = np.asarray(freqs)
142         self.observer = np.asarray(observer, dtype=float)
143         self.meta = meta
144         # observer distance in coordinate units (sets the flux and angular scale)
145         self.distance = float(distance if distance is not None else np.linalg.norm(self.
    ↪ observer))
146         self.length_unit = float(length_unit)
147
148     @property
```

L.133 Stores intensity, τ, status, step counts (all (nf, ny, nx)), pixel axes, frequencies, observer and the settings.

L.145–146 Observer distance in coordinate units (the true distance for a star, otherwise $|\mathbf{O}|$) and metres per coordinate unit. Together they set the flux density and the angular scale.

RadioImage: derived quantities

lines 149–175

```
149     def brightness_temperature(self):
150         """Rayleigh-Jeans brightness temperature  T_b = I c^2 / (2 k nu^2)  [K]."""
151         f = self.freqs[:, None, None]
152         return self.intensity * C_LIGHT**2 / (2.0 * K_B * f**2)
153 
154     @property
155     def pixel_solid_angle(self):
156         D = self.distance
157         return (self.x[1] - self.x[0]) * (self.y[1] - self.y[0]) / D**2   # [sr]
158 
159     @property
160     def flux_density(self):
161         """Total flux density at the observer [W m^-2 Hz^-1] for each frequency."""
162         return self.intensity.sum(axis=(1, 2)) * self.pixel_solid_angle
163 
164     @property
165     def flux_density_sfu(self):
166         return self.flux_density / 1e-22
167 
168     @property
169     def flux_density_mjy(self):
170         """Flux density in mJy (stellar sources)."""
171         return self.flux_density / 1e-29
172 
173     @property
174     def distance_pc(self):
175         return self.distance * self.length_unit / 3.0856775814913673e16
```

L.152 $T_b = Ic^2/(2k_B\nu^2)$.

L.155 $\Delta\Omega = \Delta X \Delta Y / D^2$: pixel area over distance squared, both in coordinate units.

L.160 $F = \sum I\,\Delta\Omega$ per frequency, in W m^{-2} Hz^{-1}.

L.165 The same in solar flux units.

L.169 The same in mJy, the natural unit for stars.

L.174 Observer distance in parsec.

RadioImage.summary()

lines 177–190

```
177     def summary(self):
178         lines = []
179         for k, f in enumerate(self.freqs):
180             st = self.status[k]
181             counts = ", ".join(f"{STATUS_NAMES[c]}={int((st == c).sum())}"
182                                for c in np.unique(st))
```

```
183             lines.append(f"{freq_label(f):>12s}: max I={self.intensity[k].max():.3e}, "
184                          f"max Tb={self.brightness_temperature[k].max():.3e} K, "
185                          f"flux={self.flux_density[k]:.3e} W/m2/Hz "
186                          f"({self.flux_density[k] / JANSKY:.3e} Jy at {self.distance_pc:.3g}
    ↪ pc); "
187                          f"rays: {counts}")
188         return "\n".join(lines)
189
190     # ----- output -------------------------------------------------------
```

L.181 For each frequency: maximum intensity and T_b, flux in SI and Jy, and how many rays ended in each way.

Output methods

lines 191–224

```
191     def save_npz(self, path):
192         np.savez_compressed(path, intensity=self.intensity, tau=self.tau,
193                             status=self.status, nstep=self.nstep, x=self.x, y=self.y,
194                             freqs=self.freqs, observer=self.observer,
195                             distance=self.distance, length_unit=self.length_unit,
196                             flux_density=self.flux_density,
197                             tb=self.brightness_temperature, meta=repr(self.meta))
198
199     def save_tecplot(self, path):
200         """ASCII Tecplot file in the same layout as BATS-R-US write_plot_radiowave."""
201         names = " ".join(f'"f={freq_label(f).replace(" ", "_")}"' for f in self.freqs)
202         nx, ny = self.x.size, self.y.size
203         with open(path, "w") as fh:
204             fh.write('TITLE="radiosynth: Radiotelescope Image"\n')
205             fh.write(f'VARIABLES = "X" "Y" {names}\n')
206             fh.write(f'ZONE T="radio", I={nx}, J={ny}, K=1, F=POINT\n')
207             X, Y = np.meshgrid(self.x, self.y)
208             data = np.column_stack([X.ravel(), Y.ravel()] +
209                                    [self.intensity[k].ravel() for k in range(len(self.freqs))
    ↪ ])
210             np.savetxt(fh, data, fmt="%14.6E")
211
212     def save_fits(self, path, quantity="intensity"):
213         """One FITS file with a (nfreq, ny, nx) cube (astropy used if installed)."""
214         from .fitsio import write_fits
215         data = self.brightness_temperature if quantity == "tb" else self.intensity
216         h = {"BUNIT": "K" if quantity == "tb" else "W m-2 Hz-1 sr-1",
217              "CTYPE1": "SOLX", "CUNIT1": "Rsun", "CRPIX1": 1.0,
218              "CRVAL1": float(self.x[0]), "CDELT1": float(self.x[1] - self.x[0]),
219              "CTYPE2": "SOLY", "CUNIT2": "Rsun", "CRPIX2": 1.0,
220              "CRVAL2": float(self.y[0]), "CDELT2": float(self.y[1] - self.y[0])}
221         for k, f in enumerate(self.freqs):
222             h[f"FREQ{k + 1}"] = float(f)
223         h["OBSX"], h["OBSY"], h["OBSZ"] = (float(v) for v in self.observer)
224         write_fits(path, data, h)
```

L.192 NPZ with all arrays, including T_b and the settings (as text).

L.199–210 ASCII Tecplot ordered zone with variables X, Y and one column per frequency, the same layout as the BATS-R-US radio image file.

L.212–224 FITS cube with a simple linear WCS in $R_\odot$ and the frequencies and observer in the header. It is written with astropy if installed, otherwise with the built-in writer in `fitsio.py`.

RadioImage.plot()

lines 226–273

```
226     def plot(self, quantity="tb", path=None, log=True, cmap="inferno", ncols=None,
227              vmin=None, vmax=None, show_limb=True):
228         import matplotlib
229         if path is not None:
230             matplotlib.use("Agg")
231         import matplotlib.pyplot as plt
232         from matplotlib.colors import LogNorm
233 
234         data = self.brightness_temperature if quantity == "tb" else self.intensity
235         label = r"$T_b$ [K]" if quantity == "tb" else r"$I_\nu$ [W m$^{-2}$ Hz$^{-1}$ sr$
    ↪ ^{-1}$]"
236         nf = len(self.freqs)
237         ncols = ncols or min(nf, 3)
238         nrows = math.ceil(nf / ncols)
239         fig, axes = plt.subplots(nrows, ncols, figsize=(4.6 * ncols, 4.0 * nrows), squeeze=
    ↪ False)
240         ext = [self.x[0] - 0.5 * (self.x[1] - self.x[0]), self.x[-1] + 0.5 * (self.x[1] -
    ↪ self.x[0]),
241                self.y[0] - 0.5 * (self.y[1] - self.y[0]), self.y[-1] + 0.5 * (self.y[1] -
    ↪ self.y[0])]
242         for k in range(nrows * ncols):
243             ax = axes.flat[k]
244             if k >= nf:
245                 ax.axis("off")
246                 continue
247             d = data[k]
248             pos = d[d > 0]
249             if log and pos.size:
250                 lo = vmin if vmin is not None else max(pos.min(), pos.max() * 1e-4)
251                 hi = vmax if vmax is not None else pos.max()
252                 norm = LogNorm(vmin=lo, vmax=hi)
253                 im = ax.imshow(np.where(d > 0, d, np.nan), origin="lower", extent=ext,
254                                cmap=cmap, norm=norm)
255             else:
256                 im = ax.imshow(d, origin="lower", extent=ext, cmap=cmap, vmin=vmin, vmax=vmax
    ↪ )
257             if show_limb:
258                 t = np.linspace(0, 2 * np.pi, 361)
259                 ax.plot(np.cos(t), np.sin(t), "w--", lw=0.7, alpha=0.8)
260                 ax.set_xlim(ext[0], ext[1])
261                 ax.set_ylim(ext[2], ext[3])
262             ax.set_title(freq_label(self.freqs[k]))
263             ax.set_xlabel(r"X [$R_\odot$]")
264             ax.set_ylabel(r"Y [$R_\odot$]")
265             fig.colorbar(im, ax=ax, label=label, shrink=0.85)
266         fig.tight_layout()
267         if path is not None:
268             fig.savefig(path, dpi=130)
269             plt.close(fig)
270         return fig
```

```
271
272
273 # ---------------------------------------------------------------------------
```

L.230 Non-interactive backend when writing a file (works on clusters without a display).

L.237–239 One panel per frequency, up to three per row.

L.240 Image extent at the pixel edges, for `imshow`.

L.249–254 Logarithmic colour scale over four decades by default; pixels with zero intensity are left blank.

L.257–259 Dashed circle marking the photospheric limb ($r = 1$).

make_radio_image(): the main entry point

lines 274–321

```
274 def make_radio_image(grid, frequencies, observer=(-215.0, 0.0, 0.0),
275                     x_size=6.0, y_size=6.0, nx=128, ny=128, *,
276                     emission="bremsstrahlung", gaunt=10.0, transfer="thin",
277                     no_refraction=False, r_integration=None, up=(0.0, 0.0, 1.0),
278                     n_workers=1, verbose=True, distance_pc=None, **trace_kw):
279     """Compute a synthetic radio image from an MHDGrid.
280
281     Parameters mirror the BATS-R-US #SAVEPLOT 'rwi' (radiowave image) input:
282     observer position ObsPos [R_sun], image size X_Size_Image/Y_Size_Image
283     [R_sun], pixel numbers n_Pix_X/n_Pix_Y, the frequency list (string such as
284     '42.3MHz, 125MHz, 17GHz' or numbers in Hz), and UseNoRefraction.
285
286     distance_pc : if given, the observer is at this distance (parsec) in the
287     direction of ``observer`` and the rays are parallel (distant star); the
288     flux density is computed for that distance.  Otherwise the rays diverge from
289     ``observer`` (Sun seen from the Earth, as in BATS-R-US).
290
291     Extra keywords are passed to raytrace.trace_rays (tolerance, step_factor,
292     tau_max, kappa_over_mu, ...).
293     """
294     freqs = parse_frequencies(frequencies)
295     em = emission if callable(emission) else EmissionModel(emission, gaunt=gaunt)
296     L = float(getattr(grid, "length_unit", R_SUN))
297     parallel = distance_pc is not None
298     xc, yc, dirs, tau_v, xi_v, normal, pix = image_geometry(observer, x_size, y_size, nx, ny,
299                                                             up, parallel=parallel)
300     diag = math.sqrt((0.5 * x_size)**2 + (0.5 * y_size)**2)
301     if r_integration is None:
302         r_integration = float(math.ceil(max(diag + 1.0, 5.0)))
303     if parallel:
304         distance = distance_pc * 3.0856775814913673e16 / L      # coordinate units
305         far = 4.0 * max(r_integration, getattr(grid, "r_outer", r_integration), diag)
306         origins = pix + far * normal
307         pos, status0 = start_points(grid, origins, dirs, r_integration)
308     else:
309         distance = float(np.linalg.norm(observer))
310         pos, status0 = start_points(grid, observer, dirs, r_integration)
311     nray = dirs.shape[0]
312     if verbose:
313         where = (f"distance {distance_pc:g} pc (parallel rays), direction "
314                  f"{tuple(round(float(v), 4) for v in normal)}" if parallel
315                  else f"observer={tuple(float(v) for v in observer)}")
```

```
316         print(f"radiosynth: {nx}x{ny} rays, {where}, "
317               f"rIntegration={r_integration} R_sun, refraction={'off' if no_refraction else '
    ↪ on'}, "
318               f"emission={em}, transfer={transfer}")
319         print(f"            {grid}")
320         if (status0 == MISSED).any():
321             print(f"            {(status0 == MISSED).sum()} rays never enter the grid box")
```

L.294 Frequency list in Hz.

L.295 A string selects the built-in `EmissionModel`; a function is used as given.

L.298 Pixel axes and ray directions.

L.300–302 `rIntegration = ceiling(max(ImagePlaneDiagRadius+1.0, 5.0))`, as in Fortran.

L.296–297 Length unit of the grid; `distance_pc` switches to the distant-observer (parallel-ray) projection.

L.303–307 Parallel projection: the distance in coordinate units sets the flux. Each ray starts far outside the model on the observer's side of its pixel, and then enters the integration sphere / domain like any other ray.

L.308–310 Perspective (Sun seen from the Earth): all rays start at the observer.

L.312–321 Report the set-up.

make_radio_image(): tracing

lines 323–348

```
323     kw = dict(transfer=transfer, no_refraction=no_refraction,
324               r_integration=r_integration, **trace_kw)
325     I = np.zeros((len(freqs), nray))
326     T = np.zeros_like(I)
327     S = np.zeros((len(freqs), nray), np.int8)
328     NS = np.zeros((len(freqs), nray), np.int64)
329
330     use_pool = n_workers and n_workers > 1 and "fork" in mp.get_all_start_methods()
331     for k, f in enumerate(freqs):
332         t0 = time.time()
333         if verbose:
334             print(f"RAYTRACE START: f = {freq_label(f)}  (n_cr = {critical_density(f):.3e} m
    ↪ ^-3)")
335         if use_pool:
336             _G.update(grid=grid, pos=pos, dirs=dirs, status0=status0, emission=em, kw=kw)
337             nchunk = 4 * n_workers
338             chunks = [(np.arange(c, nray, nchunk), f) for c in range(nchunk)]
339             with mp.get_context("fork").Pool(n_workers) as pool:
340                 for idx, i_, t_, s_, n_ in pool.imap_unordered(_worker, chunks):
341                     I[k, idx], T[k, idx], S[k, idx], NS[k, idx] = i_, t_, s_, n_
342             _G.clear()
343         else:
344             r = trace_rays(grid, pos, dirs, f, em, status0=status0, **kw)
345             I[k], T[k], S[k], NS[k] = r["intensity"], r["tau"], r["status"], r["nstep"]
346         if verbose:
347             print(f"RAYTRACE END:   {time.time() - t0:.1f} s, "
348                   f"mean steps/ray {NS[k][S[k] != MISSED].mean():.0f}")
```

L.323 Options passed to `trace_rays` (plus any advanced keywords).

L.325–328 Result arrays (frequency, ray).

L.330 Parallel only if more than one worker is requested and `fork` is available (Linux, macOS).

L.336–338 Share the data and split the rays into $4\times$ `n_workers` interleaved chunks (rays $c, c+N, c+2N, \ldots$). Rays near the disk take more steps than off-limb rays, so interleaving balances the load.

L.339–341 Run the chunks and scatter the results back by index.

L.344–345 Serial path: all rays in one call.

make_radio_image(): result

lines 350–358

```
350     shp = (len(freqs), ny, nx)
351     meta = dict(x_size=x_size, y_size=y_size, nx=nx, ny=ny, emission=repr(em),
352                 transfer=transfer, no_refraction=no_refraction, distance_pc=distance_pc,
353                 r_integration=r_integration, tau_vec=tau_v, xi_vec=xi_v, **trace_kw)
354     img = RadioImage(I.reshape(shp), T.reshape(shp), S.reshape(shp), NS.reshape(shp),
355                      xc, yc, freqs, observer, meta, distance=distance, length_unit=L)
356     if verbose:
357         print(img.summary())
358     return img
```

L.350 Reshape (ray) to (ny, nx): with X fastest, row j is image Y.

L.351 Record the settings with the image.

L.354 Build and return the result object, printing the summary if `verbose`.

trace_paths(): individual trajectories

lines 361–384

```
361 def trace_paths(grid, frequency, observer, x_pix, y_pix, *, x_size=6.0, y_size=6.0,
362                 r_integration=None, up=(0.0, 0.0, 1.0), emission="bremsstrahlung",
363                 gaunt=10.0, **trace_kw):
364     """Trace individual rays through given image-plane points (x_pix, y_pix)
365     [R_sun] and return their paths (for plots like Benkevitch+2010 Fig. 7-9)."""
366     obs = np.asarray(observer, dtype=float)
367     _, _, _, tau_v, xi_v, _, _ = image_geometry(obs, x_size, y_size, 2, 2, up)
368     xp = np.atleast_1d(x_pix).astype(float)
369     yp = np.atleast_1d(y_pix).astype(float)
370     pix = xp[:, None] * tau_v + yp[:, None] * xi_v
371     d = pix - obs
372     d /= np.linalg.norm(d, axis=1)[:, None]
373     if r_integration is None:
374         diag = math.sqrt((0.5 * x_size)**2 + (0.5 * y_size)**2)
375         r_integration = float(math.ceil(max(diag + 1.0, 5.0)))
376     pos, st0 = start_points(grid, obs, d, r_integration)
377     em = emission if callable(emission) else EmissionModel(emission, gaunt=gaunt)
378     f = parse_frequencies(frequency)[0]
379     r = trace_rays(grid, pos, d, f, em, status0=st0, r_integration=r_integration,
380                    record_ids=np.arange(len(xp)), **trace_kw)
381     return r
382
383
384 # ---------------------------------------------------------------------------
```

Traces rays through arbitrary image-plane points and records their paths. It uses the same geometry, start points and tracer as an image, so the paths are exactly those that make up the

image.

L.370–372 Directions to the requested image-plane points.

L.380 Record every ray.

compare_refraction(): refracted vs straight lines of sight

lines 385–472

```
385 REFRACTIVE_MECHANISMS = ("bremsstrahlung", "simplistic")
386
387
388 def compare_refraction(grid, frequencies, observer=(-215.0, 0.0, 0.0), x_size=6.0,
389                         y_size=6.0, nx=128, ny=128, *, emission="bremsstrahlung",
390                         quantity="tb", path=None, verbose=True, **kw):
391     """Same image with refracted rays and with straight lines of sight.
392
393     Available for the mechanisms that use the refractive ray tracer
394     ('bremsstrahlung', 'simplistic', or a user emission function).  Both images
395     use identical settings (all keywords of make_radio_image, e.g. transfer,
396     gaunt, distance_pc, n_workers); only no_refraction differs.  Gyroresonance
397     and gyrosynchrotron are computed along straight rays only and are not
398     accepted here.
399
400     quantity : 'tb' or 'intensity' for the figure
401     path     : if given, write a figure with one row per frequency: refracted,
402                straight, and the ratio refracted/straight
403
404     Returns dict(refracted=RadioImage, straight=RadioImage,
405                  flux_ratio=(nfreq,) refracted/straight flux density,
406                  changed=(nfreq,) fraction of emitting pixels changed by > 10%,
407                  figure=matplotlib figure or None)
408     """
409     if isinstance(emission, str) and emission.lower() not in REFRACTIVE_MECHANISMS:
410         raise ValueError(f"compare_refraction works for {REFRACTIVE_MECHANISMS} (refractive "
411                          f"ray tracer); '{emission}' is computed along straight rays only")
412     kw.pop("no_refraction", None)
413     common = dict(observer=observer, x_size=x_size, y_size=y_size, nx=nx, ny=ny,
414                   emission=emission, verbose=False, **kw)
415     ref = make_radio_image(grid, frequencies, no_refraction=False, **common)
416     los = make_radio_image(grid, frequencies, no_refraction=True, **common)
417     flux_ratio = ref.flux_density / np.where(los.flux_density > 0, los.flux_density, np.nan)
418     a, b = ref.intensity, los.intensity
419     with np.errstate(invalid="ignore", divide="ignore"):
420         rel = np.abs(a - b) / np.maximum(np.maximum(a, b), 1e-300)
421     emitting = np.maximum(a, b) > 1e-3 * np.maximum(a, b).max(axis=(1, 2), keepdims=True)
422     changed = np.array([(rel[k][emitting[k]] > 0.1).mean() if emitting[k].any() else 0.0
423                         for k in range(a.shape[0])])
424     if verbose:
425         for k, f in enumerate(ref.freqs):
426             print(f"{freq_label(f):>12s}: flux refracted/straight = {flux_ratio[k]:.4f}, "
427                   f"{100 * changed[k]:.1f}% of the emitting pixels change by more than 10%")
428     fig = None
429     if path is not None:
430         fig = _figure_refraction(ref, los, quantity, path)
431     return dict(refracted=ref, straight=los, flux_ratio=flux_ratio, changed=changed, figure=
    ↪ fig)
432
```

```
433
434 def _figure_refraction(ref, los, quantity, path):
435     import matplotlib
436     matplotlib.use("Agg")
437     import matplotlib.pyplot as plt
438     from matplotlib.colors import LogNorm
439     da = ref.brightness_temperature if quantity == "tb" else ref.intensity
440     db = los.brightness_temperature if quantity == "tb" else los.intensity
441     label = r"$T_b$ [K]" if quantity == "tb" else r"$I_\nu$"
442     nf = len(ref.freqs)
443     fig, axes = plt.subplots(nf, 3, figsize=(14, 4.0 * nf), squeeze=False)
444     dx, dy = ref.x[1] - ref.x[0], ref.y[1] - ref.y[0]
445     ext = [ref.x[0] - dx / 2, ref.x[-1] + dx / 2, ref.y[0] - dy / 2, ref.y[-1] + dy / 2]
446     t = np.linspace(0, 2 * np.pi, 361)
447     for k in range(nf):
448         hi = max(da[k].max(), db[k].max())
449         pos = np.concatenate([da[k][da[k] > 0], db[k][db[k] > 0]])
450         lo = max(pos.min(), hi * 1e-4) if pos.size else 1e-30
451         for j, (d, title) in enumerate(((da[k], "refracted rays"), (db[k], "straight lines of
    ↪  sight"))):
452             ax = axes[k, j]
453             im = ax.imshow(np.where(d > 0, d, np.nan), origin="lower", extent=ext, cmap="
    ↪ inferno",
454                            norm=LogNorm(vmin=lo, vmax=hi))
455             fig.colorbar(im, ax=ax, label=label, shrink=0.85)
456             ax.set_title(f"{freq_label(ref.freqs[k])}: {title}")
457         with np.errstate(invalid="ignore", divide="ignore"):
458             ratio = np.where((da[k] > lo) | (db[k] > lo), da[k] / db[k], np.nan)
459         ax = axes[k, 2]
460         im = ax.imshow(np.log10(ratio), origin="lower", extent=ext, cmap="RdBu_r", vmin=-1,
    ↪ vmax=1)
461         fig.colorbar(im, ax=ax, label="log10(refracted / straight)", shrink=0.85)
462         ax.set_title(f"flux ratio {ref.flux_density[k] / max(los.flux_density[k], 1e-300):.3f
    ↪ }")
463         for ax in axes[k]:
464             ax.plot(np.cos(t), np.sin(t), "k--", lw=0.6, alpha=0.6)
465             ax.set_xlim(ext[0], ext[1])
466             ax.set_ylim(ext[2], ext[3])
467             ax.set_xlabel(r"X [$R_\odot$]")
468             ax.set_ylabel(r"Y [$R_\odot$]")
469     fig.tight_layout()
470     fig.savefig(path, dpi=120)
471     plt.close(fig)
472     return fig
```

L.385 The mechanisms that use the refractive tracer; gyroresonance and gyrosynchrotron are straight-ray only and are refused.

L.413–416 The same call twice; only `no_refraction` differs, so the comparison isolates refraction.

L.417 Refracted / straight flux density per frequency.

L.419–422 Fraction of the emitting pixels (above 10^{-3} of the maximum) whose intensity changes by more than 10%.

L.424–430 Report and optional figure.

L.434–472 One row per frequency: refracted and straight maps on a common logarithmic scale, and $\log_{10}$ of their ratio.

Chapter 12

common/symmetric.py — 1D (spherically symmetric) models

File: `radiosynth/common/symmetric.py` — 210 lines.

Turns a radial profile into a grid, and computes images, radial brightness profiles and fluxes of a symmetric model from one row of rays.

Module docstring and imports

lines 4–30

```
4  """Spherically symmetric (1D) models.
5
6  A 1D model gives n_e(r) and T_e(r).  Assuming spherical symmetry it is turned
7  into an axisymmetric SphericalGrid with a single colatitude cell spanning 0..pi
8  (grid_from_profile, read_profile, and load_pluto for 1D spherical output).
9
10 Because the image of a symmetric model depends only on the distance p from
11 the disk centre, make_symmetric_image traces a single row of rays
12 (0 <= p <= p_max) and rotates the result into a 2D image; it also returns the
13 radial brightness profile and the flux density integrated exactly over the
14 profile, S = (2 pi / D^2) int I(p) p dp.  This is much cheaper than the full
15 2D calculation and exact for a symmetric model (also with refraction and for
16 an observer at a finite distance, since the problem is symmetric about the
17 observer-Sun axis).
18
19 1D support is provided for the mechanisms of the refractive ray tracer
20 (bremsstrahlung, simplistic).  Gyroresonance and gyrosynchrotron are not
21 offered for 1D models: a spherically symmetric field is a magnetic monopole and
22 the fast electrons of flares are not spherically symmetric, so such images
23 would not be meaningful.
24 """
25 import math
26
27 import numpy as np
28
29 from .constants import K_B, M_P, R_SUN
30 from .grid import SphericalGrid
```

Explains the axisymmetric grid with one colatitude cell, the one-row method and why only the mechanisms of the refractive tracer are offered.

grid_from_profile()

lines 33–52

```
33 def grid_from_profile(r, ne, te=None, r_edges=None, r_body=None, length_unit=R_SUN,
34                        dtype=np.float32):
35     """SphericalGrid of a spherically symmetric model.
36
37     r      : radii [coordinate units: R_sun, or R_star with length_unit]
38     ne     : electron density [m^-3];  te : temperature [K] (default 1 K, enough
```

```
            for the simplistic mechanism only)
    r_edges: cell edges (default: midpoints between the radii)
    r_body : inner boundary (default: first edge)
    """
    r = np.asarray(r, float)
    if r.ndim != 1 or r.size < 2 or np.any(np.diff(r) <= 0):
        raise ValueError("r must be 1D and strictly increasing")
    ne = np.asarray(ne, float).reshape(-1, 1)
    te_a = None if te is None else np.asarray(te, float).reshape(-1, 1)
    g = SphericalGrid(r, np.array([0.5 * np.pi]), None, ne, te_a, r_edges=r_edges,
                      theta_edges=np.array([0.0, np.pi]), r_body=r_body, dtype=dtype,
                      length_unit=length_unit)
    g.symmetric = True
    return g
```

L.43–45 Check the radii.

L.48–49 One colatitude cell with edges 0 and π and no longitude: the fields depend on r only, and every direction is inside the domain.

L.51 Mark the grid as symmetric.

read_profile(): text tables

lines 55–79

```
def read_profile(path, r_col=0, density_col=1, te_col=2, *, r_unit=1.0,
                 density="ne", density_unit="cm^-3", mu_e=1.0, te_unit=1.0, skiprows=0,
                 delimiter=None, **kw):
    """Read a 1D model from a text table (one radius per line) and return
    grid_from_profile(...).

    r_unit       : coordinate units per file unit (e.g. 1/6.96e10 for cm -> R_sun)
    density      : 'ne' (electron density) or 'rho' (mass density)
    density_unit : for 'ne': 'cm^-3' or 'm^-3'; for 'rho': 'g/cm^3' or 'kg/m^3'
    mu_e         : mass per electron in m_p for 'rho'
    te_unit      : kelvin per file unit;  te_col : None if absent
    """
    d = np.loadtxt(path, skiprows=skiprows, delimiter=delimiter, ndmin=2)
    r = d[:, r_col] * r_unit
    q = d[:, density_col]
    if density == "ne":
        ne = q * (1e6 if density_unit == "cm^-3" else 1.0)
    elif density == "rho":
        rho = q * (1e3 if density_unit == "g/cm^3" else 1.0)          # kg/m^3
        ne = rho / (mu_e * M_P)
    else:
        raise ValueError("density must be 'ne' or 'rho'")
    te = None if te_col is None else d[:, te_col] * te_unit
    order = np.argsort(r)
    return grid_from_profile(r[order], ne[order], None if te is None else te[order], **kw)
```

L.67 One radius per line; columns chosen by the caller.

L.70–76 Electron density in cm^{-3} or m^{-3}, or mass density converted with $n_e = \rho/(\mu_e m_p)$.

L.78–79 Sort by radius and build the grid.

is_symmetric()

lines 82–89

```
82 def is_symmetric(grid):
83     """True for a grid built from a 1D profile (or any spherical grid with a
84     single theta and phi cell)."""
85     return bool(getattr(grid, "symmetric", False)) or (
86         isinstance(grid, SphericalGrid) and grid.shape[1] == 1 and grid.shape[2] == 1)
87 
88 
89 # ---------------------------------------------------------------------------
```

A grid made by `grid_from_profile`, or any spherical grid with one θ and one ϕ cell (e.g. reloaded from npz or read from PLUTO 1D output).

_rotate()

lines 90–94

```
90 def _rotate(row_p, row_vals, X, Y):
91     """Values of a radial profile at the pixels of a 2D image (linear in p)."""
92     rr = np.sqrt(X**2 + Y**2)
93     out = np.interp(rr.ravel(), row_p, row_vals, right=0.0)
94     return out.reshape(rr.shape)
```

Linear interpolation of the profile at the distance $\sqrt{X^2+Y^2}$ of every pixel; zero beyond the last ray.

make_symmetric_image(): one row of rays

lines 97–145

```
97 def make_symmetric_image(grid, frequencies, mechanism="bremsstrahlung",
98                         observer=(-215.0, 0.0, 0.0), x_size=None, y_size=None, nx=256,
99                         ny=256, n_p=1000, p_max=None, verbose=True, **kw):
100    """Image of a spherically symmetric model from one row of rays.
101
102    mechanism : 'bremsstrahlung' or 'simplistic' (or an emission function)
103    x_size, y_size, nx, ny : the 2D image produced by rotation (default size:
104                2 p_max square)
105    n_p, p_max : number of rays (default 1000; the limb, where the brightness
106                 changes abruptly, needs fine sampling) and the largest distance
107                 from the centre [coordinate units] (default: half of the image
108                 diagonal, or the outer radius of the grid)
109    Other keywords go to make_radio_image (transfer, gaunt, no_refraction,
110    distance_pc, step_factor, ...), so refraction can be switched on or off.
111
112    Returns a RadioImage on the (ny, nx) pixel grid, with
113    img.profile = dict(p [coordinate units],
114    intensity (nf, n_p+1), tb (nf, n_p+1), flux (nf,) [W m^-2 Hz^-1] within the
115    circle p <= p_max (the image's own flux_density covers its square field of
116    view)).
117    """
118    from .image import make_radio_image
119    if not is_symmetric(grid):
120        print("radiosynth: WARNING - make_symmetric_image assumes a spherically symmetric "
121              "model; this grid is not built from a 1D profile")
122    if p_max is None:
```

```
123         if x_size is not None or y_size is not None:
124             p_max = 0.5 * math.hypot(x_size or y_size, y_size or x_size)
125         else:
126             p_max = float(getattr(grid, "r_outer", 5.0))
127     x_size = x_size if x_size is not None else 2.0 * p_max
128     y_size = y_size if y_size is not None else x_size
129     # one row through the centre: rays at p = k dp, k = -n_p..n_p (one exactly at the
130     # centre), 1 pixel high
131     dp = p_max / n_p
132     row = dict(x_size=(2 * n_p + 1) * dp, y_size=dp, nx=2 * n_p + 1, ny=1, observer=observer)
133     mech = mechanism.lower() if isinstance(mechanism, str) else mechanism
134     # integrate along the whole line of sight through the model (the default of the
135     # image functions is a sphere sized to the field of view)
136     kw.setdefault("r_integration", max(float(getattr(grid, "r_outer", p_max)), p_max) + 1.0)
137     if verbose:
138         print(f"radiosynth: spherically symmetric model - {n_p + 1} rays (0 <= p <= {p_max:g
    ↪ }), "
139               f"mechanism {mechanism if isinstance(mechanism, str) else 'user function'}")
140     if callable(mech) or mech in ("bremsstrahlung", "simplistic"):
141         im = make_radio_image(grid, frequencies, emission=mech, verbose=False, **row, **kw)
142     else:
143         raise ValueError("1D (spherically symmetric) images are available for 'bremsstrahlung
    ↪ ' "
144                          "and 'simplistic'; gyroresonance and gyrosynchrotron need a 2.5D or
    ↪ 3D "
145                          "magnetic field")
```

L.122–126 Default $p_{\max}$: half the image diagonal, or the outer radius of the grid.

L.131–132 A $(2n_p+1)\times 1$ image whose pixel centres are $p = k\,\Delta p$, $k = -n_p \ldots n_p$: one ray exactly at the centre.

L.136 Integrate along the whole line of sight through the model.

L.140–145 `make_radio_image` for bremsstrahlung, simplistic or a user emission function (refraction on or off as requested); other mechanisms are refused.

make_symmetric_image(): rotation and flux

lines 147–183

```
147     nf = len(im.freqs)
148     half = slice(n_p, 2 * n_p + 1)                  # pixels with p >= 0
149     p = np.abs(im.x[half])
150     p[0] = 0.0
151     # rotate every per-pixel array (nf, 1, 2 n_p) into (nf, ny, nx)
152     xc = -0.5 * x_size + (np.arange(nx) + 0.5) * x_size / nx
153     yc = -0.5 * y_size + (np.arange(ny) + 0.5) * y_size / ny
154     X, Y = np.meshgrid(xc, yc)
155     profile = {}
156     for name, val in list(vars(im).items()):
157         if not (isinstance(val, np.ndarray) and val.shape == (nf, 1, 2 * n_p + 1)):
158             continue
159         v = val[:, 0, half]
160         if np.issubdtype(v.dtype, np.floating):
161             new = np.stack([_rotate(p, v[k], X, Y) for k in range(nf)])
162         else:                                       # status codes etc.: nearest ray
163             idx = np.clip(np.rint(np.sqrt(X**2 + Y**2) / dp).astype(np.int64), 0, n_p)
164             new = v[:, idx].astype(v.dtype)
165         setattr(im, name, new)
```

```
166         if name == "intensity":
167             profile[name] = v
168     im.x, im.y = xc, yc
169     im.meta = dict(im.meta, x_size=x_size, y_size=y_size, nx=nx, ny=ny, symmetric=True,
170                    n_p=n_p, p_max=p_max)
171     # flux from the profile: S = 2 pi / D^2 int_0^p_max I(p) p dp  (all azimuths, i.e.
172     # the whole disk out to p_max, not only the square field of view of the image)
173     trap = getattr(np, "trapezoid", None) or np.trapz
174     flux = 2.0 * math.pi * trap(profile["intensity"] * p, p, axis=1) / im.distance**2
175     f2 = np.asarray(im.freqs, float)[:, None] ** 2
176     from .constants import C_LIGHT
177     profile.update(p=p, tb=profile["intensity"] * C_LIGHT**2 / (2.0 * K_B * f2), flux=flux)
178     im.profile = profile
179     if verbose:
180         for k, f in enumerate(im.freqs):
181             print(f"   {f / 1e6:10.2f} MHz: flux {flux[k]:.4e} W/m^2/Hz "
182                   f"({flux[k] / 1e-22:.4g} sfu), T_b(centre) {profile['tb'][k, 0]:.4g} K")
183     return im
```

L.148–150 The rays with $p \geq 0$.

L.156–167 Every per-pixel array of the image is rotated: floating-point quantities by linear interpolation in p, status codes from the nearest ray.

L.169 Record the 2D image layout and the symmetric settings.

L.173–174 $S = 2\pi D^{-2} \int_0^{p_{\max}} I(p)\, p\, dp$ (trapezoid rule) over the whole circle.

L.177 Profile: p, $I(p)$, $T_b(p)$, flux.

plot_profile() lines 186–210

```
186 def plot_profile(img, path=None, ax=None, units="rsun"):
187     """T_b against the distance from the disk centre for every frequency
188     (img from make_symmetric_image)."""
189     import matplotlib
190     if path is not None:
191         matplotlib.use("Agg")
192     import matplotlib.pyplot as plt
193     from .image import freq_label
194     pr = img.profile
195     if ax is None:
196         fig, ax = plt.subplots(figsize=(6.2, 4.2))
197     else:
198         fig = ax.figure
199     for k, f in enumerate(img.freqs):
200         ax.semilogy(pr["p"], np.maximum(pr["tb"][k], 1e-30), label=freq_label(f))
201     ax.axvline(1.0, color="gray", ls="--", lw=0.7)
202     ax.set_xlabel(r"distance from disk centre [$R$]")
203     ax.set_ylabel(r"$T_b$ [K]")
204     ax.legend(fontsize=8)
205     ax.grid(alpha=0.3, which="both")
206     fig.tight_layout()
207     if path is not None:
208         fig.savefig(path, dpi=130)
209         plt.close(fig)
210     return fig
```

$T_b(p)$ for every frequency on a logarithmic scale, with the limb marked.

Chapter 13

common/figures.py — figures in the style of Moschou et al. (2018)

File: `radiosynth/common/figures.py` — 742 lines.

Plotting routines for publication figures: a synthetic map compared with an observation (Fig. 2 of the paper), a single-frequency map (Fig. 3) and a time sequence of CME images at several frequencies (Fig. 5), plus an animation writer. Coordinates can be shown in $R_\odot$ or helioprojective arcsec.

Module docstring and imports

lines 4–41

```
4  """Publication-style figures in the manner of Moschou et al. (2018).
5
6  * ``figure_compare_observation``  -- Fig. 2 style: synthetic map (log scale, white
7    log contours) next to an observed map (e.g. Nobeyama 17 GHz FITS) with the
8    synthetic contours and the limb overplotted, both in helioprojective arcsec.
9  * ``figure_single_map``           -- Fig. 3 style: one frequency, rainbow colour
10   map, white log-spaced contours, limb circle, frequency label.
11 * ``figure_time_series``          -- Fig. 5 style: rows = times, columns =
12   frequencies, log images with log-spaced (green) contours, time stamps and one
13   shared colour bar.  ``animate_time_series`` writes the animation (GIF/MP4).
14 * ``make_time_series``            -- run make_radio_image on a list of MHD
15   snapshots (files or grids) and keep the images.
16 * ``observer_for_date``, ``sun_ephemeris`` -- Earth's position (B0, L0, P,
17   distance) for a date (sunpy if installed, otherwise Meeus' algorithm).
18 * ``fit_longitude_offset``        -- find the longitude offset of a model
19   frame (e.g. a PLUTO run without solar coordinates) by correlating synthetic
20   maps with an observed map.
21
22 Coordinates
23 -----------
24 radiosynth images are in R_sun in the image plane through the Sun centre.
25 For an observer at distance D [R_sun] the helioprojective angle of a point at
26 image coordinate X is  Tx = atan(X / D)  (``rsun_to_arcsec``).  If the model is
27 in a heliographic frame (z = solar rotation axis; BATS-R-US SC/IH use HGR,
28 i.e. Carrington) and the observer is placed with ``earth_observer(B0, L0)``,
29 the image x axis points to solar West and y to solar North, the same
30 orientation as standard solar images (e.g. Nobeyama, MWA, LOFAR).
31 """
32 import math
33
34 import numpy as np
35
36 ARCSEC_PER_RAD = 180.0 / math.pi * 3600.0
37
38
39 # ---------------------------------------------------------------------------
40 # coordinates
41 # ---------------------------------------------------------------------------
```

Describes the four figure functions and the coordinate conventions.

L.36 Radians to arcseconds.

earth_observer() lines 42–55

```
42 def earth_observer(b0_deg=0.0, l0_deg=0.0, distance_rsun=215.03):
43     """Observer position in a heliographic (Carrington/Stonyhurst) model frame.
44 
45     b0_deg : heliographic latitude of the observer (solar B0 angle)
46     l0_deg : heliographic longitude of the observer (Carrington L0 for an
47              HGR/Carrington model, 0 for a Stonyhurst model)
48     With up=(0,0,1) the image then has solar North up and West to the right.
49     (sunpy: ``sunpy.coordinates.sun.B0(t)``, ``sunpy.coordinates.sun.L0(t)``,
50     ``sunpy.coordinates.get_earth(t).radius`` give the values for a date.)
51     """
52     b, l_ = math.radians(b0_deg), math.radians(l0_deg)
53     return (distance_rsun * math.cos(b) * math.cos(l_),
54             distance_rsun * math.cos(b) * math.sin(l_),
55             distance_rsun * math.sin(b))
```

Places the observer at a given heliographic latitude B0 and longitude L0. For a model in a heliographic frame with z along the solar rotation axis, the image then has solar North up and West to the right, the same as observed maps.

L.53 $\mathbf{O} = D(\cos B_0 \cos L_0,\ \cos B_0 \sin L_0,\ \sin B_0)$.

_julian_day() lines 58–69

```
58 def _julian_day(time):
59     """Julian day from a datetime, an ISO string ('2014-11-01T03:00') or a number
60     (taken as a Julian day).  UTC is used for TT (error < 0.01 deg in L0)."""
61     import datetime as _dt
62     if isinstance(time, (int, float, np.floating)):
63         return float(time)
64     if isinstance(time, str):
65         time = _dt.datetime.fromisoformat(time.replace("Z", "").replace(" ", "T"))
66     if time.tzinfo is not None:
67         time = time.astimezone(_dt.timezone.utc).replace(tzinfo=None)
68     j2000 = _dt.datetime(2000, 1, 1, 12, 0, 0)
69     return 2451545.0 + (time - j2000).total_seconds() / 86400.0
```

L.62–63 A number is taken as a Julian day.

L.64–67 An ISO string is parsed; a time-zone-aware datetime is converted to UTC.

L.69 Julian day = 2451545.0 (J2000.0, 2000 Jan 1 12:00) plus elapsed days. UTC is used in place of TT; the 69 s difference changes L_0 by less than 0.01°.

sun_ephemeris(): B0, L0, P and distance lines 72–118

```
72 def sun_ephemeris(time, use_sunpy=True):
73     """Solar B0, Carrington L0, position angle P [deg] and the Sun-Earth
```

```
    distance [R_sun] at ``time``.

    Uses sunpy if it is installed; otherwise the algorithm of Meeus
    (Astronomical Algorithms, 2nd ed., ch. 25 and 29), accurate to about
    0.01 deg, which is far below the resolution of radio images."""
    from .constants import AU, R_SUN
    if use_sunpy:
        try:
            from sunpy.coordinates import sun
            import astropy.units as u
            from astropy.time import Time
            t = Time(time) if not isinstance(time, (int, float)) else Time(time, format="jd")
            return dict(B0=sun.B0(t).deg, L0=sun.L0(t).deg, P=sun.P(t).deg,
                        distance_rsun=float(sun.earth_distance(t).to(u.m).value / R_SUN),
                        source="sunpy")
        except ImportError:
            pass
    jd = _julian_day(time)
    T = (jd - 2451545.0) / 36525.0
    rad = math.radians
    # Sun's apparent longitude (Meeus ch. 25, low precision)
    Ls = 280.46646 + 36000.76983 * T + 0.0003032 * T**2
    M = 357.52911 + 35999.05029 * T - 0.0001537 * T**2
    C = ((1.914602 - 0.004817 * T - 0.000014 * T**2) * math.sin(rad(M))
         + (0.019993 - 0.000101 * T) * math.sin(rad(2 * M)) + 0.000289 * math.sin(rad(3 * M))
    ↪ )
    true_long = Ls + C
    omega = 125.04 - 1934.136 * T
    lam = true_long - 0.00569 - 0.00478 * math.sin(rad(omega))      # apparent longitude
    e = 0.016708634 - 0.000042037 * T - 0.0000001267 * T**2
    R_au = 1.000001018 * (1 - e**2) / (1 + e * math.cos(rad(M + C)))
    eps = (23.0 + 26.0 / 60 + 21.448 / 3600 - (46.815 * T) / 3600
           + 0.00256 * math.cos(rad(omega)))                         # true obliquity
    # physical ephemeris (Meeus ch. 29)
    theta = (jd - 2398220.0) * 360.0 / 25.38
    I = 7.25
    K = 73.6667 + 1.3958333 * (jd - 2396758.0) / 36525.0
    lamp = lam                                          # nutation in longitude neglected
    ↪ (<0.005 deg)
    x = math.degrees(math.atan(-math.cos(rad(lamp)) * math.tan(rad(eps))))
    y = math.degrees(math.atan(-math.cos(rad(lam - K)) * math.tan(rad(I))))
    P = x + y
    B0 = math.degrees(math.asin(math.sin(rad(lam - K)) * math.sin(rad(I))))
    eta = math.degrees(math.atan2(-math.sin(rad(lam - K)) * math.cos(rad(I)),
                                  -math.cos(rad(lam - K))))
    L0 = (eta - theta) % 360.0
    return dict(B0=B0, L0=L0, P=P, distance_rsun=R_au * AU / R_SUN, source="Meeus")
```

L.80–90 Use sunpy if it is installed.

L.92 Julian centuries since J2000.0.

L.95–99 Sun's geometric longitude (Meeus ch. 25): mean longitude L, mean anomaly M, equation of centre C.

L.100–101 Apparent longitude λ, corrected for aberration and the main nutation term.

L.102–103 Eccentricity and Sun–Earth distance in AU.

L.104 True obliquity of the ecliptic.

L.107–109 Meeus ch. 29: rotation angle θ (Carrington rate, 25.38 d sidereal), inclination $I = 7.25^\circ$

of the solar equator, and longitude K of its ascending node.

L.111–113 Position angle P of the solar rotation axis, measured from celestial north. You need it to rotate RA/Dec images to solar-north-up.

L.114 $B_0 = \arcsin(\sin(\lambda - K)\sin I)$, the heliographic latitude of the disk centre.

L.115–117 $L_0 = \eta - \theta$, the Carrington longitude of the disk centre (the sub-Earth point).

L.118 Distance converted to $R_\odot$. Checked against Meeus' worked example 29.a to 0.02°.

observer_for_date()

lines 121–141

```
121 def observer_for_date(time, phi_offset_deg=0.0, frame="carrington", distance_rsun=None,
122                      verbose=True):
123     """Observer position (Earth) in the model frame for a given date.
124 
125     frame = 'carrington' : model longitude phi = Carrington longitude (data-driven
126                            runs from synoptic maps; BATS-R-US HGR).  Observer at
127                            longitude L0(time) + phi_offset_deg.
128     frame = 'stonyhurst' : model longitude 0 faces the Earth.  Observer at
129                            longitude phi_offset_deg.
130     phi_offset_deg : extra longitude shift, e.g. when the setup maps the
131                      magnetogram with an offset (find it with fit_longitude_offset).
132     The latitude is B0(time); the distance is the true Sun-Earth distance
133     unless distance_rsun is given.  Use with up=(0, 0, 1) (solar north = +z)."""
134     eph = sun_ephemeris(time)
135     lon = (eph["L0"] if frame.lower().startswith("carr") else 0.0) + phi_offset_deg
136     D = distance_rsun or eph["distance_rsun"]
137     if verbose:
138         print(f"radiosynth: {time}: B0 = {eph['B0']:+.3f} deg, L0 = {eph['L0']:.3f} deg, "
139               f"P = {eph['P']:+.2f} deg, D = {eph['distance_rsun']:.2f} R_sun ({eph['source
    ↪ ']}); "
140               f"observer longitude in model = {lon % 360:.2f} deg")
141     return earth_observer(eph["B0"], lon, D)
```

Places the observer at the Earth's position for a date, in the model frame.

L.135 Carrington frame: longitude L_0+offset. Stonyhurst frame (model longitude 0 faces the Earth): just the offset.

L.141 Observer at latitude B_0, that longitude, and the true distance (or the given one).

rsun_to_arcsec(), image_extent(), limb_radius()

lines 144–180

```
144 def rsun_to_arcsec(x_rsun, distance_rsun):
145     """Image-plane coordinate [R_sun] -> helioprojective angle [arcsec]."""
146     return np.arctan(np.asarray(x_rsun, float) / distance_rsun) * ARCSEC_PER_RAD
147 
148 
149 def _dist(img):
150     """Observer distance in coordinate units (true distance for stellar images)."""
151     return float(getattr(img, "distance", np.linalg.norm(img.observer)))
152 
153 
154 def _unit_label(img, units):
155     if units == "arcsec":
156         return "[arcsec]"
```

```
157     if units == "mas":
158         return "[mas]"
159     from .constants import R_SUN
160     L = float(getattr(img, "length_unit", R_SUN))
161     return r"[$R_\odot$]" if abs(L / R_SUN - 1) < 1e-6 else r"[$R_\ast$]"
162 
163 
164 def image_extent(img, units="rsun"):
165     """[left, right, bottom, top] of the pixel edges in coordinate units ('rsun':
166     R_sun or R_star), 'arcsec' or 'mas' (milliarcsec, for stars / VLBI)."""
167     dx = img.x[1] - img.x[0]
168     dy = img.y[1] - img.y[0]
169     e = np.array([img.x[0] - dx / 2, img.x[-1] + dx / 2, img.y[0] - dy / 2, img.y[-1] + dy /
        ↪ 2])
170     if units in ("arcsec", "mas"):
171         e = rsun_to_arcsec(e, _dist(img)) * (1000.0 if units == "mas" else 1.0)
172     return list(e)
173 
174 
175 def limb_radius(img, units="rsun"):
176     """Apparent photospheric radius in the chosen units."""
177     D = _dist(img)
178     if units == "rsun":
179         return 1.0
180     return math.asin(1.0 / D) * ARCSEC_PER_RAD * (1000.0 if units == "mas" else 1.0)
```

L.146 Helioprojective angle $T_x = \arctan(X/D)$ of an image-plane coordinate.

L.164–172 Pixel-edge extent of a RadioImage, for `imshow`, in $R_\odot$ or arcsec.

L.175–180 Apparent radius of the photosphere: 1 $R_\odot$, or $\arcsin(1/D)$ in arcsec (about 960” at 1 AU).

Helper functions

lines 183–234

```
183 def _freq_index(img, freq):
184     if freq is None:
185         return 0
186     if isinstance(freq, (int, np.integer)) and freq < len(img.freqs):
187         return int(freq)
188     from .image import parse_frequencies
189     f = parse_frequencies(freq)[0] if isinstance(freq, str) else float(freq)
190     return int(np.argmin(np.abs(img.freqs - f)))
191 
192 
193 def _data(img, k, quantity):
194     return (img.brightness_temperature if quantity == "tb" else img.intensity)[k]
195 
196 
197 def _label(quantity):
198     return (r"$T_b$ [K]" if quantity == "tb"
199             else r"$I_\nu$ [W m$^{-2}$ Hz$^{-1}$ sr$^{-1}$]")
200 
201 
202 def smooth_to_beam(data, pixel_arcsec, fwhm_arcsec):
203     """Convolve a map with a circular Gaussian beam (FWHM in arcsec)."""
204     from scipy.ndimage import gaussian_filter
```

```
205     if not fwhm_arcsec:
206         return data
207     sig = fwhm_arcsec / (2.0 * math.sqrt(2.0 * math.log(2.0))) / pixel_arcsec
208     return gaussian_filter(data, sig)
209
210
211 def log_levels(data, n=10, decades=None, vmin=None, vmax=None):
212     """n logarithmically spaced contour levels between vmin and vmax
213     (default: the positive range of data, limited to ``decades`` if given)."""
214     pos = data[np.isfinite(data) & (data > 0)]
215     if pos.size == 0:
216         return np.array([])
217     hi = vmax if vmax is not None else pos.max()
218     lo = vmin if vmin is not None else (hi * 10.0**(-decades) if decades else pos.min())
219     return np.geomspace(lo, hi, n + 1)[:-1] if hi > lo else np.array([lo])
220
221
222 def decade_levels(data, lo_exp=None, hi_exp=None):
223     """Contours at every power of ten inside the data range (Moschou Fig. 2)."""
224     pos = data[np.isfinite(data) & (data > 0)]
225     if pos.size == 0:
226         return np.array([])
227     a = math.ceil(math.log10(pos.min())) if lo_exp is None else lo_exp
228     b = math.floor(math.log10(pos.max())) if hi_exp is None else hi_exp
229     return 10.0**np.arange(a, b + 1)
230
231
232 # ---------------------------------------------------------------------------
233 # observations
234 # ---------------------------------------------------------------------------
```

L.183 Select a frequency by index, by value in Hz, or by a string such as '125MHz' (nearest frequency in the image).

L.193 Intensity or brightness temperature for one frequency.

L.202 Convolve with a circular Gaussian beam: $\sigma = \mathrm{FWHM}/(2\sqrt{2\ln 2})$ in pixels. Use this to compare with an instrument of finite resolution.

L.211 n logarithmically spaced contour levels below the maximum, over a chosen number of decades.

L.222 One contour per power of ten, as in Fig. 2 of the paper.

read_fits_map(): observed FITS maps

lines 235–265

```
235 def read_fits_map(path, hdu=0):
236     """Read a solar FITS image (e.g. Nobeyama NoRH, MWA, LOFAR) and return
237     (data, extent_arcsec) assuming a helioprojective, solar-north-up image with a
238     linear WCS (CRPIX/CRVAL/CDELT in arcsec; CROTA/rotation is applied if present).
239
240     For maps that are not north-up, use sunpy:  ``m = sunpy.map.Map(path).rotate()``
241     and pass ``(m.data, [Tx0, Tx1, Ty0, Ty1])`` instead."""
242     from .fitsio import read_fits
243     data, h = read_fits(path, hdu)
244     data = np.squeeze(data)
245     ny, nx = data.shape[-2:]
246     scale = {"deg": 3600.0, "arcmin": 60.0, "arcsec": 1.0}
```

```
247
248     def ax(i, n):
249         cu = str(h.get(f"CUNIT{i}", "arcsec")).strip().lower()
250         f = scale.get(cu, 1.0)
251         crpix, crval = h.get(f"CRPIX{i}", (n + 1) / 2.0), h.get(f"CRVAL{i}", 0.0)
252         cdelt = h.get(f"CDELT{i}", 1.0)
253         return (np.array([0.5, n + 0.5]) - crpix) * cdelt * f + crval * f
254
255     ex, ey = ax(1, nx), ax(2, ny)
256     rot = float(h.get("CROTA2", h.get("CROTA", 0.0)) or 0.0)
257     if abs(rot) > 0.5:
258         from scipy.ndimage import rotate
259         data = rotate(data, -rot, reshape=False, order=1, mode="nearest")
260     return data, [ex[0], ex[1], ey[0], ey[1]]
261
262
263 # ---------------------------------------------------------------------------
264 # Fig. 3 style
265 # ---------------------------------------------------------------------------
```

L.242 Uses astropy when installed, otherwise the small built-in reader.

L.248–253 Extent of each axis from the linear WCS keywords: (pixel edge − CRPIX) CDELT + CRVAL, converted to arcsec using CUNIT.

L.256–259 If the map is rotated (CROTA2), rotate it so that solar North is up.

figure_single_map(): Fig. 3 style

lines 266–323

```
266 def figure_single_map(img, freq=None, quantity="intensity", units="rsun", cmap="jet",
267                      log=False, n_contours=8, contour_decades=4, contour_color="white",
268                      limb_color="black", beam_fwhm_arcsec=None, ax=None, path=None,
269                      vmin=None, vmax=None, title=None):
270     """One-frequency map in the style of Moschou et al. (2018) Fig. 3: rainbow
271     colour map (linear by default), white log-spaced intensity contours, the
272     photospheric limb and an 'f = ... MHz' label."""
273     import matplotlib
274     if path is not None and ax is None:
275         matplotlib.use("Agg")
276     import matplotlib.pyplot as plt
277     from matplotlib.colors import LogNorm
278     from .image import freq_label
279     k = _freq_index(img, freq)
280     d = _data(img, k, quantity).copy()
281     ext = image_extent(img, units)
282     if beam_fwhm_arcsec:
283         pix = rsun_to_arcsec(img.x[1] - img.x[0], _dist(img))
284         d = smooth_to_beam(d, pix, beam_fwhm_arcsec)
285     own = ax is None
286     if own:
287         fig, ax = plt.subplots(figsize=(6.2, 5.2))
288     else:
289         fig = ax.figure
290     pos = d[d > 0]
291     if log and pos.size:
292         hi = vmax or pos.max()
293         norm = LogNorm(vmin=vmin or hi * 1e-4, vmax=hi)
```

```
294         im = ax.imshow(np.where(d > 0, d, np.nan), origin="lower", extent=ext, cmap=cmap,
    ↪ norm=norm)
295     else:
296         im = ax.imshow(d, origin="lower", extent=ext, cmap=cmap, vmin=vmin, vmax=vmax)
297     lev = log_levels(d, n_contours, contour_decades)
298     if lev.size:
299         X = np.linspace(ext[0], ext[1], d.shape[1])
300         Y = np.linspace(ext[2], ext[3], d.shape[0])
301         ax.contour(X, Y, d, levels=lev, colors=contour_color, linewidths=0.8)
302     t = np.linspace(0, 2 * np.pi, 400)
303     R = limb_radius(img, units)
304     ax.plot(R * np.cos(t), R * np.sin(t), color=limb_color, lw=1.4)
305     ax.text(0.03, 0.97, title or f"f = {freq_label(img.freqs[k])}", transform=ax.transAxes,
306             ha="left", va="top", color="white", fontsize=13, fontweight="bold",
307             bbox=dict(facecolor="black", alpha=0.35, edgecolor="none"))
308     un = _unit_label(img, units)
309     ax.set_xlabel(f"X {un}")
310     ax.set_ylabel(f"Y {un}")
311     ax.set_aspect("equal")
312     fig.colorbar(im, ax=ax, label=_label(quantity), shrink=0.9)
313     if own:
314         fig.tight_layout()
315         if path is not None:
316             fig.savefig(path, dpi=150)
317             plt.close(fig)
318     return fig
319
320
321 # ---------------------------------------------------------------------------
322 # Fig. 2 style
323 # ---------------------------------------------------------------------------
```

L.279–284 Select the frequency and optionally smooth to a beam.

L.291–296 Linear colour scale by default (as in the paper), logarithmic if `log=True`.

L.297–301 White log-spaced contours.

L.303–304 Limb circle.

L.305 The 'f = ... MHz' label in the upper-left corner.

figure_compare_observation(): Fig. 2 style

lines 324–398

```
324 def figure_compare_observation(img, obs, obs_extent=None, freq=None, quantity="intensity",
325                               syn_cmap="YlOrRd_r", obs_cmap="hot", beam_fwhm_arcsec=None,
326                               levels="decades", n_contours=6, contour_color="white",
327                               syn_limb_color="black", obs_limb_color="lime",
328                               fov_arcsec=None, obs_label="Observation", path=None,
329                               obs_vmin=None, obs_vmax=None):
330     """Synthetic map next to an observed map, with the synthetic contours drawn
331     on the observation (Moschou et al. 2018, Fig. 2).
332
333     obs        : path to a FITS file, or a 2D array (then give obs_extent =
334                  [Tx_min, Tx_max, Ty_min, Ty_max] in arcsec, solar north up)
335     levels     : 'decades' (one contour per power of ten), 'log' (n_contours
336                  log-spaced levels) or an explicit list of levels
337     beam_fwhm_arcsec : smooth the synthetic map to the instrument beam before
338                  contouring (e.g. ~10 arcsec for NoRH at 17 GHz)
```

```
    """
    import matplotlib
    if path is not None:
        matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    from matplotlib.colors import LogNorm
    from .image import freq_label
    k = _freq_index(img, freq)
    syn = _data(img, k, quantity).copy()
    ext = image_extent(img, "arcsec")
    pix = rsun_to_arcsec(img.x[1] - img.x[0], _dist(img))
    syn_s = smooth_to_beam(syn, pix, beam_fwhm_arcsec)
    if isinstance(obs, str):
        odata, oext = read_fits_map(obs)
    else:
        odata, oext = np.asarray(obs, float), list(obs_extent)
    if isinstance(levels, str):
        lev = decade_levels(syn_s) if levels == "decades" else log_levels(syn_s, n_contours,
    ↪ 5)
    else:
        lev = np.asarray(levels)
    X = np.linspace(ext[0], ext[1], syn.shape[1])
    Y = np.linspace(ext[2], ext[3], syn.shape[0])
    R = limb_radius(img, "arcsec")
    t = np.linspace(0, 2 * np.pi, 400)

    fig, axs = plt.subplots(1, 2, figsize=(12.5, 5.8))
    pos = syn_s[syn_s > 0]
    norm = LogNorm(vmin=max(pos.min(), pos.max() * 1e-5), vmax=pos.max()) if pos.size else
    ↪ None
    im0 = axs[0].imshow(np.where(syn_s > 0, syn_s, np.nan), origin="lower", extent=ext,
                        cmap=syn_cmap, norm=norm)
    axs[0].contour(X, Y, syn_s, levels=lev, colors=contour_color, linewidths=0.8)
    axs[0].plot(R * np.cos(t), R * np.sin(t), color=syn_limb_color, lw=1.4)
    axs[0].set_title(f"Synthetic, {freq_label(img.freqs[k])}")
    cb = fig.colorbar(im0, ax=axs[0], shrink=0.88, label=_label(quantity))
    for lv in lev:
        cb.ax.axhline(lv, color=contour_color, lw=0.8)

    im1 = axs[1].imshow(odata, origin="lower", extent=oext, cmap=obs_cmap,
                        vmin=obs_vmin, vmax=obs_vmax)
    axs[1].contour(X, Y, syn_s, levels=lev, colors=contour_color, linewidths=0.8)
    axs[1].plot(R * np.cos(t), R * np.sin(t), color=obs_limb_color, lw=1.4)
    axs[1].set_title(f"{obs_label} with synthetic contours")
    fig.colorbar(im1, ax=axs[1], shrink=0.88, label="observed (instrument units)")
    lim = fov_arcsec or max(abs(ext[0]), abs(ext[1]))
    for a in axs:
        a.set_xlim(-lim, lim)
        a.set_ylim(-lim, lim)
        a.set_aspect("equal")
        a.set_xlabel("Solar X [arcsec]")
        a.set_ylabel("Solar Y [arcsec]")
    fig.tight_layout()
    if path is not None:
        fig.savefig(path, dpi=150)
        plt.close(fig)
    return fig

```

```
395
396 # ---------------------------------------------------------------------------
397 # Fig. 5 style
398 # ---------------------------------------------------------------------------
```

Left: the synthetic map on a log scale with its contours. Right: the observation with the same synthetic contours and the limb overplotted.

L.347–350 Synthetic map in arcsec, smoothed to the instrument beam if requested.
L.351–354 The observation is a FITS file or an array with its extent.
L.355–358 Contour levels: one per decade, log-spaced, or given explicitly.
L.367–374 Synthetic panel; the contour levels are also marked on its colour bar, as in the paper.
L.376–379 Observation panel with the synthetic contours and a green limb.
L.382–388 Common field of view and axis labels.

make_time_series()

lines 399–423

```
399 def make_time_series(snapshots, frequencies, observer=(-215.0, 0.0, 0.0), x_size=6.0,
400                     y_size=6.0, nx=128, ny=128, loader=None, save_prefix=None, **kw):
401     """Make one RadioImage per MHD snapshot.
402
403     snapshots : list (or iterable) of file names / PLUTO directories (read
404                 with radiosynth.load_mhd) or of grids; with ``loader`` given,
405                 any items, each turned into a grid by ``loader(item)`` (e.g.
406                 PLUTO output numbers: loader=lambda n: load_pluto(d, nout=n))
407     save_prefix : if given, each image is saved as f'{save_prefix}_{i:03d}.npz'
408     Remaining keywords go to make_radio_image (transfer, n_workers, ...)."""
409     from .image import make_radio_image
410     from .readers import load_mhd
411     out = []
412     for i, s in enumerate(snapshots):
413         if loader is not None:
414             g = loader(s)
415         elif isinstance(s, str):
416             g = load_mhd(s)
417         else:
418             g = s
419         img = make_radio_image(g, frequencies, observer, x_size, y_size, nx, ny, **kw)
420         if save_prefix:
421             img.save_npz(f"{save_prefix}_{i:03d}.npz")
422         out.append(img)
423     return out
```

L.412–422 For each snapshot (file, PLUTO directory, or grid): load it, make the image with the same settings and optionally save it. The result is a list of RadioImages ordered in time.

figure_time_series(): Fig. 5 style

lines 426–485

```
426 def figure_time_series(images, times=None, freqs=None, quantity="intensity", units="rsun",
427                       cmap="gray", n_contours=10, contour_decades=6, contour_color="lime",
428                       vmin=None, vmax=None, time_fmt="Time: {:.1f} min", fov=None,
```

```
                      limb=False, path=None, panel_size=3.2):
    """Grid of images, rows = times, columns = frequencies, common logarithmic
    colour scale, n log-spaced contours per panel (Moschou et al. 2018, Fig. 5).

    images : list of RadioImage (one per time), all with the same frequencies
    times  : list of times (labels) or None
    freqs  : frequencies to show (strings, Hz or indices); default all
    """
    import matplotlib
    if path is not None:
        matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    from matplotlib.colors import LogNorm
    from .image import freq_label
    ref = images[0]
    ks = list(range(len(ref.freqs))) if freqs is None else [_freq_index(ref, f) for f in
                                                            (freqs if isinstance(freqs, (
    ↪ list, tuple)) else [freqs])]
    nr, nc = len(images), len(ks)
    alld = np.concatenate([_data(im, k, quantity).ravel() for im in images for k in ks])
    pos = alld[alld > 0]
    hi = vmax or pos.max()
    lo = vmin or hi * 10.0**(-contour_decades)
    norm = LogNorm(vmin=lo, vmax=hi)
    lev = np.geomspace(lo, hi, n_contours + 1)[:-1]
    fig, axs = plt.subplots(nr, nc, figsize=(panel_size * nc + 1.2, panel_size * nr),
                            squeeze=False, gridspec_kw=dict(wspace=0.02, hspace=0.02))
    for r, im in enumerate(images):
        ext = image_extent(im, units)
        for c, k in enumerate(ks):
            a = axs[r, c]
            d = _data(im, k, quantity)
            h = a.imshow(np.where(d > 0, d, np.nan), origin="lower", extent=ext, cmap=cmap,
    ↪ norm=norm)
            a.set_facecolor("black")
            X = np.linspace(ext[0], ext[1], d.shape[1])
            Y = np.linspace(ext[2], ext[3], d.shape[0])
            a.contour(X, Y, d, levels=lev, colors=contour_color, linewidths=0.6)
            if limb:
                tt = np.linspace(0, 2 * np.pi, 300)
                R = limb_radius(im, units)
                a.plot(R * np.cos(tt), R * np.sin(tt), "w--", lw=0.6)
            a.text(0.5, 0.97, freq_label(im.freqs[k]), transform=a.transAxes, ha="center",
                   va="top", color="white", fontsize=10, fontweight="bold")
            if c == 0 and times is not None:
                a.text(0.04, 0.04, time_fmt.format(times[r]), transform=a.transAxes,
                       color="white", fontsize=10)
            if fov:
                a.set_xlim(-fov, fov)
                a.set_ylim(-fov, fov)
            a.set_xticks([])
            a.set_yticks([])
    cb = fig.colorbar(h, ax=axs, shrink=0.9, pad=0.01, label=_label(quantity))
    for lv in lev:
        cb.ax.axhline(lv, color=contour_color, lw=0.6)
    if path is not None:
        fig.savefig(path, dpi=140, bbox_inches="tight")
        plt.close(fig)
```

```
485     return fig
```

L.444 Frequencies to show (columns).

L.447–452 One logarithmic colour scale and one set of contour levels for all panels, so the panels can be compared directly.

L.455–478 Rows are times and columns frequencies: image, contours, frequency title, time stamp in the first column.

L.479–481 Shared colour bar with the contour levels marked.

animate_time_series()

lines 488–532

```
488 def animate_time_series(images, path, times=None, freq=None, quantity="intensity",
489                        units="rsun", cmap="gray", n_contours=10, contour_decades=6,
490                        contour_color="lime", fps=4, time_fmt="Time: {:.1f} min"):
491     """Write an animation (GIF via Pillow, or MP4 if path ends in .mp4 and ffmpeg
492     is installed) of one frequency through a list of RadioImages."""
493     import matplotlib
494     matplotlib.use("Agg")
495     import matplotlib.pyplot as plt
496     from matplotlib import animation
497     from matplotlib.colors import LogNorm
498     from .image import freq_label
499     k = _freq_index(images[0], freq)
500     alld = np.concatenate([_data(im, k, quantity).ravel() for im in images])
501     hi = alld.max()
502     lo = hi * 10.0**(-contour_decades)
503     lev = np.geomspace(lo, hi, n_contours + 1)[:-1]
504     fig, ax = plt.subplots(figsize=(5.5, 5))
505     ext = image_extent(images[0], units)
506 
507     def draw(i):
508         ax.clear()
509         d = _data(images[i], k, quantity)
510         ax.imshow(np.where(d > 0, d, np.nan), origin="lower", extent=ext, cmap=cmap,
511                   norm=LogNorm(vmin=lo, vmax=hi))
512         ax.set_facecolor("black")
513         X = np.linspace(ext[0], ext[1], d.shape[1])
514         Y = np.linspace(ext[2], ext[3], d.shape[0])
515         ax.contour(X, Y, d, levels=lev, colors=contour_color, linewidths=0.6)
516         ax.set_title(freq_label(images[i].freqs[k]))
517         if times is not None:
518             ax.text(0.04, 0.04, time_fmt.format(times[i]), transform=ax.transAxes, color="
    ↪ white")
519         ax.set_xticks([])
520         ax.set_yticks([])
521         return []
522 
523     ani = animation.FuncAnimation(fig, draw, frames=len(images), blit=False)
524     writer = animation.FFMpegWriter(fps=fps) if path.endswith(".mp4") else animation.
    ↪ PillowWriter(fps=fps)
525     ani.save(path, writer=writer)
526     plt.close(fig)
527     return path
528 
529 
```

```
# ---------------------------------------------------------------------------
# orientation of models without solar coordinates
# ---------------------------------------------------------------------------
```

L.507–521 Draw one frame: image, contours, frequency and time.

L.523–525 Write a GIF with Pillow, or an MP4 with ffmpeg if the file name ends in .mp4.

_regrid_to_obs()

lines 533–545

```
def _regrid_to_obs(img, k, quantity, beam_fwhm_arcsec, tx, ty):
    """Synthetic map (optionally beam-smoothed) interpolated to observed pixel
    centres tx, ty [arcsec]."""
    from scipy.interpolate import RegularGridInterpolator
    D = _dist(img)
    d = _data(img, k, quantity)
    xs = rsun_to_arcsec(img.x, D)
    ys = rsun_to_arcsec(img.y, D)
    if beam_fwhm_arcsec:
        d = smooth_to_beam(d, xs[1] - xs[0], beam_fwhm_arcsec)
    f = RegularGridInterpolator((ys, xs), d, bounds_error=False, fill_value=np.nan)
    TY, TX = np.meshgrid(ty, tx, indexing="ij")
    return f(np.column_stack([TY.ravel(), TX.ravel()])).reshape(TY.shape)
```

L.539–542 Synthetic pixel axes in arcsec; smooth to the beam.

L.543–545 Bilinear interpolation onto the observed pixel centres; NaN outside the synthetic field of view.

fit_longitude_offset()

lines 548–624

```
def fit_longitude_offset(grid, obs, frequency, b0_deg, l0_deg, offsets=None, obs_extent=None,
                         x_size=2.6, y_size=None, npix=128, distance_rsun=215.0,
                         up=(0.0, 0.0, 1.0), beam_fwhm_arcsec=None, quantity="tb",
                         use_log=True, r_mask=1.1, refine=True, n_workers=1,
                         plot_path=None, verbose=True, **image_kw):
    """Find the model longitude offset that best matches an observed map.

    For each trial offset the observer is placed at heliographic latitude b0_deg and
    longitude l0_deg + offset, a synthetic map is computed, smoothed to the beam,
    interpolated onto the observed pixels and correlated with the observation
    (Pearson correlation of log10 values inside r < r_mask apparent solar radii).

    obs        : FITS file or 2D array (then give obs_extent in arcsec), solar north up
    offsets    : trial offsets [deg]; default -60..60 in 5-deg steps
    refine     : refine the best offset with a parabola through the three best points
    Returns dict(offsets, corr, best, best_refined, image_best).
    Then use earth_observer(b0_deg, l0_deg + best_refined) (or observer_for_date(...,
    phi_offset_deg=best_refined)) for all other frequencies and times."""
    from .image import make_radio_image
    if offsets is None:
        offsets = np.arange(-60.0, 60.1, 5.0)
    offsets = np.asarray(offsets, float)
    if isinstance(obs, str):
        odata, oext = read_fits_map(obs)
```

```
572     else:
573         odata, oext = np.asarray(obs, float), list(obs_extent)
574     ny, nx = odata.shape
575     tx = oext[0] + (np.arange(nx) + 0.5) * (oext[1] - oext[0]) / nx
576     ty = oext[2] + (np.arange(ny) + 0.5) * (oext[3] - oext[2]) / ny
577     TY, TX = np.meshgrid(ty, tx, indexing="ij")
578     rlimb = math.asin(1.0 / distance_rsun) * ARCSEC_PER_RAD
579     mask = (np.hypot(TX, TY) < r_mask * rlimb) & np.isfinite(odata)
580     corr = np.full(offsets.size, np.nan)
581     best_img, best_c = None, -np.inf
582     for i, off in enumerate(offsets):
583         o = earth_observer(b0_deg, l0_deg + off, distance_rsun)
584         img = make_radio_image(grid, frequency, o, x_size, y_size or x_size, npix, npix,
585                                up=up, n_workers=n_workers, verbose=False, **image_kw)
586         syn = _regrid_to_obs(img, 0, quantity, beam_fwhm_arcsec, tx, ty)
587         m = mask & np.isfinite(syn)
588         a, b = syn[m], odata[m]
589         if use_log:
590             good = (a > 0) & (b > 0)
591             a, b = np.log10(a[good]), np.log10(b[good])
592         if a.size > 10:
593             corr[i] = np.corrcoef(a, b)[0, 1]
594         if verbose:
595             print(f"  offset {off:+7.2f} deg: correlation {corr[i]:.4f}")
596         if np.isfinite(corr[i]) and corr[i] > best_c:
597             best_c, best_img = corr[i], img
598     ib = int(np.nanargmax(corr))
599     best = float(offsets[ib])
600     best_ref = best
601     if refine and 0 < ib < offsets.size - 1 and np.all(np.isfinite(corr[ib - 1:ib + 2])):
602         x3, y3 = offsets[ib - 1:ib + 2], corr[ib - 1:ib + 2]
603         a2, a1, _ = np.polyfit(x3, y3, 2)
604         if a2 < 0:
605             best_ref = float(np.clip(-a1 / (2 * a2), x3[0], x3[-1]))
606     if verbose:
607         print(f"radiosynth: best longitude offset {best:+.2f} deg (refined {best_ref:+.2f}
    ↪ deg), "
608               f"correlation {corr[ib]:.4f}")
609     if plot_path is not None:
610         import matplotlib
611         matplotlib.use("Agg")
612         import matplotlib.pyplot as plt
613         fig, ax = plt.subplots(figsize=(5.5, 3.8))
614         ax.plot(offsets, corr, "o-")
615         ax.axvline(best_ref, color="C3", ls="--", label=f"best {best_ref:+.1f} deg")
616         ax.set_xlabel("longitude offset [deg]")
617         ax.set_ylabel("correlation with observation")
618         ax.legend()
619         ax.grid(alpha=0.3)
620         fig.tight_layout()
621         fig.savefig(plot_path, dpi=140)
622         plt.close(fig)
623     return dict(offsets=offsets, corr=corr, best=best, best_refined=best_ref,
624                 image_best=best_img)
```

L.567–573 Trial offsets (default -60° to 60° in 5° steps) and the observed map.

L.575–579 Observed pixel centres and the comparison region $r < $ `r_mask` apparent solar radii.

L.582–586 For each offset: move the observer in longitude, make the synthetic map, and put it on the observed pixels.

L.589–593 Pearson correlation of the logarithms. It is insensitive to the absolute calibration of the observation and gives faint and bright structures comparable weight.

L.598–605 Best grid offset, refined by the vertex of a parabola through the best point and its two neighbours.

L.609–622 Optional plot of correlation against offset.

compare_with_observation()

lines 627–742

```
627 def compare_with_observation(img, obs, freq=None, quantity="tb", beam_fwhm_arcsec=None,
628                             obs_extent=None, r_max=3.0, n_bins=30, sector_halfwidth=30.0,
629                             plot_path=None):
630     """Quantitative comparison of a synthetic map with an observed map.
631
632     The synthetic map is smoothed to the beam and interpolated onto the observed
633     pixels (helioprojective arcsec, solar north up).  Returns a dict with
634       syn, obs, tx, ty, r     : maps on the observed pixels; r in apparent solar radii
635       mean_disk_obs/syn       : mean value for r < 1
636       mean_limb_obs/syn       : mean value for 1 <= r < 1.3
637       corr_log                : Pearson correlation of log10 values for r < 1.3
638       ratio_disk              : mean_disk_syn / mean_disk_obs
639       r_bins, prof_obs, prof_syn : azimuthally averaged radial profiles
640       r_half_obs/syn          : outermost radius where the profile is >= half its
641                                 maximum (size of the radio Sun)
642       prof_sector_obs/syn     : radial profiles in the N, E, S, W sectors
643                                 (+- sector_halfwidth deg of position angle)
644       r_half_sector_obs/syn   : radio-Sun size in each sector
645       elongation_obs/syn      : equatorial (E,W) / polar (N,S) size; the main
646                                 measure for axisymmetric (2.5D) models
647     plot_path : optional figure with both radial profiles and the residual map.
648     """
649     k = _freq_index(img, freq)
650     if isinstance(obs, str):
651         odata, oext = read_fits_map(obs)
652     else:
653         odata, oext = np.asarray(obs, float), list(obs_extent)
654     ny, nx = odata.shape
655     tx = oext[0] + (np.arange(nx) + 0.5) * (oext[1] - oext[0]) / nx
656     ty = oext[2] + (np.arange(ny) + 0.5) * (oext[3] - oext[2]) / ny
657     syn = _regrid_to_obs(img, k, quantity, beam_fwhm_arcsec, tx, ty)
658     TY, TX = np.meshgrid(ty, tx, indexing="ij")
659     rlimb = limb_radius(img, "arcsec")
660     r = np.hypot(TX, TY) / rlimb
661     ok = np.isfinite(odata) & np.isfinite(syn)
662     disk = ok & (r < 1.0)
663     limb = ok & (r >= 1.0) & (r < 1.3)
664     inner = ok & (r < 1.3) & (odata > 0) & (syn > 0)
665     edges = np.linspace(0.0, r_max, n_bins + 1)
666
667     def prof(m):
668         return np.array([np.nanmean(m[ok & (r >= a) & (r < b)]) if np.any(ok & (r >= a) & (r
    ↪ < b))
669                          else np.nan for a, b in zip(edges[:-1], edges[1:])])
670
```

```
    def r_half(p):
        """Outermost radius where the profile is still >= half its maximum
        (size of the radio Sun; robust to limb brightening and central dips)."""
        rc = 0.5 * (edges[1:] + edges[:-1])
        half = 0.5 * np.nanmax(p)
        above = np.where(p >= half)[0]
        if above.size == 0 or above[-1] + 1 >= p.size:
            return np.nan
        j = above[-1]
        return float(np.interp(half, [p[j + 1], p[j]], [rc[j + 1], rc[j]]))

    po, ps = prof(odata), prof(syn)
    # position angle from solar north, counter-clockwise (towards East = -X)
    pa = np.degrees(np.arctan2(-TX, TY)) % 360.0
    sectors = {"N": 0.0, "E": 90.0, "S": 180.0, "W": 270.0}

    def prof_sector(m, c):
        dpa = np.abs((pa - c + 180.0) % 360.0 - 180.0)
        sel = ok & (dpa <= sector_halfwidth)
        return np.array([np.nanmean(m[sel & (r >= a) & (r < b)]) if np.any(sel & (r >= a) & (
    ↪ r < b))
                         else np.nan for a, b in zip(edges[:-1], edges[1:])])
    res = dict(syn=syn, obs=odata, tx=tx, ty=ty, r=r,
               mean_disk_obs=float(np.nanmean(odata[disk])), mean_disk_syn=float(np.nanmean(
    ↪ syn[disk])),
               mean_limb_obs=float(np.nanmean(odata[limb])), mean_limb_syn=float(np.nanmean(
    ↪ syn[limb])),
               corr_log=float(np.corrcoef(np.log10(odata[inner]), np.log10(syn[inner]))[0,
    ↪ 1])
               if inner.sum() > 10 else np.nan,
               r_bins=0.5 * (edges[1:] + edges[:-1]), prof_obs=po, prof_syn=ps,
               r_half_obs=r_half(po), r_half_syn=r_half(ps))
    res["ratio_disk"] = res["mean_disk_syn"] / res["mean_disk_obs"]
    for tag, m in (("obs", odata), ("syn", syn)):
        sp = {k_: prof_sector(m, c) for k_, c in sectors.items()}
        rh = {k_: r_half(v) for k_, v in sp.items()}
        res[f"prof_sector_{tag}"] = sp
        res[f"r_half_sector_{tag}"] = rh
        res[f"elongation_{tag}"] = (0.5 * (rh["E"] + rh["W"])) / (0.5 * (rh["N"] + rh["S"]))
    if plot_path is not None:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        fig, axs = plt.subplots(1, 3, figsize=(16, 4.4))
        rb = res["r_bins"]
        for tag, ls in (("obs", "-"), ("syn", "--")):
            sp = res[f"prof_sector_{tag}"]
            axs[2].plot(rb, 0.5 * (sp["E"] + sp["W"]), "C0" + ls, label=f"equator ({tag})")
            axs[2].plot(rb, 0.5 * (sp["N"] + sp["S"]), "C3" + ls, label=f"poles ({tag})")
        axs[2].set_xlabel(r"r [apparent $R_\odot$]")
        axs[2].set_ylabel(_label(quantity))
        axs[2].set_title(f"equator vs poles (elongation obs {res['elongation_obs']:.2f}, "
                         f"syn {res['elongation_syn']:.2f})", fontsize=10)
        axs[2].legend(fontsize=8)
        axs[2].grid(alpha=0.3)
        axs[0].plot(res["r_bins"], po, "o-", label="observation")
        axs[0].plot(res["r_bins"], ps, "s-", label="synthetic")
        axs[0].set_xlabel(r"r [apparent $R_\odot$]")
```

```
725         axs[0].set_ylabel(_label(quantity))
726         axs[0].legend()
727         axs[0].grid(alpha=0.3)
728         axs[0].set_title(f"radial profiles (r_half: obs {res['r_half_obs']:.2f}, "
729                          f"syn {res['r_half_syn']:.2f})", fontsize=10)
730         resid = syn - odata
731         v = np.nanpercentile(np.abs(resid), 99)
732         im = axs[1].imshow(resid, origin="lower", extent=oext, cmap="RdBu_r", vmin=-v, vmax=v
    ↪ )
733         t = np.linspace(0, 2 * np.pi, 300)
734         axs[1].plot(rlimb * np.cos(t), rlimb * np.sin(t), "k-", lw=0.8)
735         axs[1].set_title(f"synthetic - observed (corr_log {res['corr_log']:.3f})", fontsize
    ↪ =10)
736         axs[1].set_xlabel("Solar X [arcsec]")
737         axs[1].set_ylabel("Solar Y [arcsec]")
738         fig.colorbar(im, ax=axs[1], shrink=0.85)
739         fig.tight_layout()
740         fig.savefig(plot_path, dpi=140)
741         plt.close(fig)
742     return res
```

L.649–657 Put the beam-smoothed synthetic map on the observed pixels.

L.658–660 Distance from the disk centre in apparent solar radii.

L.662–664 Regions: disk ($r < 1$), limb ($1 \le r < 1.3$), and both, for the correlation.

L.667–669 Azimuthally averaged radial profile.

L.671–680 Size of the radio Sun: the outermost radius where the profile is still at least half its maximum. This is robust against limb brightening and central dips.

L.684 Position angle of each pixel, measured from solar north counter-clockwise (towards East, i.e. $-X$), as is usual for solar images.

L.687–689 Radial profile restricted to a sector of $\pm$ `sector_halfwidth` around a position angle; the angle difference is wrapped to $[-180°, 180°]$.

L.700–705 Profiles and radio-Sun size in the N, E, S, W sectors, and the elongation (equatorial/polar size). This is the main measure for axisymmetric (2.5D) models.

L.692–699 Collect the maps, the mean disk and limb values, the log correlation, the profiles and the sizes.

L.706–741 Optional figure: both radial profiles and the residual map (synthetic minus observed).

Chapter 14

common/stellar.py — stellar radio emission

File: `radiosynth/common/stellar.py` — 128 lines.

Tools for other stars: viewing geometry from inclination and rotational phase, images of a star at a distance in parsec (parallel rays), spectra and spectral indices, rotational modulation. Model coordinates are in stellar radii; the grid's `length_unit` holds R_* in metres.

Module docstring and constants

lines 4–24

```
4  """Stellar radio emission: distant observers, spectra, rotational modulation.
5
6  For other stars the model coordinates are in stellar radii (grid.length_unit =
7  R_star in metres, set with ``r_star=`` in the readers) and the star is seen from
8  parsecs away, so the rays are parallel and the result is mostly a flux density
9  (mJy), a spectrum and its spectral index, or a light curve over the rotation.
10 VLBI-resolved images can be shown in milliarcseconds with the figure routines
11 (units="mas").
12
13 Viewing geometry: the stellar rotation axis is the model z axis.  The observer
14 direction is given by the inclination i (angle between the rotation axis and the
15 line of sight; i = 90 deg is equator-on) and the rotational phase phi (model
16 longitude facing the observer):  n = (sin i cos phi, sin i sin phi, cos i).
17 """
18 import math
19
20 import numpy as np
21
22 from .image import make_radio_image, parse_frequencies
23
24 PC = 3.0856775814913673e16          # parsec [m]
```

Explains the conventions: rotation axis = model z; inclination i between the axis and the line of sight; phase ϕ = model longitude facing the observer.

`L.24` One parsec in metres.

stellar_observer()

lines 27–33

```
27 def stellar_observer(inclination_deg=90.0, phase_deg=0.0, distance=1.0e4):
28     """Observer position vector for inclination i and rotational phase phi
29     (direction matters; with distance_pc in make_radio_image the rays are
30     parallel and |vector| is irrelevant)."""
31     i, p = math.radians(inclination_deg), math.radians(phase_deg)
32     return (distance * math.sin(i) * math.cos(p), distance * math.sin(i) * math.sin(p),
33             distance * math.cos(i))
```

`L.32` Direction $\hat{n} = (\sin i \cos\phi,\ \sin i \sin\phi,\ \cos i)$ times a distance. With `distance_pc` the rays are parallel and only the direction matters.

_default_fov() lines 36–39

```
36 def _default_fov(grid, fov):
37     if fov is not None:
38         return float(fov)
39     return 2.0 * float(getattr(grid, "r_outer", 10.0)) / math.sqrt(2.0)
```

Default field of view: the largest square inside the model domain.

make_stellar_image() lines 42–54

```
42 def make_stellar_image(grid, frequencies, inclination_deg=90.0, phase_deg=0.0,
43                         distance_pc=10.0, fov=None, npix=128, up=(0.0, 0.0, 1.0), **kw):
44     """Synthetic image of a star at ``distance_pc`` seen at inclination and phase.
45 
46     fov : full width of the image in stellar radii (default: the largest square
47           inside the model domain).  Remaining keywords go to make_radio_image
48           (transfer, gaunt, n_workers, no_refraction, ...).  The image object has
49           flux_density_mjy and can be plotted in 'mas' with radiosynth.figures."""
50     fov = _default_fov(grid, fov)
51     kw.setdefault("transfer", "full")
52     kw.setdefault("r_integration", float(getattr(grid, "r_outer", fov)))
53     return make_radio_image(grid, frequencies, stellar_observer(inclination_deg, phase_deg),
54                             fov, fov, npix, npix, up=up, distance_pc=distance_pc, **kw)
```

L.50–52 Defaults for stars: full radiative transfer (stellar coronae and winds are often optically thick) and an integration sphere covering the whole model.

L.53 The image itself: parallel rays from the direction given by i and ϕ, flux at `distance_pc`.

spectral_index() lines 57–62

```
57 def spectral_index(freqs, flux):
58     """Local spectral index alpha = dlog S / dlog nu at the geometric mid-points."""
59     f = np.asarray(freqs, float)
60     s = np.asarray(flux, float)
61     alpha = np.diff(np.log(s)) / np.diff(np.log(f))
62     return np.sqrt(f[1:] * f[:-1]), alpha
```

L.61 $\alpha = \Delta \ln S / \Delta \ln \nu$ between neighbouring frequencies, at their geometric mean.

stellar_spectrum() lines 65–94

```
65 def stellar_spectrum(grid, frequencies, inclination_deg=90.0, phase_deg=0.0, distance_pc
       ↪ =10.0,
66                       fov=None, npix=128, plot_path=None, verbose=False, **kw):
67     """Flux-density spectrum of a star: returns dict(freqs [Hz], flux_mjy, nu_mid,
68     alpha, image).  ``frequencies`` as for make_radio_image (e.g. np.geomspace)."""
69     img = make_stellar_image(grid, frequencies, inclination_deg, phase_deg, distance_pc, fov,
70                              npix, verbose=verbose, **kw)
```

```
71      f = img.freqs
72      s = img.flux_density_mjy
73      nu_mid, alpha = spectral_index(f, s)
74      if plot_path is not None:
75          import matplotlib
76          matplotlib.use("Agg")
77          import matplotlib.pyplot as plt
78          fig, axs = plt.subplots(1, 2, figsize=(11, 4.2))
79          axs[0].loglog(f / 1e9, s, "o-")
80          axs[0].set_xlabel("frequency [GHz]")
81          axs[0].set_ylabel(f"flux density at {distance_pc:g} pc [mJy]")
82          axs[0].grid(alpha=0.3, which="both")
83          axs[1].semilogx(nu_mid / 1e9, alpha, "s-")
84          axs[1].axhline(0.6, color="gray", ls=":", label="0.6 (ionised wind)")
85          axs[1].axhline(2.0, color="gray", ls="--", label="2 (optically thick)")
86          axs[1].axhline(-0.1, color="gray", ls="-.", label="-0.1 (optically thin)")
87          axs[1].set_xlabel("frequency [GHz]")
88          axs[1].set_ylabel(r"spectral index $\alpha$ ($S\propto\nu^\alpha$)")
89          axs[1].legend(fontsize=8)
90          axs[1].grid(alpha=0.3, which="both")
91          fig.tight_layout()
92          fig.savefig(plot_path, dpi=140)
93          plt.close(fig)
94      return dict(freqs=f, flux_mjy=s, nu_mid=nu_mid, alpha=alpha, image=img)
```

L.69–73 One multi-frequency image gives the spectrum (mJy) and the local spectral index.

L.74–93 Optional figure: spectrum and spectral index, with reference values 2 (optically thick), 0.6 (ionised wind) and −0.1 (optically thin free–free).

rotational_modulation()

lines 97–124

```
97  def rotational_modulation(grid, frequency, phases_deg, inclination_deg=90.0, distance_pc
        ↪ =10.0,
98                            fov=None, npix=96, plot_path=None, verbose=False, **kw):
99      """Flux density over a stellar rotation: returns dict(phases, flux_mjy, images)."""
100     phases = np.asarray(list(phases_deg), float)
101     imgs, flux = [], []
102     for ph in phases:
103         img = make_stellar_image(grid, frequency, inclination_deg, ph, distance_pc, fov, npix
        ↪ ,
104                                  verbose=verbose, **kw)
105         imgs.append(img)
106         flux.append(img.flux_density_mjy[0])
107     flux = np.array(flux)
108     if plot_path is not None:
109         import matplotlib
110         matplotlib.use("Agg")
111         import matplotlib.pyplot as plt
112         fig, ax = plt.subplots(figsize=(6, 3.8))
113         ax.plot(phases, flux, "o-")
114         ax.set_xlabel("rotational phase [deg]")
115         ax.set_ylabel(f"flux density at {distance_pc:g} pc [mJy]")
116         ax.set_title(f"{parse_frequencies(frequency)[0] / 1e9:.3g} GHz, i = {inclination_deg:
        ↪ g} deg")
117         ax.grid(alpha=0.3)
```

```
118         fig.tight_layout()
119         fig.savefig(plot_path, dpi=140)
120         plt.close(fig)
121     return dict(phases=phases, flux_mjy=flux, images=imgs)
122
123
124 # ---------------------------------------------------------------------------
```

L.102–106 One image per rotational phase; the flux densities form the light curve.

L.108–120 Optional light-curve figure.

Re-export of the wind solution

lines 125–128

```
125 # The analytic free-free wind solution belongs to the bremsstrahlung mechanism;
126 # also available here (radiosynth.stellar.wind_*).
127 # ---------------------------------------------------------------------------
128 from ..bremsstrahlung.wind import wind_optical_depth, wind_flux_density  # noqa: E402,F401
```

L.128 The analytic wind functions now live in `bremsstrahlung/wind.py` (they are free–free physics); they are imported here so that `radiosynth.stellar.wind_flux_density` keeps working.

Chapter 15

common/fitsio.py — FITS input/output without astropy

File: `radiosynth/common/fitsio.py` — 105 lines.

Small FITS reader and writer for single images. astropy is used when it is installed; otherwise these routines handle the simple primary-HDU images used for solar radio maps and for radiosynth's own output.

Constants

lines 4–13

```
4  """Minimal FITS image I/O (primary HDU, 2D/3D images) without astropy.
5
6  astropy.io.fits is used when installed; otherwise these small routines read and
7  write simple single-HDU images (BITPIX 8/16/32/-32/-64, BSCALE/BZERO), which
8  covers Nobeyama/MWA/LOFAR-style solar maps and radiosynth's own output.
9  """
10 import numpy as np
11
12 _BLOCK = 2880
13 _DT = {8: ">u1", 16: ">i2", 32: ">i4", 64: ">i8", -32: ">f4", -64: ">f8"}
```

L.12 FITS files are written in blocks of 2880 bytes.

L.13 BITPIX code to NumPy data type (FITS is big-endian).

_parse_value()

lines 16–28

```
16 def _parse_value(v):
17     v = v.split("/")[0].strip() if not v.strip().startswith("'") else v.strip()
18     if v.startswith("'"):
19         return v[1:v.find("'", 1)].strip()
20     if v in ("T", "F"):
21         return v == "T"
22     try:
23         return int(v)
24     except ValueError:
25         try:
26             return float(v.replace("D", "E"))
27         except ValueError:
28             return v
```

Turns the value field of an 80-character header card into a string, a boolean, an integer or a float, dropping the comment after '/'.

read_fits()

lines 31–66

```
31 def read_fits(path, hdu=0):
32     """Return (data, header_dict) of an image HDU."""
33     try:
34         from astropy.io import fits
35         with fits.open(path) as hl:
36             return np.asarray(hl[hdu].data, float), dict(hl[hdu].header)
37     except ImportError:
38         pass
39     raw = open(path, "rb").read()
40     pos = 0
41     for k in range(hdu + 1):
42         hdr = {}
43         while True:
44             block = raw[pos:pos + _BLOCK].decode("ascii", "replace")
45             pos += _BLOCK
46             done = False
47             for i in range(0, _BLOCK, 80):
48                 card = block[i:i + 80]
49                 key = card[:8].strip()
50                 if key == "END":
51                     done = True
52                     break
53                 if card[8:10] == "= ":
54                     hdr[key] = _parse_value(card[10:])
55             if done:
56                 break
57         naxis = hdr.get("NAXIS", 0)
58         shape = [hdr[f"NAXIS{i}"] for i in range(1, naxis + 1)]
59         nbytes = abs(hdr["BITPIX"]) // 8 * int(np.prod(shape)) if naxis else 0
60         if k == hdu:
61             data = np.frombuffer(raw, dtype=_DT[hdr["BITPIX"]], count=int(np.prod(shape)),
62                                  offset=pos).reshape(shape[::-1]).astype(float)
63             data = data * hdr.get("BSCALE", 1.0) + hdr.get("BZERO", 0.0)
64             return data, hdr
65         pos += -(-nbytes // _BLOCK) * _BLOCK
66     raise IndexError("HDU not found")
```

L.33–38 Use astropy if available.

L.43–52 Read 2880-byte header blocks until the END card, collecting the KEY = value cards.

L.57–59 Image dimensions and size.

L.61–64 Read the data (FITS order: NAXIS1 fastest, giving a (ny, nx) array) and apply BSCALE/BZERO.

L.65 Skip to the next HDU if a later one was requested.

write_fits()

lines 69–105

```
69 def write_fits(path, data, header=None):
70     """Write a float32 image with the given header keywords."""
71     data = np.asarray(data, dtype=np.float32)
72     try:
73         from astropy.io import fits
74         h = fits.Header()
75         for k, v in (header or {}).items():
76             h[k] = v
```

```
77          fits.PrimaryHDU(data, header=h).writeto(path, overwrite=True)
78          return
79      except ImportError:
80          pass
81
82      def card(k, v):
83          if isinstance(v, bool):
84              s = "T" if v else "F"
85              return f"{k:<8}= {s:>20}".ljust(80)
86          if isinstance(v, (int, np.integer)):
87              return f"{k:<8}= {int(v):>20}".ljust(80)
88          if isinstance(v, (float, np.floating)):
89              return f"{k:<8}= {float(v):>20.12E}".ljust(80)
90          s = "'" + str(v).replace("'", "''")[:66].ljust(8) + "'"
91          return f"{k:<8}= {s:<20}".ljust(80)
92
93      cards = [card("SIMPLE", True), card("BITPIX", -32), card("NAXIS", data.ndim)]
94      cards += [card(f"NAXIS{i + 1}", n) for i, n in enumerate(data.shape[::-1])]
95      for k, v in (header or {}).items():
96          if k not in ("SIMPLE", "BITPIX", "NAXIS") and not k.startswith("NAXIS"):
97              cards.append(card(k[:8].upper(), v))
98      cards.append("END".ljust(80))
99      h = "".join(cards)
100     h += " " * (-len(h) % _BLOCK)
101     body = data.astype(">f4").tobytes()
102     body += b"\0" * (-len(body) % _BLOCK)
103     with open(path, "wb") as fh:
104         fh.write(h.encode("ascii"))
105         fh.write(body)
```

L.82–91 Format one 80-character card for booleans, integers, floats and strings.

L.93–98 Mandatory keywords (SIMPLE, BITPIX=-32, NAXIS, NAXISn), the user's keywords, END.

L.100–105 Pad the header and the big-endian float32 data to whole 2880-byte blocks.

Chapter 16

common/cli.py — command-line interface

File: `radiosynth/common/cli.py` — 346 lines.

`python -m radiosynth INPUT --freq ...`: parses the options, loads the model with `load_mhd` (with the magnetic field for gyroresonance), calls the image driver of the chosen mechanism, and writes the requested files.

Module docstring and imports

lines 4–22

```
4  """Command-line interface.
5
6  Example
7  -------
8  python -m radiosynth 3d__mhd_box.dat --freq "80MHz, 150MHz, 17GHz" \
9      --observer -215 0 0 --size 6 6 --npix 256 256 --workers 8 \
10     --out radio_n001000 --plot
11
12 python -m radiosynth 3d__mhd_ar.dat --mechanism gyroresonance --freq "5GHz, 17GHz" \
13     --size 0.5 0.5 --npix 200 200 --out gr_ar --plot
14 """
15 import argparse
16 import sys
17
18 import numpy as np
19
20 from .readers import load_mhd
21 from .image import make_radio_image
22 from .mechanism import available_mechanisms, get_mechanism
```

Two examples: a free–free run and a gyroresonance run.

> L.22 For `--list-mechanisms`.

Options: image and physics

lines 25–133

```
25 def main(argv=None):
26     ap = argparse.ArgumentParser(prog="radiosynth",
27                                  description="Synthetic radio images from MHD output: "
28                                              "free-free/simplistic (refractive ray tracing, "
29                                              "BATS-R-US algorithm) or gyroresonance + free-
   ↪ free "
30                                              "(straight rays, Stokes I and V)")
31     ap.add_argument("input", nargs="?", help="MHD output: .dat/.tec Tecplot ASCII, .plt (
   ↪ pytecplot), "
32                                   ".out BATS-R-US IDL, a PLUTO output directory (grid.out + "
33                                   "dbl.out/flt.out) or PLUTO .vtk, or .npz saved by --save-
   ↪ grid")
34     ap.add_argument("--freq", help="e.g. '42.3MHz, 125MHz, 17GHz'")
35     ap.add_argument("--list-mechanisms", action="store_true",
```

```
                    help="list the emission mechanisms and exit")
    ap.add_argument("--observer", nargs=3, type=float, default=[-215.0, 0.0, 0.0],
                    metavar=("X", "Y", "Z"), help="observer position [R_sun]")
    ap.add_argument("--date", help="observation time, e.g. 2014-11-01T03:00: places the observer "
                                   "at Earth (B0, L0, distance) instead of --observer")
    ap.add_argument("--frame", default="carrington", choices=["carrington", "stonyhurst"],
                    help="with --date: model longitude = Carrington longitude, or 0 facing Earth")
    ap.add_argument("--phi-offset", type=float, default=0.0,
                    help="with --date: extra longitude offset of the model frame [deg]")
    st = ap.add_argument_group("stars")
    st.add_argument("--r-star", type=float, help="stellar radius in R_sun; coordinates are "
                                                 "then in stellar radii (BATS-R-US #STAR runs)")
    st.add_argument("--distance-pc", type=float, help="distance of the star [pc]: parallel rays, "
                                                      "flux at that distance (prints mJy)" )
    st.add_argument("--inclination", type=float, help="inclination of the rotation axis (z) to "
                                                      "the line of sight [deg]; sets the view")
    st.add_argument("--phase", type=float, default=0.0, help="rotational phase [deg]")
    ap.add_argument("--size", nargs=2, type=float, default=[6.0, 6.0],
                    metavar=("XSIZE", "YSIZE"), help="image size [R_sun]")
    ap.add_argument("--npix", nargs=2, type=int, default=[128, 128], metavar=("NX", "NY"))
    ap.add_argument("--up", nargs=3, type=float, default=[0.0, 0.0, 1.0],
                    help="vector defining image 'up' (BATS-R-US uses +z)")
    ap.add_argument("--mechanism", "--emission", dest="emission", default="bremsstrahlung",
                    choices=["bremsstrahlung", "simplistic", "gyroresonance", "gyrosynchrotron",
                             "combined"],
                    help="emission mechanism (folder of radiosynth); gyroresonance, "
                         "gyrosynchrotron and combined (ff + gr + gs in one transfer) read "
                         "the magnetic field and use straight rays")
    ap.add_argument("--gaunt", default=None,
                    help="Gaunt factor (number) or 'dulk' (default 10; 'dulk' for gyroresonance)")
    gr = ap.add_argument_group("gyroresonance")
    gr.add_argument("--gr-mechanisms", default="gr+ff", choices=["gr+ff", "gr", "ff"],
                    help="layers + free-free (default), layers only, or magnetised free-free" )
    gr.add_argument("--s-max", type=int, default=10, help="highest harmonic")
    gr.add_argument("--coupling", default="weak", choices=["weak", "strong"],
                    help="mode coupling through quasi-transverse regions")
    gr.add_argument("--ds", type=float, help="step along the rays (default half the smallest cell)")
    gr.add_argument("--t-body", type=float, help="brightness temperature of the body [K]")
    gr.add_argument("--b-unit", help="unit of B in the file if not given there: G, nT, T "
                                     "or gauss per unit")
    gs = ap.add_argument_group("gyrosynchrotron (fast GS codes; build once with "
                               "python -m radiosynth.gyrosynchrotron.library --build)")
    gs.add_argument("--nb", type=float, default=1e6,
                    help="fast-electron density [cm^-3] (peak value of the rule)")
    gs.add_argument("--nb-rule", default="sphere",
                    choices=["sphere", "uniform", "thermal-fraction", "current"],
                    help="where the fast electrons are: gaussian blob (--nb-center, "
```

```
                        "--nb-radius), everywhere, n_b = fraction x n_e (--nb as the "
                        "fraction), or proportional to |J| (n_b = nb at max |J|)")
    gs.add_argument("--nb-center", type=float, nargs=3, metavar=("X", "Y", "Z"))
    gs.add_argument("--nb-radius", type=float, default=0.02, help="blob radius [R_sun]")
    gs.add_argument("--delta", type=float, default=4.0, help="power-law index")
    gs.add_argument("--emin-kev", type=float, default=10.0)
    gs.add_argument("--emax-mev", type=float, default=10.0)
    gs.add_argument("--pitch", default="iso", choices=["iso", "gau", "elc", "gab", "sga"])
    gs.add_argument("--loss-cone", type=float, default=60.0,
                    help="loss-cone angle / beam direction [deg]")
    gs.add_argument("--dmu", type=float, default=0.2, help="loss-cone / beam width")
    gs.add_argument("--gs-coupling", default="exact", choices=["exact", "weak", "strong"])
    gs.add_argument("--gs-library", help="path of the compiled GS codes library")
    gs.add_argument("--gs-diagnostics", action="store_true",
                    help="also compute free-free alone (fast-electron fraction map)")
    gs.add_argument("--ff-refraction", action="store_true",
                    help="combined: correct the free-free part for refraction (refractive "
                         "ray tracer); gyroresonance and gyrosynchrotron stay on straight
    ↪ rays")
    gs.add_argument("--include", nargs="+", default=["ff", "gr", "gs"],
                    choices=["ff", "gr", "gs"],
                    help="combined: mechanisms to include (default all three; 'gs' needs --nb
    ↪  > 0)")
    ap.add_argument("--transfer", default="thin", choices=["thin", "full"],
                    help="thin: I=sum(kappa B ds) as in BATS-R-US; full: with absorption")
    ap.add_argument("--no-refraction", action="store_true", help="straight lines of sight")
    ap.add_argument("--compare-refraction", action="store_true",
                    help="bremsstrahlung/simplistic: compute the image with refracted rays
    ↪ and "
                         "with straight lines of sight; writes PREFIX_refracted.*, "
                         "PREFIX_straight.* and PREFIX_refraction.png")
    ap.add_argument("--kappa-over-mu", action="store_true",
                    help="use kappa/mu (refractive-index correction to absorption)")
    ap.add_argument("--r-integration", type=float, default=None)
    ap.add_argument("--r-body", type=float, default=None,
                    help="inner boundary [R_sun] (default 1, or PLUTO inner radius)")
    ap.add_argument("--step-factor", type=float, default=1.0)
    ap.add_argument("--tolerance", type=float, default=1e-2)
    ap.add_argument("--workers", type=int, default=1)
    # variable selection
    ap.add_argument("--density-var")
    ap.add_argument("--density-unit", help="e.g. g/cm^3, kg/m^3, cm^-3, m^-3")
    ap.add_argument("--te-var")
    ap.add_argument("--pe-var")
    ap.add_argument("--p-var")
    ap.add_argument("--te-const", type=float)
    ap.add_argument("--mu", type=float, default=1.0,
                    help="mean mass per electron in m_p (n_e = rho/(mu m_p)); BATS-R-US uses
    ↪ 1")
    ap.add_argument("--resample", type=int, nargs="+",
                    help="resample scattered/AMR points to this uniform shape (1 or 3 ints)")
    ap.add_argument("--resample-box", type=float, nargs=6,
                    metavar=("XMIN", "XMAX", "YMIN", "YMAX", "ZMIN", "ZMAX"))
    ap.add_argument("--resample-method", default="linear", choices=["linear", "nearest"])
    ap.add_argument("--save-grid", help="save the n_e/T_e grid as .npz for fast reuse")
```

L.31 Model file or PLUTO directory (optional only so that `--list-mechanisms` works alone).

L.34–56 Frequencies, observer (or `--date`, `--frame`, `--phi-offset` to place it at the Earth for a given time), stellar options, image size, pixels and up vector.

L.58–64 Mechanism (`--emission` is kept as an alias) and Gaunt factor (default 10, or 'dulk' for gyroresonance).

L.66–74 Gyroresonance options: which contributions, highest harmonic, mode coupling, step, body temperature, unit of B.

L.76–96 Gyrosynchrotron options: fast-electron density and its spatial rule, energy spectrum, pitch-angle distribution, mode coupling, library path, diagnostics.

L.98–101 Combined image: refraction correction of the free–free part and the processes to include.

L.104–118 Transfer mode, refraction switches (`--no-refraction`, `--compare-refraction`) and numerical controls of the refractive tracer.

L.120–133 Input interpretation for BATS-R-US and generic files, and resampling.

Options: 1D models

lines 134–150

```
134     sy = ap.add_argument_group("1D (spherically symmetric) models")
135     sy.add_argument("--profile", action="store_true",
136                     help="INPUT is a text table of a 1D model (one radius per line), "
137                          "assumed spherically symmetric")
138     sy.add_argument("--profile-cols", type=int, nargs="+", default=[0, 1, 2], metavar="COL",
139                     help="columns of r, density, T_e (default 0 1 2); "
140                          "give 2 columns for no temperature")
141     sy.add_argument("--profile-density", default="ne", choices=["ne", "rho"])
142     sy.add_argument("--profile-density-unit", default=None,
143                     help="cm^-3 / m^-3 for ne (default cm^-3), g/cm^3 / kg/m^3 for rho "
144                          "(default g/cm^3)")
145     sy.add_argument("--profile-r-unit", type=float, default=1.0,
146                     help="coordinate units (R_sun or R_star) per file unit of r")
147     sy.add_argument("--symmetric", action="store_true",
148                     help="spherically symmetric model: trace one row of rays and rotate "
149                          "(fast); writes the T_b profile (PREFIX_profile.txt/.png)")
150     sy.add_argument("--n-p", type=int, default=1000, help="number of rays for --symmetric")
```

L.134–150 Text-table input of a 1D model (columns, density kind and unit, radius unit), and `--symmetric` (one row of rays) with its number of rays.

Options: PLUTO and output

lines 151–168

```
151     pl = ap.add_argument_group("PLUTO input")
152     pl.add_argument("--nout", type=int, default=-1, help="PLUTO output number (default last)"
        ↪ )
153     pl.add_argument("--pluto-format", choices=["dbl", "flt"], help="default: autodetect")
154     pl.add_argument("--definitions", help="path to definitions.h (for UNIT_*)")
155     pl.add_argument("--unit-density", type=float, help="g/cm^3")
156     pl.add_argument("--unit-length", type=float, help="cm (R_sun = 6.96e10)")
157     pl.add_argument("--unit-velocity", type=float, help="cm/s")
158     pl.add_argument("--mu-e", type=float, default=1.0, help="n_e = rho/(mu_e m_p)")
159     pl.add_argument("--pypluto", nargs="?", const="", default=None, metavar="DIR",
160                     help="read PLUTO data with pyPLUTO (optionally give the directory "
161                          "containing pyPLUTO.py)")
```

```
162     pl.add_argument("--mmw", type=float, default=0.5,
163                     help="mean molecular weight for T = p mu m_p/(rho k)")
164     # output
165     ap.add_argument("--out", default="radio_image", help="output prefix")
166     ap.add_argument("--formats", nargs="+", default=["npz", "tec"],
167                     choices=["npz", "tec", "fits"])
168     ap.add_argument("--plot", action="store_true", help="save PNG of T_b and intensity")
```

L.152–162 PLUTO output number, format, definitions file, units and plasma parameters.

L.165–168 Output prefix, file formats and plots.

Running

lines 169–216

```
169     a = ap.parse_args(argv)
170     if a.list_mechanisms:
171         for name in available_mechanisms():
172             print(f"{name:16s} {get_mechanism(name).description}")
173         return 0
174     if a.input is None or a.freq is None:
175         ap.error("the input file and --freq are required")
176     use_gr = a.emission == "gyroresonance"
177     use_gs = a.emission == "gyrosynchrotron"
178     use_cb = a.emission == "combined"
179     if a.compare_refraction and (use_gr or use_gs or use_cb):
180         ap.error("--compare-refraction is available for bremsstrahlung and simplistic only "
181                  "(gyroresonance, gyrosynchrotron and combined use straight rays)")
182 
183     rs = None
184     if a.resample:
185         rs = tuple(a.resample) if len(a.resample) == 3 else (a.resample[0],) * 3
186     box = None
187     if a.resample_box:
188         b = a.resample_box
189         box = ((b[0], b[1]), (b[2], b[3]), (b[4], b[5]))
190     if a.profile:
191         from .symmetric import read_profile
192         c = a.profile_cols
193         du = a.profile_density_unit or ("cm^-3" if a.profile_density == "ne" else "g/cm^3")
194         if use_gr or use_gs or use_cb:
195             ap.error("--profile (1D models): use --mechanism bremsstrahlung or simplistic")
196         grid = read_profile(a.input, r_col=c[0], density_col=c[1],
197                             te_col=c[2] if len(c) > 2 else None, r_unit=a.profile_r_unit,
198                             density=a.profile_density, density_unit=du, mu_e=a.mu_e,
199                             r_body=a.r_body, length_unit=(a.r_star or 1.0) * 6.96e8)
200         if len(c) <= 2 and a.te_const is not None:
201             grid._flat[:, 4] = a.te_const
202     else:
203       grid = load_mhd(a.input, density_var=a.density_var, density_unit=a.density_unit,
204                       te_var=a.te_var, pe_var=a.pe_var, p_var=a.p_var, te_const=a.te_const,
205                       mu=a.mu, r_body=a.r_body, resample_shape=rs, resample_box=box,
206                       resample_method=a.resample_method, nout=a.nout, fmt=a.pluto_format,
207                       definitions=a.definitions, unit_density=a.unit_density,
208                       unit_length=a.unit_length, unit_velocity=a.unit_velocity,
209                       mu_e=a.mu_e, mean_mol_weight=a.mmw,
210                       pluto_reader="pypluto" if a.pypluto is not None else None,
211                       pypluto_path=a.pypluto or None, r_star=a.r_star,
```

```
                        with_b=(use_gr or use_gs or use_cb) or None,
                        b_unit=(float(a.b_unit) if a.b_unit and a.b_unit[0].isdigit()
                                else a.b_unit))
    if a.save_grid:
        grid.to_npz(a.save_grid)
```

L.170–173 List the registered mechanisms and stop.

L.176 Gyroresonance and gyrosynchrotron need the field and their own drivers.

L.179–181 `--compare-refraction` is refused for the straight-ray mechanisms.

L.183–189 Convert the resampling options to tuples.

L.190–201 A 1D text table is read with `read_profile` (free-free and simplistic only); without a temperature column `--te-const` sets T_e.

L.203 Load the model; `with_b` and `b_unit` only for gyroresonance.

L.216 Optionally cache the grid (with $\mathbf{B}$ if loaded).

Imaging and output

lines 217–346

```
    g_ = a.gaunt or ("dulk" if (use_gr or use_cb) else "10")
    gaunt = g_ if g_.lower() == "dulk" else float(g_)
    observer = a.observer
    if a.date:
        from .figures import observer_for_date
        observer = observer_for_date(a.date, phi_offset_deg=a.phi_offset, frame=a.frame)
    if a.inclination is not None:
        from .stellar import stellar_observer
        observer = stellar_observer(a.inclination, a.phase)
    if a.symmetric:
        from .symmetric import make_symmetric_image, plot_profile
        if use_gr or use_gs or use_cb:
            ap.error("--symmetric (1D models): use --mechanism bremsstrahlung or simplistic")
        mech_kw = dict(gaunt=gaunt, transfer=a.transfer, no_refraction=a.no_refraction,
                       step_factor=a.step_factor, tolerance=a.tolerance,
                       kappa_over_mu=a.kappa_over_mu)
        if a.r_integration is not None:
            mech_kw["r_integration"] = a.r_integration
        img = make_symmetric_image(grid, a.freq, mechanism=a.emission, observer=observer,
    ↪ x_size=a.size[0], y_size=a.size[1],
                                   nx=a.npix[0], ny=a.npix[1], n_p=a.n_p,
                                   distance_pc=a.distance_pc, **mech_kw)
        pr = img.profile
        np.savetxt(a.out + "_profile.txt",
                   np.column_stack([pr["p"]] + [pr["tb"][k] for k in range(len(img.freqs))]),
                   header="p [R] " + " ".join(f"Tb({f:.6g}Hz)[K]" for f in img.freqs))
        plot_profile(img, path=a.out + "_profile.png")
        print("flux density from the profile [W m^-2 Hz^-1]:", " ".join(f"{v:.4e}" for v in
    ↪ pr["flux"]))
    elif a.compare_refraction:
        from .image import compare_refraction
        res = compare_refraction(grid, a.freq, observer=observer, x_size=a.size[0],
                                 y_size=a.size[1], nx=a.npix[0], ny=a.npix[1], up=a.up,
                                 emission=a.emission, gaunt=gaunt, transfer=a.transfer,
                                 r_integration=a.r_integration, n_workers=a.workers,
                                 step_factor=a.step_factor, tolerance=a.tolerance,
                                 kappa_over_mu=a.kappa_over_mu, distance_pc=a.distance_pc,
```

```
                                path=a.out + "_refraction.png")
        for tag in ("refracted", "straight"):
            im = res[tag]
            if "npz" in a.formats:
                im.save_npz(f"{a.out}_{tag}.npz")
            if "tec" in a.formats:
                im.save_tecplot(f"{a.out}_{tag}.dat")
            if "fits" in a.formats:
                im.save_fits(f"{a.out}_{tag}_Tb.fits", quantity="tb")
        print("written:", a.out + "_refracted.*, " + a.out + "_straight.*, " +
              a.out + "_refraction.png")
        return 0
    el = None
    if use_gs or (use_cb and "gs" in a.include and a.nb > 0):
        from ..gyrosynchrotron import (FastElectrons, uniform, fraction_of_thermal, power_of,
                                       in_sphere)
        if a.nb_rule == "sphere":
            if a.nb_center is None:
                ap.error("--nb-rule sphere needs --nb-center X Y Z")
            rule = in_sphere(a.nb_center, a.nb_radius, a.nb)
        elif a.nb_rule == "uniform":
            rule = uniform(a.nb)
        elif a.nb_rule == "thermal-fraction":
            rule = fraction_of_thermal(a.nb)
        else:
            from .fields import current_density
            j = current_density(grid)
            rule = power_of("j", a.nb, float(j.max()), 1.0, floor=0.1)
        el = FastElectrons(nb=rule, delta=a.delta, emin_mev=a.emin_kev / 1e3,
                           emax_mev=a.emax_mev, pitch=a.pitch, alpha_c=a.loss_cone, dmu=a.dmu
    ↪ )
    if a.symmetric:
        pass                                    # image already made from the 1D profile
    elif use_cb:
        from ..combined import make_combined_image
        img = make_combined_image(grid, a.freq, el, observer=observer, x_size=a.size[0],
                                  y_size=a.size[1], nx=a.npix[0], ny=a.npix[1], up=a.up,
                                  include=a.include, gaunt=gaunt, s_max=a.s_max,
                                  coupling=a.coupling, ds=a.ds, t_body=a.t_body,
                                  r_integration=a.r_integration, distance_pc=a.distance_pc,
                                  library=a.gs_library, ff_refraction=a.ff_refraction,
                                  trace_kw=dict(n_workers=a.workers, step_factor=a.
    ↪ step_factor,
                                                tolerance=a.tolerance))
    elif use_gs:
        from ..gyrosynchrotron import make_gs_image
        img = make_gs_image(grid, a.freq, el, observer=observer, x_size=a.size[0],
                            y_size=a.size[1], nx=a.npix[0], ny=a.npix[1], up=a.up,
                            coupling=a.gs_coupling, ds=a.ds, t_body=a.t_body,
                            r_integration=a.r_integration, distance_pc=a.distance_pc,
                            library=a.gs_library, diagnostics=a.gs_diagnostics,
                            abundance="dulk" if (a.gaunt or "").lower() == "dulk" else "
    ↪ coronal")
    elif use_gr:
        from ..gyroresonance import make_gr_image
        img = make_gr_image(grid, a.freq, observer=observer, x_size=a.size[0],
                            y_size=a.size[1], nx=a.npix[0], ny=a.npix[1], up=a.up,
                            mechanisms=a.gr_mechanisms, gaunt=gaunt, s_max=a.s_max,
```

```
307                               coupling=a.coupling, ds=a.ds, t_body=a.t_body,
308                               r_integration=a.r_integration, distance_pc=a.distance_pc)
309     else:
310         img = make_radio_image(grid, a.freq, observer=observer, x_size=a.size[0],
311                                y_size=a.size[1], nx=a.npix[0], ny=a.npix[1], up=a.up,
312                                emission=a.emission, gaunt=gaunt, transfer=a.transfer,
313                                no_refraction=a.no_refraction, r_integration=a.r_integration,
314                                n_workers=a.workers, step_factor=a.step_factor,
315                                tolerance=a.tolerance, kappa_over_mu=a.kappa_over_mu,
316                                distance_pc=a.distance_pc)
317     if "npz" in a.formats:
318         img.save_npz(a.out + ".npz")
319     if "tec" in a.formats:
320         img.save_tecplot(a.out + ".dat")
321     if "fits" in a.formats:
322         img.save_fits(a.out + "_I.fits")
323         img.save_fits(a.out + "_Tb.fits", quantity="tb")
324         if use_gr or use_gs or use_cb:
325             img.save_fits(a.out + "_V.fits", quantity="v")
326             img.save_fits(a.out + "_TbV.fits", quantity="tb_v")
327     if a.plot:
328         img.plot("tb", path=a.out + "_Tb.png")
329         img.plot("intensity", path=a.out + "_I.png")
330         if use_gr or use_gs or use_cb:
331             img.plot("pol", path=a.out + "_pol.png")
332         if use_gr or use_cb:
333             img.plot("harmonic", path=a.out + "_harmonic.png")
334         if use_cb:
335             img.plot("mechanism", path=a.out + "_mechanism.png")
336         if use_gs and a.gs_diagnostics:
337             img.plot("gs_fraction", path=a.out + "_gsfraction.png")
338     if a.distance_pc:
339         for f, fl in zip(img.freqs, img.flux_density_mjy):
340             print(f"flux density at {a.distance_pc:g} pc, {f / 1e9:.4g} GHz: {fl:.4g} mJy")
341     print("written:", a.out + ".*")
342     return 0
343
344
345 if __name__ == "__main__":
346     sys.exit(main())
```

L.220–222 With `--date`, the observer is computed by `observer_for_date`.

L.223–225 With `--inclination`, the view direction comes from the inclination and phase.

L.226–243 `--symmetric`: the image from one row of rays, the radial profile written as text and figure, and the flux.

L.244–263 `--compare-refraction`: both images with identical options, the comparison figure, and the files of both.

L.265–271 Gyrosynchrotron or combined: build the n_b rule from the options (blob, uniform, fraction of n_e, or proportional to $|\mathbf{J}|$ computed by `current_density`) and the `FastElectrons` prescription.

L.284–293 Combined image (with the optional refraction correction of the free–free part); nothing more to do after `--symmetric`.

L.294–301 Gyrosynchrotron image.

L.302–308 Otherwise `make_gr_image` or `make_radio_image`.

L.255–337 Write the requested files; for gyroresonance and gyrosynchrotron also Stokes V FITS files and polarisation maps, the harmonic map (gyroresonance) and the fast-electron fraction (gyrosynchrotron).

L.338–340 For a star, print the flux density at each frequency in mJy.

Chapter 17

simplistic/emission.py — the simplistic mechanism

File: `radiosynth/simplistic/emission.py` — 26 lines.

The ad-hoc emissivity of Benkevitch et al. (2010), kept in BATS-R-US.

Module docstring and imports

lines 4–17

```
4  """The 'simplistic' radio emission mechanism of Benkevitch et al. (2010).
5
6  An ad-hoc emissivity kept in BATS-R-US (ModRadioWaveImage, 'simplistic'):
7
8      S = 1,   kappa * L = (n/n_cr)^2 (0.5 - n/n_cr)^2
9
10 with L the normalisation length (R_sun, or R_star for stars).  The intensity
11 is dimensionless; the emissivity peaks where n_e = n_cr/4 and vanishes at
12 n_e = n_cr/2, so the image shows the layer close to the critical surface.
13 It needs neither T_e nor B and is useful to visualise refraction.
14 """
15 import numpy as np
16
17 from ..common.constants import R_SUN, critical_density
```

States the formula and what the mechanism is for: a dimensionless map of the layer near the critical density, useful to see refraction.

simplistic_coefficients()

lines 20–26

```
20 def simplistic_coefficients(ne, te, freq, length_unit=R_SUN):
21     """(kappa [1/length], S=1) of the simplistic mechanism at n_e [m^-3]."""
22     ne = np.asarray(ne, dtype=float)
23     r = ne / critical_density(freq)
24     # BATS-R-US defines it per normalised length (R_sun or R_star)
25     kappa = r**2 * (0.5 - r)**2 / length_unit
26     return kappa, np.ones_like(ne)
```

`L.23` $r = n_e / n_{\mathrm{cr}}$.

`L.25` $\kappa = r^2(0.5 - r)^2$ per normalised length. Dividing by `length_unit` ($R_\odot$, or R_* for a star) gives a per-metre coefficient, so that $\kappa\,\Delta\ell[\mathrm{m}]$ equals the Fortran product of κ and the normalised step.

`L.26` $S = 1$, a proxy as in Fortran.

Chapter 18

simplistic/__init__.py — package interface

File: `radiosynth/simplistic/__init__.py` — 34 lines.

Registers the mechanism and provides `make_image`.

Mechanism registration

lines 4–34

```
4  """Simplistic mechanism (Benkevitch et al. 2010): dimensionless emissivity
5  peaking near the critical layer.  Uses the refractive ray tracer in common/.
6
7  >>> from radiosynth import simplistic
8  >>> img = simplistic.make_image(grid, "80MHz", observer=(-215, 0, 0))
9  """
10 from ..common.mechanism import Mechanism, register
11 from .emission import simplistic_coefficients
12
13
14 def make_image(grid, frequencies, **kw):
15     """Refractive ray-traced image with the simplistic emissivity
16     (same keywords as radiosynth.make_radio_image)."""
17     from ..common.image import make_radio_image
18     kw.pop("emission", None)
19     return make_radio_image(grid, frequencies, emission="simplistic", **kw)
20
21
22 def compare_refraction(grid, frequencies, **kw):
23     """Refracted vs straight-line image (see radiosynth.compare_refraction)."""
24     from ..common.image import compare_refraction as _cmp
25     kw.pop("emission", None)
26     return _cmp(grid, frequencies, emission="simplistic", **kw)
27
28
29 MECHANISM = register(Mechanism(
30     "simplistic", coefficients=simplistic_coefficients, make_image=make_image,
31     description="ad-hoc emissivity near the critical layer (Benkevitch+2010); "
32                 "dimensionless intensity"))
33
34 __all__ = ["simplistic_coefficients", "make_image", "compare_refraction", "MECHANISM"]
```

- **L.14** `make_radio_image` with `emission='simplistic'`; every other keyword is passed through.
- **L.22** Refracted vs straight-line images with the simplistic emissivity (common/image.py `compare_refraction`).
- **L.29** Entry in the registry: name, coefficient function, image driver and a one-line description (shown by `--list-mechanisms`).

Chapter 19

bremsstrahlung/emission.py — thermal free–free coefficients

File: `radiosynth/bremsstrahlung/emission.py` — 42 lines.

The bremsstrahlung coefficients of `ModUserAwsom.f90` (Moschou et al. 2018), or of Dulk (1985).

Module docstring and imports

lines 4–17

```
4  """Thermal free-free (bremsstrahlung) emission coefficients.
5
6  Exactly as in ModUserAwsom.f90 (Moschou et al. 2018, Eqs. 15-16), with a
7  constant Gaunt factor (default <g_ff> = 10, as in the paper):
8
9      S     = 2 nu^2 k_B T_e / c^2                       (Rayleigh-Jeans)
10     kappa = 4/3 sqrt(2 pi/3) [e^2/sqrt(k_B T_e m_e)]^3 (n_e/nu)^2 g_ff / c
11
12 ``gaunt='dulk'`` instead uses the Coulomb-logarithm form of Dulk (1985,
13 ARA&A 23, 169, Eq. 21), more accurate than g = 10.
14 """
15 import numpy as np
16
17 from ..common.constants import PI, K_B, M_E, C_LIGHT, E2_JM, R_SUN
```

The formulas are written out in the User Manual, Part II.

The prefactor

lines 19–19

```
19 _BREMS_PREF = 4.0 / 3.0 * np.sqrt(2.0 * PI / 3.0) * E2_JM**3 / C_LIGHT
```

L.19 Pre-computes the constant part of κ: $\frac{4}{3}\sqrt{2\pi/3}\,(e^2/4\pi\varepsilon_0)^3/c$. This is the Fortran expression `4/3*sqrt(2*cPi/3)*(cElectronChargeSquaredJm/ ...)**3 /cLightSpeed` with the temperature factor taken out.

bremsstrahlung_coefficients()

lines 22–42

```
22 def bremsstrahlung_coefficients(ne, te, freq, length_unit=R_SUN, gaunt=10.0):
23     """(kappa [1/m], S [W m^-2 Hz^-1 sr^-1]) of thermal free-free emission.
24
25     length_unit is accepted for a uniform interface and not used (SI units)."""
26     ne = np.asarray(ne, dtype=float)
27     te = np.maximum(np.asarray(te, dtype=float), 1.0)
28     S = 2.0 * freq**2 * K_B * te / C_LIGHT**2
29     if isinstance(gaunt, str):
30         if gaunt.lower() != "dulk":
31             raise ValueError("gaunt must be a number or 'dulk'")
```

```
32          # Dulk (1985) Eq. 21, cgs:  kappa = 9.78e-3 n^2 / (nu^2 T^1.5) * G  [cm^-1]
33          n_cc = ne * 1e-6
34          G = np.where(te < 2e5,
35                       18.2 + np.log(te**1.5) - np.log(freq),
36                       24.5 + np.log(te) - np.log(freq))
37          G = np.maximum(G, 1.0)
38          kappa = 9.78e-3 * n_cc**2 / (freq**2 * te**1.5) * G * 100.0  # -> 1/m
39      else:
40          kappa = (_BREMS_PREF / (K_B * te * M_E)**1.5
41                   * (ne / freq)**2 * float(gaunt))
42      return kappa, S
```

L.26 Accept lists or arrays; all arithmetic in double precision.

L.27 Protects against zero or negative temperatures from bad cells. $T_e^{-3/2}$ would otherwise diverge.

L.28 Rayleigh–Jeans source function $B_\nu = 2\nu^2 k_B T_e / c^2$ (`PlanckOut_W`).

L.29–32 Dulk (1985) form: density converted to cm^{-3}, Coulomb logarithm G chosen by temperature regime and floored at 1, κ in cm^{-1} times 100 to give m^{-1}.

L.40–41 BATS-R-US form: prefactor $\times (k_B T_e m_e)^{-3/2} \times (n_e/\nu)^2 \times \langle g \rangle$, the same expression as `OpacityEmissionOut_W`, but in m^{-1} (the Fortran multiplies by $R_\odot$ to get a per-normalised-length value).

L.42 Both arrays have the shape of the input. `length_unit` is accepted only so that all mechanisms share one signature.

Chapter 20

bremsstrahlung/magnetized.py — free–free emission of the two magneto-ionic modes

File: `radiosynth/bremsstrahlung/magnetized.py` — 49 lines.

Free–free absorption coefficients of the o- and x-modes in a magnetic field, used by the polarised straight-ray transfer of the gyroresonance package (for `mechanisms='gr+ff'` and `'ff'`).

Module docstring and imports

lines 4–27

```
"""Free-free absorption of the two magneto-ionic modes in a magnetic field.

In a magnetised plasma the free-free absorption coefficient differs for the
ordinary (o) and extraordinary (x) modes.  For Y = nu_B/nu < 1 and away from
quasi-transverse propagation the standard approximation (e.g. Zheleznyakov
1970; Dulk 1985; Gelfreikh 2004) is

    kappa_sigma = kappa_ff / (1 + sigma Y |cos(theta)|)^2,
    sigma = +1 (o-mode), -1 (x-mode)

with kappa_ff the unmagnetised coefficient (bremsstrahlung/emission.py) and
source function S_sigma = B_nu/2 = nu^2 k_B T_e / c^2 for each mode
(Kirchhoff's law per mode, so that I_o + I_x = B_nu in the thick limit).
In an optically thin source this gives the circular polarisation degree
P = (I_x - I_o)/(I_x + I_o) = 2 Y |cos(theta)| (for Y << 1); in an optically
thick isothermal source P -> 0.  Where X >= 1 - Y the x-mode is cut off.
"""
import numpy as np

from ..common.constants import C_LIGHT, K_B, R_SUN
from ..common.magnetoionic import magnetoionic_params
from .emission import bremsstrahlung_coefficients

__all__ = ["magnetized_ff_coefficients", "magnetoionic_params"]
```

Gives the approximation $\kappa_\sigma = \kappa_{\rm ff}/(1+\sigma Y|\cos\theta|)^2$ ($\sigma=+1$ o-mode, -1 x-mode), Kirchhoff's law per mode with $S_\sigma = B_\nu/2$, and the resulting thin-source polarisation $P = 2Y|\cos\theta|$.

magnetized_ff_coefficients()

lines 30–44

```
def magnetized_ff_coefficients(ne, te, b_gauss, cos_theta, freq, gaunt=10.0,
                               length_unit=R_SUN, y_max=0.9):
    """Return (kappa_o, kappa_x, S_mode) [1/m, 1/m, W m^-2 Hz^-1 sr^-1].

    b_gauss   : |B| [G];  cos_theta : cosine of the angle between B and the
                line of sight; y_max caps Y |cos| in the denominator (the
                approximation fails as Y -> 1, i.e. near the s=1 layer).
    """
```

```
38      kappa, S = bremsstrahlung_coefficients(ne, te, freq, length_unit=length_unit, gaunt=gaunt
        ↪ )
39      _, Y = magnetoionic_params(ne, b_gauss, freq)
40      q = np.minimum(Y * np.abs(np.asarray(cos_theta, dtype=float)), y_max)
41      kappa_o = kappa / (1.0 + q) ** 2
42      kappa_x = kappa / (1.0 - q) ** 2
43      S_mode = 0.5 * S
44      return kappa_o, kappa_x, S_mode
```

L.38 The unmagnetised coefficient (Gaunt factor 10 or Dulk) and B_ν.

L.39 $Y = \nu_B/\nu$ from $|B|$.

L.40 $q = Y|\cos\theta|$, capped at `y_max` $= 0.9$: the expansion is only valid for $Y < 1$, i.e. away from the $s = 1$ layer, where gyroresonance dominates anyway.

L.41–42 o-mode less, x-mode more absorbing. For $q \ll 1$, $(\kappa_x - \kappa_o)/(\kappa_x + \kappa_o) = 2q/(1+q^2)$ — tested in `tests/test_gyroresonance.py`.

L.43 Each mode carries half of the Rayleigh–Jeans intensity, so $I_o + I_x = B_\nu$ when thick.

rayleigh_jeans_mode()

lines 47–49

```
47  def rayleigh_jeans_mode(te, freq):
48      """Source function of one mode, nu^2 k T / c^2 [W m^-2 Hz^-1 sr^-1]."""
49      return np.asarray(freq, dtype=float) ** 2 * K_B * np.asarray(te, dtype=float) / C_LIGHT
        ↪ **2
```

$\nu^2 k_B T/c^2$, the source function of one mode.

Chapter 21

bremsstrahlung/wind.py — analytic stellar-wind solution

File: `radiosynth/bremsstrahlung/wind.py` — 53 lines.

The analytic free–free optical depth and flux of an isothermal wind $n = n_0(r_0/r)^2$, used to validate the ray tracer for stars (also available as `radiosynth.stellar.wind_*`).

Module docstring and imports

lines 4–21

```
4  """Analytic reference for thermal free-free emission from a stellar wind.
5
6  Isothermal, fully ionised spherical wind  n = n0 (r0/r)^2  with a constant
7  Gaunt factor (Wright & Barlow 1975; Panagia & Felli 1975).  Used to validate
8  the ray tracer (tests/test_stellar.py) and as a quick estimate of the radio
9  flux of a wind.
10 """
11 import math
12
13 import numpy as np
14
15 from ..common.constants import C_LIGHT, K_B
16 from ..common.emission import EmissionModel
17
18
19 # ---------------------------------------------------------------------------
20 # analytic reference: isothermal, fully ionised spherical wind n = n0 (r0/r)^2
21 # ---------------------------------------------------------------------------
```

The functions use `EmissionModel('bremsstrahlung')` to get κ/n_e^2, so they share the Gaunt-factor options of the images.

wind_optical_depth(): analytic reference

lines 22–40

```
22 def wind_optical_depth(p, n0, r0_m, te, freq, gaunt=10.0, r_body_m=None):
23     """Free-free optical depth along a straight line at impact parameter p [m]
24     through n = n0 (r0/r)^2 (constant T, constant Gaunt factor):
25         tau(p) = kappa1 n0^2 r0^4 * pi / (2 p^3)       (infinite line, p > r_body)
26     kappa1 = kappa / n_e^2.  For p < r_body the line stops at the stellar
27     surface (front half only), computed numerically."""
28     k1 = EmissionModel("bremsstrahlung", gaunt=gaunt)(np.array([1.0]), np.array([te]), freq)
       ↪ [0][0]
29     A = k1 * n0**2 * r0_m**4
30     p = np.atleast_1d(np.asarray(p, float))
31     tau = A * math.pi / (2.0 * p**3)
32     if r_body_m is not None:
33         inside = p < r_body_m
34         if np.any(inside):
35             # integral of dl/(p^2+l^2)^2 from l0=sqrt(rb^2-p^2) to infinity
```

```
            pp = p[inside]
            l0 = np.sqrt(r_body_m**2 - pp**2)
            F = lambda l, q: l / (2 * q**2 * (q**2 + l**2)) + np.arctan(l / q) / (2 * q**3)
    ↪ # noqa: E731
            tau[inside] = A * (math.pi / (4 * pp**3) - F(l0, pp))
    return tau
```

L.28 $\kappa_1 = \kappa / n_e^2$ for the given T_e, frequency and Gaunt factor.

L.31 Along an infinite straight line at impact parameter p: $\tau = \kappa_1 n_0^2 r_0^4 \int dl/(p^2+l^2)^2 = \kappa_1 n_0^2 r_0^4 \, \pi/(2p^3)$.

L.32–39 Rays that hit the star see only the wind in front of it. The integral then runs from $l_0 = \sqrt{r_*^2 - p^2}$, using the antiderivative $l/(2p^2(p^2+l^2)) + \arctan(l/p)/(2p^3)$.

wind_flux_density(): analytic flux of an optically thick wind

lines 43–53

```
def wind_flux_density(n0, r0_m, te, freq, distance_m, gaunt=10.0):
    """Analytic flux density [W m^-2 Hz^-1] of an infinite isothermal wind
    n = n0 (r0/r)^2 with constant Gaunt factor, neglecting the stellar disk
    (valid when the tau = 1 radius is >> r0):
        S = pi Gamma(1/3) B_nu a^(2/3) / D^2,   tau(p) = a / p^3
    which gives the classic spectrum S ~ nu^(2/3) (nu^0.6 when the Gaunt factor's
    weak frequency dependence is included; Wright & Barlow 1975, Panagia & Felli 1975)."""
    k1 = EmissionModel("bremsstrahlung", gaunt=gaunt)(np.array([1.0]), np.array([te]), freq)
    ↪ [0][0]
    a = k1 * n0**2 * r0_m**4 * math.pi / 2.0
    B = 2.0 * freq**2 * K_B * te / C_LIGHT**2
    return math.pi * math.gamma(1.0 / 3.0) * B * a**(2.0 / 3.0) / distance_m**2
```

With $\tau(p) = a/p^3$ the flux is $S = 2\pi B \int_0^\infty (1 - e^{-a/p^3}) p \, dp / D^2 = \pi \Gamma(1/3) \, B \, a^{2/3} / D^2$. Since $a \propto \nu^{-2}$ and $B \propto \nu^2$, this gives $S \propto \nu^{2/3}$, the classic wind spectrum (Wright & Barlow 1975; Panagia & Felli 1975). It becomes $\nu^{0.6}$ when the Gaunt factor's weak frequency dependence is included.

L.53 The closed form; checked against direct quadrature to 10^{-7}.

Chapter 22

bremsstrahlung/__init__.py — package interface

File: `radiosynth/bremsstrahlung/__init__.py` — 43 lines.

Registers thermal free–free emission and collects its functions.

Mechanism registration

lines 4–43

```
4  """Thermal free-free (bremsstrahlung) emission.
5
6  * emission.py   : kappa and source function (Moschou et al. 2018, g=10 or
7                    Dulk 1985); used by the refractive ray tracer in common/
8  * magnetized.py : two-mode (o/x) free-free absorption in a magnetic field,
9                    used with the polarized straight-ray transfer (gyroresonance)
10 * wind.py       : analytic isothermal-wind solution (validation, stars)
11
12 >>> from radiosynth import bremsstrahlung as ff
13 >>> img = ff.make_image(grid, "80MHz, 17GHz", observer=(-215, 0, 0), transfer="full")
14 """
15 from ..common.mechanism import Mechanism, register
16 from .emission import bremsstrahlung_coefficients
17 from .magnetized import magnetized_ff_coefficients, magnetoionic_params
18 from .wind import wind_optical_depth, wind_flux_density
19
20
21 def make_image(grid, frequencies, **kw):
22     """Refractive ray-traced free-free image (same keywords as
23     radiosynth.make_radio_image; gaunt=10 or 'dulk', transfer='thin'/'full')."""
24     from ..common.image import make_radio_image
25     kw.pop("emission", None)
26     return make_radio_image(grid, frequencies, emission="bremsstrahlung", **kw)
27
28
29 def compare_refraction(grid, frequencies, **kw):
30     """Refracted vs straight-line free-free image (see radiosynth.compare_refraction)."""
31     from ..common.image import compare_refraction as _cmp
32     kw.pop("emission", None)
33     return _cmp(grid, frequencies, emission="bremsstrahlung", **kw)
34
35
36 MECHANISM = register(Mechanism(
37     "bremsstrahlung", coefficients=bremsstrahlung_coefficients, make_image=make_image,
38     description="thermal free-free emission (Moschou+2018), refractive ray tracing",
39     options=("gaunt",)))
40
41 __all__ = ["bremsstrahlung_coefficients", "magnetized_ff_coefficients",
42            "magnetoionic_params", "wind_optical_depth", "wind_flux_density",
43            "make_image", "compare_refraction", "MECHANISM"]
```

L.16–18 Coefficients, two-mode coefficients, wind solution.

L.21 `make_radio_image` with `emission='bremsstrahlung'` (refractive ray tracing; `gaunt`, `transfer` and all other keywords passed through).

L.29 Refracted vs straight-line free–free images (common/image.py `compare_refraction`).

L.36 Registry entry; `options` lists the mechanism-specific keyword (`gaunt`).

Chapter 23

gyroresonance/opacity.py — optical depth of a harmonic layer

File: `radiosynth/gyroresonance/opacity.py` — 85 lines.

The thermal gyroresonance opacity in the thin-layer form used by gyroresonance codes (Zheleznyakov 1970; White & Kundu 1997; Fleishman & Kuznetsov 2014).

Module docstring and imports

lines 4–43

```
"""Thermal gyroresonance opacity of a harmonic layer.

A thermal electron gyrating in a field B radiates and absorbs at the harmonics
s nu_B of the gyrofrequency (nu_B = 2.80 MHz B[G]).  For a Maxwellian plasma
the absorption at frequency nu is confined to thin layers where nu = s nu_B,
broadened only by the Doppler width  ~ nu beta_T |cos(theta)|.  Integrating
the absorption coefficient across such a layer gives the optical depth
(Zheleznyakov 1970; Melrose 1980; White & Kundu 1997; Fleishman & Kuznetsov
2014, ApJ 781, 77, Eq. for tau_s^M):

    tau_s^sigma = (pi e^2 n_e / (m_e c nu)) L_B (s^2 / s!)
                  * (s^2 beta_T^2 sin^2(theta) / 2)^(s-1) * F_sigma(theta)

    beta_T^2 = k_B T_e / (m_e c^2),     L_B = |B / (dB/dl)|  along the ray,
    F_sigma  = (1 + T_sigma cos(theta))^2 / (1 + T_sigma^2)

with T_sigma the axial ratio of mode sigma (common/magnetoionic.py), signed so
that T_x cos(theta) > 0 and T_o = -1/T_x.  This is the small-Larmor-radius
(lowest order in beta_T) result: the x-mode, which co-rotates with the
electrons, has F_x -> 2 along B, and the o-mode F_o -> 0; at perpendicular
propagation F_x = 1 and F_o vanishes at this order (the o-mode opacity is then
of higher order in beta_T and is underestimated).  The refractive index is
set to 1 (valid for X = (nu_p/nu)^2 << 1, normally true where gyroresonance
emission forms) and the longitudinal part of the mode polarisation is
neglected.

Kirchhoff's law holds per mode: each layer emits the source function
S = nu^2 k_B T_e / c^2 (half the Rayleigh-Jeans intensity), so an optically
thick layer has T_b = T_e in that mode.

In SI the prefactor pi e^2/(m_e c) is pi (e^2/4 pi eps0)/(m_e c) = 2.654e-6 m^2/s.
"""
import math

import numpy as np

from ..common.constants import C_LIGHT, E2_JM, K_B, M_E, PI
from ..common.magnetoionic import axial_ratio_x

#: pi e^2 / (m_e c) in SI [m^2 s^-1]
```

Gives the formula, the definitions of β_T, L_B and the mode factor F_σ, and the approximations (lowest order in β_T, refractive index 1, longitudinal polarisation neglected). User Manual, Chapter 20, derives it.

Prefactor

lines 44–44

```
44 GR_PREFACTOR = PI * E2_JM / (M_E * C_LIGHT)
```

L.44 $\pi e^2/(m_e c)$ in SI, $\pi(e^2/4\pi\varepsilon_0)/(m_e c) = 2.654 \times 10^{-6}$ m^2 s^{-1} (0.02654 cm^2 s^{-1} in cgs).

mode_factor()

lines 47–55

```
47 def mode_factor(X, Y, cos_theta, mode):
48     """F_sigma = (1 + T_sigma cos)^2/(1 + T_sigma^2) for mode 'x' or 'o'."""
49     c = np.abs(np.asarray(cos_theta, dtype=float))
50     tx = axial_ratio_x(X, Y, c)
51     if mode == "x":
52         return (1.0 + tx * c) ** 2 / (1.0 + tx * tx)
53     if mode == "o":
54         return (tx - c) ** 2 / (1.0 + tx * tx)
55     raise ValueError("mode must be 'x' or 'o'")
```

L.50 $|T_x|$ at the layer, where $Y = 1/s$.

L.52 x-mode: $F_x = (1 + |T_x||\cos\theta|)^2/(1 + T_x^2)$; 2 along $\mathbf{B}$, 1 across.

L.54 o-mode with $T_o = -1/T_x$: $F_o = (|T_x| - |\cos\theta|)^2/(1 + T_x^2)$; 0 along and (at this order) across $\mathbf{B}$.

layer_optical_depth()

lines 58–79

```
58 def layer_optical_depth(s, ne, te, cos_theta, l_b, freq, mode, X=None):
59     """Optical depth of the s-th harmonic layer.
60
61     s         : harmonic number (int >= 1)
62     ne        : electron density at the layer [m^-3]
63     te        : electron temperature [K]
64     cos_theta : cosine of the angle between B and the line of sight
65     l_b       : magnetic scale length |B/(dB/dl)| along the ray [m]
66     freq      : observing frequency [Hz]
67     mode      : 'x' or 'o'
68     X         : (nu_p/nu)^2 at the layer (for the mode polarisation; default 0)
69     """
70     s = int(s)
71     ne = np.asarray(ne, dtype=float)
72     te = np.asarray(te, dtype=float)
73     c = np.asarray(cos_theta, dtype=float)
74     beta2 = K_B * te / (M_E * C_LIGHT**2)
75     sin2 = np.maximum(1.0 - c * c, 0.0)
76     X = np.zeros_like(ne) if X is None else np.asarray(X, dtype=float)
77     F = mode_factor(X, 1.0 / s, c, mode)
78     return (GR_PREFACTOR * ne * np.asarray(l_b, dtype=float) / freq
79             * (s * s / math.factorial(s)) * (0.5 * s * s * beta2 * sin2) ** (s - 1) * F)
```

L.74 $\beta_T^2 = k_B T_e/(m_e c^2)$.

L.75 $\sin^2\theta$.

L.77 Mode factor with $Y = 1/s$.

L.78–79 $\tau_s^\sigma = \frac{\pi e^2 n_e L_B}{m_e c \nu}\,\frac{s^2}{s!}\left(\frac{s^2\beta_T^2\sin^2\theta}{2}\right)^{s-1} F_\sigma$.

harmonic_field()

lines 82–85

```
82 def harmonic_field(s, freq):
83     """|B| [G] of the s-th harmonic layer at frequency freq [Hz]."""
84     from ..common.magnetoionic import NU_B_PER_GAUSS
85     return freq / (s * NU_B_PER_GAUSS)
```

$|B| = \nu/(s\,\nu_B/\mathrm{G})$: the field strength of the s-th layer, e.g. 595 G for $s = 3$ at 5 GHz.

Chapter 24

gyroresonance/transfer.py — polarised transfer along straight rays

File: `radiosynth/gyroresonance/transfer.py` — 225 lines.

Combines harmonic layers and two-mode free–free absorption along each sampled line of sight and returns Stokes I and V and diagnostics.

Module docstring

lines 4–32

```
"""Polarised radiative transfer along straight rays: gyroresonance layers plus
(magnetised) free-free absorption.

For each line of sight, sampled at nodes 0..nk from the observer side
(common/straight.py), and each step k between nodes k and k+1:

* free-free: optical depth  dtau_ff = kappa_sigma(mid-point) dl  for the
  o- and x-modes (bremsstrahlung/magnetized.py);
* gyroresonance: for every harmonic s whose layer |B| = nu/(s nu_B/G) lies
  inside the step (sign change of s nu_B - nu), the layer optical depth
  tau_s (opacity.py) with n_e, T_e, theta interpolated to the crossing and
  L_B = |B| dl / |B_{k+1} - B_k|;
* the step emits S_eff (1 - exp(-dtau)) attenuated by exp(-tau in front),
  with dtau = dtau_ff + sum_s tau_s and S_eff the dtau-weighted mean of
  nu^2 k_B T / c^2 over its free-free part and its layers.

Mode coupling.  Along a ray B . k may change sign (quasi-transverse region).
coupling='weak' (default; usual at microwave frequencies): the sense of
circular polarisation is preserved, so the transfer is done for right- and
left-hand circular polarisation R and L, each using the x-mode coefficients
where it is the x-mode (R: B . k > 0) and the o-mode ones elsewhere;
I = R + L, V = R - L.  coupling='strong': mode identity is preserved, the
transfer is done for the x- and o-modes and V = +/-(I_x - I_o) with the sign
of B . k at the observer end of the ray.

Cutoffs: a polarisation stops where its mode cannot propagate (o: X >= 1,
x: X >= 1 - Y); nothing behind the cutoff contributes.
"""
import numpy as np
```

Describes the step-by-step scheme, the treatment of mode coupling (weak: transfer of right/left circular polarisation; strong: transfer of the modes) and of cutoffs.

Imports

lines 34–39

```
from ..common.constants import C_LIGHT, K_B
from ..common.magnetoionic import NU_B_PER_GAUSS, magnetoionic_params
from ..bremsstrahlung.magnetized import magnetized_ff_coefficients
from .opacity import layer_optical_depth
```

```
38
39 _SRC = K_B / C_LIGHT**2      # S = nu^2 k T / c^2 per mode
```

L.39 $S = \nu^2 k_B T / c^2$ per mode.

transfer_chunk(): fields along the rays

lines 42–90

```
42 def transfer_chunk(sm, dirs, freq, length_unit, *, mechanisms="gr+ff", gaunt="dulk",
43                     s_max=10, coupling="weak", t_body=None, hit_body=None,
44                     x_warn=0.25, extra=None):
45     """Polarised transfer for one chunk of rays from iter_samples.
46
47     Returns dict of per-ray arrays: I1, I2 (the two polarisations: R/L or x/o),
48     tau1, tau2 (total optical depth), V, cutoff (bool), dominant_s (0 = free-free,
49     -1 = no emission), radius (intensity-weighted emission radius), gr_fraction,
50     x_flag (a significant layer had X > x_warn).
51
52     extra : optional additional emission/absorption per step in the circular
53             polarisations (used by radiosynth.combined for gyrosynchrotron):
54             dict of (nr, nk) arrays 'dtau_R', 'dtau_L' (optical depth of the
55             step) and 'st_R', 'st_L' (optical depth x effective temperature
56             S/(nu^2 k/c^2) [K]).  Then out['gs_fraction'] is also returned."""
57     use_gr = "gr" in mechanisms
58     use_ff = "ff" in mechanisms
59     nr, nn = sm["ne"].shape
60     ne, te, b, dl = sm["ne"], sm["te"], sm["b"], sm["dl"] * length_unit
61     nobs = -dirs[sm["idx"]][:, None, :]                        # towards the observer
62     bmag = np.sqrt(np.einsum("ijk,ijk->ij", b, b))
63     cos = np.where(bmag > 0, np.einsum("ijk,ijk->ij", b, nobs) / np.maximum(bmag, 1e-300),
       ↪ 0.0)
64     X, Y = magnetoionic_params(ne, bmag, freq)
65     nu_b = NU_B_PER_GAUSS * bmag
66     live = sm["inside"]
67
68     # ---- cutoffs at the nodes (polarisation-dependent in weak coupling)
69     cut_o = X >= 1.0
70     cut_x = X >= 1.0 - Y
71     # ---- mid-step quantities
72     m = lambda a: 0.5 * (a[:, 1:] + a[:, :-1])                      # noqa: E731
73     ne_m, te_m, cos_m, b_m = m(ne), m(te), m(cos), m(bmag)
74     step_ok = (dl > 0) & live[:, 1:] & live[:, :-1]
75     if use_ff:
76         k_o, k_x, _ = magnetized_ff_coefficients(ne_m, te_m, b_m, cos_m, freq, gaunt=gaunt)
77         dtau_o = np.where(step_ok, k_o * dl, 0.0)
78         dtau_x = np.where(step_ok, k_x * dl, 0.0)
79     else:
80         dtau_o = np.zeros_like(dl)
81         dtau_x = np.zeros_like(dl)
82     st_o = dtau_o * te_m                                        # sum tau*T (source weight)
83     st_x = dtau_x * te_m
84     gr_o = np.zeros_like(dl)
85     gst_x = np.zeros_like(dl)                                   # sum tau*T of the layers
86     gst_o = np.zeros_like(dl)
87     gr_x = np.zeros_like(dl)
88     best_tau = np.zeros_like(dl)                                # largest layer tau in step
89     best_s = np.zeros(dl.shape, np.int16)
```

```
90      x_flag_step = np.zeros(dl.shape, bool)
```

L.57 Which contributions are included (`mechanisms`).

L.61 Unit vector towards the observer, $\mathbf{k}$ direction.

L.62–63 $|B|$ and $\cos\theta = \mathbf{B}\cdot\hat{\mathbf{k}}/|B|$ at the nodes.

L.64–65 X, Y and ν_B at the nodes.

L.69–70 Where each mode is cut off.

L.72–74 Mid-step values; a step counts if both nodes are inside the domain and it has non-zero length.

L.75–81 Free–free optical depth of each step for both modes, $\kappa_\sigma \Delta\ell$ at the mid-point.

L.82–90 Accumulators: $\sum \tau T$ (to build the step's source function), layer optical depths, the strongest harmonic of each step and the validity flag.

transfer_chunk(): harmonic layers

lines 92–120

```
92      if use_gr:
93          f0, f1 = nu_b[:, :-1], nu_b[:, 1:]
94          for s in range(1, int(s_max) + 1):
95              cross = step_ok & ((s * f0 - freq) * (s * f1 - freq) < 0)
96              if not cross.any():
97                  continue
98              ii, kk = np.nonzero(cross)
99              a0, a1 = f0[ii, kk], f1[ii, kk]
100             u = (freq / s - a0) / (a1 - a0)
101             lerp = lambda a: a[ii, kk] + u * (a[ii, kk + 1] - a[ii, kk])   # noqa: E731
102             ne_c, te_c, cos_c, X_c = lerp(ne), lerp(te), lerp(cos), lerp(X)
103             l_b = (freq / (s * NU_B_PER_GAUSS)) * dl[ii, kk] / np.abs(
104                 bmag[ii, kk + 1] - bmag[ii, kk])
105             t_x = layer_optical_depth(s, ne_c, te_c, cos_c, l_b, freq, "x", X=X_c)
106             t_o = layer_optical_depth(s, ne_c, te_c, cos_c, l_b, freq, "o", X=X_c)
107             # the x-mode does not exist at the s = 1 layer when X > 0 (cutoff X >= 1 - Y = 0)
108             if s == 1:
109                 t_x = np.where(X_c > 0, 0.0, t_x)
110             np.add.at(gr_x, (ii, kk), t_x)
111             np.add.at(gr_o, (ii, kk), t_o)
112             np.add.at(st_x, (ii, kk), t_x * te_c)
113             np.add.at(st_o, (ii, kk), t_o * te_c)
114             np.add.at(gst_x, (ii, kk), t_x * te_c)
115             np.add.at(gst_o, (ii, kk), t_o * te_c)
116             tmax = np.maximum(t_x, t_o)
117             better = tmax > best_tau[ii, kk]
118             best_tau[ii[better], kk[better]] = tmax[better]
119             best_s[ii[better], kk[better]] = s
120             x_flag_step[ii, kk] |= (X_c > x_warn) & (tmax > 0.1)
```

L.95 Layer s lies in the step if $s\nu_B - \nu$ changes sign between its nodes.

L.99–102 Fractional position of the crossing and linear interpolation of n_e, T_e, $\cos\theta$, X to it.

L.103 $L_B = |B|/|dB/d\ell|$ with $|B|$ = the layer value and the gradient from the two nodes.

L.105–106 Optical depths of the layer in both modes (opacity.py).

L.108–109 At $s = 1$ ($Y = 1$) the x-mode is cut off for any $X > 0$; it never reaches its fundamental layer.

L.110–113 Add to the step totals (several harmonics can fall in one step).

L.116–120 Remember the strongest harmonic of the step, and flag layers with $\tau > 0.1$ at $X > 0.25$, where $n_\sigma \approx 1$ is a poor approximation.

transfer_chunk(): choice of the two polarisations

lines 122–148

```
122     dtx = dtau_x + gr_x
123     dto = dtau_o + gr_o
124     if coupling == "weak":
125         pos = cos_m > 0                       # R is the x-mode where B.k > 0
126         d1, d2 = np.where(pos, dtx, dto), np.where(pos, dto, dtx)
127         s1, s2 = np.where(pos, st_x, st_o), np.where(pos, st_o, st_x)
128         g1, g2 = np.where(pos, gr_x, gr_o), np.where(pos, gr_o, gr_x)
129         h1, h2 = np.where(pos, gst_x, gst_o), np.where(pos, gst_o, gst_x)
130         posn = cos > 0
131         c1, c2 = np.where(posn, cut_x, cut_o), np.where(posn, cut_o, cut_x)
132     elif coupling == "strong":
133         d1, d2, s1, s2, g1, g2, c1, c2 = dtx, dto, st_x, st_o, gr_x, gr_o, cut_x, cut_o
134         h1, h2 = gst_x, gst_o
135     else:
136         raise ValueError("coupling must be 'weak' or 'strong'")
137     e1 = e2 = None
138     if extra is not None:
139         if coupling == "weak":                    # polarisation 1 = R, 2 = L
140             e1d, e2d, e1s, e2s = extra["dtau_R"], extra["dtau_L"], extra["st_R"], extra["st_L
    ↪ "]
141         else:                                     # 1 = x (= R where B.k > 0), 2 = o
142             pm = cos_m > 0
143             e1d = np.where(pm, extra["dtau_R"], extra["dtau_L"])
144             e2d = np.where(pm, extra["dtau_L"], extra["dtau_R"])
145             e1s = np.where(pm, extra["st_R"], extra["st_L"])
146             e2s = np.where(pm, extra["st_L"], extra["st_R"])
147         d1, d2, s1, s2 = d1 + e1d, d2 + e2d, s1 + e1s, s2 + e2s
148         e1, e2 = e1s, e2s
```

`h1`, `h2` carry $\sum \tau T$ of the layers (needed for the shares when an extra source is present).

L.137–148 Optional extra contribution (gyrosynchrotron in the combined image): optical depth and $\tau T_{\rm eff}$ in R/L, mapped to the two polarisations (in strong coupling $x = R$ where $\mathbf{B}\cdot\mathbf{k} > 0$) and added to the step sums.

L.124–131 Weak coupling: polarisation 1 = right-hand circular, which is the x-mode where $\mathbf{B}\cdot\mathbf{k} > 0$ and the o-mode elsewhere; polarisation 2 = left-hand. The cutoffs are chosen per node in the same way.

L.132–133 Strong coupling: polarisation 1 = x-mode, 2 = o-mode.

transfer_chunk(): formal solution

lines 150–191

```
150     out = {}
151     contrib_tot = np.zeros_like(dl)
152     contrib_gr = np.zeros_like(dl)
153     contrib_gs = np.zeros_like(dl)
154     cut_any = np.zeros(nr, bool)
```

```
155     for tag, d, sw, g, c, e, h in (("1", d1, s1, g1, c1, e1, h1), ("2", d2, s2, g2, c2, e2,
        ↪ h2)):
156         # polarisation stops at its first cutoff node (steps beyond contribute nothing)
157         alive = np.cumprod(~(c & live), axis=1).astype(bool)
158         ok = alive[:, 1:]
159         d = np.where(ok, d, 0.0)
160         sw = np.where(ok, sw, 0.0)
161         g = np.where(ok, g, 0.0)
162         tau_front = np.cumsum(d, axis=1) - d
163         with np.errstate(invalid="ignore", divide="ignore"):
164             S = np.where(d > 0, sw / np.where(d > 0, d, 1.0), 0.0) * _SRC * freq**2
165         w = np.exp(-tau_front) * -np.expm1(-d)
166         contrib = S * w
167         I = contrib.sum(axis=1)
168         tau = d.sum(axis=1)
169         cut_ray = ~alive[:, -1]
170         if t_body is not None and hit_body is not None:
171             body = hit_body[sm["idx"]] & ~cut_ray
172             I = I + np.where(body, _SRC * freq**2 * float(t_body) * np.exp(-tau), 0.0)
173         out["I" + tag], out["tau" + tag] = I, tau
174         contrib_tot += contrib
175         with np.errstate(invalid="ignore", divide="ignore"):
176             gsp = 0.0
177             if e is None:
178                 grp = np.where(d > 0, contrib * g / np.where(d > 0, d, 1.0), 0.0)
179             else:
180                 # with an extra source the shares are those of the source function
181                 hh = np.where(ok, h, 0.0)
182                 grp = np.where(sw > 0, contrib * hh / np.where(sw > 0, sw, 1.0), 0.0)
183                 # share of the step's source function due to the extra part:
184                 # S = sum(tau_i T_i)/sum(tau_i), so the share is st_extra / sw
185                 ee = np.where(ok, e, 0.0)
186                 gsp = np.where(sw > 0, contrib * ee / np.where(sw > 0, sw, 1.0), 0.0)
187                 contrib_gs += gsp
188             contrib_gr += grp
189         # free-free part of this polarisation (as seen by the observer)
190         out["ff" + tag] = (contrib - grp - gsp).sum(axis=1)
191         cut_any |= cut_ray
```

L.157–161 A polarisation stops at its first cutoff node: later steps are switched off.

L.162 Optical depth between the observer and the start of each step.

L.164 Source function of the step: the τ-weighted mean of $\nu^2 k_B T/c^2$ over its free–free part and its layers.

L.165–167 $I = \sum_k S_k(1-e^{-\Delta\tau_k})e^{-\tau_{<k}}$, the exact solution for piecewise-constant S.

L.170–172 Optional body (chromosphere) at temperature `t_body` behind rays that hit it.

L.174–188 Per-step contributions, total and from the layers; with an extra source the shares are those of the source function, $\Delta\tau_i S_i/\sum\Delta\tau S$, for the layers and the extra part.

L.190 The free–free part of the polarisation (used by the refraction correction of the combined image).

transfer_chunk(): Stokes V and diagnostics

lines 193–225

```
193     if coupling == "weak":
194         out["V"] = out["I1"] - out["I2"]
```

```
195         out["vsign"] = np.ones(nr)
196     else:
197         has_b = bmag > 0
198         first = np.argmax(has_b, axis=1)
199         sgn = np.sign(cos[np.arange(nr), first])
200         out["V"] = sgn * (out["I1"] - out["I2"])
201         out["vsign"] = sgn
202 
203     tot = contrib_tot.sum(axis=1)
204     r_m = np.linalg.norm(0.5 * (sm["p"][:, 1:] + sm["p"][:, :-1]), axis=-1)
205     with np.errstate(invalid="ignore", divide="ignore"):
206         out["radius"] = np.where(tot > 0, (contrib_tot * r_m).sum(axis=1) / tot, np.nan)
207         out["gr_fraction"] = np.where(tot > 0, contrib_gr.sum(axis=1) / tot, 0.0)
208         if extra is not None:
209             out["gs_fraction"] = np.where(tot > 0, contrib_gs.sum(axis=1) / tot, 0.0)
210     # dominant harmonic: the s whose layers carry most of the intensity
211     dom = np.full(nr, -1, np.int16)
212     if use_gr:
213         share = np.zeros((int(s_max) + 1, nr))
214         np.add.at(share, (best_s.ravel(), np.repeat(np.arange(nr), dl.shape[1])),
215                   contrib_gr.ravel())
216         share[0] = contrib_tot.sum(axis=1) - contrib_gr.sum(axis=1)   # free-free part
217         if extra is not None:
218             share[0] -= contrib_gs.sum(axis=1)
219         dom = np.where(tot > 0, np.argmax(share, axis=0), -1).astype(np.int16)
220     elif use_ff:
221         dom = np.where(tot > 0, 0, -1).astype(np.int16)
222     out["dominant_s"] = dom
223     out["cutoff"] = cut_any
224     out["x_flag"] = x_flag_step.any(axis=1)
225     return out
```

L.194 Weak coupling: $V = I_R - I_L$ (`vsign` $= 1$).

L.197–200 Strong coupling: $V = \pm(I_x - I_o)$, sign of $\mathbf{B}\cdot\mathbf{k}$ at the observer end of the ray.

L.203–207 Intensity-weighted emission radius and the fraction of I coming from layers.

L.211–222 Dominant harmonic: the s whose layers contribute most; 0 if free–free dominates, -1 if there is no emission.

Chapter 25

gyroresonance/image.py — make_gr_image and GRImage

File: `radiosynth/gyroresonance/image.py` — 270 lines.

The driver: the same image geometry and observers as the refractive images, straight rays, polarised transfer, and a `RadioImage` subclass with Stokes V and diagnostics.

Module docstring and imports

lines 4–23

```
4  """Gyroresonance + free-free images with circular polarisation.
5
6  make_gr_image uses the same image-plane geometry, observers (Earth-based,
7  dated, stellar/parallel) and output container as the refractive free-free
8  images (common/image.py), but straight lines of sight (common/straight.py)
9  and the polarised transfer of transfer.py.
10 """
11 import math
12 import time
13
14 import numpy as np
15
16 from ..common.constants import R_SUN, critical_density
17 from ..common.image import (RadioImage, freq_label, image_geometry, parse_frequencies,
18                             start_points)
19 from ..common.raytrace import HIT_BODY, MISSED, OVERDENSE
20 from ..common.straight import default_step, iter_samples, segment_ends
21 from .transfer import transfer_chunk
22
23 PC = 3.0856775814913673e16
```

GRImage: attributes

lines 26–51

```
26 class GRImage(RadioImage):
27     """RadioImage with Stokes V and gyroresonance diagnostics.
28
29     intensity   : Stokes I = I_1 + I_2      (nfreq, ny, nx) [W m^-2 Hz^-1 sr^-1]
30     stokes_v    : Stokes V (IAU/IEEE: V > 0 right-hand circular)
31     pol1, pol2  : intensities of the two polarisations: R, L (coupling='weak')
32                   or x, o (coupling='strong')
33     tau         : optical depth of polarisation 1;  tau2 that of polarisation 2
34     dominant_s  : harmonic carrying most of the intensity (0 = free-free,
35                   -1 = no emission)
36     radius      : intensity-weighted emission radius [coordinate units]
37     gr_fraction : fraction of I from gyroresonance layers
38     x_flag      : True where a significant layer had (nu_p/nu)^2 > 0.25 (the
39                   n = 1 approximation of the layer opacity is then rough)
40     """
```

```
41
42      def __init__(self, *a, stokes_v=None, pol1=None, pol2=None, tau2=None, dominant_s=None,
43                   radius=None, gr_fraction=None, x_flag=None, pol_names=("R", "L"), **kw):
44          super().__init__(*a, **kw)
45          self.stokes_v = stokes_v
46          self.pol1, self.pol2, self.tau2 = pol1, pol2, tau2
47          self.dominant_s = dominant_s
48          self.radius = radius
49          self.gr_fraction = gr_fraction
50          self.x_flag = x_flag
51          self.pol_names = tuple(pol_names)
```

Inherits intensity, brightness temperature, flux density, FITS/Tecplot output from `RadioImage`; adds Stokes V, the two polarisations, their optical depths and the diagnostics.

`L.43` ('R','L') for weak, ('x','o') for strong coupling.

GRImage: derived quantities

lines 53–82

```
53      def _tb(self, a):
54          return a * self.brightness_temperature_factor
55
56      @property
57      def brightness_temperature_factor(self):
58          from ..common.constants import C_LIGHT, K_B
59          return C_LIGHT**2 / (2.0 * K_B * self.freqs[:, None, None] ** 2)
60
61      @property
62      def tb_v(self):
63          """Stokes V as a brightness temperature [K]."""
64          return self._tb(self.stokes_v)
65
66      @property
67      def tb_pol(self):
68          """Brightness temperatures of the two polarisations, T_p = I_p c^2/(k nu^2)
69          (each polarisation carries half of the Rayleigh-Jeans intensity, so
70          T_p = T_e for an optically thick isothermal source and
71          T_b(I) = (T_1 + T_2)/2)."""
72          return 2.0 * self._tb(self.pol1), 2.0 * self._tb(self.pol2)
73
74      @property
75      def polarization(self):
76          """Degree of circular polarisation V/I (NaN where I = 0)."""
77          with np.errstate(invalid="ignore", divide="ignore"):
78              return np.where(self.intensity > 0, self.stokes_v / self.intensity, np.nan)
79
80      @property
81      def flux_density_v(self):
82          return self.stokes_v.sum(axis=(1, 2)) * self.pixel_solid_angle
```

`L.62` Stokes V as a brightness temperature.

`L.67` Per-polarisation $T_p = I_p c^2/(k_B \nu^2)$, equal to T_e for a thick isothermal source.

`L.75` V/I.

`L.81` Flux density of Stokes V.

GRImage: summary and output

lines 84–122

```
84      def summary(self):
85          lines = []
86          for k, f in enumerate(self.freqs):
87              I = self.flux_density[k]
88              V = self.flux_density_v[k]
89              tb = self.brightness_temperature[k]
90              ds = self.dominant_s[k]
91              harm = ", ".join(f"s={s}:{int((ds == s).sum())}" for s in np.unique(ds) if s > 0)
92              lines.append(f"{freq_label(f):>12s}: max Tb={tb.max():.3e} K, flux={I / 1e-22:.3e
    ↪ } sfu"
93                           f" ({I / 1e-26:.3e} Jy), V/I={V / I if I > 0 else 0:+.3f}, "
94                           f"GR fraction of flux={np.nansum(self.gr_fraction[k] * self.
    ↪ intensity[k]) / max(self.intensity[k].sum(), 1e-300):.2f}"
95                           f"; dominant harmonic pixels: {harm or 'none'}")
96          return "\n".join(lines)
97
98      def save_npz(self, path):
99          np.savez_compressed(path, intensity=self.intensity, stokes_v=self.stokes_v,
100                              pol1=self.pol1, pol2=self.pol2, pol_names=np.array(self.pol_names
    ↪ ),
101                              tau=self.tau, tau2=self.tau2, status=self.status, x=self.x, y=
    ↪ self.y,
102                              freqs=self.freqs, observer=self.observer, distance=self.distance,
103                              length_unit=self.length_unit, flux_density=self.flux_density,
104                              flux_density_v=self.flux_density_v, tb=self.
    ↪ brightness_temperature,
105                              tb_v=self.tb_v, dominant_s=self.dominant_s, radius=self.radius,
106                              gr_fraction=self.gr_fraction, x_flag=self.x_flag, meta=repr(self.
    ↪ meta))
107
108     def save_fits(self, path, quantity="intensity"):
109         """quantity: 'intensity', 'tb', 'v' (Stokes V intensity), 'tb_v', 'pol'."""
110         from ..common.fitsio import write_fits
111         if quantity in ("intensity", "tb"):
112             return super().save_fits(path, quantity)
113         data = {"v": self.stokes_v, "tb_v": self.tb_v, "pol": self.polarization}[quantity]
114         unit = {"v": "W m-2 Hz-1 sr-1", "tb_v": "K", "pol": ""}[quantity]
115         h = {"BUNIT": unit, "STOKES": "V" if quantity != "pol" else "V/I",
116              "CTYPE1": "SOLX", "CUNIT1": "Rsun", "CRPIX1": 1.0,
117              "CRVAL1": float(self.x[0]), "CDELT1": float(self.x[1] - self.x[0]),
118              "CTYPE2": "SOLY", "CUNIT2": "Rsun", "CRPIX2": 1.0,
119              "CRVAL2": float(self.y[0]), "CDELT2": float(self.y[1] - self.y[0])}
120         for k, f in enumerate(self.freqs):
121             h[f"FREQ{k + 1}"] = float(f)
122         write_fits(path, np.nan_to_num(data), h)
```

L.84 One line per frequency: peak T_b, flux in sfu and Jy, V/I of the total flux, GR fraction and the dominant harmonics.

L.98 All arrays, including V, the polarisations and the diagnostics.

L.108 `'intensity'`, `'tb'` (as `RadioImage`), or `'v'`, `'tb_v'`, `'pol'`.

GRImage.plot()

lines 124–178

```
    def plot(self, quantity="tb", path=None, log=None, cmap=None, ncols=None,
             vmin=None, vmax=None, show_limb=True):
        """quantity: 'tb', 'intensity', 'v' (T_b of Stokes V), 'pol' (V/I),
        'harmonic' (dominant s), 'radius', 'gr_fraction'."""
        if quantity in ("tb", "intensity"):
            return super().plot(quantity, path=path, log=True if log is None else log,
                                cmap=cmap or "inferno", ncols=ncols, vmin=vmin, vmax=vmax,
                                show_limb=show_limb)
        import matplotlib
        if path is not None:
            matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        data, label, cm, lo, hi = {
            "v": (self.tb_v, r"$T_b(V)$ [K]", "RdBu_r", None, None),
            "pol": (self.polarization, "V/I", "RdBu_r", -1.0, 1.0),
            "harmonic": (np.where(self.dominant_s >= 0, self.dominant_s, np.nan).astype(float
    ↪ ),
                         "dominant harmonic s (0 = free-free)", "viridis", None, None),
            "radius": (self.radius, r"emission radius [$R$]", "viridis", None, None),
            "gr_fraction": (self.gr_fraction, getattr(self, "fraction_label", "gyroresonance
    ↪ fraction"),
                            "magma", 0.0, 1.0),
        }[quantity]
        nf = len(self.freqs)
        ncols = ncols or min(nf, 3)
        nrows = math.ceil(nf / ncols)
        fig, axes = plt.subplots(nrows, ncols, figsize=(4.6 * ncols, 4.0 * nrows), squeeze=
    ↪ False)
        dx, dy = self.x[1] - self.x[0], self.y[1] - self.y[0]
        ext = [self.x[0] - dx / 2, self.x[-1] + dx / 2, self.y[0] - dy / 2, self.y[-1] + dy /
    ↪  2]
        for k in range(nrows * ncols):
            ax = axes.flat[k]
            if k >= nf:
                ax.axis("off")
                continue
            d = data[k]
            a, b = (vmin if vmin is not None else lo), (vmax if vmax is not None else hi)
            if quantity == "v" and a is None:
                m = np.nanmax(np.abs(d)) or 1.0
                a, b = -m, m
            im = ax.imshow(d, origin="lower", extent=ext, cmap=cmap or cm, vmin=a, vmax=b)
            if show_limb:
                t = np.linspace(0, 2 * np.pi, 361)
                ax.plot(np.cos(t), np.sin(t), "k--", lw=0.7, alpha=0.6)
            ax.set_xlim(ext[0], ext[1])
            ax.set_ylim(ext[2], ext[3])
            ax.set_title(freq_label(self.freqs[k]))
            ax.set_xlabel(r"X [$R_\odot$]")
            ax.set_ylabel(r"Y [$R_\odot$]")
            fig.colorbar(im, ax=ax, label=label, shrink=0.85)
        fig.tight_layout()
        if path is not None:
            fig.savefig(path, dpi=130)
            plt.close(fig)
        return fig

```

```
177
178 # ---------------------------------------------------------------------------
```

`'tb'`, `'intensity'` as before; `'v'` (symmetric colour scale), `'pol'` (−1 to 1), `'harmonic'`, `'radius'`, `'gr_fraction'`.

make_gr_image(): set-up

lines 179–231

```
179 def make_gr_image(grid, frequencies, observer=(-215.0, 0.0, 0.0), x_size=2.0, y_size=2.0,
180                   nx=128, ny=128, *, mechanisms="gr+ff", gaunt="dulk", s_max=10,
181                   coupling="weak", ds=None, t_body=None, r_integration=None,
182                   up=(0.0, 0.0, 1.0), distance_pc=None, max_points=400000, verbose=True):
183     """Gyroresonance (+ free-free) image with Stokes I and V along straight rays.
184
185     grid        : MHDGrid / SphericalGrid with a magnetic field (add_field or
186                   load_mhd(..., with_b=True)); coordinates and B in gauss
187     frequencies : '5GHz, 17GHz' or numbers in Hz
188     observer, x_size, y_size, nx, ny, up, distance_pc, r_integration : as in
189                   make_radio_image (the default field of view is 2 x 2 R_sun)
190     mechanisms  : 'gr+ff' (default: both, absorbing each other), 'gr' (layers
191                   only) or 'ff' (magnetised free-free only, straight rays)
192     gaunt       : free-free Gaunt factor (number or 'dulk', default 'dulk')
193     s_max       : highest harmonic considered (default 10)
194     coupling    : 'weak' (polarisation sense conserved through quasi-transverse
195                   regions; default) or 'strong' (mode conserved)
196     ds          : step along the rays [coordinate units] (default half the
197                   smallest cell); the layers are found between the nodes, so
198                   ds only has to resolve the variation of n_e, T_e and B
199     t_body      : brightness temperature of the body (e.g. 1e4 K for the
200                   chromosphere) seen by rays that reach r_body; default None
201                   (no emission, as in the refractive tracer)
202     """
203     if not getattr(grid, "has_field", False):
204         raise ValueError("gyroresonance needs the magnetic field: grid.add_field(bx, by, bz)
    ↪ "
205                          "or load_mhd(..., with_b=True)")
206     mechanisms = mechanisms.lower().replace(" ", "")
207     if mechanisms not in ("gr+ff", "ff+gr", "gr", "ff"):
208         raise ValueError("mechanisms must be 'gr+ff', 'gr' or 'ff'")
209     freqs = parse_frequencies(frequencies)
210     L = float(getattr(grid, "length_unit", R_SUN))
211     parallel = distance_pc is not None
212     xc, yc, dirs, tau_v, xi_v, normal, pix = image_geometry(observer, x_size, y_size, nx, ny,
213                                                             up, parallel=parallel)
214     diag = math.sqrt((0.5 * x_size) ** 2 + (0.5 * y_size) ** 2)
215     if r_integration is None:
216         r_integration = float(math.ceil(max(diag + 1.0, 5.0)))
217     if parallel:
218         distance = distance_pc * PC / L
219         far = 4.0 * max(r_integration, getattr(grid, "r_outer", r_integration), diag)
220         pos, status0 = start_points(grid, pix + far * normal, dirs, r_integration)
221     else:
222         distance = float(np.linalg.norm(observer))
223         pos, status0 = start_points(grid, observer, dirs, r_integration)
224     t_end, status = segment_ends(grid, pos, dirs, status0, r_integration)
225     step = default_step(grid, t_end, ds)
```

```
226     hit_body = status == HIT_BODY
227     nray = dirs.shape[0]
228     if verbose:
229         print(f"radiosynth.gyroresonance: {nx}x{ny} straight rays, mechanisms={mechanisms}, "
230               f"coupling={coupling}, s_max={s_max}, step={step:.3g}, gaunt={gaunt}")
231         print(f"            {grid}")
```

L.203–208 Check that the grid has a field and that `mechanisms` is valid.

L.212–223 Image geometry and start points exactly as in `make_radio_image`, including the parallel (stellar) projection.

L.224–226 Length of each line and the step.

make_gr_image(): loop over frequencies and chunks

lines 233–270

```
233     nf = len(freqs)
234     keys = ("I1", "I2", "tau1", "tau2", "V", "radius", "gr_fraction")
235     res = {k: np.zeros((nf, nray)) for k in keys}
236     res["radius"][:] = np.nan
237     dom = np.full((nf, nray), -1, np.int16)
238     cut = np.zeros((nf, nray), bool)
239     xf = np.zeros((nf, nray), bool)
240     for k, f in enumerate(freqs):
241         t0 = time.time()
242         for sm in iter_samples(grid, pos, dirs, t_end, step, max_points=max_points):
243             o = transfer_chunk(sm, dirs, f, L, mechanisms=mechanisms, gaunt=gaunt,
244                                s_max=s_max, coupling=coupling, t_body=t_body,
245                                hit_body=hit_body)
246             i = sm["idx"]
247             for key in keys:
248                 res[key][k, i] = o[key]
249             dom[k, i], cut[k, i], xf[k, i] = o["dominant_s"], o["cutoff"], o["x_flag"]
250         if verbose:
251             print(f"   {freq_label(f)}: {time.time() - t0:.1f} s  (n_cr = {critical_density(f
    ↪ ):.3e} m^-3)")
252 
253     shp = (nf, ny, nx)
254     st = np.broadcast_to(status, (nf, nray)).copy()
255     st[cut & (st != MISSED)] = OVERDENSE
256     meta = dict(x_size=x_size, y_size=y_size, nx=nx, ny=ny, mechanisms=mechanisms,
257                 gaunt=gaunt, s_max=s_max, coupling=coupling, ds=step, t_body=t_body,
258                 distance_pc=distance_pc, r_integration=r_integration,
259                 tau_vec=tau_v, xi_vec=xi_v, emission="gyroresonance", transfer="full")
260     R = lambda a: a.reshape(shp)                                    # noqa: E731
261     img = GRImage(R(res["I1"] + res["I2"]), R(res["tau1"]), R(st),
262                   np.full(shp, int(np.ceil(np.max(t_end, initial=0) / step))), xc, yc, freqs,
263                   observer, meta, distance=distance, length_unit=L,
264                   stokes_v=R(res["V"]), pol1=R(res["I1"]), pol2=R(res["I2"]),
265                   tau2=R(res["tau2"]), dominant_s=R(dom), radius=R(res["radius"]),
266                   gr_fraction=R(res["gr_fraction"]), x_flag=R(xf),
267                   pol_names=("R", "L") if coupling == "weak" else ("x", "o"))
268     if verbose:
269         print(img.summary())
270     return img
```

L.242–249 Sample a chunk, do the transfer, store the per-ray results.

L.254–255 Rays stopped by a cutoff get the status 'over-dense'.

L.261–267 Assemble the image: $I = I_1 + I_2$, τ of polarisation 1 in `tau`, everything reshaped to (nfreq, ny, nx).

Chapter 26

gyroresonance/__init__.py — package interface

File: `radiosynth/gyroresonance/__init__.py` — 36 lines.

Registers the mechanism and exports its functions.

Mechanism registration

lines 4–36

```
4  """Thermal gyroresonance emission (with magnetised free-free emission).
5
6  * opacity.py  : optical depth of the harmonic layers nu = s nu_B for the x- and
7                  o-modes (Zheleznyakov 1970; White & Kundu 1997)
8  * transfer.py : polarised transfer along straight rays: layers + two-mode
9                  free-free, Stokes I and V, weak or strong mode coupling
10 * image.py    : make_gr_image and the GRImage container (I, V, per-polarisation
11                 T_b, dominant harmonic, emission radius, GR fraction)
12
13 Uses from common/: grids (with the magnetic field), readers, image-plane
14 geometry, straight-ray sampler, magneto-ionic theory, figures; from
15 bremsstrahlung/: the magnetised free-free coefficients.
16
17 >>> import radiosynth as rs
18 >>> grid = rs.load_mhd("3d__mhd_ar.dat", with_b=True)
19 >>> img = rs.gyroresonance.make_image(grid, "5GHz, 17GHz", x_size=1, y_size=1)
20 >>> img.plot("tb"); img.plot("pol"); img.plot("harmonic")
21 """
22 from ..common.mechanism import Mechanism, register
23 from .opacity import layer_optical_depth, mode_factor, harmonic_field, GR_PREFACTOR
24 from .transfer import transfer_chunk
25 from .image import make_gr_image, GRImage
26
27 make_image = make_gr_image
28
29 MECHANISM = register(Mechanism(
30     "gyroresonance", coefficients=None, make_image=make_gr_image,
31     description="thermal gyroresonance layers + magnetised free-free, straight rays, "
32                 "Stokes I and V (needs B)",
33     options=("mechanisms", "s_max", "coupling", "gaunt", "t_body", "ds")))
34
35 __all__ = ["layer_optical_depth", "mode_factor", "harmonic_field", "GR_PREFACTOR",
36            "transfer_chunk", "make_gr_image", "make_image", "GRImage", "MECHANISM"]
```

The docstring lists the three modules and what they take from `common/` and `bremsstrahlung/`.

`L.27` Same name as in the other folders.

`L.29` `coefficients=None`: gyroresonance cannot be used inside the refractive tracer.

Chapter 27

gyrosynchrotron/library.py — the fast GS codes as a library

File: `radiosynth/gyrosynchrotron/library.py` — 200 lines.

Finds, builds and loads the public fast gyrosynchrotron codes (Kuznetsov & Fleishman 2021). The codes are GPL-3 and are not part of radiosynth; this module only calls them through `ctypes`.

Module docstring and imports

lines 4–39

```
4  """Access to the public "Ultimate fast gyrosynchrotron codes".
5
6  The gyrosynchrotron emission is computed by the fast GS codes of Fleishman &
7  Kuznetsov (2010, ApJ 721, 1127) and Kuznetsov & Fleishman (2021, ApJ 922, 103),
8  the engine also used by GX Simulator:
9
10     https://github.com/kuznetsov-radio/gyrosynchrotron      (GPL-3.0)
11
12 radiosynth does not include that code.  It loads the compiled library
13 (MWTransferArr.so / .dll) with ctypes and calls its two Python entry points,
14 pyGET_MW (one line of sight) and pyGET_MW_SLICE (many lines of sight, OpenMP).
15 The calling conventions are those of the code's CallingConventions.pdf.
16
17 Where the library is looked for (first match wins):
18   1. the ``path`` argument of load_library() / make_gs_image(library=...);
19   2. the environment variable RADIOSYNTH_GSCODES (file, or directory that
20      contains the library);
21   3. ~/.radiosynth/gyrosynchrotron/binaries/ (where build_library() puts it).
22
23 build_library() clones the repository and compiles it with make (g++ and
24 OpenMP; on macOS "brew install libomp").  If compilation fails it can fall
25 back to the prebuilt binaries shipped in the repository.
26 """
27 import ctypes
28 import os
29 import platform
30 import shutil
31 import subprocess
32
33 import numpy as np
34 from numpy.ctypeslib import ndpointer
35
36 REPO_URL = "https://github.com/kuznetsov-radio/gyrosynchrotron.git"
37 DEFAULT_HOME = os.path.join(os.path.expanduser("~"), ".radiosynth", "gyrosynchrotron")
38
39 _CACHE = {}
```

Describes the origin of the codes, where the library is searched for, and how `build_library` works.

`L.36–37` Repository and default installation folder (`~/.radiosynth/gyrosynchrotron`).

_candidate_names()

lines 42–53

```
42 def _candidate_names():
43     sysname = platform.system()
44     mach = platform.machine().lower()
45     if sysname == "Windows":
46         return ["MWTransferArr64.dll" if platform.architecture()[0] == "64bit"
47                 else "MWTransferArr32.dll"]
48     if sysname == "Darwin":
49         return (["MWTransferArr_arm64.so"] if mach in ("arm64", "aarch64")
50                 else ["MWTransferArr_x86_64.so", "MWTransferArr.so"])
51     if mach in ("arm64", "aarch64"):
52         return ["MWTransferArr.so", "MWTransferArr_arm64.so"]
53     return ["MWTransferArr.so"]
```

Library file names produced by the repository's makefile or shipped prebuilt, for Windows (32/64-bit DLL), macOS (x86-64, arm64) and Linux.

find_library()

lines 56–79

```
56 def find_library(path=None):
57     """Return the path of the GS-codes shared library, or raise FileNotFoundError
58     with instructions."""
59     cands = []
60     for base in (path, os.environ.get("RADIOSYNTH_GSCODES"),
61                  os.path.join(DEFAULT_HOME, "binaries")):
62         if not base:
63             continue
64         base = os.path.expanduser(base)
65         if os.path.isfile(base):
66             return base
67         for d in (base, os.path.join(base, "binaries")):
68             for n in _candidate_names():
69                 cands.append(os.path.join(d, n))
70     for c in cands:
71         if os.path.isfile(c):
72             return c
73     raise FileNotFoundError(
74         "The fast gyrosynchrotron codes (Kuznetsov & Fleishman 2021) were not found.\n"
75         "Build them once with\n"
76         "    python -m radiosynth.gyrosynchrotron.library --build\n"
77         "(clones " + REPO_URL + " into ~/.radiosynth/gyrosynchrotron and runs make),\n"
78         "or set RADIOSYNTH_GSCODES to the compiled MWTransferArr library / its folder.\n"
79         "Looked for: " + ", ".join(cands))
```

L.60–69 Search order: explicit path, `RADIOSYNTH_GSCODES`, the default folder; each may be the file itself or a folder (or its `binaries/` subfolder).

L.57 If nothing is found, explain how to build or point to the library.

GSLibrary: ctypes binding

lines 82–129

```
82 class GSLibrary:
83     """ctypes binding of pyGET_MW and pyGET_MW_SLICE."""
```

```
84
85      def __init__(self, path):
86          self.path = path
87          self._lib = ctypes.CDLL(path)
88          ip = ndpointer(dtype=np.int32, flags="F")
89          dp = ndpointer(dtype=np.float64, flags="F")
90          # on Linux/macOS the exported functions return double, on Windows int
91          rt = ctypes.c_int if platform.system() == "Windows" else ctypes.c_double
92          for name in ("pyGET_MW", "pyGET_MW_SLICE"):
93              f = getattr(self._lib, name)
94              f.argtypes = [ip, dp, dp, dp, dp, dp, dp]
95              f.restype = rt
96          self._mw = self._lib.pyGET_MW
97          self._slice = self._lib.pyGET_MW_SLICE
98
99      def __repr__(self):
100         return f"GSLibrary({self.path})"
101
102     @staticmethod
103     def _arrays(*a):
104         return [np.asfortranarray(x) for x in a]
105
106     def get_mw(self, Lparms, Rparms, Parms, E_arr, mu_arr, f_arr, RL):
107         """One line of sight (see CallingConventions.pdf).  RL is modified in place."""
108         Lparms = np.asfortranarray(Lparms, dtype=np.int32)
109         Rparms, Parms, E_arr, mu_arr, f_arr = (np.asfortranarray(x, dtype=np.float64)
110                                                for x in (Rparms, Parms, E_arr, mu_arr, f_arr)
    ↪ )
111         if not (RL.flags.f_contiguous and RL.dtype == np.float64):
112             raise ValueError("RL must be a Fortran-ordered float64 array")
113         res = int(self._mw(Lparms, Rparms, Parms, E_arr, mu_arr, f_arr, RL))
114         if res != 0:
115             raise RuntimeError(f"GET_MW returned error code {res}")
116         return RL
117
118     def get_mw_slice(self, Lparms_M, Rparms_M, Parms_M, E_arr, mu_arr, f_arr_M, RL_M):
119         """Many lines of sight (multi-threaded).  RL_M is modified in place."""
120         Lparms_M = np.asfortranarray(Lparms_M, dtype=np.int32)
121         Rparms_M, Parms_M, E_arr, mu_arr, f_arr_M = (
122             np.asfortranarray(x, dtype=np.float64)
123             for x in (Rparms_M, Parms_M, E_arr, mu_arr, f_arr_M))
124         if not (RL_M.flags.f_contiguous and RL_M.dtype == np.float64):
125             raise ValueError("RL_M must be a Fortran-ordered float64 array")
126         res = int(self._slice(Lparms_M, Rparms_M, Parms_M, E_arr, mu_arr, f_arr_M, RL_M))
127         if res != 0:
128             raise RuntimeError(f"GET_MW_SLICE returned error code {res}")
129         return RL_M
```

L.88–89 Argument types: Fortran-ordered int32 and float64 arrays, as required by the codes.

L.91–95 The exported `pyGET_MW` and `pyGET_MW_SLICE` return `double` on Linux/macOS and `int` on Windows (see the library's PythonInterface.cpp).

L.106–116 One line of sight: inputs converted to the right types and order, `RL` modified in place; a non-zero return code raises an error.

L.118–129 Many lines of sight (OpenMP inside the library).

load_library()

lines 132–137

```
132 def load_library(path=None):
133     """Load (once) and return the GSLibrary."""
134     p = find_library(path)
135     if p not in _CACHE:
136         _CACHE[p] = GSLibrary(p)
137     return _CACHE[p]
```

Loads each library file once and caches the binding.

build_library()

lines 140–176

```
140 def build_library(dest=DEFAULT_HOME, repo=REPO_URL, ref=None, use_prebuilt=True,
141                   verbose=True):
142     """Clone the fast GS codes into ``dest`` and compile them with make.
143 
144     ref          : git branch/tag/commit to check out (default: latest)
145     use_prebuilt : if compilation fails, use the binary shipped in the
146                    repository for this platform (if any)
147     Returns the path of the library."""
148     dest = os.path.expanduser(dest)
149     if not os.path.isdir(os.path.join(dest, "source")):
150         if shutil.which("git") is None:
151             raise RuntimeError("git is needed to download the GS codes "
152                                f"(or download {repo} by hand into {dest})")
153         os.makedirs(os.path.dirname(dest), exist_ok=True)
154         cmd = ["git", "clone", repo, dest] if ref else ["git", "clone", "--depth", "1", repo,
    ↪  dest]
155         if verbose:
156             print(" ".join(cmd))
157         subprocess.run(cmd, check=True)
158         if ref:
159             subprocess.run(["git", "-C", dest, "checkout", ref], check=True)
160     built = False
161     if platform.system() != "Windows" and shutil.which("make"):
162         if verbose:
163             print(f"compiling in {os.path.join(dest, 'source')} ...")
164         r = subprocess.run(["make"], cwd=os.path.join(dest, "source"),
165                            capture_output=not verbose)
166         built = r.returncode == 0
167     if not built:
168         if not use_prebuilt:
169             raise RuntimeError("compilation of the GS codes failed")
170         if verbose:
171             print("compilation not possible/failed: using the prebuilt binary of the
    ↪ repository")
172     path = find_library(dest)
173     _CACHE.pop(path, None)
174     if verbose:
175         print("GS codes library:", path)
176     return path
```

L.149–159 Clone the repository (optionally a given commit or tag, for reproducibility).

L.160–166 Compile with the repository's makefile.

L.167–171 Otherwise fall back to the prebuilt binary of the repository, if allowed.

library_version()

lines 179–187

```
179 def library_version(path=None):
180     """git commit of the library checkout (if it is a git clone), else None."""
181     p = find_library(path)
182     d = os.path.dirname(os.path.dirname(p))
183     if shutil.which("git") and os.path.isdir(os.path.join(d, ".git")):
184         r = subprocess.run(["git", "-C", d, "log", "-1", "--format=%h %cd", "--date=short"],
185                            capture_output=True, text=True)
186         return r.stdout.strip() or None
187     return None
```

The git commit and date of the checkout, printed with every image so that results can be reproduced.

Command line

lines 190–200

```
190 if __name__ == "__main__":                                  # pragma: no cover
191     import argparse
192     ap = argparse.ArgumentParser(description="Fast gyrosynchrotron codes for radiosynth")
193     ap.add_argument("--build", action="store_true", help="download and compile the codes")
194     ap.add_argument("--dest", default=DEFAULT_HOME)
195     ap.add_argument("--ref", default=None, help="git branch/tag/commit")
196     a = ap.parse_args()
197     if a.build:
198         build_library(a.dest, ref=a.ref)
199     else:
200         print(find_library())
```

`python -m radiosynth.gyrosynchrotron.library --build [--ref COMMIT]` builds the codes; without options it prints the path found.

Chapter 28

gyrosynchrotron/electrons.py — the fast-electron prescription

File: `radiosynth/gyrosynchrotron/electrons.py` — 212 lines.

`FastElectrons` holds the description of the fast electrons; the rule functions build spatial distributions of their density.

Module docstring

lines 4–42

```
"""Description of the fast (non-thermal) electrons.

An MHD model contains no fast electrons, so they have to be prescribed.
FastElectrons holds that prescription: at every sample point of a line of
sight it returns the parameters of an analytical distribution function of
the fast GS codes (Kuznetsov & Fleishman 2021, AnalyticalDistributions.pdf):

    f(E, mu) = u(E) g(mu),   2 pi int u dE = n_b,   int g dmu = 1

energy distributions u(E)   ('energy'):
    'plw'  power law in kinetic energy, N(E) ~ E^-delta, E_min < E < E_max
    'dpl'  double power law (delta below, delta2 above E_break)
    'kap'  kappa distribution (uses kappa; replaces the thermal electrons)
    'tnt'  thermal + power-law tail matched at eps (thermal/non-thermal)
    'tpl'  isotropic thermal + power law (thermal gyro emission included)
    'plp', 'plg'  power law in momentum / Lorentz factor
pitch-angle distributions g(mu)   ('pitch'):
    'iso'  isotropic
    'gau'  gaussian loss cone (alpha_c = loss-cone angle, dmu = width)
    'elc'  exponential loss cone
    'gab'  gaussian beam (alpha_c = beam direction, dmu = width)
    'sga'  super-gaussian beam (also a4)

Every parameter may be a number or a *rule*: a callable taking the dict of
sampled quantities

    pos (N,3) [coordinate units], r (N,), ne [m^-3], te [K], b (N,3) [G],
    bmag [G], and any grid scalar fields (e.g. 'j' [A m^-2] after
    radiosynth.common.fields.current_density(grid))

and returning an (N,) array.  The helper functions below build common rules
for the fast-electron density n_b [cm^-3].

Alternatively, an array-defined distribution can be given (from_array): a
tabulated shape f(E, mu), normalised to unit density, multiplied at every
point by the local n_b (for instance from test-particle or Fokker-Planck
results); it is passed to the codes as their numerically defined
distribution.
"""
```

Lists the analytical energy and pitch-angle distributions of the codes and explains rules (functions

of the sampled fields) and array-defined distributions.

Index tables

lines 43–51

```
43 import math
44
45 import numpy as np
46
47 ENERGY_INDEX = {"fff": 0, "thm": 2, "plw": 3, "dpl": 4, "tnt": 5, "kap": 6, "plp": 7,
48                 "plg": 8, "tnp": 9, "tng": 10, "tpl": 11, "tdp": 12}
49 PITCH_INDEX = {"iso": 0, "elc": 2, "gau": 3, "gab": 4, "sga": 5}
50 MEC2_MEV = 0.51099895
51 _trapz = getattr(np, "trapezoid", None) or np.trapz
```

L.47–49 Names of the distributions and the indices the codes expect in `Parms[6]` and `Parms[14]` (AnalyticalDistributions.pdf).

L.51 Trapezoidal integration (numpy 1 and 2).

FastElectrons: constructor

lines 54–89

```
54 class FastElectrons:
55     """Prescription of the fast electrons (see module docstring).
56
57     nb       : fast-electron density [cm^-3] (number or rule); 0 = none
58     energy   : energy distribution ('plw', 'dpl', 'kap', 'tnt', 'tpl', ...)
59     delta    : power-law index (delta_1 for 'dpl')
60     emin_mev, emax_mev : energy cutoffs [MeV] (default 10 keV - 10 MeV)
61     delta2, ebreak_mev : high-energy index and break energy ('dpl', 'tdp')
62     kappa    : kappa index ('kap');  eps : matching parameter ('tnt', 'tnp', 'tng')
63     pitch    : pitch-angle distribution ('iso', 'gau', 'elc', 'gab', 'sga')
64     alpha_c  : loss-cone angle or beam direction [deg];  dmu : width;  a4 : 'sga'
65     nb_min   : n_b below this [cm^-3] is set to 0, so that the (expensive)
66                gyrosynchrotron calculation is skipped there (default 1 cm^-3)
67     """
68
69     def __init__(self, nb=0.0, energy="plw", delta=4.0, emin_mev=0.01, emax_mev=10.0,
70                  delta2=None, ebreak_mev=None, kappa=None, eps=None, pitch="iso",
71                  alpha_c=60.0, dmu=0.2, a4=10.0, nb_min=1.0):
72         energy, pitch = energy.lower(), pitch.lower()
73         if energy not in ENERGY_INDEX:
74             raise ValueError(f"energy must be one of {sorted(ENERGY_INDEX)}")
75         if pitch not in PITCH_INDEX:
76             raise ValueError(f"pitch must be one of {sorted(PITCH_INDEX)}")
77         if energy in ("dpl", "tdp") and (delta2 is None or ebreak_mev is None):
78             raise ValueError("'dpl'/'tdp' need delta2 and ebreak_mev")
79         if energy == "kap" and kappa is None:
80             raise ValueError("'kap' needs kappa")
81         self.nb, self.energy, self.delta = nb, energy, delta
82         self.emin_mev, self.emax_mev = emin_mev, emax_mev
83         self.delta2, self.ebreak_mev = delta2, ebreak_mev
84         self.kappa, self.eps = kappa, eps
85         self.pitch, self.alpha_c, self.dmu, self.a4 = pitch, alpha_c, dmu, a4
86         self.nb_min = float(nb_min)
87         self.array = None                    # (E [MeV], mu, f normalised) if from_array
```

```
88
89      @classmethod
```

L.72–80 Check the distribution names and the parameters each one needs.
L.81–87 Store everything; numbers and rules are both allowed.

FastElectrons.from_array()

lines 90–111

```
90      def from_array(cls, E_mev, mu, f, nb, pitch_average=False):
91          """Array-defined distribution: shape f(E, mu) [MeV^-1, arbitrary units] on
92          the grid E_mev (NE >= 3, increasing) x mu (Nmu >= 3, from -1 to 1),
93          rescaled at every point to the local density n_b [cm^-3] (number or rule)."""
94          E = np.asarray(E_mev, float)
95          m = np.asarray(mu, float)
96          F = np.asarray(f, float)
97          if F.shape != (E.size, m.size) or E.size < 3 or m.size < 3:
98              raise ValueError("f must have shape (NE, Nmu) with NE, Nmu >= 3")
99          norm = 2 * np.pi * _trapz(_trapz(F, m, axis=1), E)
100         obj = cls(nb=nb)
101         obj.array = (E, m, F / norm)
102         obj.pitch_average = pitch_average
103         return obj
104
105     def __repr__(self):
106         if self.array is not None:
107             return f"FastElectrons(array-defined, NE={self.array[0].size}, Nmu={self.array
    ↪ [1].size})"
108         return (f"FastElectrons(energy={self.energy!r}, delta={self.delta}, "
109                 f"E={self.emin_mev}-{self.emax_mev} MeV, pitch={self.pitch!r})")
110
111     @staticmethod
```

L.99–101 A tabulated $f(E,\mu)$ normalised so that $2\pi\iint f\,d\mu\,dE = 1$; it is multiplied by the local n_b when passed to the codes.

Evaluation of numbers and rules

lines 112–129

```
112     def _eval(v, fields, n):
113         if callable(v):
114             return np.broadcast_to(np.asarray(v(fields), float), (n,)).astype(float)
115         return np.full(n, float(v))
116
117     def evaluate(self, fields):
118         """Dict of per-point parameters for the given sampled fields."""
119         n = fields["ne"].shape[0]
120         e = lambda v: self._eval(v, fields, n)                          # noqa: E731
121         nb = e(self.nb)
122         out = dict(nb=np.where(nb >= self.nb_min, nb, 0.0), delta=e(self.delta),
123                    emin=e(self.emin_mev), emax=e(self.emax_mev),
124                    alpha_c=e(self.alpha_c), dmu=e(self.dmu), a4=e(self.a4))
125         out["delta2"] = e(self.delta2) if self.delta2 is not None else np.zeros(n)
126         out["ebreak"] = e(self.ebreak_mev) if self.ebreak_mev is not None else np.zeros(n)
127         k = self.kappa if self.energy == "kap" else self.eps
```

```
128         out["eps_kappa"] = e(k) if k is not None else np.zeros(n)
129         return out
```

L.112 A rule is called with the dict of sampled fields; a number is broadcast.

L.117–129 All parameters for the voxels of a chunk; n_b below `nb_min` is set to zero, so the expensive gyrosynchrotron calculation is skipped there.

Energy check

lines 131–165

```
131     def energy_density(self, fields):
132         """Kinetic energy density of the fast electrons [erg cm^-3] (power-law
133         cases 'plw', 'dpl' analytically; other cases are not estimated: 0)."""
134         p = self.evaluate(fields)
135         nb, d, a, b = p["nb"], p["delta"], p["emin"], p["emax"]
136         if self.array is not None:
137             E, m, F = self.array
138             mean = 2 * np.pi * _trapz(_trapz(F, m, axis=1) * E, E)
139             return nb * mean * 1.602176634e-6
140         if self.energy != "plw":
141             return np.zeros_like(nb)
142 
143         def moment(k):            # int E^(k-delta) dE between a and b
144             q = k + 1 - d
145             return np.where(np.abs(q) > 1e-9, (b**q - a**q) / np.where(np.abs(q) > 1e-9, q,
    ↪ 1),
146                             np.log(b / a))
147         mean = moment(1) / moment(0)                                  # MeV
148         return nb * mean * 1.602176634e-6
149 
150     def check_energy(self, fields, bmag, limit=0.1, verbose=True):
151         """Ratio of the fast-electron to the magnetic energy density (B^2/8pi);
152         warns where it exceeds ``limit``.  Returns the max ratio."""
153         w = self.energy_density(fields)
154         wb = np.maximum(bmag, 1e-30) ** 2 / (8 * math.pi)
155         r = np.where(w > 0, w / wb, 0.0)
156         rmax = float(r.max(initial=0.0))
157         if verbose and rmax > limit:
158             print(f"WARNING: fast-electron energy density reaches {rmax:.2g} x B^2/8pi "
159                   f"(> {limit}); the prescription is probably unphysical")
160         return rmax
161 
162 
163 # ---------------------------------------------------------------------------
164 # rules for n_b [cm^-3]
165 # ---------------------------------------------------------------------------
```

L.143–147 For a power law, the mean energy $\int E^{1-\delta}dE / \int E^{-\delta}dE$ between the cutoffs (logarithm for the exponent -1).

L.139 Energy density $n_b\langle E\rangle$ in erg cm^{-3}.

L.150–160 Ratio to $B^2/8\pi$; a warning above `limit` (default 0.1).

Rules for n_b

lines 166–212

```
166 def uniform(nb):
167     """Constant n_b wherever there is plasma."""
168     return lambda f: np.where(f["ne"] > 0, float(nb), 0.0)
169
170
171 def fraction_of_thermal(fraction):
172     """n_b = fraction x n_e (thermal electron density)."""
173     return lambda f: float(fraction) * f["ne"] * 1e-6
174
175
176 def power_of(field, nb0, ref, power=1.0, floor=0.0):
177     """n_b = nb0 (q/ref)^power for a sampled quantity q = field ('bmag', 'ne',
178     'te', 'r' or a grid scalar such as 'j'); values below ``floor`` x ref give 0."""
179     def rule(f):
180         q = np.maximum(np.asarray(f[field], float), 0.0) / float(ref)
181         return np.where(q >= floor, float(nb0) * q ** float(power), 0.0)
182     return rule
183
184
185 def in_sphere(center, radius, nb, profile="gaussian", cutoff=3.0):
186     """Fast electrons in a blob around ``center`` [coordinate units]:
187     profile 'gaussian' n_b = nb exp(-d^2/radius^2) for d < cutoff x radius
188     (0 beyond), or 'top-hat' (d < radius)."""
189     c = np.asarray(center, float)
190
191     def rule(f):
192         d2 = np.sum((f["pos"] - c) ** 2, axis=1) / float(radius) ** 2
193         if profile == "gaussian":
194             return np.where(d2 < cutoff**2, nb * np.exp(-d2), 0.0)
195         return nb * (d2 < 1.0)
196     return rule
197
198
199 def in_box(lo, hi, nb):
200     """Constant n_b inside the box lo <= pos <= hi."""
201     lo, hi = np.asarray(lo, float), np.asarray(hi, float)
202     return lambda f: np.where(np.all((f["pos"] >= lo) & (f["pos"] <= hi), axis=1), float(nb),
      ↪ 0.0)
203
204
205 def product(*rules):
206     """Product of rules (e.g. in_sphere(...) x power_of('bmag', ...) / nb0)."""
207     def rule(f):
208         out = 1.0
209         for r in rules:
210             out = out * (r(f) if callable(r) else float(r))
211         return out
212     return rule
```

L.166 Constant wherever there is plasma.

L.171 n_b = fraction $\times n_e$ (with n_e in cm^{-3}).

L.176 $n_b = n_{b0}(q/q_{\mathrm{ref}})^p$ for any sampled quantity (e.g. `'bmag'`, `'j'`), zero below `floor`.

L.185 Gaussian blob truncated at `cutoff` radii, or a top-hat sphere.

L.199 Constant inside a box.

L.205 Product of rules and numbers.

Chapter 29

gyrosynchrotron/image.py — make_gs_image and GSImage

File: `radiosynth/gyrosynchrotron/image.py` — 286 lines.

Builds the voxel arrays for the codes, calls them for chunks of lines of sight, and assembles the image.

Module docstring and imports

lines 4–42

```
4  """Gyrosynchrotron (+ free-free) images computed with the fast GS codes.
5
6  radiosynth supplies the geometry and the plasma along every line of sight
7  (common/image.py, common/straight.py): the voxels of each straight ray with
8  their length, n_e, T_e, |B|, the angle between B and the line of sight, and the
9  fast-electron parameters from a FastElectrons prescription.  The radiative
10 transfer along the ray - gyrosynchrotron emission and absorption of the fast
11 electrons, free-free emission of the thermal plasma, the two magneto-ionic
12 modes and their coupling - is done by the fast GS codes (GET_MW_SLICE,
13 Kuznetsov & Fleishman 2021), many lines of sight at once.
14
15 Conventions checked against the library (tests/test_gyrosynchrotron.py):
16   * the viewing angle theta is the angle between B and the direction to the
17     observer; voxel 0 of a line of sight is the far end;
18   * RL rows: 1/2 left/right (weak coupling), 3/4 (strong), 5/6 (exact);
19     right-hand circular polarisation dominates the x-mode emission where B
20     points towards the observer, so V = R - L follows the IAU/IEEE convention
21     used everywhere in radiosynth;
22   * intensities come out as flux densities [sfu] at 1 AU from a source of area
23     Rparms[0]; with the pixel area as source area this gives the intensity
24     I = F AU^2 / S, independent of the actual observer distance.
25 """
26 import math
27 import time
28
29 import numpy as np
30
31 from ..common.constants import AU, C_LIGHT, K_B, R_SUN
32 from ..common.image import freq_label, image_geometry, parse_frequencies, start_points
33 from ..common.raytrace import HIT_BODY
34 from ..common.straight import default_step, iter_samples, segment_ends
35 from ..gyroresonance.image import GRImage
36 from .electrons import ENERGY_INDEX, PITCH_INDEX, FastElectrons
37 from .library import load_library, library_version
38
39 PC = 3.0856775814913673e16
40 SFU = 1e-22
41 COUPLING_ROWS = {"weak": (1, 2), "strong": (3, 4), "exact": (5, 6)}
42 ABUNDANCE = {"dulk": -1, "coronal": 0, "photospheric": 1}
```

States the division of work between radiosynth and the codes and the conventions (viewing angle,

voxel order, output rows, sfu → intensity) that the tests check.

L.41–42 Rows of the output array for each coupling and the abundance keys.

GSImage

lines 45–75

```
45 class GSImage(GRImage):
46     """Gyrosynchrotron image: Stokes I and V (pol1 = R, pol2 = L).
47 
48     gs_fraction (alias gr_fraction): 1 - I_ff/I, the fraction of the intensity
49     due to the fast electrons (only with diagnostics=True, else NaN).
50     tau is not returned by the library (NaN)."""
51 
52     fraction_label = "fast-electron fraction"
53 
54     @property
55     def gs_fraction(self):
56         return self.gr_fraction
57 
58     def summary(self):
59         lines = []
60         for k, f in enumerate(self.freqs):
61             I = self.flux_density[k]
62             V = self.flux_density_v[k]
63             star = self.meta.get("distance_pc") is not None
64             fl = (f"{I / 1e-29:.4g} mJy at {self.distance_pc:.3g} pc" if star
65                   else f"{I / SFU:.4g} sfu")
66             lines.append(f"{freq_label(f):>12s}: max Tb={self.brightness_temperature[k].max()
    ↪ :.3e} K, "
67                          f"flux={fl}, V/I={V / I if I > 0 else 0:+.3f}")
68         return "\n".join(lines)
69 
70     def plot(self, quantity="tb", **kw):
71         if quantity == "gs_fraction":
72             quantity = "gr_fraction"
73         if quantity in ("harmonic", "radius"):
74             raise ValueError(f"'{quantity}' is not available for gyrosynchrotron images")
75         return super().plot(quantity, **kw)
```

A `GRImage` (Stokes I and V, polarisations, plotting, FITS); `gs_fraction` is the fraction of the intensity due to the fast electrons; harmonic and radius maps are not defined here.

L.58 One line per frequency (sfu for the Sun, mJy for a star).

_fields(): voxel quantities

lines 78–88

```
78 def _fields(sm, grid, idx_scalars):
79     """Mid-point quantities of every voxel of a chunk (flattened)."""
80     m = lambda a: 0.5 * (a[:, 1:] + a[:, :-1])                    # noqa: E731
81     pos = m(sm["p"]).reshape(-1, 3)
82     f = dict(pos=pos, r=np.linalg.norm(pos, axis=1),
83              ne=m(sm["ne"]).ravel(), te=m(sm["te"]).ravel(),
84              b=m(sm["b"]).reshape(-1, 3))
85     f["bmag"] = np.linalg.norm(f["b"], axis=1)
```

```
    for name in idx_scalars:
        f[name] = grid.sample_scalar(name, pos)
    return f
```

Mid-point values of position, n_e, T_e, $\mathbf{B}$ and every grid scalar for all voxels of a chunk; this is the dict passed to the rules.

voxel_parms(): the parameters of the codes

lines 91–112

```
def voxel_parms(fl, par, dl_cm, valid, nb, theta, free_free, eidx, pidx, abund, is_array):
    """The 24 voxel parameters of the fast GS codes (CallingConventions.pdf) for
    voxels with quantities fl (from _fields), fast-electron parameters par,
    lengths dl_cm [cm], validity mask, n_b [cm^-3] and viewing angle theta [deg]."""
    P = np.zeros((24, dl_cm.size))
    P[0] = np.where(valid, dl_cm, 0.0)
    P[1] = np.where(valid, np.maximum(fl["te"], 1.0), 1e6)
    P[2] = np.where(valid, fl["ne"] * 1e-6, 0.0)
    P[3] = np.maximum(fl["bmag"], 1e-3)
    P[4] = np.where(fl["bmag"] > 0, theta, 90.0)
    P[5] = (0 if free_free else 2) + 4                # e-neutrals off (no n_HI given)
    P[6] = eidx
    P[7] = nb
    P[8] = par["eps_kappa"]
    P[9], P[10], P[11] = par["emin"], par["emax"], par["ebreak"]
    P[12], P[13] = par["delta"], par["delta2"]
    P[14] = pidx
    P[15], P[16], P[17] = par["alpha_c"], par["dmu"], par["a4"]
    P[18] = P[2]                                      # n_p != 0: n0 is n_e also below 1e5 K
    P[21] = np.where(nb > 0, 2 if is_array else 1, 3)  # analytic / array / free-free only
    P[22] = abund
    return P
```

L.95–111 The 24 voxel parameters of the codes (CallingConventions.pdf): length, T, n_e, B, θ, mechanism flags (free–free on or off; electron–neutral collisions off), distribution indices and parameters, $n_p = n_e$, and the local key: analytical, array-defined, or free–free only where $n_b = 0$. Shared with the combined image.

make_gs_image(): set-up

lines 115–202

```
def make_gs_image(grid, frequencies, electrons, observer=(-215.0, 0.0, 0.0), x_size=2.0,
                  y_size=2.0, nx=64, ny=64, *, coupling="exact", free_free=True,
                  gyrosynchrotron=True, abundance="coronal", ds=None, t_body=None,
                  r_integration=None, up=(0.0, 0.0, 1.0), distance_pc=None, library=None,
                  fc=12.0, fwh=12.0, nnodes=0, q_optimization=True, diagnostics=False,
                  max_voxels=300000, verbose=True):
    """Gyrosynchrotron + free-free image (Stokes I, V) with the fast GS codes.

    grid        : MHDGrid / SphericalGrid with the magnetic field (gauss)
    frequencies : '3GHz, 17GHz' or numbers [Hz]
    electrons   : FastElectrons (radiosynth.gyrosynchrotron.electrons)
    observer, x_size, y_size, nx, ny, up, distance_pc, r_integration :
                  as in make_radio_image / make_gr_image (straight rays)
    coupling    : 'exact' (default), 'weak' or 'strong' mode coupling (library)
```

```
    free_free   : include free-free (electron-ion) emission/absorption
    gyrosynchrotron : include the fast electrons (False: library free-free only)
    abundance   : 'coronal' (Feldman 1992), 'photospheric' (Scott+2015) or
                  'dulk' (Dulk 1985 formulas) for the free-free opacity
    ds          : voxel length along the rays [coordinate units] (default: half
                  the smallest cell)
    t_body      : brightness temperature of the body seen behind the plasma
    library     : path of the compiled GS codes (see library.py)
    fc, fwh     : switch frequencies of the hybrid GS code, in units of the local
                  gyrofrequency (library defaults: 12, 12)
    nnodes, q_optimization : accuracy controls of the continuous GS code
    diagnostics : also compute the free-free-only image -> gs_fraction
    max_voxels  : voxels passed to the library at once (memory control)
    """
    if not getattr(grid, "has_field", False):
        raise ValueError("gyrosynchrotron needs the magnetic field: grid.add_field(...) "
                         "or load_mhd(..., with_b=True)")
    if not isinstance(electrons, FastElectrons):
        raise TypeError("electrons must be a FastElectrons instance")
    if coupling not in COUPLING_ROWS:
        raise ValueError("coupling must be 'exact', 'weak' or 'strong'")
    lib = load_library(library)
    freqs = parse_frequencies(frequencies)
    nf = freqs.size
    L = float(getattr(grid, "length_unit", R_SUN))
    parallel = distance_pc is not None
    xc, yc, dirs, tau_v, xi_v, normal, pix = image_geometry(observer, x_size, y_size, nx, ny,
                                                            up, parallel=parallel)
    diag = math.sqrt((0.5 * x_size) ** 2 + (0.5 * y_size) ** 2)
    if r_integration is None:
        r_integration = float(math.ceil(max(diag + 1.0, 5.0)))
    if parallel:
        distance = distance_pc * PC / L
        far = 4.0 * max(r_integration, getattr(grid, "r_outer", r_integration), diag)
        pos, status0 = start_points(grid, pix + far * normal, dirs, r_integration)
    else:
        distance = float(np.linalg.norm(observer))
        pos, status0 = start_points(grid, observer, dirs, r_integration)
    t_end, status = segment_ends(grid, pos, dirs, status0, r_integration)
    step = default_step(grid, t_end, ds)
    hit_body = status == HIT_BODY
    nray = dirs.shape[0]

    # pixel (source) area in m^2 and the sfu <-> intensity conversion
    s_pix = (x_size / nx * L) * (y_size / ny * L)
    to_int = SFU * AU**2 / s_pix                      # sfu -> W m^-2 Hz^-1 sr^-1

    arr = electrons.array
    ne_, nmu = (arr[0].size, arr[1].size) if arr is not None else (1, 1)
    E_arr = arr[0] if arr is not None else np.zeros(1)
    mu_arr = arr[1] if arr is not None else np.zeros(1)
    log_key = 0
    if arr is not None:
        r = E_arr[1:] / E_arr[:-1]
        log_key = 0 if np.allclose(r, r[0], rtol=1e-3) else 1
    scal = sorted(getattr(grid, "scalars", {}).keys())
    eidx = ENERGY_INDEX[electrons.energy]
    pidx = PITCH_INDEX[electrons.pitch]
```

```
    abund = ABUNDANCE[abundance] if isinstance(abundance, str) else int(abundance)
    if verbose:
        ver = library_version(library)
        print(f"radiosynth.gyrosynchrotron: {nx}x{ny} straight rays, coupling={coupling}, "
              f"free-free={'on' if free_free else 'off'}, voxel={step:.3g}, {electrons}")
        print(f"            GS codes: {lib.path}" + (f" ({ver})" if ver else ""))
        print(f"            {grid}")

    I_R = np.zeros((nf, nray))
    I_L = np.zeros((nf, nray))
    I_ff = np.full((nf, nray), np.nan)
    e_ratio = 0.0
    nb_max = 0.0
    t0 = time.time()
    # chunks of rays; each chunk is one GET_MW_SLICE call (OpenMP over rays)
    per_voxel = max(1, (ne_ * nmu) // 24) if arr is not None else 1
```

L.143–149 Checks.

L.150 Load the codes (error message with build instructions if missing).

L.155–169 Geometry, start points, line lengths and voxel size, as in the gyroresonance driver.

L.173–174 Pixel area and the conversion $I = F\,\mathrm{AU}^2/S$ of the codes' output.

L.176–183 Energy and pitch-angle grids of an array-defined distribution; logarithmic or linear energy spacing.

make_gs_image(): one chunk of lines of sight

lines 203–264

```
    for sm in iter_samples(grid, pos, dirs, t_end, step,
                           max_points=max(1000, max_voxels // per_voxel)):
        idx = sm["idx"]
        npix, nz = sm["dl"].shape
        fl = _fields(sm, grid, scal)
        par = electrons.evaluate(fl)
        dl_cm = (sm["dl"] * L * 100.0).ravel()
        valid = (dl_cm > 0) & (fl["ne"] > 0)
        nb = np.where(valid, par["nb"], 0.0) if gyrosynchrotron else np.zeros_like(dl_cm)
        nb_max = max(nb_max, float(nb.max(initial=0.0)))
        if gyrosynchrotron and nb.max(initial=0) > 0:
            sel = nb > 0
            sub = {k: v[sel] for k, v in fl.items()}
            e_ratio = max(e_ratio, electrons.check_energy(sub, fl["bmag"][sel], verbose=False
    ↪ ))
        nobs = np.repeat(-dirs[idx], nz, axis=0)
        cos = np.einsum("ij,ij->i", fl["b"], nobs) / np.maximum(fl["bmag"], 1e-30)
        theta = np.degrees(np.arccos(np.clip(cos, -1.0, 1.0)))

        P = voxel_parms(fl, par, dl_cm, valid, nb, theta, free_free, eidx, pidx, abund,
                        arr is not None)
        # library order: voxel 0 = far end of the line of sight
        P = P.reshape(24, npix, nz)[:, :, ::-1].transpose(0, 2, 1)
        Parms_M = np.asfortranarray(P)
        if arr is not None:
            fz = (arr[2][:, :, None, None] *
                  nb.reshape(npix, nz)[:, ::-1].T[None, None, :, :])
            f_arr = np.asfortranarray(fz)
        else:
```

```
231             f_arr = np.zeros((1, 1, 1, 1), order="F")
232         Lp = np.array([npix, nz, nf, ne_ if arr is not None else 0, nmu if arr is not None
    ↪ else 0,
233                        int(nnodes), 0, 0 if q_optimization else 1,
234                        1 if arr is None else 2, log_key, 0, 0], np.int32)
235         Rp = np.zeros((5, npix), order="F")
236         Rp[0] = s_pix * 1e4
237         Rp[3], Rp[4] = fc, fwh
238 
239         def run(Pm):
240             RL = np.zeros((7, nf, npix), order="F")
241             RL[0] = (freqs / 1e9)[:, None]
242             if t_body is not None:
243                 body = hit_body[idx]
244                 Ib = freqs[:, None] ** 2 * K_B * float(t_body) / C_LIGHT**2 / to_int
245                 RL[1:7] = np.where(body[None, :], Ib, 0.0)[None]
246             lib.get_mw_slice(Lp, Rp, Pm, E_arr, mu_arr, f_arr, RL)
247             return RL
248 
249         RL = run(Parms_M)
250         lr, rr = COUPLING_ROWS[coupling]
251         I_L[:, idx] = RL[lr] * to_int
252         I_R[:, idx] = RL[rr] * to_int
253         if diagnostics:
254             P2 = Parms_M.copy()
255             P2[21] = 3
256             RL2 = run(np.asfortranarray(P2))
257             I_ff[:, idx] = (RL2[lr] + RL2[rr]) * to_int
258 
259     if verbose:
260         print(f"   {nf} frequencies: {time.time() - t0:.1f} s; max n_b = {nb_max:.3g} cm^-3;
    ↪ "
261               f"max fast-electron/magnetic energy density = {e_ratio:.2g}")
262         if e_ratio > 0.1:
263             print("WARNING: the fast electrons carry more than 10% of the magnetic energy "
264                   "density somewhere - check the prescription")
```

L.207–211 Voxel quantities and fast-electron parameters; empty or zero-length voxels get no electrons.

L.213–216 Energy check where there are fast electrons.

L.217–219 Viewing angle between $\mathbf{B}$ and the direction to the observer.

L.221 The 24 voxel parameters (`voxel_parms`, above).

L.224 Reverse the voxel order: the codes integrate from the far end towards the observer.

L.226–231 Array-defined distribution: the normalised shape times n_b in every voxel, in the (NE, Nmu, Nz, Npix) layout.

L.232–237 Global integer and real parameters: dimensions, accuracy keys, source area, switch frequencies.

L.239–247 One call of GET_MW_SLICE; frequencies given explicitly; a body behind the plasma enters as the intensity at the start of the line of sight.

L.249–252 Left and right intensities for the chosen coupling.

L.253–257 Optional second call without fast electrons.

make_gs_image(): the image

lines 266–286

```
266     shp = (nf, ny, nx)
267     I = I_R + I_L
268     with np.errstate(invalid="ignore", divide="ignore"):
269         frac = np.where(I > 0, 1.0 - I_ff / I, 0.0) if diagnostics else np.full_like(I, np.
        ↪ nan)
270     meta = dict(x_size=x_size, y_size=y_size, nx=nx, ny=ny, coupling=coupling,
271                 free_free=free_free, gyrosynchrotron=gyrosynchrotron, abundance=abundance,
272                 ds=step, t_body=t_body, distance_pc=distance_pc, r_integration=r_integration,
273                 electrons=repr(electrons), library=lib.path, fc=fc, fwh=fwh,
274                 tau_vec=tau_v, xi_vec=xi_v, emission="gyrosynchrotron", transfer="full",
275                 energy_ratio_max=e_ratio)
276     R_ = lambda a: a.reshape(shp)                                    # noqa: E731
277     nanarr = np.full(shp, np.nan)
278     img = GSImage(R_(I), nanarr, R_(np.broadcast_to(status, (nf, nray)).copy()),
279                   np.full(shp, int(np.ceil(np.max(t_end, initial=0) / step))), xc, yc, freqs,
280                   observer, meta, distance=distance, length_unit=L,
281                   stokes_v=R_(I_R - I_L), pol1=R_(I_R), pol2=R_(I_L), tau2=nanarr.copy(),
282                   dominant_s=np.full(shp, -1, np.int16), radius=nanarr.copy(),
283                   gr_fraction=R_(frac), x_flag=np.zeros(shp, bool), pol_names=("R", "L"))
284     if verbose:
285         print(img.summary())
286     return img
```

L.269 Fast-electron fraction $1 - I_{\rm ff}/I$.

L.278–283 $I = I_R + I_L$, $V = I_R - I_L$; optical depths are not available (NaN).

Chapter 30

gyrosynchrotron/__init__.py — package interface

File: `radiosynth/gyrosynchrotron/__init__.py` — 43 lines.

Registers the mechanism and exports its functions.

Mechanism registration

lines 4–43

```
4  """Gyrosynchrotron emission of fast electrons, computed with the public fast
5  GS codes (Fleishman & Kuznetsov 2010; Kuznetsov & Fleishman 2021 - the engine
6  of GX Simulator), which are called as a compiled library.
7
8  * library.py   : finds, builds (git clone + make) and loads the GS codes
9                   (https://github.com/kuznetsov-radio/gyrosynchrotron, GPL-3.0,
10                  not bundled with radiosynth)
11 * electrons.py : FastElectrons - the prescription of the fast electrons
12                  (distribution type, n_b rules, pitch-angle distribution,
13                  array-defined distributions, energy check)
14 * image.py     : make_gs_image and GSImage (Stokes I and V)
15
16 Uses from common/: grids (with B and scalar fields such as the current
17 density, common/fields.py), readers, image geometry, straight-ray sampler.
18
19 >>> import radiosynth as rs
20 >>> from radiosynth.gyrosynchrotron import FastElectrons, in_sphere
21 >>> rs.gyrosynchrotron.build_library()                    # once
22 >>> e = FastElectrons(nb=in_sphere((-1.05, 0, 0), 0.02, 1e7), delta=3.5)
23 >>> img = rs.gyrosynchrotron.make_image(grid, "3GHz, 10GHz, 17GHz", e,
24 ...                                     x_size=0.4, y_size=0.4, nx=80, ny=80)
25 """
26 from ..common.mechanism import Mechanism, register
27 from .electrons import (FastElectrons, uniform, fraction_of_thermal, power_of, in_sphere,
28                         in_box, product)
29 from .library import build_library, find_library, load_library, library_version
30 from .image import make_gs_image, GSImage
31
32 make_image = make_gs_image
33
34 MECHANISM = register(Mechanism(
35     "gyrosynchrotron", coefficients=None, make_image=make_gs_image,
36     description="fast-electron gyrosynchrotron + free-free via the public fast GS codes "
37                 "(Kuznetsov & Fleishman 2021), straight rays, Stokes I and V (needs B "
38                 "and a FastElectrons prescription)",
39     options=("electrons", "coupling", "free_free", "abundance", "t_body", "ds", "library")))
40
41 __all__ = ["FastElectrons", "uniform", "fraction_of_thermal", "power_of", "in_sphere",
42            "in_box", "product", "build_library", "find_library", "load_library",
43            "library_version", "make_gs_image", "make_image", "GSImage", "MECHANISM"]
```

The docstring lists the three modules, the licence of the codes and a short example.

L.32 Same name as in the other folders.

L.34 Registry entry; `coefficients=None` (not usable in the refractive tracer).

Chapter 31

combined/coefficients.py — gyrosynchrotron coefficients per voxel

File: `radiosynth/combined/coefficients.py` — 90 lines.

Extracts the gyrosynchrotron optical depth and emission of single voxels from the fast GS codes, which by themselves only return intensities at the end of a line of sight.

Module docstring and constants

lines 4–30

```
4  """Per-voxel gyrosynchrotron coefficients from the fast GS codes.
5
6  The fast GS codes (Kuznetsov & Fleishman 2021) return intensities at the end
7  of a line of sight, not absorption and emission coefficients.  For a single
8  homogeneous voxel of optical depth tau and source function S (per circular
9  polarisation) the formal solution is
10
11     I_out = I_in exp(-tau) + S (1 - exp(-tau)) = I_in exp(-tau) + E.
12
13 Calling the codes for every voxel as a one-voxel line of sight, once with
14 I_in = 0 (gives E) and once with a large known I_in (gives exp(-tau)), yields
15 tau and E for both circular polarisations.  Only the fast electrons are
16 included in these calls (the codes' free-free is switched off and the thermal
17 plasma enters through radiosynth's own free-free and gyroresonance
18 coefficients), so the three mechanisms can be combined in one transfer.
19
20 All voxels of a chunk are sent in one GET_MW_SLICE call (one "line of sight"
21 per voxel; OpenMP inside the codes).  The weak-coupling output rows are used:
22 for a single voxel there is no mode coupling to treat.
23 """
24 import numpy as np
25
26 from ..common.constants import AU, C_LIGHT, K_B
27
28 SFU = 1e-22
29 _SRC = K_B / C_LIGHT**2
30 S_CM2 = 1e20                        # nominal source area of the one-voxel calls [cm^2]
```

Explains the two one-voxel calls ($I_{\rm in} = 0$ and a known $I_{\rm in}$) and why the codes' free–free is switched off.

`L.30` Nominal source area of the calls; it cancels in the intensity.

gs_voxel_coefficients()

lines 33–76

```
33 def gs_voxel_coefficients(lib, P, freqs, E_arr=None, mu_arr=None, f_arr=None, fc=12.0,
34                           fwh=12.0, nnodes=0, q_optimization=True, log_key=0):
35     """Optical depth and emitted intensity of single voxels.
36
```

```
    lib   : GSLibrary;  P : (24, Nvox) voxel parameters (gyrosynchrotron.voxel_parms,
            with free-free off);  freqs : [Hz]
    E_arr, mu_arr, f_arr : array-defined distribution (f_arr: (NE, Nmu, Nvox)) or None
    Returns dict tau_R, tau_L, E_R, E_L of shape (nf, Nvox); E in W m^-2 Hz^-1 sr^-1.
    """
    nv = P.shape[1]
    nf = freqs.size
    out = {k: np.zeros((nf, nv)) for k in ("tau_R", "tau_L", "E_R", "E_L")}
    if nv == 0:
        return out
    to_int = SFU * AU**2 / (S_CM2 * 1e-4)
    arr = f_arr is not None
    ne_ = E_arr.size if arr else 0
    nmu = mu_arr.size if arr else 0
    Lp = np.array([nv, 1, nf, ne_, nmu, int(nnodes), 0, 0 if q_optimization else 1,
                   2 if arr else 1, log_key, 0, 0], np.int32)
    Rp = np.zeros((5, nv), order="F")
    Rp[0], Rp[3], Rp[4] = S_CM2, fc, fwh
    Pm = np.asfortranarray(P.reshape(24, 1, nv))
    Ea = E_arr if arr else np.zeros(1)
    ma = mu_arr if arr else np.zeros(1)
    fa = np.asfortranarray(f_arr.reshape(ne_, nmu, 1, nv)) if arr else np.zeros((1, 1, 1, 1),
                                                                                order="F")

    def run(I0):
        RL = np.zeros((7, nf, nv), order="F")
        RL[0] = (freqs / 1e9)[:, None]
        if I0 is not None:
            RL[1:7] = I0[None]
        lib.get_mw_slice(Lp, Rp, Pm, Ea, ma, fa, RL)
        return RL[1].copy(), RL[2].copy()            # L, R (weak coupling rows)

    E_L, E_R = run(None)
    I0 = 1e6 * np.maximum(np.maximum(E_L, E_R), 1e-30) + 1e-20
    O_L, O_R = run(I0)
    for pol, E, O in (("L", E_L, O_L), ("R", E_R, O_R)):
        att = np.clip((O - E) / I0, 1e-300, 1.0)
        out["tau_" + pol] = np.minimum(-np.log(att), 700.0)
        out["E_" + pol] = np.maximum(E, 0.0) * to_int
    return out
```

L.51–58 One GET_MW_SLICE call treats every voxel as its own one-voxel line of sight; array-defined distributions are passed per voxel.

L.61–67 One call, with or without incoming intensity; rows 1/2 (left/right, weak coupling: no coupling inside one voxel).

L.69–71 Emission E alone, then with $I_{\text{in}} = 10^6 E$ (large, so that $e^{-\tau}$ is resolved to round-off).

L.73–75 $e^{-\tau} = (I_{\text{out}} - E)/I_{\text{in}}$, τ capped at 700; E converted from sfu to intensity.

source_weights()

lines 79–90

```
def source_weights(coef, freqs):
    """tau x effective temperature [K] of every voxel and polarisation, the
    quantity added to the step sums of the polarised transfer:
        tau T_eff = tau E / ((1 - exp(-tau)) nu^2 k/c^2)   (-> E/(nu^2 k/c^2) as tau -> 0).
    """
```

```
    f2 = (freqs ** 2)[:, None] * _SRC
    out = {}
    for pol in ("R", "L"):
        t, E = coef["tau_" + pol], coef["E_" + pol]
        with np.errstate(invalid="ignore", divide="ignore"):
            fac = np.where(t > 1e-12, t / -np.expm1(-np.maximum(t, 1e-300)), 1.0)
        out["st_" + pol] = E * fac / f2
    return out
```

$\tau T_{\mathrm{eff}} = \tau E/[(1-e^{-\tau})\,\nu^2 k_B/c^2]$, the quantity added to the step sums of the transfer (limit $E/(\nu^2 k_B/c^2)$ for $\tau \to 0$).

Chapter 32

combined/image.py — make_combined_image and CombinedImage

File: `radiosynth/combined/image.py` — 335 lines.

The combined driver: straight rays, radiosynth's free–free and layers, the gyrosynchrotron coefficients of the voxels with fast electrons, one polarised transfer, and the optional refraction correction of the free–free part.

Module docstring and imports

lines 4–49

```
4  """Combined free-free + gyroresonance + gyrosynchrotron images.
5
6  Along every straight line of sight the three mechanisms are put into one
7  polarised radiative transfer (gyroresonance/transfer.py), so that they emit
8  and absorb together:
9
10   * free-free (magnetised, o/x modes)        bremsstrahlung/magnetized.py
11   * gyroresonance layers nu = s nu_B         gyroresonance/opacity.py
12   * gyrosynchrotron of the fast electrons    per-voxel tau and emission from
13                                              the fast GS codes (coefficients.py)
14
15 In each step the optical depths add and the source function is the
16 optical-depth-weighted mean of the contributions (S = sum j / sum kappa), so
17 e.g. an optically thick gyroresonance layer in front of a gyrosynchrotron
18 source hides it, and dense plasma absorbs what lies behind.
19
20 Refraction: gyroresonance and gyrosynchrotron are always computed along
21 straight rays.  The free-free part can optionally (ff_refraction=True) be
22 corrected for refraction: the refractive ray tracer (as in
23 make_radio_image) is run with and without refraction, and in every pixel and
24 polarisation the free-free contribution of the combined transfer is multiplied
25 by the ratio refracted/straight of the free-free intensity,
26
27     I_p -> I_p + I_ff,p (R - 1),   R = I_ff(refracted) / I_ff(straight),
28
29 i.e. the free-free emission keeps the absorption by the layers and the fast
30 electrons computed along the straight ray.  This is exact where refraction is
31 negligible (R = 1) and approximate otherwise; where the straight free-free
32 intensity vanishes, half of the difference I_ff(refracted) - I_ff(straight)
33 is added to each polarisation.
34 """
35 import math
36 import time
37
38 import numpy as np
39
40 from ..common.constants import R_SUN
41 from ..common.image import freq_label, image_geometry, parse_frequencies, start_points
42 from ..common.raytrace import HIT_BODY, MISSED, OVERDENSE
43 from ..common.straight import default_step, iter_samples, segment_ends
```

```python
from ..gyroresonance.image import GRImage
from ..gyroresonance.transfer import transfer_chunk
from .coefficients import gs_voxel_coefficients, source_weights

PC = 3.0856775814913673e16
MECH_NAMES = {-1: "none", 0: "free-free", 1: "gyroresonance", 2: "gyrosynchrotron"}
```

Describes how the three processes are combined and the refraction correction $I_p \to I_p + I_{\mathrm{ff},p}(R-1)$ of the free–free part.

L.49 Codes of the dominant-mechanism map.

CombinedImage

lines 52–142

```python
class CombinedImage(GRImage):
    """GRImage with the contributions of the three mechanisms.

    ff_fraction, gr_fraction, gs_fraction : share of the intensity from
        free-free, gyroresonance layers and gyrosynchrotron (sum = 1)
    dominant_mechanism : 0 free-free, 1 gyroresonance, 2 gyrosynchrotron, -1 none
    dominant_s : dominant harmonic among the gyroresonance layers (as GRImage)
    """

    fraction_label = "gyroresonance fraction"

    def __init__(self, *a, gs_fraction=None, **kw):
        super().__init__(*a, **kw)
        self.gs_fraction = gs_fraction
        self.ff_fraction = np.clip(1.0 - self.gr_fraction - gs_fraction, 0.0, 1.0)
        st = np.stack([self.ff_fraction, self.gr_fraction, gs_fraction])
        self.dominant_mechanism = np.where(self.intensity > 0, np.argmax(st, axis=0),
                                           -1).astype(np.int16)

    def summary(self):
        lines = []
        for k, f in enumerate(self.freqs):
            I = self.flux_density[k]
            V = self.flux_density_v[k]
            w = self.intensity[k]
            sh = [float((fr[k] * w).sum() / max(w.sum(), 1e-300))
                  for fr in (self.ff_fraction, self.gr_fraction, self.gs_fraction)]
            star = self.meta.get("distance_pc") is not None
            fl = (f"{I / 1e-29:.4g} mJy" if star else f"{I / 1e-22:.4g} sfu")
            lines.append(f"{freq_label(f):>12s}: max Tb={self.brightness_temperature[k].max()
    ↪ :.3e} K, "
                         f"flux={fl}, V/I={V / I if I > 0 else 0:+.3f}; flux from "
                         f"ff/gr/gs = {sh[0]:.2f}/{sh[1]:.2f}/{sh[2]:.2f}")
        return "\n".join(lines)

    def plot(self, quantity="tb", path=None, **kw):
        """Also 'mechanism' (dominant mechanism map), 'ff_fraction', 'gs_fraction'."""
        if quantity in ("mechanism", "ff_fraction", "gs_fraction"):
            return self._plot_extra(quantity, path, **kw)
        return super().plot(quantity, path=path, **kw)

    def _plot_extra(self, quantity, path, ncols=None, show_limb=True, **_):
```

```
93          import matplotlib
94          if path is not None:
95              matplotlib.use("Agg")
96          import matplotlib.pyplot as plt
97          from matplotlib.colors import ListedColormap, BoundaryNorm
98          nf = len(self.freqs)
99          ncols = ncols or min(nf, 3)
100         nrows = math.ceil(nf / ncols)
101         fig, axes = plt.subplots(nrows, ncols, figsize=(4.6 * ncols, 4.0 * nrows), squeeze=
        ↪ False)
102         dx, dy = self.x[1] - self.x[0], self.y[1] - self.y[0]
103         ext = [self.x[0] - dx / 2, self.x[-1] + dx / 2, self.y[0] - dy / 2, self.y[-1] + dy /
        ↪  2]
104         for k in range(nrows * ncols):
105             ax = axes.flat[k]
106             if k >= nf:
107                 ax.axis("off")
108                 continue
109             if quantity == "mechanism":
110                 cmap = ListedColormap(["#d9d9d9", "#4c72b0", "#dd8452", "#c44e52"])
111                 norm = BoundaryNorm([-1.5, -0.5, 0.5, 1.5, 2.5], cmap.N)
112                 im = ax.imshow(self.dominant_mechanism[k], origin="lower", extent=ext,
113                                cmap=cmap, norm=norm)
114                 cb = fig.colorbar(im, ax=ax, ticks=[-1, 0, 1, 2], shrink=0.85)
115                 cb.ax.set_yticklabels(["none", "free-free", "gyroresonance", "gyrosynchrotron
        ↪ "])
116             else:
117                 d = getattr(self, quantity)[k]
118                 im = ax.imshow(d, origin="lower", extent=ext, cmap="magma", vmin=0, vmax=1)
119                 fig.colorbar(im, ax=ax, label=quantity.replace("_", " "), shrink=0.85)
120             if show_limb:
121                 t = np.linspace(0, 2 * np.pi, 361)
122                 ax.plot(np.cos(t), np.sin(t), "k--", lw=0.7, alpha=0.6)
123             ax.set_xlim(ext[0], ext[1])
124             ax.set_ylim(ext[2], ext[3])
125             ax.set_title(freq_label(self.freqs[k]))
126             ax.set_xlabel(r"X [$R_\odot$]")
127             ax.set_ylabel(r"Y [$R_\odot$]")
128         fig.tight_layout()
129         if path is not None:
130             fig.savefig(path, dpi=130)
131             plt.close(fig)
132         return fig
133
134     def save_npz(self, path):
135         super().save_npz(path)
136         z = dict(np.load(path, allow_pickle=True))
137         z.update(ff_fraction=self.ff_fraction, gs_fraction=self.gs_fraction,
138                  dominant_mechanism=self.dominant_mechanism)
139         np.savez_compressed(path, **z)
140
141
142 # ---------------------------------------------------------------------------
```

L.63–69 Free–free share = 1− (layers + gyrosynchrotron); the dominant process is the largest share.

L.71 Flux, V/I and the flux-weighted shares of the three processes per frequency.

L.86–90 Extra plots: `mechanism`, `ff_fraction`, `gs_fraction`.

L.92–132 Categorical map of the dominant process, or a fraction map.

L.134 Adds the shares and the dominant-process map to the saved file.

make_combined_image(): set-up

lines 143–231

```
143 def make_combined_image(grid, frequencies, electrons=None, observer=(-215.0, 0.0, 0.0),
144                        x_size=2.0, y_size=2.0, nx=128, ny=128, *,
145                        include=("ff", "gr", "gs"), gaunt="dulk", s_max=10, coupling="weak",
146                        ds=None, t_body=None, r_integration=None, up=(0.0, 0.0, 1.0),
147                        distance_pc=None, library=None, fc=12.0, fwh=12.0, nnodes=0,
148                        q_optimization=True, max_points=200000, verbose=True,
149                        ff_refraction=False, trace_kw=None):
150     """Free-free + gyroresonance + gyrosynchrotron in one polarised transfer.
151
152     grid        : grid with the magnetic field (load_mhd(..., with_b=True))
153     frequencies : '3GHz, 17GHz' or numbers [Hz]
154     electrons   : FastElectrons prescription (radiosynth.gyrosynchrotron); None
155                   = no fast electrons (then the result equals make_gr_image)
156     include     : any subset of ('ff', 'gr', 'gs')
157     gaunt, s_max, coupling, ds, t_body : as in make_gr_image
158     library, fc, fwh, nnodes, q_optimization : as in make_gs_image
159     observer, x_size, y_size, nx, ny, up, distance_pc, r_integration : as usual
160     ff_refraction : False (default; free-free along the straight rays like the other
161                     two) or True: correct the free-free part for refraction with the
162                     refractive ray tracer (see the module docstring); gyroresonance and
163                     gyrosynchrotron always use straight rays
164     trace_kw    : options of the refractive tracer for ff_refraction (tolerance,
165                     step_factor, n_workers, ...)
166     """
167     if not getattr(grid, "has_field", False):
168         raise ValueError("the combined image needs the magnetic field: grid.add_field(...) "
169                          "or load_mhd(..., with_b=True)")
170     include = tuple(m.lower() for m in include)
171     bad = set(include) - {"ff", "gr", "gs"}
172     if bad:
173         raise ValueError(f"unknown mechanisms {bad}; use 'ff', 'gr', 'gs'")
174     use_gs = "gs" in include and electrons is not None
175     mech = "+".join(m for m in ("gr", "ff") if m in include)
176     if use_gs:
177         from ..gyrosynchrotron.electrons import ENERGY_INDEX, PITCH_INDEX, FastElectrons
178         from ..gyrosynchrotron.image import _fields, voxel_parms
179         from ..gyrosynchrotron.library import load_library, library_version
180         if not isinstance(electrons, FastElectrons):
181             raise TypeError("electrons must be a FastElectrons instance")
182         lib = load_library(library)
183         eidx = ENERGY_INDEX[electrons.energy]
184         pidx = PITCH_INDEX[electrons.pitch]
185         arr = electrons.array
186         scal = sorted(getattr(grid, "scalars", {}).keys())
187         log_key = 0
188         if arr is not None:
189             r = arr[0][1:] / arr[0][:-1]
190             log_key = 0 if np.allclose(r, r[0], rtol=1e-3) else 1
191     freqs = parse_frequencies(frequencies)
192     nf = freqs.size
```

```
193     L = float(getattr(grid, "length_unit", R_SUN))
194     parallel = distance_pc is not None
195     xc, yc, dirs, tau_v, xi_v, normal, pix = image_geometry(observer, x_size, y_size, nx, ny,
196                                                             up, parallel=parallel)
197     diag = math.sqrt((0.5 * x_size) ** 2 + (0.5 * y_size) ** 2)
198     if r_integration is None:
199         r_integration = float(math.ceil(max(diag + 1.0, 5.0)))
200     if parallel:
201         distance = distance_pc * PC / L
202         far = 4.0 * max(r_integration, getattr(grid, "r_outer", r_integration), diag)
203         pos, status0 = start_points(grid, pix + far * normal, dirs, r_integration)
204     else:
205         distance = float(np.linalg.norm(observer))
206         pos, status0 = start_points(grid, observer, dirs, r_integration)
207     t_end, status = segment_ends(grid, pos, dirs, status0, r_integration)
208     step = default_step(grid, t_end, ds)
209     hit_body = status == HIT_BODY
210     nray = dirs.shape[0]
211     if verbose:
212         print(f"radiosynth.combined: {nx}x{ny} straight rays"
213               f"{' (free-free corrected for refraction)' if ff_refraction else ''}, "
214               f"include={'+'.join(include)}, "
215               f"coupling={coupling}, step={step:.3g}"
216               + (f", {electrons}" if use_gs else ", no fast electrons"))
217         if use_gs:
218             ver = library_version(library)
219             print(f"            GS codes: {lib.path}" + (f" ({ver})" if ver else ""))
220         print(f"            {grid}")
221 
222     keys = ("I1", "I2", "tau1", "tau2", "V", "radius", "gr_fraction", "gs_fraction",
223             "ff1", "ff2", "vsign")
224     res = {k: np.zeros((nf, nray)) for k in keys}
225     res["radius"][:] = np.nan
226     dom = np.full((nf, nray), -1, np.int16)
227     cut = np.zeros((nf, nray), bool)
228     xf = np.zeros((nf, nray), bool)
229     t0 = time.time()
230     nvox_gs = 0
231     e_ratio = 0.0
```

L.170–175 Which processes; the free–free and layer part is passed to the gyroresonance transfer as its `mechanisms` string.

L.176–190 Fast-electron machinery from the gyrosynchrotron folder (library, distribution indices, array-defined distribution).

L.195–209 Geometry, straight lines and step, exactly as in `make_gr_image` (so that without fast electrons the result is identical).

make_combined_image(): one chunk

lines 232–284

```
232     for sm in iter_samples(grid, pos, dirs, t_end, step, max_points=max_points):
233         idx = sm["idx"]
234         nr, nk = sm["dl"].shape
235         extra_all = None
236         if use_gs:
237             fl = _fields(sm, grid, scal)
```

```
238             par = electrons.evaluate(fl)
239             dl_cm = (sm["dl"] * L * 100.0).ravel()
240             valid = (dl_cm > 0) & (fl["ne"] > 0)
241             nb = np.where(valid, par["nb"], 0.0)
242             sel = np.flatnonzero(nb > 0)
243             if sel.size:
244                 nvox_gs += sel.size
245                 sub = {k: v[sel] for k, v in fl.items()}
246                 e_ratio = max(e_ratio, electrons.check_energy(sub, fl["bmag"][sel], verbose=
    ↪ False))
247                 nobs = np.repeat(-dirs[idx], nk, axis=0)[sel]
248                 cos = np.einsum("ij,ij->i", sub["b"], nobs) / np.maximum(sub["bmag"], 1e-30)
249                 theta = np.degrees(np.arccos(np.clip(cos, -1.0, 1.0)))
250                 psub = {k: v[sel] for k, v in par.items()}
251                 P = voxel_parms(sub, psub, dl_cm[sel], np.ones(sel.size, bool), nb[sel],
    ↪ theta,
252                                 False, eidx, pidx, -1, arr is not None)
253                 if arr is not None:
254                     f_arr = arr[2][:, :, None] * nb[sel][None, None, :]
255                     coef = gs_voxel_coefficients(lib, P, freqs, arr[0], arr[1], f_arr, fc,
    ↪ fwh,
256                                                  nnodes, q_optimization, log_key)
257                 else:
258                     coef = gs_voxel_coefficients(lib, P, freqs, fc=fc, fwh=fwh, nnodes=nnodes
    ↪ ,
259                                                  q_optimization=q_optimization)
260                 sw = source_weights(coef, freqs)
261                 extra_all = {}
262                 for key, val in (("dtau_R", coef["tau_R"]), ("dtau_L", coef["tau_L"]),
263                                  ("st_R", sw["st_R"]), ("st_L", sw["st_L"])):
264                     a = np.zeros((nf, nr * nk))
265                     a[:, sel] = val
266                     extra_all[key] = a.reshape(nf, nr, nk)
267         for k, f in enumerate(freqs):
268             extra = None if extra_all is None else {key: v[k] for key, v in extra_all.items()
    ↪ }
269             if extra is None and use_gs:
270                 z = np.zeros((nr, nk))
271                 extra = dict(dtau_R=z, dtau_L=z, st_R=z, st_L=z)
272             o = transfer_chunk(sm, dirs, f, L, mechanisms=mech, gaunt=gaunt, s_max=s_max,
273                                coupling=coupling, t_body=t_body, hit_body=hit_body, extra=
    ↪ extra)
274             for key in keys:
275                 if key in o:
276                     res[key][k, idx] = o[key]
277             dom[k, idx], cut[k, idx], xf[k, idx] = o["dominant_s"], o["cutoff"], o["x_flag"]
278     if verbose:
279         print(f"   {nf} frequencies: {time.time() - t0:.1f} s"
280               + (f"; {nvox_gs} voxels with fast electrons, max fast-electron/magnetic "
281                  f"energy density = {e_ratio:.2g}" if use_gs else ""))
282         if e_ratio > 0.1:
283             print("WARNING: the fast electrons carry more than 10% of the magnetic energy "
284                   "density somewhere - check the prescription")
```

L.237–242 Voxels with fast electrons.

L.251–252 Their parameters for the codes, with free–free switched off.

L.253–259 Per-voxel coefficients for all frequencies in one go.

L.260–266 Scatter them onto the (frequency, ray, step) grid.

L.267–277 One polarised transfer per frequency with the extra contribution.

make_combined_image(): refraction correction of the free-free part

lines 286–312

```
286     ff_factor = None
287     if ff_refraction and "ff" in include:
288         from ..common.image import make_radio_image
289         t1 = time.time()
290         common = dict(observer=observer, x_size=x_size, y_size=y_size, nx=nx, ny=ny,
291                       emission="bremsstrahlung", gaunt=gaunt, transfer="full", up=up,
292                       r_integration=r_integration, distance_pc=distance_pc, verbose=False,
293                       **(trace_kw or {}))
294         i_ref = make_radio_image(grid, freqs, **common).intensity.reshape(nf, nray)
295         i_los = make_radio_image(grid, freqs, no_refraction=True, **common).intensity.reshape
    ↪ (nf, nray)
296         good = i_los > 1e-6 * np.maximum(i_los.max(axis=1, keepdims=True), 1e-300)
297         with np.errstate(invalid="ignore", divide="ignore"):
298             ff_factor = np.where(good, i_ref / np.where(good, i_los, 1.0), np.nan)
299         I_old = res["I1"] + res["I2"]
300         d1 = np.where(good, res["ff1"] * (ff_factor - 1.0), 0.5 * (i_ref - i_los))
301         d2 = np.where(good, res["ff2"] * (ff_factor - 1.0), 0.5 * (i_ref - i_los))
302         res["I1"] = np.maximum(res["I1"] + d1, 0.0)
303         res["I2"] = np.maximum(res["I2"] + d2, 0.0)
304         res["V"] = res["V"] + res["vsign"] * (d1 - d2)
305         I_new = res["I1"] + res["I2"]
306         with np.errstate(invalid="ignore", divide="ignore"):
307             for key in ("gr_fraction", "gs_fraction"):
308                 res[key] = np.where(I_new > 0, res[key] * I_old / I_new, 0.0)
309         if verbose:
310             fr_ = i_ref.sum(axis=1) / np.maximum(i_los.sum(axis=1), 1e-300)
311             print(f"   free-free refraction correction: {time.time() - t1:.1f} s; refracted/
    ↪ straight "
312                   f"free-free flux = {np.round(fr_, 4)}")
```

L.294–295 Free–free images from the refractive tracer, with and without refraction (same Gaunt factor, full transfer).

L.298 R per pixel where the straight free–free image is not negligible.

L.300–304 $I_p \to I_p + I_{\mathrm{ff},p}(R-1)$; where R is undefined, half of the difference is added to each polarisation; V follows.

L.307–308 The shares are renormalised to the new intensity.

make_combined_image(): the image

lines 314–335

```
314     shp = (nf, ny, nx)
315     st = np.broadcast_to(status, (nf, nray)).copy()
316     st[cut & (st != MISSED)] = OVERDENSE
317     meta = dict(x_size=x_size, y_size=y_size, nx=nx, ny=ny, include=include,
318                 mechanisms=mech, gaunt=gaunt, s_max=s_max, coupling=coupling, ds=step,
319                 t_body=t_body, distance_pc=distance_pc, r_integration=r_integration,
320                 electrons=repr(electrons), tau_vec=tau_v, xi_vec=xi_v, emission="combined",
321                 ff_refraction=bool(ff_refraction and "ff" in include),
322                 transfer="full", energy_ratio_max=e_ratio)
```

```
323     R = lambda a: a.reshape(shp)                                  # noqa: E731
324     img = CombinedImage(R(res["I1"] + res["I2"]), R(res["tau1"]), R(st),
325                         np.full(shp, int(np.ceil(np.max(t_end, initial=0) / step))), xc, yc,
326                         freqs, observer, meta, distance=distance, length_unit=L,
327                         stokes_v=R(res["V"]), pol1=R(res["I1"]), pol2=R(res["I2"]),
328                         tau2=R(res["tau2"]), dominant_s=R(dom), radius=R(res["radius"]),
329                         gr_fraction=R(res["gr_fraction"]), x_flag=R(xf),
330                         pol_names=("R", "L") if coupling == "weak" else ("x", "o"),
331                         gs_fraction=R(res["gs_fraction"]))
332     img.ff_refraction_factor = R(ff_factor) if ff_factor is not None else None
333     if verbose:
334         print(img.summary())
335     return img
```

`L.324` Assemble the image; `ff_refraction_factor` holds R.

Chapter 33

combined/__init__.py — package interface

File: `radiosynth/combined/__init__.py` — 35 lines.

Registers the combined mechanism.

Mechanism registration

lines 4–35

```
4  """Combined image: free-free + gyroresonance + gyrosynchrotron in one transfer.
5
6  * coefficients.py : per-voxel gyrosynchrotron optical depth and emission
7                      extracted from the fast GS codes (one-voxel calls)
8  * image.py        : make_combined_image and CombinedImage (Stokes I and V,
9                      contribution of each mechanism, dominant-mechanism map)
10
11 Uses the polarised straight-ray transfer of gyroresonance/ (with the
12 magnetised free-free of bremsstrahlung/) and the fast-electron prescription
13 and library interface of gyrosynchrotron/.
14
15 >>> import radiosynth as rs
16 >>> from radiosynth.gyrosynchrotron import FastElectrons, in_sphere
17 >>> e = FastElectrons(nb=in_sphere((-1.04, 0, 0), 0.015, 1e7), delta=3.5)
18 >>> img = rs.combined.make_image(grid, "3GHz, 10GHz, 17GHz", e, x_size=0.4, y_size=0.4)
19 >>> img.plot("tb"); img.plot("mechanism"); img.plot("pol")
20 """
21 from ..common.mechanism import Mechanism, register
22 from .coefficients import gs_voxel_coefficients, source_weights
23 from .image import make_combined_image, CombinedImage, MECH_NAMES
24
25 make_image = make_combined_image
26
27 MECHANISM = register(Mechanism(
28     "combined", coefficients=None, make_image=make_combined_image,
29     description="free-free + gyroresonance + gyrosynchrotron in one polarised transfer "
30                 "(straight rays; gyrosynchrotron needs the fast GS codes and a "
31                 "FastElectrons prescription)",
32     options=("electrons", "include", "coupling", "s_max", "gaunt", "t_body", "ds", "library")
    ↪ ))
33
34 __all__ = ["gs_voxel_coefficients", "source_weights", "make_combined_image", "make_image",
35            "CombinedImage", "MECH_NAMES", "MECHANISM"]
```

The docstring lists the modules and what they use from the other folders.

L.25 Same name as in the other folders.

L.27 Registry entry.

Chapter 34

radiosynth/__init__.py — package interface

File: `radiosynth/__init__.py` — 81 lines.

Defines what `import radiosynth as rs` provides, and the short module paths (`radiosynth.grid`, ...).

Docstring: the package layout

lines 4–44

```
"""radiosynth - Synthetic Radio Imaging of MHD Solar-Stellar Coronal Model Output
and Comparison with Observations.

Package layout
--------------
radiosynth/
    common/          machinery used by every mechanism: grids, readers (BATS-R-US,
                     PLUTO, pyPLUTO), refractive ray tracer, straight-ray sampler,
                     image driver, magneto-ionic theory, figures, solar orientation
                     and comparison with observations, stellar observers, FITS, CLI
    simplistic/      Benkevitch et al. (2010) ad-hoc emissivity
    bremsstrahlung/  thermal free-free emission (Moschou et al. 2018), two-mode
                     magnetised free-free, analytic wind solution
    gyroresonance/   thermal gyroresonance (harmonic layers s nu_B = nu) combined
                     with magnetised free-free, Stokes I and V
    gyrosynchrotron/ fast-electron gyrosynchrotron + free-free, computed with the
                     public fast GS codes (Kuznetsov & Fleishman 2021) as a library
    combined/        free-free + gyroresonance + gyrosynchrotron in one polarised
                     radiative transfer

Works with BATS-R-US (Tecplot / IDL output), PLUTO (.dbl/.flt/.vtk, Cartesian
or spherical) and any model given as numpy arrays, for the Sun and for other
stars.  A Python re-implementation of the BATS-R-US radio-wave imaging tool
(Benkevitch et al. 2010, ApJ 725, 447; Moschou et al. 2018, ApJ 867, 51) that
works on saved MHD output instead of running inside BATS-R-US.

Quick start
-----------
>>> import radiosynth as rs
>>> grid = rs.load_mhd("box_mhd_t001000.dat")              # or rs.MHDGrid(x, y, z, ne, te)
>>> img = rs.make_radio_image(grid, "80MHz, 150MHz, 17GHz",
...                           observer=(-215, 0, 0), x_size=6, y_size=6,
...                           nx=256, ny=256, n_workers=8)
>>> img.plot("tb", path="radio.png"); img.save_npz("radio.npz")

Gyroresonance (needs the magnetic field):
>>> grid = rs.load_mhd("box_mhd_t001000.dat", with_b=True)
>>> gr = rs.gyroresonance.make_image(grid, "5GHz, 17GHz", observer=(-215, 0, 0),
...                                  x_size=1, y_size=1, nx=200, ny=200)
>>> gr.plot("tb"); gr.plot("v")
"""
```

Describes the four folders and gives a free–free and a gyroresonance quick start.

Public names

lines 45–69

```
45 import sys as _sys
46
47 from .common import (constants, emission, grid, raytrace, image, readers, pluto,
48                       fitsio, figures, stellar, mechanism, magnetoionic, straight, fields,
49                       symmetric)
50 from .common.constants import critical_density, plasma_frequency, R_SUN, AU_RSUN
51 from .common.emission import EmissionModel
52 from .common.grid import MHDGrid, SphericalGrid
53 from .common.raytrace import trace_rays, STATUS_NAMES
54 from .common.image import (make_radio_image, RadioImage, trace_paths, image_geometry,
55                            parse_frequencies, compare_refraction)
56 from .common.readers import (load_mhd, build_grid, grid_from_arrays, read_tecplot_ascii,
57                              read_pytecplot, read_batsrus_idl, derive_ne_te, points_to_grid)
58 from .common.pluto import (load_pluto, read_pluto_binary, read_pluto_vtk, read_pluto_pypluto,
59                            parse_definitions)
60 from .common.stellar import (stellar_observer, make_stellar_image, stellar_spectrum,
61                              rotational_modulation, spectral_index)
62 from .common.mechanism import get_mechanism, available_mechanisms
63 from . import simplistic, bremsstrahlung, gyroresonance, gyrosynchrotron, combined
64 from .combined import make_combined_image, CombinedImage
65 from .gyroresonance import make_gr_image, GRImage
66 from .gyrosynchrotron import make_gs_image, GSImage, FastElectrons
67 from .common.fields import current_density
68 from .common.symmetric import (grid_from_profile, read_profile, make_symmetric_image,
69                                plot_profile)
```

L.47 The shared modules, also available as `rs.grid`, `rs.figures`, ...

L.50–62 The main classes and functions.

L.63 Importing the five folders registers their mechanisms (the gyrosynchrotron library itself is loaded only when an image is made).

L.64 The combined driver at top level.

L.65–67 The gyroresonance and gyrosynchrotron drivers, `FastElectrons` and `current_density` at top level.

Short module paths

lines 71–81

```
71 # short module paths: "import radiosynth.grid", "from radiosynth.figures import ..."
72 for _name, _mod in (("constants", constants), ("emission", emission), ("grid", grid),
73                     ("raytrace", raytrace), ("image", image), ("readers", readers),
74                     ("pluto", pluto), ("fitsio", fitsio), ("figures", figures),
75                     ("stellar", stellar)):
76     _sys.modules.setdefault(f"{__name__}.{_name}", _mod)
77 from .common import contrib as _contrib  # noqa: E402
78 _sys.modules.setdefault(f"{__name__}.contrib", _contrib)
79 del _name, _mod
80
81 __version__ = "2.2.0"
```

L.72–78 Registers the old module names (`radiosynth.grid`, `radiosynth.figures`, `radiosynth.contrib`, ...) as aliases of the modules in `common/`, so `from radiosynth.constants import R_SUN` and similar short imports work.

`L.81` Package version (2.2.0).

Chapter 35

radiosynth/__main__.py — python -m radiosynth

File: `radiosynth/__main__.py` — 10 lines.

Three lines: runs `common/cli.main`.

Entry point

lines 4–10

```
4  """python -m radiosynth ...  (see radiosynth/common/cli.py)"""
5  import sys
6
7  from .common.cli import main
8
9  if __name__ == "__main__":
10     sys.exit(main())
```